

GNAT Reference Manual

GNAT, The GNU Ada Compiler
For GCC version 4.5.0

(GCC)

Copyright © 1995-2008, Free Software Foundation, Inc.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, with the Front-Cover Texts being “GNAT Reference Manual”, and with no Back-Cover Texts. A copy of the license is included in the section entitled “GNU Free Documentation License”.

About This Guide

This manual contains useful information in writing programs using the GNAT compiler. It includes information on implementation dependent characteristics of GNAT, including all the information required by Annex M of the Ada language standard.

GNAT implements Ada 95 and Ada 2005, and it may also be invoked in Ada 83 compatibility mode. By default, GNAT assumes Ada 2005, but you can override with a compiler switch to explicitly specify the language version. (Please refer to [Section “Compiling Different Versions of Ada”](#) in *GNAT User’s Guide*, for details on these switches.) Throughout this manual, references to “Ada” without a year suffix apply to both the Ada 95 and Ada 2005 versions of the language.

Ada is designed to be highly portable. In general, a program will have the same effect even when compiled by different compilers on different platforms. However, since Ada is designed to be used in a wide variety of applications, it also contains a number of system dependent features to be used in interfacing to the external world.

Note: Any program that makes use of implementation-dependent features may be non-portable. You should follow good programming practice and isolate and clearly document any sections of your program that make use of these features in a non-portable manner.

What This Reference Manual Contains

This reference manual contains the following chapters:

- [Chapter 1 \[Implementation Defined Pragmas\]](#), [page 5](#), lists GNAT implementation-dependent pragmas, which can be used to extend and enhance the functionality of the compiler.
- [Chapter 2 \[Implementation Defined Attributes\]](#), [page 63](#), lists GNAT implementation-dependent attributes which can be used to extend and enhance the functionality of the compiler.
- [Chapter 3 \[Implementation Advice\]](#), [page 75](#), provides information on generally desirable behavior which are not requirements that all compilers must follow since it cannot be provided on all systems, or which may be undesirable on some systems.
- [Chapter 4 \[Implementation Defined Characteristics\]](#), [page 101](#), provides a guide to minimizing implementation dependent features.
- [Chapter 5 \[Intrinsic Subprograms\]](#), [page 129](#), describes the intrinsic subprograms implemented by GNAT, and how they can be imported into user application programs.
- [Chapter 6 \[Representation Clauses and Pragmas\]](#), [page 133](#), describes in detail the way that GNAT represents data, and in particular the exact set of representation clauses and pragmas that is accepted.
- [Chapter 7 \[Standard Library Routines\]](#), [page 157](#), provides a listing of packages and a brief description of the functionality that is provided by Ada’s extensive set of standard library routines as implemented by GNAT.
- [Chapter 8 \[The Implementation of Standard I/O\]](#), [page 167](#), details how the GNAT implementation of the input-output facilities.
- [Chapter 9 \[The GNAT Library\]](#), [page 183](#), is a catalog of packages that complement the Ada predefined library.

- [Chapter 10 \[Interfacing to Other Languages\]](#), [page 199](#), describes how programs written in Ada using GNAT can be interfaced to other programming languages.
- [Chapter 11 \[Specialized Needs Annexes\]](#), [page 201](#), describes the GNAT implementation of all of the specialized needs annexes.
- [Chapter 12 \[Implementation of Specific Ada Features\]](#), [page 203](#), discusses issues related to GNAT’s implementation of machine code insertions, tasking, and several other features.
- [Chapter 13 \[Project File Reference\]](#), [page 213](#), presents the syntax and semantics of project files.
- [Chapter 14 \[Obsolescent Features\]](#), [page 229](#) documents implementation dependent features, including pragmas and attributes, which are considered obsolescent, since there are other preferred ways of achieving the same results. These obsolescent forms are retained for backwards compatibility.

This reference manual assumes a basic familiarity with the Ada 95 language, as described in the International Standard ANSI/ISO/IEC-8652:1995, January 1995. It does not require knowledge of the new features introduced by Ada 2005, (officially known as ISO/IEC 8652:1995 with Technical Corrigendum 1 and Amendment 1). Both reference manuals are included in the GNAT documentation package.

Conventions

Following are examples of the typographical and graphic conventions used in this guide:

- Functions, utility program names, standard names, and classes.
- Option flags
- ‘File names’, ‘button names’, and ‘field names’.
- Variables, environment variables, and *metasyntactic variables*.
- *Emphasis*.
- [optional information or parameters]
- Examples are described by text
and then shown this way.

Commands that are entered by the user are preceded in this manual by the characters ‘\$ ’ (dollar sign followed by space). If your system uses this sequence as a prompt, then the commands will appear exactly as you see them in the manual. If your system uses some other prompt, then the command will appear with the ‘\$’ replaced by whatever prompt character you are using.

Related Information

See the following documents for further information on GNAT:

- See [Section “About This Guide” in *GNAT User’s Guide*](#), which provides information on how to use the GNAT compiler system.
- *Ada 95 Reference Manual*, which contains all reference material for the Ada 95 programming language.

- *Ada 95 Annotated Reference Manual*, which is an annotated version of the Ada 95 standard. The annotations describe detailed aspects of the design decision, and in particular contain useful sections on Ada 83 compatibility.
- *Ada 2005 Reference Manual*, which contains all reference material for the Ada 2005 programming language.
- *Ada 2005 Annotated Reference Manual*, which is an annotated version of the Ada 2005 standard. The annotations describe detailed aspects of the design decision, and in particular contain useful sections on Ada 83 and Ada 95 compatibility.
- *DEC Ada, Technical Overview and Comparison on DIGITAL Platforms*, which contains specific information on compatibility between GNAT and DEC Ada 83 systems.
- *DEC Ada, Language Reference Manual, part number AA-PYZAB-TK* which describes in detail the pragmas and attributes provided by the DEC Ada 83 compiler system.

1 Implementation Defined Pragmas

Ada defines a set of pragmas that can be used to supply additional information to the compiler. These language defined pragmas are implemented in GNAT and work as described in the Ada Reference Manual.

In addition, Ada allows implementations to define additional pragmas whose meaning is defined by the implementation. GNAT provides a number of these implementation-defined pragmas, which can be used to extend and enhance the functionality of the compiler. This section of the GNAT Reference Manual describes these additional pragmas.

Note that any program using these pragmas might not be portable to other compilers (although GNAT implements this set of pragmas on all platforms). Therefore if portability to other compilers is an important consideration, the use of these pragmas should be minimized.

Pragma Abort_Defer

Syntax:

```
pragma Abort_Defer;
```

This pragma must appear at the start of the statement sequence of a handled sequence of statements (right after the **begin**). It has the effect of deferring aborts for the sequence of statements (but not for the declarations or handlers, if any, associated with this statement sequence).

Pragma Ada_83

Syntax:

```
pragma Ada_83;
```

A configuration pragma that establishes Ada 83 mode for the unit to which it applies, regardless of the mode set by the command line switches. In Ada 83 mode, GNAT attempts to be as compatible with the syntax and semantics of Ada 83, as defined in the original Ada 83 Reference Manual as possible. In particular, the keywords added by Ada 95 and Ada 2005 are not recognized, optional package bodies are allowed, and generics may name types with unknown discriminants without using the (<>) notation. In addition, some but not all of the additional restrictions of Ada 83 are enforced.

Ada 83 mode is intended for two purposes. Firstly, it allows existing Ada 83 code to be compiled and adapted to GNAT with less effort. Secondly, it aids in keeping code backwards compatible with Ada 83. However, there is no guarantee that code that is processed correctly by GNAT in Ada 83 mode will in fact compile and execute with an Ada 83 compiler, since GNAT does not enforce all the additional checks required by Ada 83.

Pragma Ada_95

Syntax:

```
pragma Ada_95;
```

A configuration pragma that establishes Ada 95 mode for the unit to which it applies, regardless of the mode set by the command line switches. This mode is set automatically for the **Ada** and **System** packages and their children, so you need not specify it in these

contexts. This pragma is useful when writing a reusable component that itself uses Ada 95 features, but which is intended to be usable from either Ada 83 or Ada 95 programs.

Pragma Ada_05

Syntax:

```
pragma Ada_05;
```

A configuration pragma that establishes Ada 2005 mode for the unit to which it applies, regardless of the mode set by the command line switches. This mode is set automatically for the `Ada` and `System` packages and their children, so you need not specify it in these contexts. This pragma is useful when writing a reusable component that itself uses Ada 2005 features, but which is intended to be usable from either Ada 83 or Ada 95 programs.

Pragma Ada_2005

Syntax:

```
pragma Ada_2005;
```

This configuration pragma is a synonym for `pragma Ada_05` and has the same syntax and effect.

Pragma Annotate

Syntax:

```
pragma Annotate (IDENTIFIER [,IDENTIFIER] {, ARG});

ARG ::= NAME | EXPRESSION
```

This pragma is used to annotate programs. *identifier* identifies the type of annotation. GNAT verifies that it is an identifier, but does not otherwise analyze it. The second optional identifier is also left unanalyzed, and by convention is used to control the action of the tool to which the annotation is addressed. The remaining *arg* arguments can be either string literals or more generally expressions. String literals are assumed to be either of type `Standard.String` or else `Wide_String` or `Wide_Wide_String` depending on the character literals they contain. All other kinds of arguments are analyzed as expressions, and must be unambiguous.

The analyzed pragma is retained in the tree, but not otherwise processed by any part of the GNAT compiler. This pragma is intended for use by external tools, including ASIS.

Pragma Assert

Syntax:

```
pragma Assert (
  boolean_EXPRESSION
  [, string_EXPRESSION]);
```

The effect of this pragma depends on whether the corresponding command line switch is set to activate assertions. The pragma expands into code equivalent to the following:

```
if assertions-enabled then
  if not boolean_EXPRESSION then
    System.Assertions.Raise_Assert_Failure
      (string_EXPRESSION);
```



```

    end if;
end if;

```

The string argument, if given, is the message that will be associated with the exception occurrence if the exception is raised. If no second argument is given, the default message is ‘*file:nnn*’, where *file* is the name of the source file containing the assert, and *nnn* is the line number of the assert. A pragma is not a statement, so if a statement sequence contains nothing but a pragma assert, then a null statement is required in addition, as in:

```

...
if J > 3 then
  pragma Assert (K > 3, "Bad value for K");
  null;
end if;

```

Note that, as with the `if` statement to which it is equivalent, the type of the expression is either `Standard.Boolean`, or any type derived from this standard type.

If assertions are disabled (switch ‘`-gnata`’ not used), then there is no run-time effect (and in particular, any side effects from the expression will not occur at run time). (The expression is still analyzed at compile time, and may cause types to be frozen if they are mentioned here for the first time).

If assertions are enabled, then the given expression is tested, and if it is `False` then `System.Assertions.Raise_Assert_Failure` is called which results in the raising of `Assert_Failure` with the given message.

You should generally avoid side effects in the expression arguments of this pragma, because these side effects will turn on and off with the setting of the assertions mode, resulting in assertions that have an effect on the program. However, the expressions are analyzed for semantic correctness whether or not assertions are enabled, so turning assertions on and off cannot affect the legality of a program.

Pragma Assume_No_Invalid_Values

Syntax:

```
pragma Assume_No_Invalid_Values (On | Off);
```

This is a configuration pragma that controls the assumptions made by the compiler about the occurrence of invalid representations (invalid values) in the code.

The default behavior (corresponding to an `Off` argument for this pragma), is to assume that values may in general be invalid unless the compiler can prove they are valid. Consider the following example:

```

V1 : Integer range 1 .. 10;
V2 : Integer range 11 .. 20;
...
for J in V2 .. V1 loop
  ...
end loop;

```

if `V1` and `V2` have valid values, then the loop is known at compile time not to execute since the lower bound must be greater than the upper bound. However in default mode, no such assumption is made, and the loop may execute. If `Assume_No_Invalid_Values (On)` is given, the compiler will assume that any occurrence of a variable other than in an explicit ‘`Valid`’ test always has a valid value, and the loop above will be optimized away.

The use of `Assume_No_Invalid_Values (On)` is appropriate if you know your code is free of uninitialized variables and other possible sources of invalid representations, and may result in more efficient code. A program that accesses an invalid representation with this pragma in effect is erroneous, so no guarantees can be made about its behavior.

It is peculiar though permissible to use this pragma in conjunction with validity checking (`-gnatVa`). In such cases, accessing invalid values will generally give an exception, though formally the program is erroneous so there are no guarantees that this will always be the case, and it is recommended that these two options not be used together.

Pragma `Ast_Entry`

Syntax:

```
pragma AST_Entry (entry_IDENTIFIER);
```

This pragma is implemented only in the OpenVMS implementation of GNAT. The argument is the simple name of a single entry; at most one `AST_Entry` pragma is allowed for any given entry. This pragma must be used in conjunction with the `AST_Entry` attribute, and is only allowed after the entry declaration and in the same task type specification or single task as the entry to which it applies. This pragma specifies that the given entry may be used to handle an OpenVMS asynchronous system trap (AST) resulting from an OpenVMS system service call. The pragma does not affect normal use of the entry. For further details on this pragma, see the DEC Ada Language Reference Manual, section 9.12a.

Pragma `C_Pass_By_Copy`

Syntax:

```
pragma C_Pass_By_Copy
  ([Max_Size =>] static_integer_EXPRESSION);
```

Normally the default mechanism for passing C convention records to C convention subprograms is to pass them by reference, as suggested by RM B.3(69). Use the configuration pragma `C_Pass_By_Copy` to change this default, by requiring that record formal parameters be passed by copy if all of the following conditions are met:

- The size of the record type does not exceed the value specified for `Max_Size`.
- The record type has `Convention C`.
- The formal parameter has this record type, and the subprogram has a foreign (non-Ada) convention.

If these conditions are met the argument is passed by copy, i.e. in a manner consistent with what C expects if the corresponding formal in the C prototype is a struct (rather than a pointer to a struct).

You can also pass records by copy by specifying the convention `C_Pass_By_Copy` for the record type, or by using the extended `Import` and `Export` pragmas, which allow specification of passing mechanisms on a parameter by parameter basis.

Pragma `Check`

Syntax:

```
pragma Check (
  [Name    =>] Identifier,
  [Check   =>] Boolean_EXPRESSION
  [, [Message =>] string_EXPRESSION] );
```

This pragma is similar to the predefined pragma **Assert** except that an extra identifier argument is present. In conjunction with pragma **Check_Policy**, this can be used to define groups of assertions that can be independently controlled. The identifier **Assertion** is special, it refers to the normal set of pragma **Assert** statements. The identifiers **Precondition** and **Postcondition** correspond to the pragmas of these names, so these three names would normally not be used directly in a pragma **Check**.

Checks introduced by this pragma are normally deactivated by default. They can be activated either by the command line option ‘-gnata’, which turns on all checks, or individually controlled using pragma **Check_Policy**.

Pragma Check_Name

Syntax:

```
pragma Check_Name (check_name_IDENTIFIER);
```

This is a configuration pragma that defines a new implementation defined check name (unless IDENTIFIER matches one of the predefined check names, in which case the pragma has no effect). Check names are global to a partition, so if two or more configuration pragmas are present in a partition mentioning the same name, only one new check name is introduced.

An implementation defined check name introduced with this pragma may be used in only three contexts: **pragma Suppress**, **pragma Unsuppress**, and as the prefix of a **Check_Name’Enabled** attribute reference. For any of these three cases, the check name must be visible. A check name is visible if it is in the configuration pragmas applying to the current unit, or if it appears at the start of any unit that is part of the dependency set of the current unit (e.g., units that are mentioned in **with** clauses).

Pragma Check_Policy

Syntax:

```
pragma Check_Policy
  ([Name    =>] Identifier,
   [Policy =>] POLICY_IDENTIFIER);

POLICY_IDENTIFIER ::= On | Off | Check | Ignore
```

This pragma is similar to the predefined pragma **Assertion_Policy**, except that it controls sets of named assertions introduced using the **Check** pragmas. It can be used as a configuration pragma or (unlike **Assertion_Policy**) can be used within a declarative part, in which case it controls the status to the end of the corresponding construct (in a manner identical to pragma **Suppress**).

The identifier given as the first argument corresponds to a name used in associated **Check** pragmas. For example, if the pragma:

```
pragma Check_Policy (Critical_Error, Off);
```

is given, then subsequent **Check** pragmas whose first argument is also **Critical_Error** will be disabled. The special identifier **Assertion** controls the behavior of normal **Assert** pragmas (thus a pragma **Check_Policy** with this identifier is similar to the normal **Assertion_Policy** pragma except that it can appear within a declarative part).

The special identifiers **Precondition** and **Postcondition** control the status of preconditions and postconditions. If a **Precondition** pragma is encountered, it is ignored if turned off by a **Check_Policy** specifying that **Precondition** checks are **Off** or **Ignored**. Similarly use of the name **Postcondition** controls whether **Postcondition** pragmas are recognized.

The check policy is **Off** to turn off corresponding checks, and **On** to turn on corresponding checks. The default for a set of checks for which no **Check_Policy** is given is **Off** unless the compiler switch ‘-gnata’ is given, which turns on all checks by default.

The check policy settings **Check** and **Ignore** are also recognized as synonyms for **On** and **Off**. These synonyms are provided for compatibility with the standard **Assertion_Policy** pragma.

Pragma Comment

Syntax:

```
pragma Comment (static_string_EXPRESSION);
```

This is almost identical in effect to pragma **Ident**. It allows the placement of a comment into the object file and hence into the executable file if the operating system permits such usage. The difference is that **Comment**, unlike **Ident**, has no limitations on placement of the pragma (it can be placed anywhere in the main source unit), and if more than one pragma is used, all comments are retained.

Pragma Common_Object

Syntax:

```
pragma Common_Object (
    [Internal =>] LOCAL_NAME
    [, [External =>] EXTERNAL_SYMBOL]
    [, [Size    =>] EXTERNAL_SYMBOL] );

EXTERNAL_SYMBOL ::=
    IDENTIFIER
    | static_string_EXPRESSION
```

This pragma enables the shared use of variables stored in overlaid linker areas corresponding to the use of **COMMON** in Fortran. The single object *LOCAL_NAME* is assigned to the area designated by the *External* argument. You may define a record to correspond to a series of fields. The *Size* argument is syntax checked in GNAT, but otherwise ignored.

Common_Object is not supported on all platforms. If no support is available, then the code generator will issue a message indicating that the necessary attribute for implementation of this pragma is not available.

Pragma Compile_Time_Error

Syntax:

```
pragma Compile_Time_Error
    (boolean_EXPRESSION, static_string_EXPRESSION);
```

This pragma can be used to generate additional compile time error messages. It is particularly useful in generics, where errors can be issued for specific problematic instantiations. The first parameter is a boolean expression. The pragma is effective only if the value of this expression is known at compile time, and has the value True. The set of expressions whose values are known at compile time includes all static boolean expressions, and also other values which the compiler can determine at compile time (e.g., the size of a record type set by an explicit size representation clause, or the value of a variable which was initialized to a constant and is known not to have been modified). If these conditions are met, an error message is generated using the value given as the second argument. This string value may contain embedded ASCII.LF characters to break the message into multiple lines.

Pragma Compile_Time_Warning

Syntax:

```
pragma Compile_Time_Warning
    (boolean_EXPRESSION, static_string_EXPRESSION);
```

Same as pragma Compile_Time_Error, except a warning is issued instead of an error message. Note that if this pragma is used in a package that is with'ed by a client, the client will get the warning even though it is issued by a with'ed package (normally warnings in with'ed units are suppressed, but this is a special exception to that rule).

One typical use is within a generic where compile time known characteristics of formal parameters are tested, and warnings given appropriately. Another use with a first parameter of True is to warn a client about use of a package, for example that it is not fully implemented.

Pragma Compiler_Unit

Syntax:

```
pragma Compiler_Unit;
```

This pragma is intended only for internal use in the GNAT run-time library. It indicates that the unit is used as part of the compiler build. The effect is to disallow constructs (raise with message, conditional expressions etc) that would cause trouble when bootstrapping using an older version of GNAT. For the exact list of restrictions, see the compiler sources and references to Is_Compiler_Unit.

Pragma Complete_Representation

Syntax:

```
pragma Complete_Representation;
```

This pragma must appear immediately within a record representation clause. Typical placements are before the first component clause or after the last component clause. The effect is to give an error message if any component is missing a component clause. This pragma may be used to ensure that a record representation clause is complete, and that this invariant is maintained if fields are added to the record in the future.

Pragma Complex_Representation

Syntax:

```
pragma Complex_Representation
  ([Entity =>] LOCAL_NAME);
```

The *Entity* argument must be the name of a record type which has two fields of the same floating-point type. The effect of this pragma is to force gcc to use the special internal complex representation form for this record, which may be more efficient. Note that this may result in the code for this type not conforming to standard ABI (application binary interface) requirements for the handling of record types. For example, in some environments, there is a requirement for passing records by pointer, and the use of this pragma may result in passing this type in floating-point registers.

Pragma Component_Alignment

Syntax:

```
pragma Component_Alignment (
  [Form =>] ALIGNMENT_CHOICE
  [, [Name =>] type_LOCAL_NAME]);

ALIGNMENT_CHOICE ::=
  Component_Size
| Component_Size_4
| Storage_Unit
| Default
```

Specifies the alignment of components in array or record types. The meaning of the *Form* argument is as follows:

Component_Size

Aligns scalar components and subcomponents of the array or record type on boundaries appropriate to their inherent size (naturally aligned). For example, 1-byte components are aligned on byte boundaries, 2-byte integer components are aligned on 2-byte boundaries, 4-byte integer components are aligned on 4-byte boundaries and so on. These alignment rules correspond to the normal rules for C compilers on all machines except the VAX.

Component_Size_4

Naturally aligns components with a size of four or fewer bytes. Components that are larger than 4 bytes are placed on the next 4-byte boundary.

Storage_Unit

Specifies that array or record components are byte aligned, i.e. aligned on boundaries determined by the value of the constant `System.Storage_Unit`.

Default

Specifies that array or record components are aligned on default boundaries, appropriate to the underlying hardware or operating system or both. For Open-VMS VAX systems, the `Default` choice is the same as the `Storage_Unit` choice (byte alignment). For all other systems, the `Default` choice is the same as `Component_Size` (natural alignment).

If the *Name* parameter is present, *type_LOCAL_NAME* must refer to a local record or array type, and the specified alignment choice applies to the specified type. The use of `Component_Alignment` together with a pragma `Pack` causes the `Component_Alignment` pragma to be

ignored. The use of `Component_Alignment` together with a record representation clause is only effective for fields not specified by the representation clause.

If the `Name` parameter is absent, the pragma can be used as either a configuration pragma, in which case it applies to one or more units in accordance with the normal rules for configuration pragmas, or it can be used within a declarative part, in which case it applies to types that are declared within this declarative part, or within any nested scope within this declarative part. In either case it specifies the alignment to be applied to any record or array type which has otherwise standard representation.

If the alignment for a record or array type is not specified (using pragma `Pack`, pragma `Component_Alignment`, or a record rep clause), the GNAT uses the default alignment as described previously.

Pragma `Convention_Identifier`

Syntax:

```
pragma Convention_Identifier (
    [Name =>]      IDENTIFIER,
    [Convention =>] convention_IDENTIFIER);
```

This pragma provides a mechanism for supplying synonyms for existing convention identifiers. The `Name` identifier can subsequently be used as a synonym for the given convention in other pragmas (including for example pragma `Import` or another `Convention_Identifier` pragma). As an example of the use of this, suppose you had legacy code which used `Fortran77` as the identifier for Fortran. Then the pragma:

```
pragma Convention_Identifier (Fortran77, Fortran);
```

would allow the use of the convention identifier `Fortran77` in subsequent code, avoiding the need to modify the sources. As another example, you could use this to parametrize convention requirements according to systems. Suppose you needed to use `Stdcall` on windows systems, and `C` on some other system, then you could define a convention identifier `Library` and use a single `Convention_Identifier` pragma to specify which convention would be used system-wide.

Pragma `CPP_Class`

Syntax:

```
pragma CPP_Class ([Entity =>] LOCAL_NAME);
```

The argument denotes an entity in the current declarative region that is declared as a record type. It indicates that the type corresponds to an externally declared C++ class type, and is to be laid out the same way that C++ would lay out the type. If the C++ class has virtual primitives then the record must be declared as a tagged record type.

Types for which `CPP_Class` is specified do not have assignment or equality operators defined (such operations can be imported or declared as subprograms as required). Initialization is allowed only by constructor functions (see pragma `CPP_Constructor`). Such types are implicitly limited if not explicitly declared as limited or derived from a limited type, and an error is issued in that case.

Pragma `CPP_Class` is intended primarily for automatic generation using an automatic binding generator tool. See [Section 10.2 \[Interfacing to C++\]](#), page 200 for related information.

Note: `Pragma CPP_Class` is currently obsolete. It is supported for backward compatibility but its functionality is available using `pragma Import` with `Convention = CPP`.

Pragma CPP_Constructor

Syntax:

```
pragma CPP_Constructor ([Entity =>] LOCAL_NAME
  [, [External_Name =>] static_string_EXPRESSION ]
  [, [Link_Name      =>] static_string_EXPRESSION ]);
```

This pragma identifies an imported function (imported in the usual way with `pragma Import`) as corresponding to a C++ constructor. If `External_Name` and `Link_Name` are not specified then the `Entity` argument is a name that must have been previously mentioned in a `pragma Import` with `Convention = CPP`. Such name must be of one of the following forms:

- `function Fname return T`
- `function Fname return T'Class`
- `function Fname (...) return T`
- `function Fname (...) return T'Class`

where *T* is a limited record type imported from C++ with `pragma Import` and `Convention = CPP`.

The first two forms import the default constructor, used when an object of type *T* is created on the Ada side with no explicit constructor. The latter two forms cover all the non-default constructors of the type. See the GNAT users guide for details.

If no constructors are imported, it is impossible to create any objects on the Ada side and the type is implicitly declared abstract.

`Pragma CPP_Constructor` is intended primarily for automatic generation using an automatic binding generator tool. See [Section 10.2 \[Interfacing to C++\]](#), page 200 for more related information.

Note: The use of functions returning class-wide types for constructors is currently obsolete. They are supported for backward compatibility. The use of functions returning the type *T* leave the Ada sources more clear because the imported C++ constructors always return an object of type *T*; that is, they never return an object whose type is a descendant of type *T*.

Pragma CPP_Virtual

This pragma is now obsolete has no effect because GNAT generates the same object layout than the G++ compiler.

See [Section 10.2 \[Interfacing to C++\]](#), page 200 for related information.

Pragma CPP_Vtable

This pragma is now obsolete has no effect because GNAT generates the same object layout than the G++ compiler.

See [Section 10.2 \[Interfacing to C++\]](#), page 200 for related information.

Pragma Debug

Syntax:

```
pragma Debug ([CONDITION, ]PROCEDURE_CALL_WITHOUT_SEMICOLON);

PROCEDURE_CALL_WITHOUT_SEMICOLON ::=
  PROCEDURE_NAME
  | PROCEDURE_PREFIX ACTUAL_PARAMETER_PART
```

The procedure call argument has the syntactic form of an expression, meeting the syntactic requirements for pragmas.

If debug pragmas are not enabled or if the condition is present and evaluates to False, this pragma has no effect. If debug pragmas are enabled, the semantics of the pragma is exactly equivalent to the procedure call statement corresponding to the argument with a terminating semicolon. Pragmas are permitted in sequences of declarations, so you can use `pragma Debug` to intersperse calls to debug procedures in the middle of declarations. Debug pragmas can be enabled either by use of the command line switch ‘`-gnata`’ or by use of the configuration pragma `Debug_Policy`.

Pragma Debug_Policy

Syntax:

```
pragma Debug_Policy (CHECK | IGNORE);
```

If the argument is `CHECK`, then pragma `DEBUG` is enabled. If the argument is `IGNORE`, then pragma `DEBUG` is ignored. This pragma overrides the effect of the ‘`-gnata`’ switch on the command line.

Pragma Detect_Blocking

Syntax:

```
pragma Detect_Blocking;
```

This is a configuration pragma that forces the detection of potentially blocking operations within a protected operation, and to raise `Program_Error` if that happens.

Pragma Elaboration_Checks

Syntax:

```
pragma Elaboration_Checks (Dynamic | Static);
```

This is a configuration pragma that provides control over the elaboration model used by the compilation affected by the pragma. If the parameter is `Dynamic`, then the dynamic elaboration model described in the Ada Reference Manual is used, as though the ‘`-gnatE`’ switch had been specified on the command line. If the parameter is `Static`, then the default GNAT static model is used. This configuration pragma overrides the setting of the command line. For full details on the elaboration models used by the GNAT compiler, see [Section “Elaboration Order Handling in GNAT” in *GNAT User’s Guide*](#).

Pragma Eliminate

Syntax:

```

pragma Eliminate (
  [Unit_Name =>] IDENTIFIER |
    SELECTED_COMPONENT);

pragma Eliminate (
  [Unit_Name      =>] IDENTIFIER |
    SELECTED_COMPONENT,
  [Entity         =>] IDENTIFIER |
    SELECTED_COMPONENT |
    STRING_LITERAL
  [,OVERLOADING_RESOLUTION]);

OVERLOADING_RESOLUTION ::= PARAMETER_AND_RESULT_TYPE_PROFILE |
  SOURCE_LOCATION

PARAMETER_AND_RESULT_TYPE_PROFILE ::= PROCEDURE_PROFILE |
  FUNCTION_PROFILE

PROCEDURE_PROFILE ::= Parameter_Types => PARAMETER_TYPES

FUNCTION_PROFILE ::= [Parameter_Types => PARAMETER_TYPES,]
  Result_Type => result_SUBTYPE_NAME]

PARAMETER_TYPES ::= (SUBTYPE_NAME {, SUBTYPE_NAME})
SUBTYPE_NAME    ::= STRING_VALUE

SOURCE_LOCATION ::= Source_Location => SOURCE_TRACE
SOURCE_TRACE    ::= STRING_VALUE

STRING_VALUE ::= STRING_LITERAL {& STRING_LITERAL}

```

This pragma indicates that the given entity is not used outside the compilation unit it is defined in. The entity must be an explicitly declared subprogram; this includes generic subprogram instances and subprograms declared in generic package instances.

If the entity to be eliminated is a library level subprogram, then the first form of pragma `Eliminate` is used with only a single argument. In this form, the `Unit_Name` argument specifies the name of the library level unit to be eliminated.

In all other cases, both `Unit_Name` and `Entity` arguments are required. If item is an entity of a library package, then the first argument specifies the unit name, and the second argument specifies the particular entity. If the second argument is in string form, it must correspond to the internal manner in which GNAT stores entity names (see compilation unit `Namet` in the compiler sources for details).

The remaining parameters (`OVERLOADING_RESOLUTION`) are optionally used to distinguish between overloaded subprograms. If a pragma does not contain the `OVERLOADING_RESOLUTION` parameter(s), it is applied to all the overloaded subprograms denoted by the first two parameters.

Use `PARAMETER_AND_RESULT_TYPE_PROFILE` to specify the profile of the subprogram to be eliminated in a manner similar to that used for the extended `Import` and `Export` pragmas, except that the subtype names are always given as strings. At the moment, this form of distinguishing overloaded subprograms is implemented only partially, so we do not recommend using it for practical subprogram elimination.

Note that in case of a parameterless procedure its profile is represented as `Parameter_Types => (")`

Alternatively, the `Source_Location` parameter is used to specify which overloaded alternative is to be eliminated by pointing to the location of the `DEFINING_PROGRAM_UNIT_NAME` of this subprogram in the source text. The string literal (or concatenation of string literals) given as `SOURCE_TRACE` must have the following format:

```
SOURCE_TRACE ::= SOURCE_LOCATION{LBRACKET SOURCE_LOCATION RBRACKET}

LBRACKET ::= [
RBRACKET ::= ]

SOURCE_LOCATION ::= FILE_NAME:LINE_NUMBER
FILE_NAME      ::= STRING_LITERAL
LINE_NUMBER    ::= DIGIT {DIGIT}
```

`SOURCE_TRACE` should be the short name of the source file (with no directory information), and `LINE_NUMBER` is supposed to point to the line where the defining name of the subprogram is located.

For the subprograms that are not a part of generic instantiations, only one `SOURCE_LOCATION` is used. If a subprogram is declared in a package instantiation, `SOURCE_TRACE` contains two `SOURCE_LOCATION`s, the first one is the location of the (`DEFINING_PROGRAM_UNIT_NAME` of the) instantiation, and the second one denotes the declaration of the corresponding subprogram in the generic package. This approach is recursively used to create `SOURCE_LOCATION`s in case of nested instantiations.

The effect of the pragma is to allow the compiler to eliminate the code or data associated with the named entity. Any reference to an eliminated entity outside the compilation unit it is defined in, causes a compile time or link time error.

The intention of pragma **Eliminate** is to allow a program to be compiled in a system independent manner, with unused entities eliminated, without the requirement of modifying the source text. Normally the required set of **Eliminate** pragmas is constructed automatically using the `gnatelim` tool. Elimination of unused entities local to a compilation unit is automatic, without requiring the use of pragma **Eliminate**.

Note that the reason this pragma takes string literals where names might be expected is that a pragma **Eliminate** can appear in a context where the relevant names are not visible.

Note that any change in the source files that includes removing, splitting of adding lines may make the set of **Eliminate** pragmas using `SOURCE_LOCATION` parameter illegal.

It is legal to use pragma **Eliminate** where the referenced entity is a dispatching operation, but it is not clear what this would mean, since in general the call does not know which entity is actually being called. Consequently, a pragma **Eliminate** for a dispatching operation is ignored.

Pragma `Export_Exception`

Syntax:

```
pragma Export_Exception (
    [Internal =>] LOCAL_NAME
    [, [External =>] EXTERNAL_SYMBOL]
    [, [Form    =>] Ada | VMS]
    [, [Code    =>] static_integer_EXPRESSION]);
```

```
EXTERNAL_SYMBOL ::=
  IDENTIFIER
| static_string_EXPRESSION
```

This pragma is implemented only in the OpenVMS implementation of GNAT. It causes the specified exception to be propagated outside of the Ada program, so that it can be handled by programs written in other OpenVMS languages. This pragma establishes an external name for an Ada exception and makes the name available to the OpenVMS Linker as a global symbol. For further details on this pragma, see the DEC Ada Language Reference Manual, section 13.9a3.2.

Pragma Export_Function

Syntax:

```
pragma Export_Function (
  [Internal          =>] LOCAL_NAME
  [, [External       =>] EXTERNAL_SYMBOL]
  [, [Parameter_Types =>] PARAMETER_TYPES]
  [, [Result_Type    =>] result_SUBTYPE_MARK]
  [, [Mechanism      =>] MECHANISM]
  [, [Result_Mechanism =>] MECHANISM_NAME]);

EXTERNAL_SYMBOL ::=
  IDENTIFIER
| static_string_EXPRESSION
| ""

PARAMETER_TYPES ::=
  null
| TYPE_DESIGNATOR {, TYPE_DESIGNATOR}

TYPE_DESIGNATOR ::=
  subtype_NAME
| subtype_Name ' Access

MECHANISM ::=
  MECHANISM_NAME
| (MECHANISM_ASSOCIATION {, MECHANISM_ASSOCIATION})

MECHANISM_ASSOCIATION ::=
  [formal_parameter_NAME =>] MECHANISM_NAME

MECHANISM_NAME ::=
  Value
| Reference
| Descriptor [[Class =>] CLASS_NAME]]
| Short_Descriptor [[Class =>] CLASS_NAME]]

CLASS_NAME ::= ubs | ubsb | uba | s | sb | a
```

Use this pragma to make a function externally callable and optionally provide information on mechanisms to be used for passing parameter and result values. We recommend, for the purposes of improving portability, this pragma always be used in conjunction with a separate pragma `Export`, which must precede the pragma `Export_Function`. GNAT does not require a separate pragma `Export`, but if none is present, `Convention Ada` is assumed, which is usually not what is wanted, so it is usually appropriate to use this pragma in conjunction

with a **Export** or **Convention** pragma that specifies the desired foreign convention. Pragma **Export_Function** (and **Export**, if present) must appear in the same declarative region as the function to which they apply.

internal_name must uniquely designate the function to which the pragma applies. If more than one function name exists of this name in the declarative part you must use the **Parameter_Types** and **Result_Type** parameters is mandatory to achieve the required unique designation. *subtype_marks* in these parameters must exactly match the subtypes in the corresponding function specification, using positional notation to match parameters with subtype marks. The form with an **'Access** attribute can be used to match an anonymous access parameter.

Passing by descriptor is supported only on the OpenVMS ports of GNAT. The default behavior for **Export_Function** is to accept either 64bit or 32bit descriptors unless **short_descriptor** is specified, then only 32bit descriptors are accepted.

Special treatment is given if the **EXTERNAL** is an explicit null string or a static string expressions that evaluates to the null string. In this case, no external name is generated. This form still allows the specification of parameter mechanisms.

Pragma **Export_Object**

Syntax:

```
pragma Export_Object
    [Internal =>] LOCAL_NAME
    [, [External =>] EXTERNAL_SYMBOL]
    [, [Size      =>] EXTERNAL_SYMBOL]

EXTERNAL_SYMBOL ::=
    IDENTIFIER
  | static_string_EXPRESSION
```

This pragma designates an object as exported, and apart from the extended rules for external symbols, is identical in effect to the use of the normal **Export** pragma applied to an object. You may use a separate **Export** pragma (and you probably should from the point of view of portability), but it is not required. *Size* is syntax checked, but otherwise ignored by GNAT.

Pragma **Export_Procedure**

Syntax:

```
pragma Export_Procedure (
    [Internal      =>] LOCAL_NAME
    [, [External    =>] EXTERNAL_SYMBOL]
    [, [Parameter_Types =>] PARAMETER_TYPES]
    [, [Mechanism    =>] MECHANISM]);

EXTERNAL_SYMBOL ::=
    IDENTIFIER
  | static_string_EXPRESSION
  | ""

PARAMETER_TYPES ::=
    null
  | TYPE_DESIGNATOR {, TYPE_DESIGNATOR}
```

```

TYPE_DESIGNATOR ::=
  subtype_NAME
| subtype_Name ' Access

MECHANISM ::=
  MECHANISM_NAME
| (MECHANISM_ASSOCIATION {, MECHANISM_ASSOCIATION})

MECHANISM_ASSOCIATION ::=
  [formal_parameter_NAME =>] MECHANISM_NAME

MECHANISM_NAME ::=
  Value
| Reference
| Descriptor [[Class =>] CLASS_NAME]
| Short_Descriptor [[Class =>] CLASS_NAME]

CLASS_NAME ::= ubc | ubcb | uba | s | sb | a

```

This pragma is identical to `Export_Function` except that it applies to a procedure rather than a function and the parameters `Result_Type` and `Result_Mechanism` are not permitted. GNAT does not require a separate pragma `Export`, but if none is present, `Convention Ada` is assumed, which is usually not what is wanted, so it is usually appropriate to use this pragma in conjunction with a `Export` or `Convention` pragma that specifies the desired foreign convention.

Passing by descriptor is supported only on the OpenVMS ports of GNAT. The default behavior for `Export_Procedure` is to accept either 64bit or 32bit descriptors unless `short_descriptor` is specified, then only 32bit descriptors are accepted.

Special treatment is given if the `EXTERNAL` is an explicit null string or a static string expressions that evaluates to the null string. In this case, no external name is generated. This form still allows the specification of parameter mechanisms.

Pragma Export_Value

Syntax:

```

pragma Export_Value (
  [Value      =>] static_integer_EXPRESSION,
  [Link_Name =>] static_string_EXPRESSION);

```

This pragma serves to export a static integer value for external use. The first argument specifies the value to be exported. The `Link_Name` argument specifies the symbolic name to be associated with the integer value. This pragma is useful for defining a named static value in Ada that can be referenced in assembly language units to be linked with the application. This pragma is currently supported only for the AAMP target and is ignored for other targets.

Pragma Export_Valued_Procedure

Syntax:

```

pragma Export_Valued_Procedure (
  [Internal      =>] LOCAL_NAME
  [, [External   =>] EXTERNAL_SYMBOL]

```

```

    [, [Parameter_Types =>] PARAMETER_TYPES]
    [, [Mechanism      =>] MECHANISM]);

EXTERNAL_SYMBOL ::=
    IDENTIFIER
  | static_string_EXPRESSION
  | ""

PARAMETER_TYPES ::=
    null
  | TYPE_DESIGNATOR {, TYPE_DESIGNATOR}

TYPE_DESIGNATOR ::=
    subtype_NAME
  | subtype_Name ' Access

MECHANISM ::=
    MECHANISM_NAME
  | (MECHANISM_ASSOCIATION {, MECHANISM_ASSOCIATION})

MECHANISM_ASSOCIATION ::=
    [formal_parameter_NAME =>] MECHANISM_NAME

MECHANISM_NAME ::=
    Value
  | Reference
  | Descriptor [(Class =>] CLASS_NAME)]
  | Short_Descriptor [(Class =>] CLASS_NAME)]

CLASS_NAME ::= ubs | ubsb | uba | s | sb | a

```

This pragma is identical to `Export_Procedure` except that the first parameter of *LOCAL_NAME*, which must be present, must be of mode `OUT`, and externally the subprogram is treated as a function with this parameter as the result of the function. GNAT provides for this capability to allow the use of `OUT` and `IN OUT` parameters in interfacing to external functions (which are not permitted in Ada functions). GNAT does not require a separate pragma `Export`, but if none is present, `Convention Ada` is assumed, which is almost certainly not what is wanted since the whole point of this pragma is to interface with foreign language functions, so it is usually appropriate to use this pragma in conjunction with a `Export` or `Convention` pragma that specifies the desired foreign convention.

Passing by descriptor is supported only on the OpenVMS ports of GNAT. The default behavior for `Export_Valued_Procedure` is to accept either 64bit or 32bit descriptors unless `short_descriptor` is specified, then only 32bit descriptors are accepted.

Special treatment is given if the `EXTERNAL` is an explicit null string or a static string expressions that evaluates to the null string. In this case, no external name is generated. This form still allows the specification of parameter mechanisms.

Pragma Extend_System

Syntax:

```
pragma Extend_System ([Name =>] IDENTIFIER);
```

This pragma is used to provide backwards compatibility with other implementations that extend the facilities of package `System`. In GNAT, `System` contains only the definitions

that are present in the Ada RM. However, other implementations, notably the DEC Ada 83 implementation, provide many extensions to package **System**.

For each such implementation accommodated by this pragma, GNAT provides a package **Aux_xxx**, e.g. **Aux_DEC** for the DEC Ada 83 implementation, which provides the required additional definitions. You can use this package in two ways. You can **with** it in the normal way and access entities either by selection or using a **use** clause. In this case no special processing is required.

However, if existing code contains references such as **System.xxx** where **xxx** is an entity in the extended definitions provided in package **System**, you may use this pragma to extend visibility in **System** in a non-standard way that provides greater compatibility with the existing code. Pragma **Extend_System** is a configuration pragma whose single argument is the name of the package containing the extended definition (e.g. **Aux_DEC** for the DEC Ada case). A unit compiled under control of this pragma will be processed using special visibility processing that looks in package **System.Aux_xxx** where **Aux_xxx** is the pragma argument for any entity referenced in package **System**, but not found in package **System**.

You can use this pragma either to access a predefined **System** extension supplied with the compiler, for example **Aux_DEC** or you can construct your own extension unit following the above definition. Note that such a package is a child of **System** and thus is considered part of the implementation. To compile it you will have to use the appropriate switch for compiling system units. See [Section “About This Guide”](#) in *gnat-ugn*, for details.

Pragma External

Syntax:

```
pragma External (
  [ Convention    =>] convention_IDENTIFIER,
  [ Entity        =>] LOCAL_NAME
  [, [External_Name =>] static_string_EXPRESSION ]
  [, [Link_Name    =>] static_string_EXPRESSION ]);
```

This pragma is identical in syntax and semantics to pragma **Export** as defined in the Ada Reference Manual. It is provided for compatibility with some Ada 83 compilers that used this pragma for exactly the same purposes as pragma **Export** before the latter was standardized.

Pragma External_Name_Casing

Syntax:

```
pragma External_Name_Casing (
  Uppercase | Lowercase
  [, Uppercase | Lowercase | As_Is]);
```

This pragma provides control over the casing of external names associated with **Import** and **Export** pragmas. There are two cases to consider:

Implicit external names

Implicit external names are derived from identifiers. The most common case arises when a standard Ada **Import** or **Export** pragma is used with only two arguments, as in:

```
pragma Import (C, C_Routine);
```


Since Ada is a case-insensitive language, the spelling of the identifier in the Ada source program does not provide any information on the desired casing of the external name, and so a convention is needed. In GNAT the default treatment is that such names are converted to all lower case letters. This corresponds to the normal C style in many environments. The first argument of pragma `External_Name_Casing` can be used to control this treatment. If `Uppercase` is specified, then the name will be forced to all uppercase letters. If `Lowercase` is specified, then the normal default of all lower case letters will be used.

This same implicit treatment is also used in the case of extended DEC Ada 83 compatible Import and Export pragmas where an external name is explicitly specified using an identifier rather than a string.

Explicit external names

Explicit external names are given as string literals. The most common case arises when a standard Ada Import or Export pragma is used with three arguments, as in:

```
pragma Import (C, C_Routine, "C_routine");
```

In this case, the string literal normally provides the exact casing required for the external name. The second argument of pragma `External_Name_Casing` may be used to modify this behavior. If `Uppercase` is specified, then the name will be forced to all uppercase letters. If `Lowercase` is specified, then the name will be forced to all lowercase letters. A specification of `As_Is` provides the normal default behavior in which the casing is taken from the string provided.

This pragma may appear anywhere that a pragma is valid. In particular, it can be used as a configuration pragma in the `'gnat.adc'` file, in which case it applies to all subsequent compilations, or it can be used as a program unit pragma, in which case it only applies to the current unit, or it can be used more locally to control individual Import/Export pragmas.

It is primarily intended for use with OpenVMS systems, where many compilers convert all symbols to upper case by default. For interfacing to such compilers (e.g. the DEC C compiler), it may be convenient to use the pragma:

```
pragma External_Name_Casing (Uppercase, Uppercase);
```

to enforce the upper casing of all external symbols.

Pragma `Fast_Math`

Syntax:

```
pragma Fast_Math;
```

This is a configuration pragma which activates a mode in which speed is considered more important for floating-point operations than absolutely accurate adherence to the requirements of the standard. Currently the following operations are affected:

Complex Multiplication

The normal simple formula for complex multiplication can result in intermediate overflows for numbers near the end of the range. The Ada standard requires that this situation be detected and corrected by scaling, but in `Fast_Math` mode such cases will simply result in overflow. Note that to take advantage of this you

must instantiate your own version of `Ada.Numerics.Generic_Complex_Types` under control of the pragma, rather than use the preinstantiated versions.

Pragma Favor_Top_Level

Syntax:

```
pragma Favor_Top_Level (type_NAME);
```

The named type must be an access-to-subprogram type. This pragma is an efficiency hint to the compiler, regarding the use of `'Access` or `'Unrestricted_Access` on nested (non-library-level) subprograms. The pragma means that nested subprograms are not used with this type, or are rare, so that the generated code should be efficient in the top-level case. When this pragma is used, dynamically generated trampolines may be used on some targets for nested subprograms. See also the `No_Implicit_Dynamic_Code` restriction.

Pragma Finalize_Storage_Only

Syntax:

```
pragma Finalize_Storage_Only (first_subtype_LOCAL_NAME);
```

This pragma allows the compiler not to emit a `Finalize` call for objects defined at the library level. This is mostly useful for types where finalization is only used to deal with storage reclamation since in most environments it is not necessary to reclaim memory just before terminating execution, hence the name.

Pragma Float_Representation

Syntax:

```
pragma Float_Representation (FLOAT_REP[, float_type_LOCAL_NAME]);
```

```
FLOAT_REP ::= VAX_Float | IEEE_Float
```

In the one argument form, this pragma is a configuration pragma which allows control over the internal representation chosen for the predefined floating point types declared in the packages `Standard` and `System`. On all systems other than OpenVMS, the argument must be `IEEE_Float` and the pragma has no effect. On OpenVMS, the argument may be `VAX_Float` to specify the use of the VAX float format for the floating-point types in `Standard`. This requires that the standard runtime libraries be recompiled. See [Section “The GNAT Run-Time Library Builder gnatlbr” in *GNAT User's Guide OpenVMS*](#), for a description of the `GNAT LIBRARY` command.

The two argument form specifies the representation to be used for the specified floating-point type. On all systems other than OpenVMS, the argument must be `IEEE_Float` and the pragma has no effect. On OpenVMS, the argument may be `VAX_Float` to specify the use of the VAX float format, as follows:

- For digits values up to 6, F float format will be used.
- For digits values from 7 to 9, G float format will be used.
- For digits values from 10 to 15, F float format will be used.
- Digits values above 15 are not allowed.

Pragma Ident

Syntax:

```
pragma Ident (static_string_EXPRESSION);
```

This pragma provides a string identification in the generated object file, if the system supports the concept of this kind of identification string. This pragma is allowed only in the outermost declarative part or declarative items of a compilation unit. If more than one **Ident** pragma is given, only the last one processed is effective. On OpenVMS systems, the effect of the pragma is identical to the effect of the DEC Ada 83 pragma of the same name. Note that in DEC Ada 83, the maximum allowed length is 31 characters, so if it is important to maintain compatibility with this compiler, you should obey this length limit.

Pragma Implemented_By_Entry

Syntax:

```
pragma Implemented_By_Entry (LOCAL_NAME);
```

This is a representation pragma which applies to protected, synchronized and task interface primitives. If the pragma is applied to primitive operation Op of interface Iface, it is illegal to override Op in a type that implements Iface, with anything other than an entry.

```
type Iface is protected interface;
procedure Do_Something (Object : in out Iface) is abstract;
pragma Implemented_By_Entry (Do_Something);
```

```
protected type P is new Iface with
  procedure Do_Something; -- Illegal
end P;
```

```
task type T is new Iface with
  entry Do_Something;      -- Legal
end T;
```

NOTE: The pragma is still in its design stage by the Ada Rapporteur Group. It is intended to be used in conjunction with dispatching requeue statements as described in AI05-0030. Should the ARG decide on an official name and syntax, this pragma will become language-defined rather than GNAT-specific.

Pragma Implicit_Packing

Syntax:

```
pragma Implicit_Packing;
```

This is a configuration pragma that requests implicit packing for packed arrays for which a size clause is given but no explicit pragma **Pack** or specification of **Component_Size** is present. It also applies to records where no record representation clause is present. Consider this example:

```
type R is array (0 .. 7) of Boolean;
for R'Size use 8;
```

In accordance with the recommendation in the RM (RM 13.3(53)), a **Size** clause does not change the layout of a composite object. So the **Size** clause in the above example is normally rejected, since the default layout of the array uses 8-bit components, and thus the array requires a minimum of 64 bits.

If this declaration is compiled in a region of code covered by an occurrence of the configuration pragma `Implicit_Packing`, then the `Size` clause in this and similar examples will cause implicit packing and thus be accepted. For this implicit packing to occur, the type in question must be an array of small components whose size is known at compile time, and the `Size` clause must specify the exact size that corresponds to the length of the array multiplied by the size in bits of the component type.

Similarly, the following example shows the use in the record case

```
type r is record
  a, b, c, d, e, f, g, h : boolean;
  chr                    : character;
end record;
for r'size use 16;
```

Without a pragma `Pack`, each Boolean field requires 8 bits, so the minimum size is 72 bits, but with a pragma `Pack`, 16 bits would be sufficient. The use of pragma `Implicit_Packing` allows this record declaration to compile without an explicit pragma `Pack`.

Pragma Import_Exception

Syntax:

```
pragma Import_Exception (
  [Internal =>] LOCAL_NAME
  [, [External =>] EXTERNAL_SYMBOL]
  [, [Form    =>] Ada | VMS]
  [, [Code    =>] static_integer_EXPRESSION]);

EXTERNAL_SYMBOL ::=
  IDENTIFIER
| static_string_EXPRESSION
```

This pragma is implemented only in the OpenVMS implementation of GNAT. It allows OpenVMS conditions (for example, from OpenVMS system services or other OpenVMS languages) to be propagated to Ada programs as Ada exceptions. The pragma specifies that the exception associated with an exception declaration in an Ada program be defined externally (in non-Ada code). For further details on this pragma, see the DEC Ada Language Reference Manual, section 13.9a.3.1.

Pragma Import_Function

Syntax:

```
pragma Import_Function (
  [Internal          =>] LOCAL_NAME,
  [, [External       =>] EXTERNAL_SYMBOL]
  [, [Parameter_Types =>] PARAMETER_TYPES]
  [, [Result_Type    =>] SUBTYPE_MARK]
  [, [Mechanism      =>] MECHANISM]
  [, [Result_Mechanism =>] MECHANISM_NAME]
  [, [First_Optional_Parameter =>] IDENTIFIER]);

EXTERNAL_SYMBOL ::=
  IDENTIFIER
| static_string_EXPRESSION
```

```

PARAMETER_TYPES ::=
    null
  | TYPE_DESIGNATOR {, TYPE_DESIGNATOR}

TYPE_DESIGNATOR ::=
    subtype_NAME
  | subtype_Name ' Access

MECHANISM ::=
    MECHANISM_NAME
  | (MECHANISM_ASSOCIATION {, MECHANISM_ASSOCIATION})

MECHANISM_ASSOCIATION ::=
    [formal_parameter_NAME =>] MECHANISM_NAME

MECHANISM_NAME ::=
    Value
  | Reference
  | Descriptor [[Class =>] CLASS_NAME]]
  | Short_Descriptor [[Class =>] CLASS_NAME]]

CLASS_NAME ::= ubs | ubsb | uba | s | sb | a | nca

```

This pragma is used in conjunction with a pragma **Import** to specify additional information for an imported function. The pragma **Import** (or equivalent pragma **Interface**) must precede the **Import_Function** pragma and both must appear in the same declarative part as the function specification.

The *Internal* argument must uniquely designate the function to which the pragma applies. If more than one function name exists of this name in the declarative part you must use the **Parameter_Types** and *Result_Type* parameters to achieve the required unique designation. Subtype marks in these parameters must exactly match the subtypes in the corresponding function specification, using positional notation to match parameters with subtype marks. The form with an **'Access** attribute can be used to match an anonymous access parameter.

You may optionally use the *Mechanism* and *Result_Mechanism* parameters to specify passing mechanisms for the parameters and result. If you specify a single mechanism name, it applies to all parameters. Otherwise you may specify a mechanism on a parameter by parameter basis using either positional or named notation. If the mechanism is not specified, the default mechanism is used.

Passing by descriptor is supported only on the OpenVMS ports of GNAT. The default behavior for **Import_Function** is to pass a 64bit descriptor unless **short_descriptor** is specified, then a 32bit descriptor is passed.

First_Optional_Parameter applies only to OpenVMS ports of GNAT. It specifies that the designated parameter and all following parameters are optional, meaning that they are not passed at the generated code level (this is distinct from the notion of optional parameters in Ada where the parameters are passed anyway with the designated optional parameters). All optional parameters must be of mode **IN** and have default parameter values that are either known at compile time expressions, or uses of the **'Null_Parameter** attribute.

Pragma **Import_Object**

Syntax:

```

pragma Import_Object
  [Internal =>] LOCAL_NAME
  [, [External =>] EXTERNAL_SYMBOL]
  [, [Size      =>] EXTERNAL_SYMBOL]);

EXTERNAL_SYMBOL ::=
  IDENTIFIER
  | static_string_EXPRESSION

```

This pragma designates an object as imported, and apart from the extended rules for external symbols, is identical in effect to the use of the normal **Import** pragma applied to an object. Unlike the subprogram case, you need not use a separate **Import** pragma, although you may do so (and probably should do so from a portability point of view). *size* is syntax checked, but otherwise ignored by GNAT.

Pragma Import_Procedure

Syntax:

```

pragma Import_Procedure (
  [Internal      =>] LOCAL_NAME
  [, [External   =>] EXTERNAL_SYMBOL]
  [, [Parameter_Types =>] PARAMETER_TYPES]
  [, [Mechanism   =>] MECHANISM]
  [, [First_Optional_Parameter =>] IDENTIFIER]);

EXTERNAL_SYMBOL ::=
  IDENTIFIER
  | static_string_EXPRESSION

PARAMETER_TYPES ::=
  null
  | TYPE_DESIGNATOR {, TYPE_DESIGNATOR}

TYPE_DESIGNATOR ::=
  subtype_NAME
  | subtype_Name ' Access

MECHANISM ::=
  MECHANISM_NAME
  | (MECHANISM_ASSOCIATION {, MECHANISM_ASSOCIATION})

MECHANISM_ASSOCIATION ::=
  [formal_parameter_NAME =>] MECHANISM_NAME

MECHANISM_NAME ::=
  Value
  | Reference
  | Descriptor [(Class =>] CLASS_NAME)]
  | Short_Descriptor [(Class =>] CLASS_NAME)]

CLASS_NAME ::= ubs | ubsb | uba | s | sb | a | nca

```

This pragma is identical to **Import_Function** except that it applies to a procedure rather than a function and the parameters **Result_Type** and **Result_Mechanism** are not permitted.

Pragma Import_Valued_Procedure

Syntax:

```
pragma Import_Valued_Procedure (
    [Internal                      =>] LOCAL_NAME
  [, [External                    =>] EXTERNAL_SYMBOL]
  [, [Parameter_Types            =>] PARAMETER_TYPES]
  [, [Mechanism                  =>] MECHANISM]
  [, [First_Optional_Parameter =>] IDENTIFIER]);

EXTERNAL_SYMBOL ::=
    IDENTIFIER
  | static_string_EXPRESSION

PARAMETER_TYPES ::=
    null
  | TYPE_DESIGNATOR {, TYPE_DESIGNATOR}

TYPE_DESIGNATOR ::=
    subtype_NAME
  | subtype_Name ' Access

MECHANISM ::=
    MECHANISM_NAME
  | (MECHANISM_ASSOCIATION {, MECHANISM_ASSOCIATION})

MECHANISM_ASSOCIATION ::=
    [formal_parameter_NAME =>] MECHANISM_NAME

MECHANISM_NAME ::=
    Value
  | Reference
  | Descriptor [([Class =>] CLASS_NAME)]
  | Short_Descriptor [([Class =>] CLASS_NAME)]

CLASS_NAME ::= ubs | ubsb | uba | s | sb | a | nca
```

This pragma is identical to `Import_Procedure` except that the first parameter of *LOCAL_NAME*, which must be present, must be of mode `OUT`, and externally the subprogram is treated as a function with this parameter as the result of the function. The purpose of this capability is to allow the use of `OUT` and `IN OUT` parameters in interfacing to external functions (which are not permitted in Ada functions). You may optionally use the **Mechanism** parameters to specify passing mechanisms for the parameters. If you specify a single mechanism name, it applies to all parameters. Otherwise you may specify a mechanism on a parameter by parameter basis using either positional or named notation. If the mechanism is not specified, the default mechanism is used.

Note that it is important to use this pragma in conjunction with a separate pragma `Import` that specifies the desired convention, since otherwise the default convention is `Ada`, which is almost certainly not what is required.

Pragma InitializeScalars

Syntax:

```
pragma InitializeScalars;
```

This pragma is similar to `Normalize_Scalars` conceptually but has two important differences. First, there is no requirement for the pragma to be used uniformly in all units of a partition, in particular, it is fine to use this just for some or all of the application units of a partition, without needing to recompile the run-time library.

In the case where some units are compiled with the pragma, and some without, then a declaration of a variable where the type is defined in package `Standard` or is locally declared will always be subject to initialization, as will any declaration of a scalar variable. For composite variables, whether the variable is initialized may also depend on whether the package in which the type of the variable is declared is compiled with the pragma.

The other important difference is that you can control the value used for initializing scalar objects. At bind time, you can select several options for initialization. You can initialize with invalid values (similar to `Normalize_Scalars`, though for `Initialize_Scalars` it is not always possible to determine the invalid values in complex cases like signed component fields with non-standard sizes). You can also initialize with high or low values, or with a specified bit pattern. See the users guide for binder options for specifying these cases.

This means that you can compile a program, and then without having to recompile the program, you can run it with different values being used for initializing otherwise uninitialized values, to test if your program behavior depends on the choice. Of course the behavior should not change, and if it does, then most likely you have an erroneous reference to an uninitialized value.

It is even possible to change the value at execution time eliminating even the need to rebind with a different switch using an environment variable. See the GNAT users guide for details.

Note that pragma `Initialize_Scalars` is particularly useful in conjunction with the enhanced validity checking that is now provided in GNAT, which checks for invalid values under more conditions. Using this feature (see description of the ‘`-gnatV`’ flag in the users guide) in conjunction with pragma `Initialize_Scalars` provides a powerful new tool to assist in the detection of problems caused by uninitialized variables.

Note: the use of `Initialize_Scalars` has a fairly extensive effect on the generated code. This may cause your code to be substantially larger. It may also cause an increase in the amount of stack required, so it is probably a good idea to turn on stack checking (see description of stack checking in the GNAT users guide) when using this pragma.

Pragma `Inline_Always`

Syntax:

```
pragma Inline_Always (NAME [, NAME]);
```

Similar to pragma `Inline` except that inlining is not subject to the use of option ‘`-gnatn`’ and the inlining happens regardless of whether this option is used.

Pragma `Inline_Generic`

Syntax:

```
pragma Inline_Generic (generic_package_NAME);
```

This is implemented for compatibility with DEC Ada 83 and is recognized, but otherwise ignored, by GNAT. All generic instantiations are inlined by default when using GNAT.

Pragma Interface

Syntax:

```
pragma Interface (
    [Convention    =>] convention_identifier,
    [Entity        =>] local_NAME
    [, [External_Name =>] static_string_expression]
    [, [Link_Name   =>] static_string_expression]);
```

This pragma is identical in syntax and semantics to the standard Ada pragma `Import`. It is provided for compatibility with Ada 83. The definition is upwards compatible both with pragma `Interface` as defined in the Ada 83 Reference Manual, and also with some extended implementations of this pragma in certain Ada 83 implementations.

Pragma Interface_Name

Syntax:

```
pragma Interface_Name (
    [Entity        =>] LOCAL_NAME
    [, [External_Name =>] static_string_EXPRESSION]
    [, [Link_Name   =>] static_string_EXPRESSION]);
```

This pragma provides an alternative way of specifying the interface name for an interfaced subprogram, and is provided for compatibility with Ada 83 compilers that use the pragma for this purpose. You must provide at least one of *External_Name* or *Link_Name*.

Pragma Interrupt_Handler

Syntax:

```
pragma Interrupt_Handler (procedure_LOCAL_NAME);
```

This program unit pragma is supported for parameterless protected procedures as described in Annex C of the Ada Reference Manual. On the AAMP target the pragma can also be specified for nonprotected parameterless procedures that are declared at the library level (which includes procedures declared at the top level of a library package). In the case of AAMP, when this pragma is applied to a nonprotected procedure, the instruction `IERET` is generated for returns from the procedure, enabling maskable interrupts, in place of the normal return instruction.

Pragma Interrupt_State

Syntax:

```
pragma Interrupt_State
    ([Name =>] value,
     [State =>] SYSTEM | RUNTIME | USER);
```

Normally certain interrupts are reserved to the implementation. Any attempt to attach an interrupt causes `Program_Error` to be raised, as described in RM C.3.2(22). A typical example is the `SIGINT` interrupt used in many systems for an *Ctrl-C* interrupt. Normally this interrupt is reserved to the implementation, so that *Ctrl-C* can be used to interrupt execution. Additionally, signals such as `SIGSEGV`, `SIGABRT`, `SIGFPE` and `SIGILL` are often mapped to specific Ada exceptions, or used to implement run-time functions such as the `abort` statement and stack overflow checking.

Pragma `Interrupt_State` provides a general mechanism for overriding such uses of interrupts. It subsumes the functionality of pragma `Unreserve_All_Interrupts`. Pragma `Interrupt_State` is not available on OS/2, Windows or VMS. On all other platforms than VxWorks, it applies to signals; on VxWorks, it applies to vectored hardware interrupts and may be used to mark interrupts required by the board support package as reserved.

Interrupts can be in one of three states:

- **System**
The interrupt is reserved (no Ada handler can be installed), and the Ada run-time may not install a handler. As a result you are guaranteed standard system default action if this interrupt is raised.
- **Runtime**
The interrupt is reserved (no Ada handler can be installed). The run time is allowed to install a handler for internal control purposes, but is not required to do so.
- **User**
The interrupt is unreserved. The user may install a handler to provide some other action.

These states are the allowed values of the `State` parameter of the pragma. The `Name` parameter is a value of the type `Ada.Interrupts.Interrupt_ID`. Typically, it is a name declared in `Ada.Interrupts.Names`.

This is a configuration pragma, and the binder will check that there are no inconsistencies between different units in a partition in how a given interrupt is specified. It may appear anywhere a pragma is legal.

The effect is to move the interrupt to the specified state.

By declaring interrupts to be `SYSTEM`, you guarantee the standard system action, such as a core dump.

By declaring interrupts to be `USER`, you guarantee that you can install a handler.

Note that certain signals on many operating systems cannot be caught and handled by applications. In such cases, the pragma is ignored. See the operating system documentation, or the value of the array `Reserved` declared in the spec of package `System.OS_Interface`.

Overriding the default state of signals used by the Ada runtime may interfere with an application's runtime behavior in the cases of the synchronous signals, and in the case of the signal used to implement the `abort` statement.

Pragma `Keep_Names`

Syntax:

```
pragma Keep_Names ([On =>] enumeration_first_subtype_LOCAL_NAME);
```

The `LOCAL_NAME` argument must refer to an enumeration first subtype in the current declarative part. The effect is to retain the enumeration literal names for use by `Image` and `Value` even if a global `Discard_Names` pragma applies. This is useful when you want to generally suppress enumeration literal names and for example you therefore use a `Discard_Names` pragma in the `'gnat.adc'` file, but you want to retain the names for specific enumeration types.

Pragma License

Syntax:

```
pragma License (Unrestricted | GPL | Modified_GPL | Restricted);
```

This pragma is provided to allow automated checking for appropriate license conditions with respect to the standard and modified GPL. A pragma `License`, which is a configuration pragma that typically appears at the start of a source file or in a separate ‘`gnat.adc`’ file, specifies the licensing conditions of a unit as follows:

- **Unrestricted** This is used for a unit that can be freely used with no license restrictions. Examples of such units are public domain units, and units from the Ada Reference Manual.
- **GPL** This is used for a unit that is licensed under the unmodified GPL, and which therefore cannot be `with`’ed by a restricted unit.
- **Modified_GPL** This is used for a unit licensed under the GNAT modified GPL that includes a special exception paragraph that specifically permits the inclusion of the unit in programs without requiring the entire program to be released under the GPL.
- **Restricted** This is used for a unit that is restricted in that it is not permitted to depend on units that are licensed under the GPL. Typical examples are proprietary code that is to be released under more restrictive license conditions. Note that restricted units are permitted to `with` units which are licensed under the modified GPL (this is the whole point of the modified GPL).

Normally a unit with no `License` pragma is considered to have an unknown license, and no checking is done. However, standard GNAT headers are recognized, and license information is derived from them as follows.

A GNAT license header starts with a line containing 78 hyphens. The following comment text is searched for the appearance of any of the following strings.

If the string “GNU General Public License” is found, then the unit is assumed to have GPL license, unless the string “As a special exception” follows, in which case the license is assumed to be modified GPL.

If one of the strings “This specification is adapted from the Ada Semantic Interface” or “This specification is derived from the Ada Reference Manual” is found then the unit is assumed to be unrestricted.

These default actions means that a program with a restricted license pragma will automatically get warnings if a GPL unit is inappropriately `with`’ed. For example, the program:

```
with Sem_Ch3;
with GNAT.Sockets;
procedure Secret_Stuff is
...
end Secret_Stuff
```

if compiled with `pragma License (Restricted)` in a ‘`gnat.adc`’ file will generate the warning:

```
1.  with Sem_Ch3;
    |
    >>> license of withed unit "Sem_Ch3" is incompatible

2.  with GNAT.Sockets;
```

```
3. procedure Secret_Stuff is
```

Here we get a warning on `Sem_Ch3` since it is part of the GNAT compiler and is licensed under the GPL, but no warning for `GNAT.Sockets` which is part of the GNAT run time, and is therefore licensed under the modified GPL.

Pragma Link_With

Syntax:

```
pragma Link_With (static_string_EXPRESSION {,static_string_EXPRESSION});
```

This pragma is provided for compatibility with certain Ada 83 compilers. It has exactly the same effect as pragma `Linker_Options` except that spaces occurring within one of the string expressions are treated as separators. For example, in the following case:

```
pragma Link_With ("-labc -ldef");
```

results in passing the strings `-labc` and `-ldef` as two separate arguments to the linker. In addition pragma `Link_With` allows multiple arguments, with the same effect as successive pragmas.

Pragma Linker_Alias

Syntax:

```
pragma Linker_Alias (
  [Entity =>] LOCAL_NAME,
  [Target =>] static_string_EXPRESSION);
```

LOCAL_NAME must refer to an object that is declared at the library level. This pragma establishes the given entity as a linker alias for the given target. It is equivalent to `__attribute__((alias))` in GNU C and causes *LOCAL_NAME* to be emitted as an alias for the symbol *static_string_EXPRESSION* in the object file, that is to say no space is reserved for *LOCAL_NAME* by the assembler and it will be resolved to the same address as *static_string_EXPRESSION* by the linker.

The actual linker name for the target must be used (e.g. the fully encoded name with qualification in Ada, or the mangled name in C++), or it must be declared using the C convention with pragma `Import` or pragma `Export`.

Not all target machines support this pragma. On some of them it is accepted only if pragma `Weak_External` has been applied to *LOCAL_NAME*.

```
-- Example of the use of pragma Linker_Alias
```

```
package p is
  i : Integer := 1;
  pragma Export (C, i);

  new_name_for_i : Integer;
  pragma Linker_Alias (new_name_for_i, "i");
end p;
```

Pragma Linker_Constructor

Syntax:

```
pragma Linker_Constructor (procedure_LOCAL_NAME);
```

procedure_LOCAL_NAME must refer to a parameterless procedure that is declared at the library level. A procedure to which this pragma is applied will be treated as an initialization routine by the linker. It is equivalent to `__attribute__((constructor))` in GNU C and causes *procedure_LOCAL_NAME* to be invoked before the entry point of the executable is called (or immediately after the shared library is loaded if the procedure is linked in a shared library), in particular before the Ada run-time environment is set up.

Because of these specific contexts, the set of operations such a procedure can perform is very limited and the type of objects it can manipulate is essentially restricted to the elementary types. In particular, it must only contain code to which pragma Restrictions (No_Elaboration_Code) applies.

This pragma is used by GNAT to implement auto-initialization of shared Stand Alone Libraries, which provides a related capability without the restrictions listed above. Where possible, the use of Stand Alone Libraries is preferable to the use of this pragma.

Pragma Linker_Destructor

Syntax:

```
pragma Linker_Destructor (procedure_LOCAL_NAME);
```

procedure_LOCAL_NAME must refer to a parameterless procedure that is declared at the library level. A procedure to which this pragma is applied will be treated as a finalization routine by the linker. It is equivalent to `__attribute__((destructor))` in GNU C and causes *procedure_LOCAL_NAME* to be invoked after the entry point of the executable has exited (or immediately before the shared library is unloaded if the procedure is linked in a shared library), in particular after the Ada run-time environment is shut down.

See `pragma Linker_Constructor` for the set of restrictions that apply because of these specific contexts.

Pragma Linker_Section

Syntax:

```
pragma Linker_Section (
  [Entity =>] LOCAL_NAME,
  [Section =>] static_string_EXPRESSION);
```

LOCAL_NAME must refer to an object that is declared at the library level. This pragma specifies the name of the linker section for the given entity. It is equivalent to `__attribute__((section))` in GNU C and causes *LOCAL_NAME* to be placed in the *static_string_EXPRESSION* section of the executable (assuming the linker doesn't rename the section).

The compiler normally places library-level objects in standard sections depending on their type: procedures and functions generally go in the `.text` section, initialized variables in the `.data` section and uninitialized variables in the `.bss` section.

Other, special sections may exist on given target machines to map special hardware, for example I/O ports or flash memory. This pragma is a means to defer the final layout of the executable to the linker, thus fully working at the symbolic level with the compiler.

Some file formats do not support arbitrary sections so not all target machines support this pragma. The use of this pragma may cause a program execution to be erroneous if it

is used to place an entity into an inappropriate section (e.g. a modified variable into the `.text` section). See also `pragma Persistent_BSS`.

```
-- Example of the use of pragma Linker_Section

package IO_Card is
  Port_A : Integer;
  pragma Volatile (Port_A);
  pragma Linker_Section (Port_A, ".bss.port_a");

  Port_B : Integer;
  pragma Volatile (Port_B);
  pragma Linker_Section (Port_B, ".bss.port_b");
end IO_Card;
```

Pragma Long_Float

Syntax:

```
pragma Long_Float (FLOAT_FORMAT);

FLOAT_FORMAT ::= D_Float | G_Float
```

This pragma is implemented only in the OpenVMS implementation of GNAT. It allows control over the internal representation chosen for the predefined type `Long_Float` and for floating point type representations with `digits` specified in the range 7 through 15. For further details on this pragma, see the *DEC Ada Language Reference Manual*, section 3.5.7b. Note that to use this pragma, the standard runtime libraries must be recompiled. See [Section “The GNAT Run-Time Library Builder gnatlbr” in *GNAT User’s Guide OpenVMS*](#), for a description of the GNAT LIBRARY command.

Pragma Machine_Attribute

Syntax:

```
pragma Machine_Attribute (
  [Entity      =>] LOCAL_NAME,
  [Attribute_Name =>] static_string_EXPRESSION
[, [Info      =>] static_EXPRESSION] );
```

Machine-dependent attributes can be specified for types and/or declarations. This pragma is semantically equivalent to `__attribute__((attribute_name))` (if *info* is not specified) or `__attribute__((attribute_name(info)))` in GNU C, where *attribute_name* is recognized by the compiler middle-end or the `TARGET_ATTRIBUTE_TABLE` machine specific macro. A string literal for the optional parameter *info* is transformed into an identifier, which may make this pragma unusable for some attributes. See [Section “Defining target-specific uses of `\_\_attribute\_\_`” in *GNU Compiler Collection \(GCC\) Internals*](#), further information.

Pragma Main

Syntax:

```
pragma Main
  (MAIN_OPTION [, MAIN_OPTION]);

MAIN_OPTION ::=
  [Stack_Size      =>] static_integer_EXPRESSION
| [Task_Stack_Size_Default =>] static_integer_EXPRESSION
```

```
| [Time_Slicing_Enabled    =>] static_boolean_EXPRESSION
```

This pragma is provided for compatibility with OpenVMS VAX Systems. It has no effect in GNAT, other than being syntax checked.

Pragma Main_Storage

Syntax:

```
pragma Main_Storage
  (MAIN_STORAGE_OPTION [, MAIN_STORAGE_OPTION]);

MAIN_STORAGE_OPTION ::=
  [WORKING_STORAGE =>] static_SIMPLE_EXPRESSION
| [TOP_GUARD      =>] static_SIMPLE_EXPRESSION
```

This pragma is provided for compatibility with OpenVMS VAX Systems. It has no effect in GNAT, other than being syntax checked. Note that the pragma also has no effect in DEC Ada 83 for OpenVMS Alpha Systems.

Pragma No_Body

Syntax:

```
pragma No_Body;
```

There are a number of cases in which a package spec does not require a body, and in fact a body is not permitted. GNAT will not permit the spec to be compiled if there is a body around. The pragma `No_Body` allows you to provide a body file, even in a case where no body is allowed. The body file must contain only comments and a single `No_Body` pragma. This is recognized by the compiler as indicating that no body is logically present.

This is particularly useful during maintenance when a package is modified in such a way that a body needed before is no longer needed. The provision of a dummy body with a `No_Body` pragma ensures that there is no interference from earlier versions of the package body.

Pragma No_Return

Syntax:

```
pragma No_Return (procedure_LOCAL_NAME {, procedure_LOCAL_NAME});
```

Each *procedure_LOCAL_NAME* argument must refer to one or more procedure declarations in the current declarative part. A procedure to which this pragma is applied may not contain any explicit `return` statements. In addition, if the procedure contains any implicit returns from falling off the end of a statement sequence, then execution of that implicit return will cause `Program.Error` to be raised.

One use of this pragma is to identify procedures whose only purpose is to raise an exception. Another use of this pragma is to suppress incorrect warnings about missing returns in functions, where the last statement of a function statement sequence is a call to such a procedure.

Note that in Ada 2005 mode, this pragma is part of the language, and is identical in effect to the pragma as implemented in Ada 95 mode.

Pragma No_Strict_Aliasing

Syntax:

```
pragma No_Strict_Aliasing [(Entity =>] type_LOCAL_NAME)];
```

type_LOCAL_NAME must refer to an access type declaration in the current declarative part. The effect is to inhibit strict aliasing optimization for the given type. The form with no arguments is a configuration pragma which applies to all access types declared in units to which the pragma applies. For a detailed description of the strict aliasing optimization, and the situations in which it must be suppressed, see [Section “Optimization and Strict Aliasing” in *GNAT User’s Guide*](#).

Pragma Normalize_Scalars

Syntax:

```
pragma Normalize_Scalars;
```

This is a language defined pragma which is fully implemented in GNAT. The effect is to cause all scalar objects that are not otherwise initialized to be initialized. The initial values are implementation dependent and are as follows:

Standard.Character

Objects whose root type is `Standard.Character` are initialized to `Character’Last` unless the subtype range excludes NUL (in which case NUL is used). This choice will always generate an invalid value if one exists.

Standard.Wide_Character

Objects whose root type is `Standard.Wide_Character` are initialized to `Wide_Character’Last` unless the subtype range excludes NUL (in which case NUL is used). This choice will always generate an invalid value if one exists.

Standard.Wide_Wide_Character

Objects whose root type is `Standard.Wide_Wide_Character` are initialized to the invalid value `16#FFFF_FFFF#` unless the subtype range excludes NUL (in which case NUL is used). This choice will always generate an invalid value if one exists.

Integer types

Objects of an integer type are treated differently depending on whether negative values are present in the subtype. If no negative values are present, then all one bits is used as the initial value except in the special case where zero is excluded from the subtype, in which case all zero bits are used. This choice will always generate an invalid value if one exists.

For subtypes with negative values present, the largest negative number is used, except in the unusual case where this largest negative number is in the subtype, and the largest positive number is not, in which case the largest positive value is used. This choice will always generate an invalid value if one exists.

Floating-Point Types

Objects of all floating-point types are initialized to all 1-bits. For standard IEEE format, this corresponds to a NaN (not a number) which is indeed an invalid value.

Fixed-Point Types

Objects of all fixed-point types are treated as described above for integers, with the rules applying to the underlying integer value used to represent the fixed-point value.

Modular types

Objects of a modular type are initialized to all one bits, except in the special case where zero is excluded from the subtype, in which case all zero bits are used. This choice will always generate an invalid value if one exists.

Enumeration types

Objects of an enumeration type are initialized to all one-bits, i.e. to the value $2^{**} \text{typ'Size} - 1$ unless the subtype excludes the literal whose Pos value is zero, in which case a code of zero is used. This choice will always generate an invalid value if one exists.

Pragma Obsolescent

Syntax:

```
pragma Obsolescent;

pragma Obsolescent (
  [Message =>] static_string_EXPRESSION
  [, [Version =>] Ada_05]);

pragma Obsolescent (
  [Entity =>] NAME
  [, [Message =>] static_string_EXPRESSION
  [, [Version =>] Ada_05]] );
```

This pragma can occur immediately following a declaration of an entity, including the case of a record component. If no Entity argument is present, then this declaration is the one to which the pragma applies. If an Entity parameter is present, it must either match the name of the entity in this declaration, or alternatively, the pragma can immediately follow an enumeration type declaration, where the Entity argument names one of the enumeration literals.

This pragma is used to indicate that the named entity is considered obsolescent and should not be used. Typically this is used when an API must be modified by eventually removing or modifying existing subprograms or other entities. The pragma can be used at an intermediate stage when the entity is still present, but will be removed later.

The effect of this pragma is to output a warning message on a reference to an entity thus marked that the subprogram is obsolescent if the appropriate warning option in the compiler is activated. If the Message parameter is present, then a second warning message is given containing this text. In addition, a reference to the entity is considered to be a violation of pragma Restrictions (No_Obsolescent_Features).

This pragma can also be used as a program unit pragma for a package, in which case the entity name is the name of the package, and the pragma indicates that the entire package is considered obsolescent. In this case a client `with`'ing such a package violates the restriction, and the `with` statement is flagged with warnings if the warning option is set.

If the Version parameter is present (which must be exactly the identifier `Ada_05`, no other argument is allowed), then the indication of obsolescence applies only when compiling in

Ada 2005 mode. This is primarily intended for dealing with the situations in the predefined library where subprograms or packages have become defined as obsolescent in Ada 2005 (e.g. in `Ada.Characters.Handling`), but may be used anywhere.

The following examples show typical uses of this pragma:

```
package p is
  pragma Obsolescent (p, Message => "use pp instead of p");
end p;

package q is
  procedure q2;
  pragma Obsolescent ("use q2new instead");

  type R is new integer;
  pragma Obsolescent
    (Entity => R,
     Message => "use RR in Ada 2005",
     Version => Ada_05);

  type M is record
    F1 : Integer;
    F2 : Integer;
    pragma Obsolescent;
    F3 : Integer;
  end record;

  type E is (a, bc, 'd', quack);
  pragma Obsolescent (Entity => bc)
  pragma Obsolescent (Entity => 'd')

  function "+"
    (a, b : character) return character;
  pragma Obsolescent (Entity => "+");
end;
```

Note that, as for all pragmas, if you use a pragma argument identifier, then all subsequent parameters must also use a pragma argument identifier. So if you specify "Entity =>" for the Entity argument, and a Message argument is present, it must be preceded by "Message =>".

Pragma Optimize_Alignment

Syntax:

```
pragma Optimize_Alignment (TIME | SPACE | OFF);
```

This is a configuration pragma which affects the choice of default alignments for types where no alignment is explicitly specified. There is a time/space trade-off in the selection of these values. Large alignments result in more efficient code, at the expense of larger data space, since sizes have to be increased to match these alignments. Smaller alignments save space, but the access code is slower. The normal choice of default alignments (which is what you get if you do not use this pragma, or if you use an argument of OFF), tries to balance these two requirements.

Specifying SPACE causes smaller default alignments to be chosen in two cases. First any packed record is given an alignment of 1. Second, if a size is given for the type, then the alignment is chosen to avoid increasing this size. For example, consider:

```

type R is record
  X : Integer;
  Y : Character;
end record;

for R'Size use 5*8;

```

In the default mode, this type gets an alignment of 4, so that access to the Integer field X are efficient. But this means that objects of the type end up with a size of 8 bytes. This is a valid choice, since sizes of objects are allowed to be bigger than the size of the type, but it can waste space if for example fields of type R appear in an enclosing record. If the above type is compiled in `Optimize_Alignment (Space)` mode, the alignment is set to 1.

Specifying `TIME` causes larger default alignments to be chosen in the case of small types with sizes that are not a power of 2. For example, consider:

```

type R is record
  A : Character;
  B : Character;
  C : Boolean;
end record;

pragma Pack (R);
for R'Size use 17;

```

The default alignment for this record is normally 1, but if this type is compiled in `Optimize_Alignment (Time)` mode, then the alignment is set to 4, which wastes space for objects of the type, since they are now 4 bytes long, but results in more efficient access when the whole record is referenced.

As noted above, this is a configuration pragma, and there is a requirement that all units in a partition be compiled with a consistent setting of the optimization setting. This would normally be achieved by use of a configuration pragma file containing the appropriate setting. The exception to this rule is that units with an explicit configuration pragma in the same file as the source unit are excluded from the consistency check, as are all predefined units. The latter are compiled by default in pragma `Optimize_Alignment (Off)` mode if no pragma appears at the start of the file.

Pragma Passive

Syntax:

```
pragma Passive [(Semaphore | No)];
```

Syntax checked, but otherwise ignored by GNAT. This is recognized for compatibility with DEC Ada 83 implementations, where it is used within a task definition to request that a task be made passive. If the argument `Semaphore` is present, or the argument is omitted, then DEC Ada 83 treats the pragma as an assertion that the containing task is passive and that optimization of context switch with this task is permitted and desired. If the argument `No` is present, the task must not be optimized. GNAT does not attempt to optimize any tasks in this manner (since protected objects are available in place of passive tasks).

Pragma Persistent_BSS

Syntax:

```
pragma Persistent_BSS [(LOCAL_NAME)]
```

This pragma allows selected objects to be placed in the `.persistent_bss` section. On some targets the linker and loader provide for special treatment of this section, allowing a program to be reloaded without affecting the contents of this data (hence the name persistent).

There are two forms of usage. If an argument is given, it must be the local name of a library level object, with no explicit initialization and whose type is potentially persistent. If no argument is given, then the pragma is a configuration pragma, and applies to all library level objects with no explicit initialization of potentially persistent types.

A potentially persistent type is a scalar type, or a non-tagged, non-discriminated record, all of whose components have no explicit initialization and are themselves of a potentially persistent type, or an array, all of whose constraints are static, and whose component type is potentially persistent.

If this pragma is used on a target where this feature is not supported, then the pragma will be ignored. See also `pragma Linker_Section`.

Pragma Polling

Syntax:

```
pragma Polling (ON | OFF);
```

This pragma controls the generation of polling code. This is normally off. If `pragma Polling (ON)` is used then periodic calls are generated to the routine `Ada.Exceptions.Poll`. This routine is a separate unit in the runtime library, and can be found in file `'a-excpol.adb'`.

`Pragma Polling` can appear as a configuration pragma (for example it can be placed in the `'gnat.adc'` file) to enable polling globally, or it can be used in the statement or declaration sequence to control polling more locally.

A call to the polling routine is generated at the start of every loop and at the start of every subprogram call. This guarantees that the `Poll` routine is called frequently, and places an upper bound (determined by the complexity of the code) on the period between two `Poll` calls.

The primary purpose of the polling interface is to enable asynchronous aborts on targets that cannot otherwise support it (for example Windows NT), but it may be used for any other purpose requiring periodic polling. The standard version is null, and can be replaced by a user program. This will require re-compilation of the `Ada.Exceptions` package that can be found in files `'a-except.ads'` and `'a-except.adb'`.

A standard alternative unit (in file `'4wexcpol.adb'` in the standard GNAT distribution) is used to enable the asynchronous abort capability on targets that do not normally support the capability. The version of `Poll` in this file makes a call to the appropriate runtime routine to test for an abort condition.

Note that polling can also be enabled by use of the `'-gnatP'` switch. See [Section “Switches for gcc”](#) in *GNAT User's Guide*, for details.

Pragma Postcondition

Syntax:

```
pragma Postcondition (
  [Check    =>] Boolean_Expression
  [, [Message =>] String_Expression]);
```

The **Postcondition** pragma allows specification of automatic postcondition checks for subprograms. These checks are similar to assertions, but are automatically inserted just prior to the return statements of the subprogram with which they are associated (including implicit returns at the end of procedure bodies and associated exception handlers).

In addition, the boolean expression which is the condition which must be true may contain references to function'Result in the case of a function to refer to the returned value.

Postcondition pragmas may appear either immediately following the (separate) declaration of a subprogram, or at the start of the declarations of a subprogram body. Only other pragmas may intervene (that is appear between the subprogram declaration and its postconditions, or appear before the postcondition in the declaration sequence in a subprogram body). In the case of a postcondition appearing after a subprogram declaration, the formal arguments of the subprogram are visible, and can be referenced in the postcondition expressions.

The postconditions are collected and automatically tested just before any return (implicit or explicit) in the subprogram body. A postcondition is only recognized if postconditions are active at the time the pragma is encountered. The compiler switch '**gnata**' turns on all postconditions by default, and pragma **Check_Policy** with an identifier of **Postcondition** can also be used to control whether postconditions are active.

The general approach is that postconditions are placed in the spec if they represent functional aspects which make sense to the client. For example we might have:

```
function Direction return Integer;
pragma Postcondition
  (Direction'Result = +1
   or else
   Direction'Result = -1);
```

which serves to document that the result must be +1 or -1, and will test that this is the case at run time if postcondition checking is active.

Postconditions within the subprogram body can be used to check that some internal aspect of the implementation, not visible to the client, is operating as expected. For instance if a square root routine keeps an internal counter of the number of times it is called, then we might have the following postcondition:

```
Sqrt_Calls : Natural := 0;

function Sqrt (Arg : Float) return Float is
  pragma Postcondition
    (Sqrt_Calls = Sqrt_Calls'Old + 1);
  ...
end Sqrt
```

As this example, shows, the use of the **Old** attribute is often useful in postconditions to refer to the state on entry to the subprogram.

Note that postconditions are only checked on normal returns from the subprogram. If an abnormal return results from raising an exception, then the postconditions are not checked.

If a postcondition fails, then the exception **System.Assertions.Assert_Failure** is raised. If a message argument was supplied, then the given string will be used as the

exception message. If no message argument was supplied, then the default message has the form "Postcondition failed at file:line". The exception is raised in the context of the subprogram body, so it is possible to catch postcondition failures within the subprogram body itself.

Within a package spec, normal visibility rules in Ada would prevent forward references within a postcondition pragma to functions defined later in the same package. This would introduce undesirable ordering constraints. To avoid this problem, all postcondition pragmas are analyzed at the end of the package spec, allowing forward references.

The following example shows that this even allows mutually recursive postconditions as in:

```
package Parity_Functions is
  function Odd (X : Natural) return Boolean;
  pragma Postcondition
    (Odd'Result =
      (x = 1
       or else
        (x /= 0 and then Even (X - 1))));

  function Even (X : Natural) return Boolean;
  pragma Postcondition
    (Even'Result =
      (x = 0
       or else
        (x /= 1 and then Odd (X - 1))));

end Parity_Functions;
```

There are no restrictions on the complexity or form of conditions used within `Postcondition` pragmas. The following example shows that it is even possible to verify performance behavior.

```
package Sort is

  Performance : constant Float;
  -- Performance constant set by implementation
  -- to match target architecture behavior.

  procedure Treesort (Arg : String);
  -- Sorts characters of argument using N*logN sort
  pragma Postcondition
    (Float (Clock - Clock'Old) <=
      Float (Arg'Length) *
      log (Float (Arg'Length)) *
      Performance);

end Sort;
```

Note: postcondition pragmas associated with subprograms that are marked as `Inline_Always`, or those marked as `Inline` with front-end inlining (`-gnatN` option set) are accepted and legality-checked by the compiler, but are ignored at run-time even if postcondition checking is enabled.

Pragma Precondition

Syntax:

```
pragma Precondition (
```

```
[Check =>] Boolean_Expression
[, [Message =>] String_Expression];
```

The `Precondition` pragma is similar to `Postcondition` except that the corresponding checks take place immediately upon entry to the subprogram, and if a precondition fails, the exception is raised in the context of the caller, and the attribute `'Result` cannot be used within the precondition expression.

Otherwise, the placement and visibility rules are identical to those described for postconditions. The following is an example of use within a package spec:

```
package Math_Functions is
...
function Sqrt (Arg : Float) return Float;
pragma Precondition (Arg >= 0.0)
...
end Math_Functions;
```

`Precondition` pragmas may appear either immediately following the (separate) declaration of a subprogram, or at the start of the declarations of a subprogram body. Only other pragmas may intervene (that is appear between the subprogram declaration and its postconditions, or appear before the postcondition in the declaration sequence in a subprogram body).

Note: postcondition pragmas associated with subprograms that are marked as `Inline_Always`, or those marked as `Inline` with front-end inlining (`-gnatN` option set) are accepted and legality-checked by the compiler, but are ignored at run-time even if postcondition checking is enabled.

Pragma Profile (Ravenscar)

Syntax:

```
pragma Profile (Ravenscar);
```

A configuration pragma that establishes the following set of configuration pragmas:

Task_Dispatching_Policy (FIFO_Within_Priorities)

[RM D.2.2] Tasks are dispatched following a preemptive priority-ordered scheduling policy.

Locking_Policy (Ceiling_Locking)

[RM D.3] While tasks and interrupts execute a protected action, they inherit the ceiling priority of the corresponding protected object.

plus the following set of restrictions:

Max_Entry_Queue_Length = 1

Defines the maximum number of calls that are queued on a (protected) entry. Note that this restriction is checked at run time. Violation of this restriction results in the raising of `Program_Error` exception at the point of the call. For the Profile (Ravenscar) the value of `Max_Entry_Queue_Length` is always 1 and hence no task can be queued on a protected entry.

Max_Protected_Entries = 1

[RM D.7] Specifies the maximum number of entries per protected type. The bounds of every entry family of a protected unit shall be static, or shall be defined by a discriminant of a subtype whose corresponding bound is static. For the Profile (Ravenscar) the value of `Max_Protected_Entries` is always 1.

Max_Task_Entries = 0

[RM D.7] Specifies the maximum number of entries per task. The bounds of every entry family of a task unit shall be static, or shall be defined by a discriminant of a subtype whose corresponding bound is static. A value of zero indicates that no rendezvous are possible. For the Profile (Ravenscar), the value of Max_Task_Entries is always 0 (zero).

No_Abort_Statements

[RM D.7] There are no abort_statements, and there are no calls to Task_Identification.Abort_Task.

No_Asynchronous_Control

There are no semantic dependences on the package Asynchronous_Task_Control.

No_Calendar

There are no semantic dependencies on the package Ada.Calendar.

No_Dynamic_Attachment

There is no call to any of the operations defined in package Ada.Interrupts (Is_Reserved, Is_Attached, Current_Handler, Attach_Handler, Exchange_Handler, Detach_Handler, and Reference).

No_Dynamic_Priorities

[RM D.7] There are no semantic dependencies on the package Dynamic_Priorities.

No_Implicit_Heap_Allocations

[RM D.7] No constructs are allowed to cause implicit heap allocation.

No_Local_Protected_Objects

Protected objects and access types that designate such objects shall be declared only at library level.

No_Local_Timing_Events

[RM D.7] All objects of type Ada.Timing_Events.Timing_Event are declared at the library level.

No_Protected_Type_Allocators

There are no allocators for protected types or types containing protected sub-components.

No_Relative_Delay

There are no delay_relative statements.

No_Requeue_Statements

Requeue statements are not allowed.

No_Select_Statements

There are no select_statements.

No_Specific_Termination_Handlers

[RM D.7] There are no calls to Ada.Task_Termination.Set_Specific_Handler or to Ada.Task_Termination.Specific_Handler.

No_Task_Allocators

[RM D.7] There are no allocators for task types or types containing task sub-components.

No_Task_Attributes_Package

There are no semantic dependencies on the `Ada.Task_Attributes` package.

No_Task_Hierarchy

[RM D.7] All (non-environment) tasks depend directly on the environment task of the partition.

No_Task_Termination

Tasks which terminate are erroneous.

No_Unchecked_Conversion

There are no semantic dependencies on the `Ada.Unchecked_Conversion` package.

No_Unchecked_Deallocation

There are no semantic dependencies on the `Ada.Unchecked_Deallocation` package.

Simple_Barriers

Entry barrier condition expressions shall be either static boolean expressions or boolean objects which are declared in the protected type which contains the entry.

This set of configuration pragmas and restrictions correspond to the definition of the “Ravenscar Profile” for limited tasking, devised and published by the *International Real-Time Ada Workshop*, 1997, and whose most recent description is available at <http://www-users.cs.york.ac.uk/~burns/ravenscar.ps>.

The original definition of the profile was revised at subsequent IRTAW meetings. It has been included in the *ISO Guide for the Use of the Ada Programming Language in High Integrity Systems*, and has been approved by ISO/IEC/SC22/WG9 for inclusion in the next revision of the standard. The formal definition given by the Ada Rapporteur Group (ARG) can be found in two Ada Issues (AI-249 and AI-305) available at <http://www.ada-auth.org/cgi-bin/cvsweb.cgi/AIs/AI-00249.TXT> and <http://www.ada-auth.org/cgi-bin/cvsweb.cgi/AIs/AI-00305.TXT> respectively.

The above set is a superset of the restrictions provided by pragma `Profile (Restricted)`, it includes six additional restrictions (`Simple_Barriers`, `No_Select_Statements`, `No_Calendar`, `No_Implicit_Heap_Allocations`, `No_Relative_Delay` and `No_Task_Termination`). This means that pragma `Profile (Ravenscar)`, like the pragma `Profile (Restricted)`, automatically causes the use of a simplified, more efficient version of the tasking run-time system.

Pragma Profile (Restricted)

Syntax:

```
pragma Profile (Restricted);
```

A configuration pragma that establishes the following set of restrictions:

- No_Abort_Statements
- No_Entry_Queue
- No_Task_Hierarchy
- No_Task_Allocators
- No_Dynamic_Priorities
- No_Terminate_Alternatives
- No_Dynamic_Attachment
- No_Protected_Type_Allocators
- No_Local_Protected_Objects
- No_Requeue_Statements
- No_Task_Attributes_Package
- Max_Asynchronous_Select_Nesting = 0
- Max_Task_Entries = 0
- Max_Protected_Entries = 1
- Max_Select_Alternatives = 0

This set of restrictions causes the automatic selection of a simplified version of the run time that provides improved performance for the limited set of tasking functionality permitted by this set of restrictions.

Pragma Psect_Object

Syntax:

```
pragma Psect_Object (
  [Internal =>] LOCAL_NAME,
  [, [External =>] EXTERNAL_SYMBOL]
  [, [Size      =>] EXTERNAL_SYMBOL]);

EXTERNAL_SYMBOL ::=
  IDENTIFIER
  | static_string_EXPRESSION
```

This pragma is identical in effect to pragma `Common_Object`.

Pragma Pure_Function

Syntax:

```
pragma Pure_Function ([Entity =>] function_LOCAL_NAME);
```

This pragma appears in the same declarative part as a function declaration (or a set of function declarations if more than one overloaded declaration exists, in which case the pragma applies to all entities). It specifies that the function `Entity` is to be considered pure for the purposes of code generation. This means that the compiler can assume that there are no side effects, and in particular that two calls with identical arguments produce the same result. It also means that the function can be used in an address clause.

Note that, quite deliberately, there are no static checks to try to ensure that this promise is met, so `Pure_Function` can be used with functions that are conceptually pure, even if they do modify global variables. For example, a square root function that is instrumented

to count the number of times it is called is still conceptually pure, and can still be optimized, even though it modifies a global variable (the count). Memo functions are another example (where a table of previous calls is kept and consulted to avoid re-computation).

Note: Most functions in a **Pure** package are automatically pure, and there is no need to use pragma **Pure_Function** for such functions. One exception is any function that has at least one formal of type **System.Address** or a type derived from it. Such functions are not considered pure by default, since the compiler assumes that the **Address** parameter may be functioning as a pointer and that the referenced data may change even if the address value does not. Similarly, imported functions are not considered to be pure by default, since there is no way of checking that they are in fact pure. The use of pragma **Pure_Function** for such a function will override these default assumption, and cause the compiler to treat a designated subprogram as pure in these cases.

Note: If pragma **Pure_Function** is applied to a renamed function, it applies to the underlying renamed function. This can be used to disambiguate cases of overloading where some but not all functions in a set of overloaded functions are to be designated as pure.

If pragma **Pure_Function** is applied to a library level function, the function is also considered pure from an optimization point of view, but the unit is not a Pure unit in the categorization sense. So for example, a function thus marked is free to **with** non-pure units.

Pragma Restriction_Warnings

Syntax:

```
pragma Restriction_Warnings
  (restriction_IDENTIFIER {, restriction_IDENTIFIER});
```

This pragma allows a series of restriction identifiers to be specified (the list of allowed identifiers is the same as for pragma **Restrictions**). For each of these identifiers the compiler checks for violations of the restriction, but generates a warning message rather than an error message if the restriction is violated.

Pragma Shared

This pragma is provided for compatibility with Ada 83. The syntax and semantics are identical to pragma **Atomic**.

Pragma Short_Circuit_And_Or

This configuration pragma causes any occurrence of the **AND** operator applied to operands of type **Standard.Boolean** to be short-circuited (i.e. the **AND** operator is treated as if it were **AND THEN**). **Or** is similarly treated as **OR ELSE**. This may be useful in the context of certification protocols requiring the use of short-circuited logical operators. If this configuration pragma occurs locally within the file being compiled, it applies only to the file being compiled. There is no requirement that all units in a partition use this option.

semantics are identical to pragma **Atomic**.

Pragma Source_File_Name

Syntax:

```
pragma Source_File_Name (
  [Unit_Name    =>] unit_NAME,
  Spec_File_Name =>  STRING_LITERAL,
  [Index => INTEGER_LITERAL]);

pragma Source_File_Name (
  [Unit_Name    =>] unit_NAME,
  Body_File_Name =>  STRING_LITERAL,
  [Index => INTEGER_LITERAL]);
```

Use this to override the normal naming convention. It is a configuration pragma, and so has the usual applicability of configuration pragmas (i.e. it applies to either an entire partition, or to all units in a compilation, or to a single unit, depending on how it is used. *unit_name* is mapped to *file_name_literal*. The identifier for the second argument is required, and indicates whether this is the file name for the spec or for the body.

The optional *Index* argument should be used when a file contains multiple units, and when you do not want to use **gnatchop** to separate them into multiple files (which is the recommended procedure to limit the number of recompilations that are needed when some sources change). For instance, if the source file `'source.adb'` contains

```
package B is
...
end B;

with B;
procedure A is
begin
  ..
end A;
```

you could use the following configuration pragmas:

```
pragma Source_File_Name
  (B, Spec_File_Name => "source.adb", Index => 1);
pragma Source_File_Name
  (A, Body_File_Name => "source.adb", Index => 2);
```

Note that the **gnatname** utility can also be used to generate those configuration pragmas.

Another form of the **Source_File_Name** pragma allows the specification of patterns defining alternative file naming schemes to apply to all files.

```
pragma Source_File_Name
  ( [Spec_File_Name =>] STRING_LITERAL
    [, [Casing      =>] CASING_SPEC]
    [, [Dot_Replacement =>] STRING_LITERAL]);

pragma Source_File_Name
  ( [Body_File_Name =>] STRING_LITERAL
    [, [Casing      =>] CASING_SPEC]
    [, [Dot_Replacement =>] STRING_LITERAL]);

pragma Source_File_Name
  ( [Subunit_File_Name =>] STRING_LITERAL
    [, [Casing      =>] CASING_SPEC]
    [, [Dot_Replacement =>] STRING_LITERAL]);
```

CASING_SPEC ::= Lowercase | Uppercase | Mixedcase

The first argument is a pattern that contains a single asterisk indicating the point at which the unit name is to be inserted in the pattern string to form the file name. The second

argument is optional. If present it specifies the casing of the unit name in the resulting file name string. The default is lower case. Finally the third argument allows for systematic replacement of any dots in the unit name by the specified string literal.

A pragma `Source_File_Name` cannot appear after a [\[Pragma Source_File_Name_Project\]](#), [page 51](#).

For more details on the use of the `Source_File_Name` pragma, See [Section “Using Other File Names”](#) in *GNAT User’s Guide*, and [Section “Alternative File Naming Schemes”](#) in *GNAT User’s Guide*.

Pragma Source_File_Name_Project

This pragma has the same syntax and semantics as pragma `Source_File_Name`. It is only allowed as a stand alone configuration pragma. It cannot appear after a [\[Pragma Source_File_Name\]](#), [page 49](#), and most importantly, once pragma `Source_File_Name_Project` appears, no further `Source_File_Name` pragmas are allowed.

The intention is that `Source_File_Name_Project` pragmas are always generated by the Project Manager in a manner consistent with the naming specified in a project file, and when naming is controlled in this manner, it is not permissible to attempt to modify this naming scheme using `Source_File_Name` pragmas (which would not be known to the project manager).

Pragma Source_Reference

Syntax:

```
pragma Source_Reference (INTEGER_LITERAL, STRING_LITERAL);
```

This pragma must appear as the first line of a source file. *integer_literal* is the logical line number of the line following the pragma line (for use in error messages and debugging information). *string_literal* is a static string constant that specifies the file name to be used in error messages and debugging information. This is most notably used for the output of `gnatchop` with the ‘-r’ switch, to make sure that the original unchopped source file is the one referred to.

The second argument must be a string literal, it cannot be a static string expression other than a string literal. This is because its value is needed for error messages issued by all phases of the compiler.

Pragma Stream_Convert

Syntax:

```
pragma Stream_Convert (
  [Entity =>] type_LOCAL_NAME,
  [Read   =>] function_NAME,
  [Write  =>] function_NAME);
```

This pragma provides an efficient way of providing stream functions for types defined in packages. Not only is it simpler to use than declaring the necessary functions with attribute representation clauses, but more significantly, it allows the declaration to be made in such a way that the stream packages are not loaded unless they are needed. The use of the

Stream.Convert pragma adds no overhead at all, unless the stream attributes are actually used on the designated type.

The first argument specifies the type for which stream functions are provided. The second parameter provides a function used to read values of this type. It must name a function whose argument type may be any subtype, and whose returned type must be the type given as the first argument to the pragma.

The meaning of the *Read* parameter is that if a stream attribute directly or indirectly specifies reading of the type given as the first parameter, then a value of the type given as the argument to the Read function is read from the stream, and then the Read function is used to convert this to the required target type.

Similarly the *Write* parameter specifies how to treat write attributes that directly or indirectly apply to the type given as the first parameter. It must have an input parameter of the type specified by the first parameter, and the return type must be the same as the input type of the Read function. The effect is to first call the Write function to convert to the given stream type, and then write the result type to the stream.

The Read and Write functions must not be overloaded subprograms. If necessary renamings can be supplied to meet this requirement. The usage of this attribute is best illustrated by a simple example, taken from the GNAT implementation of package Ada.Strings.Unbounded:

```
function To_Unbounded (S : String)
    return Unbounded_String
    renames To_Unbounded_String;

pragma Stream_Convert
    (Unbounded_String, To_Unbounded, To_String);
```

The specifications of the referenced functions, as given in the Ada Reference Manual are:

```
function To_Unbounded_String (Source : String)
    return Unbounded_String;

function To_String (Source : Unbounded_String)
    return String;
```

The effect is that if the value of an unbounded string is written to a stream, then the representation of the item in the stream is in the same format that would be used for `Standard.String'Output`, and this same representation is expected when a value of this type is read from the stream. Note that the value written always includes the bounds, even for `Unbounded_String'Write`, since `Unbounded_String` is not an array type.

Pragma Style_Checks

Syntax:

```
pragma Style_Checks (string_LITERAL | ALL_CHECKS |
    On | Off [, LOCAL_NAME]);
```

This pragma is used in conjunction with compiler switches to control the built in style checking provided by GNAT. The compiler switches, if set, provide an initial setting for the switches, and this pragma may be used to modify these settings, or the settings may be provided entirely by the use of the pragma. This pragma can be used anywhere that a pragma is legal, including use as a configuration pragma (including use in the `'gnat.adc'` file).

The form with a string literal specifies which style options are to be activated. These are additive, so they apply in addition to any previously set style check options. The codes for the options are the same as those used in the ‘`-gnaty`’ switch to `gcc` or `gnatmake`. For example the following two methods can be used to enable layout checking:

- ```
pragma Style_Checks ("l");
```
- ```
gcc -c -gnatyl ...
```

The form `ALL_CHECKS` activates all standard checks (its use is equivalent to the use of the `gnaty` switch with no options. See [Section “About This Guide” in GNAT User’s Guide](#), for details.

The forms with `Off` and `On` can be used to temporarily disable style checks as shown in the following example:

```
pragma Style_Checks ("k"); -- requires keywords in lower case
pragma Style_Checks (Off); -- turn off style checks
NULL;                     -- this will not generate an error message
pragma Style_Checks (On);  -- turn style checks back on
NULL;                     -- this will generate an error message
```

Finally the two argument form is allowed only if the first argument is `On` or `Off`. The effect is to turn of semantic style checks for the specified entity, as shown in the following example:

```
pragma Style_Checks ("r"); -- require consistency of identifier casing
Arg : Integer;
Rf1 : Integer := ARG;      -- incorrect, wrong case
pragma Style_Checks (Off, Arg);
Rf2 : Integer := ARG;      -- OK, no error
```

Pragma Subtitle

Syntax:

```
pragma Subtitle ([Subtitle =>] STRING_LITERAL);
```

This pragma is recognized for compatibility with other Ada compilers but is ignored by GNAT.

Pragma Suppress

Syntax:

```
pragma Suppress (Identifier [, [On =>] Name]);
```

This is a standard pragma, and supports all the check names required in the RM. It is included here because GNAT recognizes one additional check name: `Alignment_Check` which can be used to suppress alignment checks on addresses used in address clauses. Such checks can also be suppressed by suppressing range checks, but the specific use of `Alignment_Check` allows suppression of alignment checks without suppressing other range checks.

Note that pragma `Suppress` gives the compiler permission to omit checks, but does not require the compiler to omit checks. The compiler will generate checks if they are essentially

free, even when they are suppressed. In particular, if the compiler can prove that a certain check will necessarily fail, it will generate code to do an unconditional “raise”, even if checks are suppressed. The compiler warns in this case.

Of course, run-time checks are omitted whenever the compiler can prove that they will not fail, whether or not checks are suppressed.

Pragma Suppress_All

Syntax:

```
pragma Suppress_All;
```

This pragma can only appear immediately following a compilation unit. The effect is to apply **Suppress (All_Checks)** to the unit which it follows. This pragma is implemented for compatibility with DEC Ada 83 usage. The use of pragma **Suppress (All_Checks)** as a normal configuration pragma is the preferred usage in GNAT.

Pragma Suppress_Exception_Locations

Syntax:

```
pragma Suppress_Exception_Locations;
```

In normal mode, a raise statement for an exception by default generates an exception message giving the file name and line number for the location of the raise. This is useful for debugging and logging purposes, but this entails extra space for the strings for the messages. The configuration pragma **Suppress_Exception_Locations** can be used to suppress the generation of these strings, with the result that space is saved, but the exception message for such raises is null. This configuration pragma may appear in a global configuration pragma file, or in a specific unit as usual. It is not required that this pragma be used consistently within a partition, so it is fine to have some units within a partition compiled with this pragma and others compiled in normal mode without it.

Pragma Suppress_Initialization

Syntax:

```
pragma Suppress_Initialization ([Entity =>] type_Name);
```

This pragma suppresses any implicit or explicit initialization associated with the given type name for all variables of this type.

Pragma Task_Info

Syntax

```
pragma Task_Info (EXPRESSION);
```

This pragma appears within a task definition (like pragma **Priority**) and applies to the task in which it appears. The argument must be of type **System.Task_Info.Task_Info_Type**. The **Task_Info** pragma provides system dependent control over aspects of tasking implementation, for example, the ability to map tasks to specific processors. For details on the facilities available for the version of GNAT that you are using, see the documentation in the spec of package **System.Task_Info** in the runtime library.

Pragma Task_Name

Syntax

```
pragma Task_Name (string_EXPRESSION);
```

This pragma appears within a task definition (like pragma `Priority`) and applies to the task in which it appears. The argument must be of type `String`, and provides a name to be used for the task instance when the task is created. Note that this expression is not required to be static, and in particular, it can contain references to task discriminants. This facility can be used to provide different names for different tasks as they are created, as illustrated in the example below.

The task name is recorded internally in the run-time structures and is accessible to tools like the debugger. In addition the routine `Ada.Task_Identification.Image` will return this string, with a unique task address appended.

```
-- Example of the use of pragma Task_Name

with Ada.Task_Identification;
use Ada.Task_Identification;
with Text_IO; use Text_IO;
procedure t3 is

    type Astring is access String;

    task type Task_Typ (Name : access String) is
        pragma Task_Name (Name.all);
    end Task_Typ;

    task body Task_Typ is
        Nam : constant String := Image (Current_Task);
    begin
        Put_Line ("-->" & Nam (1 .. 14) & "<--");
    end Task_Typ;

    type Ptr_Task is access Task_Typ;
    Task_Var : Ptr_Task;

begin
    Task_Var :=
        new Task_Typ (new String'("This is task 1"));
    Task_Var :=
        new Task_Typ (new String'("This is task 2"));
end;
```

Pragma Task_Storage

Syntax:

```
pragma Task_Storage (
    [Task_Type =>] LOCAL_NAME,
    [Top_Guard =>] static_integer_EXPRESSION);
```

This pragma specifies the length of the guard area for tasks. The guard area is an additional storage area allocated to a task. A value of zero means that either no guard area is created or a minimal guard area is created, depending on the target. This pragma can appear anywhere a `Storage_Size` attribute definition clause is allowed for a task type.

Pragma Thread_Local_Storage

Syntax:

```
pragma Thread_Local_Storage ([Entity =>] LOCAL_NAME);
```

This pragma specifies that the specified entity, which must be a variable declared in a library level package, is to be marked as "Thread Local Storage" (TLS). On systems supporting this (which include Solaris, GNU/Linux and VxWorks 6), this causes each thread (and hence each Ada task) to see a distinct copy of the variable.

The variable may not have default initialization, and if there is an explicit initialization, it must be either `null` for an access variable, or a static expression for a scalar variable. This provides a low level mechanism similar to that provided by the `Ada.Task_Attributes` package, but much more efficient and is also useful in writing interface code that will interact with foreign threads.

If this pragma is used on a system where TLS is not supported, then an error message will be generated and the program will be rejected.

Pragma Time_Slice

Syntax:

```
pragma Time_Slice (static_duration_EXPRESSION);
```

For implementations of GNAT on operating systems where it is possible to supply a time slice value, this pragma may be used for this purpose. It is ignored if it is used in a system that does not allow this control, or if it appears in other than the main program unit. Note that the effect of this pragma is identical to the effect of the DEC Ada 83 pragma of the same name when operating under OpenVMS systems.

Pragma Title

Syntax:

```
pragma Title (TITLING_OPTION [, TITLING_OPTION]);
```

```
TITLING_OPTION ::=
  [Title    =>] STRING_LITERAL,
  | [Subtitle =>] STRING_LITERAL
```

Syntax checked but otherwise ignored by GNAT. This is a listing control pragma used in DEC Ada 83 implementations to provide a title and/or subtitle for the program listing. The program listing generated by GNAT does not have titles or subtitles.

Unlike other pragmas, the full flexibility of named notation is allowed for this pragma, i.e. the parameters may be given in any order if named notation is used, and named and positional notation can be mixed following the normal rules for procedure calls in Ada.

Pragma Unchecked_Union

Syntax:

```
pragma Unchecked_Union (first_subtype_LOCAL_NAME);
```

This pragma is used to specify a representation of a record type that is equivalent to a C union. It was introduced as a GNAT implementation defined pragma in the GNAT Ada 95 mode. Ada 2005 includes an extended version of this pragma, making it language defined,

and GNAT fully implements this extended version in all language modes (Ada 83, Ada 95, and Ada 2005). For full details, consult the Ada 2005 Reference Manual, section B.3.3.

Pragma Unimplemented_Unit

Syntax:

```
pragma Unimplemented_Unit;
```

If this pragma occurs in a unit that is processed by the compiler, GNAT aborts with the message ‘xxx not implemented’, where xxx is the name of the current compilation unit. This pragma is intended to allow the compiler to handle unimplemented library units in a clean manner.

The abort only happens if code is being generated. Thus you can use specs of unimplemented packages in syntax or semantic checking mode.

Pragma Universal_Aliasing

Syntax:

```
pragma Universal_Aliasing [(Entity =>] type_LOCAL_NAME)];
```

type_LOCAL_NAME must refer to a type declaration in the current declarative part. The effect is to inhibit strict type-based aliasing optimization for the given type. In other words, the effect is as though access types designating this type were subject to pragma No_Strict_Aliasing. For a detailed description of the strict aliasing optimization, and the situations in which it must be suppressed, See [Section “Optimization and Strict Aliasing” in GNAT User’s Guide](#).

Pragma Universal_Data

Syntax:

```
pragma Universal_Data [(library_unit_Name)];
```

This pragma is supported only for the AAMP target and is ignored for other targets. The pragma specifies that all library-level objects (Counter 0 data) associated with the library unit are to be accessed and updated using universal addressing (24-bit addresses for AAMP5) rather than the default of 16-bit Data Environment (DENV) addressing. Use of this pragma will generally result in less efficient code for references to global data associated with the library unit, but allows such data to be located anywhere in memory. This pragma is a library unit pragma, but can also be used as a configuration pragma (including use in the ‘gnat.adc’ file). The functionality of this pragma is also available by applying the -univ switch on the compilations of units where universal addressing of the data is desired.

Pragma Unmodified

Syntax:

```
pragma Unmodified (LOCAL_NAME {, LOCAL_NAME});
```

This pragma signals that the assignable entities (variables, out parameters, in out parameters) whose names are listed are deliberately not assigned in the current source unit. This suppresses warnings about the entities being referenced but not assigned, and in addition a warning will be generated if one of these entities is in fact assigned in the same unit as the pragma (or in the corresponding body, or one of its subunits).

This is particularly useful for clearly signaling that a particular parameter is not modified, even though the spec suggests that it might be.

Pragma Unreferenced

Syntax:

```
pragma Unreferenced (LOCAL_NAME {, LOCAL_NAME});
pragma Unreferenced (library_unit_NAME {, library_unit_NAME});
```

This pragma signals that the entities whose names are listed are deliberately not referenced in the current source unit. This suppresses warnings about the entities being unreferenced, and in addition a warning will be generated if one of these entities is in fact referenced in the same unit as the pragma (or in the corresponding body, or one of its subunits).

This is particularly useful for clearly signaling that a particular parameter is not referenced in some particular subprogram implementation and that this is deliberate. It can also be useful in the case of objects declared only for their initialization or finalization side effects.

If `LOCAL_NAME` identifies more than one matching homonym in the current scope, then the entity most recently declared is the one to which the pragma applies. Note that in the case of accept formals, the pragma `Unreferenced` may appear immediately after the keyword `do` which allows the indication of whether or not accept formals are referenced or not to be given individually for each accept statement.

The left hand side of an assignment does not count as a reference for the purpose of this pragma. Thus it is fine to assign to an entity for which pragma `Unreferenced` is given.

Note that if a warning is desired for all calls to a given subprogram, regardless of whether they occur in the same unit as the subprogram declaration, then this pragma should not be used (calls from another unit would not be flagged); pragma `Obsolescent` can be used instead for this purpose, see See [\[Pragma Obsolescent\]](#), page 39.

The second form of pragma `Unreferenced` is used within a context clause. In this case the arguments must be unit names of units previously mentioned in `with` clauses (similar to the usage of pragma `Elaborate_All`). The effect is to suppress warnings about unreferenced units and unreferenced entities within these units.

Pragma Unreferenced_Objects

Syntax:

```
pragma Unreferenced_Objects (local_subtype_NAME {, local_subtype_NAME});
```

This pragma signals that for the types or subtypes whose names are listed, objects which are declared with one of these types or subtypes may not be referenced, and if no references appear, no warnings are given.

This is particularly useful for objects which are declared solely for their initialization and finalization effect. Such variables are sometimes referred to as RAII variables (Resource Acquisition Is Initialization). Using this pragma on the relevant type (most typically a limited controlled type), the compiler will automatically suppress unwanted warnings about these variables not being referenced.

Pragma Unreserve_All_Interrupts

Syntax:

```
pragma Unreserve_All_Interrupts;
```

Normally certain interrupts are reserved to the implementation. Any attempt to attach an interrupt causes `Program_Error` to be raised, as described in RM C.3.2(22). A typical example is the `SIGINT` interrupt used in many systems for a `Ctrl-C` interrupt. Normally this interrupt is reserved to the implementation, so that `Ctrl-C` can be used to interrupt execution.

If the pragma `Unreserve_All_Interrupts` appears anywhere in any unit in a program, then all such interrupts are unreserved. This allows the program to handle these interrupts, but disables their standard functions. For example, if this pragma is used, then pressing `Ctrl-C` will not automatically interrupt execution. However, a program can then handle the `SIGINT` interrupt as it chooses.

For a full list of the interrupts handled in a specific implementation, see the source code for the spec of `Ada.Interrupts.Names` in file ‘`a-intnam.ads`’. This is a target dependent file that contains the list of interrupts recognized for a given target. The documentation in this file also specifies what interrupts are affected by the use of the `Unreserve_All_Interrupts` pragma.

For a more general facility for controlling what interrupts can be handled, see pragma `Interrupt_State`, which subsumes the functionality of the `Unreserve_All_Interrupts` pragma.

Pragma Unsuppress

Syntax:

```
pragma Unsuppress (IDENTIFIER [, [On =>] NAME]);
```

This pragma undoes the effect of a previous pragma `Suppress`. If there is no corresponding pragma `Suppress` in effect, it has no effect. The range of the effect is the same as for pragma `Suppress`. The meaning of the arguments is identical to that used in pragma `Suppress`.

One important application is to ensure that checks are on in cases where code depends on the checks for its correct functioning, so that the code will compile correctly even if the compiler switches are set to suppress checks.

Pragma Use_VADS_Size

Syntax:

```
pragma Use_VADS_Size;
```

This is a configuration pragma. In a unit to which it applies, any use of the ‘`Size`’ attribute is automatically interpreted as a use of the ‘`VADS_Size`’ attribute. Note that this may result in incorrect semantic processing of valid Ada 95 or Ada 2005 programs. This is intended to aid in the handling of existing code which depends on the interpretation of `Size` as implemented in the VADS compiler. See description of the `VADS_Size` attribute for further details.

Pragma Validity_Checks

Syntax:

```
pragma Validity_Checks (string_LITERAL | ALL_CHECKS | On | Off);
```

This pragma is used in conjunction with compiler switches to control the built-in validity checking provided by GNAT. The compiler switches, if set provide an initial setting for the switches, and this pragma may be used to modify these settings, or the settings may be provided entirely by the use of the pragma. This pragma can be used anywhere that a pragma is legal, including use as a configuration pragma (including use in the ‘gnat.adc’ file).

The form with a string literal specifies which validity options are to be activated. The validity checks are first set to include only the default reference manual settings, and then a string of letters in the string specifies the exact set of options required. The form of this string is exactly as described for the ‘-gnatVx’ compiler switch (see the GNAT users guide for details). For example the following two methods can be used to enable validity checking for mode `in` and `in out` subprogram parameters:

-
- `pragma Validity_Checks ("im");`
-
- `gcc -c -gnatVim ...`

The form `ALL_CHECKS` activates all standard checks (its use is equivalent to the use of the `gnatva` switch).

The forms with `Off` and `On` can be used to temporarily disable validity checks as shown in the following example:

```
pragma Validity_Checks ("c"); -- validity checks for copies
pragma Validity_Checks (Off); -- turn off validity checks
A := B;                      -- B will not be validity checked
pragma Validity_Checks (On);  -- turn validity checks back on
A := C;                      -- C will be validity checked
```

Pragma Volatile

Syntax:

```
pragma Volatile (LOCAL_NAME);
```

This pragma is defined by the Ada Reference Manual, and the GNAT implementation is fully conformant with this definition. The reason it is mentioned in this section is that a pragma of the same name was supplied in some Ada 83 compilers, including DEC Ada 83. The Ada 95 / Ada 2005 implementation of pragma `Volatile` is upwards compatible with the implementation in DEC Ada 83.

Pragma Warnings

Syntax:

```
pragma Warnings (On | Off);
pragma Warnings (On | Off, LOCAL_NAME);
pragma Warnings (static_string_EXPRESSION);
```

```
pragma Warnings (On | Off, static_string_EXPRESSION);
```

Normally warnings are enabled, with the output being controlled by the command line switch. Warnings (**Off**) turns off generation of warnings until a Warnings (**On**) is encountered or the end of the current unit. If generation of warnings is turned off using this pragma, then no warning messages are output, regardless of the setting of the command line switches.

The form with a single argument may be used as a configuration pragma.

If the *LOCAL_NAME* parameter is present, warnings are suppressed for the specified entity. This suppression is effective from the point where it occurs till the end of the extended scope of the variable (similar to the scope of **Suppress**).

The form with a single static_string_EXPRESSION argument provides more precise control over which warnings are active. The string is a list of letters specifying which warnings are to be activated and which deactivated. The code for these letters is the same as the string used in the command line switch controlling warnings. For a brief summary, use the gnatmake command with no arguments, which will generate usage information containing the list of warnings switches supported. For full details see [Section “Warning Message Control” in GNAT User’s Guide](#).

The specified warnings will be in effect until the end of the program or another pragma Warnings is encountered. The effect of the pragma is cumulative. Initially the set of warnings is the standard default set as possibly modified by compiler switches. Then each pragma Warning modifies this set of warnings as specified. This form of the pragma may also be used as a configuration pragma.

The fourth form, with an On|Off parameter and a string, is used to control individual messages, based on their text. The string argument is a pattern that is used to match against the text of individual warning messages (not including the initial "warning: " tag).

The pattern may contain asterisks, which match zero or more characters in the message. For example, you can use `pragma Warnings (Off, "*bits of*unused")` to suppress the warning message `warning: 960 bits of "a" unused`. No other regular expression notations are permitted. All characters other than asterisk in these three specific cases are treated as literal characters in the match.

There are two ways to use this pragma. The OFF form can be used as a configuration pragma. The effect is to suppress all warnings (if any) that match the pattern string throughout the compilation.

The second usage is to suppress a warning locally, and in this case, two pragmas must appear in sequence:

```
pragma Warnings (Off, Pattern);
... code where given warning is to be suppressed
pragma Warnings (On, Pattern);
```

In this usage, the pattern string must match in the Off and On pragmas, and at least one matching warning must be suppressed.

Note: the debug flag `-gnatd.i` (`/NOWARNINGS_PRAGMAS` in VMS) can be used to cause the compiler to entirely ignore all WARNINGS pragmas. This can be useful in checking whether obsolete pragmas in existing programs are hiding real problems.

Pragma Weak_External

Syntax:

```
pragma Weak_External ([Entity =>] LOCAL_NAME);
```

LOCAL_NAME must refer to an object that is declared at the library level. This pragma specifies that the given entity should be marked as a weak symbol for the linker. It is equivalent to `__attribute__((weak))` in GNU C and causes *LOCAL_NAME* to be emitted as a weak symbol instead of a regular symbol, that is to say a symbol that does not have to be resolved by the linker if used in conjunction with a pragma Import.

When a weak symbol is not resolved by the linker, its address is set to zero. This is useful in writing interfaces to external modules that may or may not be linked in the final executable, for example depending on configuration settings.

If a program references at run time an entity to which this pragma has been applied, and the corresponding symbol was not resolved at link time, then the execution of the program is erroneous. It is not erroneous to take the Address of such an entity, for example to guard potential references, as shown in the example below.

Some file formats do not support weak symbols so not all target machines support this pragma.

```
-- Example of the use of pragma Weak_External
```

```
package External_Module is
  key : Integer;
  pragma Import (C, key);
  pragma Weak_External (key);
  function Present return boolean;
end External_Module;

with System; use System;
package body External_Module is
  function Present return boolean is
  begin
    return key'Address /= System.Null_Address;
  end Present;
end External_Module;
```

Pragma Wide_Character-Encoding

Syntax:

```
pragma Wide_Character-Encoding (IDENTIFIER | CHARACTER_LITERAL);
```

This pragma specifies the wide character encoding to be used in program source text appearing subsequently. It is a configuration pragma, but may also be used at any point that a pragma is allowed, and it is permissible to have more than one such pragma in a file, allowing multiple encodings to appear within the same file.

The argument can be an identifier or a character literal. In the identifier case, it is one of HEX, UPPER, SHIFT_JIS, EUC, UTF8, or BRACKETS. In the character literal case it is correspondingly one of the characters 'h', 'u', 's', 'e', '8', or 'b'.

Note that when the pragma is used within a file, it affects only the encoding within that file, and does not affect withed units, specs, or subunits.

2 Implementation Defined Attributes

Ada defines (throughout the Ada reference manual, summarized in Annex K), a set of attributes that provide useful additional functionality in all areas of the language. These language defined attributes are implemented in GNAT and work as described in the Ada Reference Manual.

In addition, Ada allows implementations to define additional attributes whose meaning is defined by the implementation. GNAT provides a number of these implementation-dependent attributes which can be used to extend and enhance the functionality of the compiler. This section of the GNAT reference manual describes these additional attributes.

Note that any program using these attributes may not be portable to other compilers (although GNAT implements this set of attributes on all platforms). Therefore if portability to other compilers is an important consideration, you should minimize the use of these attributes.

Abort_Signal

`Standard'Abort_Signal` (`Standard` is the only allowed prefix) provides the entity for the special exception used to signal task abort or asynchronous transfer of control. Normally this attribute should only be used in the tasking runtime (it is highly peculiar, and completely outside the normal semantics of Ada, for a user program to intercept the abort exception).

Address_Size

`Standard'Address_Size` (`Standard` is the only allowed prefix) is a static constant giving the number of bits in an `Address`. It is the same value as `System.Address'Size`, but has the advantage of being static, while a direct reference to `System.Address'Size` is non-static because `Address` is a private type.

Asm_Input

The `Asm_Input` attribute denotes a function that takes two parameters. The first is a string, the second is an expression of the type designated by the prefix. The first (string) argument is required to be a static expression, and is the constraint for the parameter, (e.g. what kind of register is required). The second argument is the value to be used as the input argument. The possible values for the constant are the same as those used in the RTL, and are dependent on the configuration file used to build the GCC back end. [Section 12.1 \[Machine Code Insertions\]](#), page 203

Asm_Output

The `Asm_Output` attribute denotes a function that takes two parameters. The first is a string, the second is the name of a variable of the type designated by the attribute prefix. The first (string) argument is required to be a static expression and designates the constraint for the parameter (e.g. what kind of register is required). The second argument is the variable to be updated with the result. The possible values for constraint are the same as those used in the RTL, and are dependent on the configuration file used to build the GCC back end. If there are no output operands, then this argument may either be omitted, or explicitly given as `No_Output_Operands`. [Section 12.1 \[Machine Code Insertions\]](#), page 203

AST_Entry

This attribute is implemented only in OpenVMS versions of GNAT. Applied to the name of an entry, it yields a value of the predefined type `AST_Handler` (declared in the predefined package `System`, as extended by the use of pragma `Extend_System (Aux_DEC)`). This value enables the given entry to be called when an AST occurs. For further details, refer to the *DEC Ada Language Reference Manual*, section 9.12a.

Bit

obj'`Bit`, where *obj* is any object, yields the bit offset within the storage unit (byte) that contains the first bit of storage allocated for the object. The value of this attribute is of the type `Universal_Integer`, and is always a non-negative number not exceeding the value of `System.Storage_Unit`.

For an object that is a variable or a constant allocated in a register, the value is zero. (The use of this attribute does not force the allocation of a variable to memory).

For an object that is a formal parameter, this attribute applies to either the matching actual parameter or to a copy of the matching actual parameter.

For an access object the value is zero. Note that *obj.all*'`Bit` is subject to an `Access_Check` for the designated object. Similarly for a record component *X.C*'`Bit` is subject to a discriminant check and *X(I).Bit* and *X(I1..I2)*'`Bit` are subject to index checks.

This attribute is designed to be compatible with the DEC Ada 83 definition and implementation of the `Bit` attribute.

Bit_Position

R.C'`Bit`, where *R* is a record object and *C* is one of the fields of the record type, yields the bit offset within the record contains the first bit of storage allocated for the object. The value of this attribute is of the type `Universal_Integer`. The value depends only on the field *C* and is independent of the alignment of the containing record *R*.

Compiler_Version

`Standard`'`Compiler_Version` (`Standard` is the only allowed prefix) yields a static string identifying the version of the compiler being used to compile the unit containing the attribute reference. A typical result would be something like "GNAT Pro 6.3.0w (20090221)".

Code_Address

The '`Address` attribute may be applied to subprograms in Ada 95 and Ada 2005, but the intended effect seems to be to provide an address value which can be used to call the subprogram by means of an address clause as in the following example:

```
procedure K is ...

procedure L;
for L'Address use K'Address;
pragma Import (Ada, L);
```

A call to *L* is then expected to result in a call to *K*. In Ada 83, where there were no access-to-subprogram values, this was a common work-around for getting the effect of an indirect

call. GNAT implements the above use of **Address** and the technique illustrated by the example code works correctly.

However, for some purposes, it is useful to have the address of the start of the generated code for the subprogram. On some architectures, this is not necessarily the same as the **Address** value described above. For example, the **Address** value may reference a subprogram descriptor rather than the subprogram itself.

The **'Code_Address** attribute, which can only be applied to subprogram entities, always returns the address of the start of the generated code of the specified subprogram, which may or may not be the same value as is returned by the corresponding **'Address** attribute.

Default_Bit_Order

Standard'Default_Bit_Order (**Standard** is the only permissible prefix), provides the value **System.Default_Bit_Order** as a **Pos** value (0 for **High_Order_First**, 1 for **Low_Order_First**). This is used to construct the definition of **Default_Bit_Order** in package **System**.

Elaborated

The prefix of the **'Elaborated** attribute must be a unit name. The value is a Boolean which indicates whether or not the given unit has been elaborated. This attribute is primarily intended for internal use by the generated code for dynamic elaboration checking, but it can also be used in user programs. The value will always be **True** once elaboration of all units has been completed. An exception is for units which need no elaboration, the value is always **False** for such units.

Elab_Body

This attribute can only be applied to a program unit name. It returns the entity for the corresponding elaboration procedure for elaborating the body of the referenced unit. This is used in the main generated elaboration procedure by the binder and is not normally used in any other context. However, there may be specialized situations in which it is useful to be able to call this elaboration procedure from Ada code, e.g. if it is necessary to do selective re-elaboration to fix some error.

Elab_Spec

This attribute can only be applied to a program unit name. It returns the entity for the corresponding elaboration procedure for elaborating the spec of the referenced unit. This is used in the main generated elaboration procedure by the binder and is not normally used in any other context. However, there may be specialized situations in which it is useful to be able to call this elaboration procedure from Ada code, e.g. if it is necessary to do selective re-elaboration to fix some error.

Emax

The **Emax** attribute is provided for compatibility with Ada 83. See the Ada 83 reference manual for an exact description of the semantics of this attribute.

Enabled

The **Enabled** attribute allows an application program to check at compile time to see if the designated check is currently enabled. The prefix is a simple identifier, referencing any predefined check name (other than **All_Checks**) or a check name introduced by pragma **Check_Name**. If no argument is given for the attribute, the check is for the general state of the check, if an argument is given, then it is an entity name, and the check indicates whether an **Suppress** or **Unsuppress** has been given naming the entity (if not, then the argument is ignored).

Note that instantiations inherit the check status at the point of the instantiation, so a useful idiom is to have a library package that introduces a check name with pragma **Check_Name**, and then contains generic packages or subprograms which use the **Enabled** attribute to see if the check is enabled. A user of this package can then issue a pragma **Suppress** or pragma **Unsuppress** before instantiating the package or subprogram, controlling whether the check will be present.

Enum_Rep

For every enumeration subtype *S*, **S'Enum_Rep** denotes a function with the following spec:

```
function S'Enum_Rep (Arg : S'Base)
  return Universal_Integer;
```

It is also allowable to apply **Enum_Rep** directly to an object of an enumeration type or to a non-overloaded enumeration literal. In this case **S'Enum_Rep** is equivalent to **typ'Enum_Rep(S)** where *typ* is the type of the enumeration literal or object.

The function returns the representation value for the given enumeration value. This will be equal to value of the **Pos** attribute in the absence of an enumeration representation clause. This is a static attribute (i.e. the result is static if the argument is static).

S'Enum_Rep can also be used with integer types and objects, in which case it simply returns the integer value. The reason for this is to allow it to be used for (<>) discrete formal arguments in a generic unit that can be instantiated with either enumeration types or integer types. Note that if **Enum_Rep** is used on a modular type whose upper bound exceeds the upper bound of the largest signed integer type, and the argument is a variable, so that the universal integer calculation is done at run time, then the call to **Enum_Rep** may raise **Constraint_Error**.

Enum_Val

For every enumeration subtype *S*, **S'Enum_Val** denotes a function with the following spec:

```
function S'Enum_Val (Arg : Universal_Integer)
  return S'Base;
```

The function returns the enumeration value whose representation matches the argument, or raises **Constraint_Error** if no enumeration literal of the type has the matching value. This will be equal to value of the **Val** attribute in the absence of an enumeration representation clause. This is a static attribute (i.e. the result is static if the argument is static).

Epsilon

The **Epsilon** attribute is provided for compatibility with Ada 83. See the Ada 83 reference manual for an exact description of the semantics of this attribute.

Fixed_Value

For every fixed-point type S , S' **Fixed_Value** denotes a function with the following specification:

```
function  $S'$ Fixed_Value (Arg : Universal_Integer)
  return  $S$ ;
```

The value returned is the fixed-point value V such that

$$V = \text{Arg} * S'\text{Small}$$

The effect is thus similar to first converting the argument to the integer type used to represent S , and then doing an unchecked conversion to the fixed-point type. The difference is that there are full range checks, to ensure that the result is in range. This attribute is primarily intended for use in implementation of the input-output functions for fixed-point values.

Has_Access_Values

The prefix of the **Has_Access_Values** attribute is a type. The result is a Boolean value which is True if the is an access type, or is a composite type with a component (at any nesting depth) that is an access type, and is False otherwise. The intended use of this attribute is in conjunction with generic definitions. If the attribute is applied to a generic private type, it indicates whether or not the corresponding actual type has access values.

Has_Discriminants

The prefix of the **Has_Discriminants** attribute is a type. The result is a Boolean value which is True if the type has discriminants, and False otherwise. The intended use of this attribute is in conjunction with generic definitions. If the attribute is applied to a generic private type, it indicates whether or not the corresponding actual type has discriminants.

Img

The **Img** attribute differs from **Image** in that it may be applied to objects as well as types, in which case it gives the **Image** for the subtype of the object. This is convenient for debugging:

```
Put_Line ("X = " & X'Img);
```

has the same meaning as the more verbose:

```
Put_Line ("X = " & T'Image (X));
```

where T is the (sub)type of the object X .

Integer_Value

For every integer type S , S' **Integer_Value** denotes a function with the following spec:

```
function  $S'$ Integer_Value (Arg : Universal_Fixed)
  return  $S$ ;
```

The value returned is the integer value V , such that

$$\text{Arg} = V * T'\text{Small}$$

where T is the type of **Arg**. The effect is thus similar to first doing an unchecked conversion from the fixed-point type to its corresponding implementation type, and then converting the result to the target integer type. The difference is that there are full range checks, to ensure

that the result is in range. This attribute is primarily intended for use in implementation of the standard input-output functions for fixed-point values.

Invalid_Value

For every scalar type *S*, *S*'Invalid_Value returns an undefined value of the type. If possible this value is an invalid representation for the type. The value returned is identical to the value used to initialize an otherwise uninitialized value of the type if pragma Initialize Scalars is used, including the ability to modify the value with the binder -Sxx flag and relevant environment variables at run time.

Large

The **Large** attribute is provided for compatibility with Ada 83. See the Ada 83 reference manual for an exact description of the semantics of this attribute.

Machine_Size

This attribute is identical to the **Object_Size** attribute. It is provided for compatibility with the DEC Ada 83 attribute of this name.

Mantissa

The **Mantissa** attribute is provided for compatibility with Ada 83. See the Ada 83 reference manual for an exact description of the semantics of this attribute.

Max_Interrupt_Priority

Standard'Max_Interrupt_Priority (*Standard* is the only permissible prefix), provides the same value as *System*.Max_Interrupt_Priority.

Max_Priority

Standard'Max_Priority (*Standard* is the only permissible prefix) provides the same value as *System*.Max_Priority.

Maximum_Alignment

Standard'Maximum_Alignment (*Standard* is the only permissible prefix) provides the maximum useful alignment value for the target. This is a static value that can be used to specify the alignment for an object, guaranteeing that it is properly aligned in all cases.

Mechanism_Code

function'Mechanism_Code yields an integer code for the mechanism used for the result of function, and *subprogram*'Mechanism_Code (*n*) yields the mechanism used for formal parameter number *n* (a static integer value with 1 meaning the first parameter) of *subprogram*. The code returned is:

- 1 by copy (value)
- 2 by reference

- 3 by descriptor (default descriptor class)
- 4 by descriptor (UBS: unaligned bit string)
- 5 by descriptor (UBSB: aligned bit string with arbitrary bounds)
- 6 by descriptor (UBA: unaligned bit array)
- 7 by descriptor (S: string, also scalar access type parameter)
- 8 by descriptor (SB: string with arbitrary bounds)
- 9 by descriptor (A: contiguous array)
- 10 by descriptor (NCA: non-contiguous array)

Values from 3 through 10 are only relevant to Digital OpenVMS implementations.

Null_Parameter

A reference `T'Null_Parameter` denotes an imaginary object of type or subtype `T` allocated at machine address zero. The attribute is allowed only as the default expression of a formal parameter, or as an actual expression of a subprogram call. In either case, the subprogram must be imported.

The identity of the object is represented by the address zero in the argument list, independent of the passing mechanism (explicit or default).

This capability is needed to specify that a zero address should be passed for a record or other composite object passed by reference. There is no way of indicating this without the `Null_Parameter` attribute.

Object_Size

The size of an object is not necessarily the same as the size of the type of an object. This is because by default object sizes are increased to be a multiple of the alignment of the object. For example, `Natural'Size` is 31, but by default objects of type `Natural` will have a size of 32 bits. Similarly, a record containing an integer and a character:

```
type Rec is record
  I : Integer;
  C : Character;
end record;
```

will have a size of 40 (that is `Rec'Size` will be 40. The alignment will be 4, because of the integer field, and so the default size of record objects for this type will be 64 (8 bytes).

Old

The attribute `Prefix'Old` can be used within a subprogram to refer to the value of the prefix on entry. So for example if you have an argument of a record type `X` called `Arg1`, you can refer to `Arg1.Field'Old` which yields the value of `Arg1.Field` on entry. The implementation simply involves generating an object declaration which captures the value on entry. Any prefix is allowed except one of a limited type (since limited types cannot be copied to capture their values) or a local variable (since it does not exist at subprogram entry time).

The following example shows the use of `'Old` to implement a test of a postcondition:

```

with Old_Pkg;
procedure Old is
begin
  Old_Pkg.Incr;
end Old;

package Old_Pkg is
  procedure Incr;
end Old_Pkg;

package body Old_Pkg is
  Count : Natural := 0;

  procedure Incr is
  begin
    ... code manipulating the value of Count

    pragma Assert (Count = Count'Old + 1);
  end Incr;
end Old_Pkg;

```

Note that it is allowed to apply `'Old` to a constant entity, but this will result in a warning, since the old and new values will always be the same.

Passed_By_Reference

`type 'Passed_By_Reference` for any subtype *type* returns a value of type `Boolean` value that is `True` if the type is normally passed by reference and `False` if the type is normally passed by copy in calls. For scalar types, the result is always `False` and is static. For non-scalar types, the result is non-static.

Pool_Address

`X'Pool_Address` for any object *X* returns the address of *X* within its storage pool. This is the same as `X'Address`, except that for an unconstrained array whose bounds are allocated just before the first component, `X'Pool_Address` returns the address of those bounds, whereas `X'Address` returns the address of the first component.

Here, we are interpreting “storage pool” broadly to mean “wherever the object is allocated”, which could be a user-defined storage pool, the global heap, on the stack, or in a static memory area. For an object created by `new`, `Ptr.all'Pool_Address` is what is passed to `Allocate` and returned from `Deallocate`.

Range_Length

`type 'Range_Length` for any discrete type *type* yields the number of values represented by the subtype (zero for a null range). The result is static for static subtypes. `Range_Length` applied to the index subtype of a one dimensional array always gives the same result as `Range` applied to the array itself.

Result

`function 'Result` can only be used with in a Postcondition pragma for a function. The prefix must be the name of the corresponding function. This is used to refer to the result

of the function in the postcondition expression. For a further discussion of the use of this attribute and examples of its use, see the description of pragma `Postcondition`.

Safe_Emax

The `Safe_Emax` attribute is provided for compatibility with Ada 83. See the Ada 83 reference manual for an exact description of the semantics of this attribute.

Safe_Large

The `Safe_Large` attribute is provided for compatibility with Ada 83. See the Ada 83 reference manual for an exact description of the semantics of this attribute.

Small

The `Small` attribute is defined in Ada 95 (and Ada 2005) only for fixed-point types. GNAT also allows this attribute to be applied to floating-point types for compatibility with Ada 83. See the Ada 83 reference manual for an exact description of the semantics of this attribute when applied to floating-point types.

Storage_Unit

`Standard'Storage_Unit` (`Standard` is the only permissible prefix) provides the same value as `System.Storage_Unit`.

Stub_Type

The GNAT implementation of remote access-to-classwide types is organized as described in AARM section E.4 (20.t): a value of an RACW type (designating a remote object) is represented as a normal access value, pointing to a "stub" object which in turn contains the necessary information to contact the designated remote object. A call on any dispatching operation of such a stub object does the remote call, if necessary, using the information in the stub object to locate the target partition, etc.

For a prefix `T` that denotes a remote access-to-classwide type, `T'Stub_Type` denotes the type of the corresponding stub objects.

By construction, the layout of `T'Stub_Type` is identical to that of type `RACW_Stub_Type` declared in the internal implementation-defined unit `System.Partition_Interface`. Use of this attribute will create an implicit dependency on this unit.

Target_Name

`Standard'Target_Name` (`Standard` is the only permissible prefix) provides a static string value that identifies the target for the current compilation. For GCC implementations, this is the standard gcc target name without the terminating slash (for example, GNAT 5.0 on windows yields "i586-pc-mingw32msv").

Tick

`Standard'Tick` (`Standard` is the only permissible prefix) provides the same value as `System.Tick`,

To_Address

The `System.To_Address` (`System` is the only permissible prefix) denotes a function identical to `System.Storage_Elements.To_Address` except that it is a static attribute. This means that if its argument is a static expression, then the result of the attribute is a static expression. The result is that such an expression can be used in contexts (e.g. preelaborable packages) which require a static expression and where the function call could not be used (since the function call is always non-static, even if its argument is static).

Type_Class

`type 'Type_Class` for any type or subtype `type` yields the value of the type class for the full type of `type`. If `type` is a generic formal type, the value is the value for the corresponding actual subtype. The value of this attribute is of type `System.Aux_DEC.Type_Class`, which has the following definition:

```
type Type_Class is
  (Type_Class_Enumeration,
   Type_Class_Integer,
   Type_Class_Fixed_Point,
   Type_Class_Floating_Point,
   Type_Class_Array,
   Type_Class_Record,
   Type_Class_Access,
   Type_Class_Task,
   Type_Class_Address);
```

Protected types yield the value `Type_Class_Task`, which thus applies to all concurrent types. This attribute is designed to be compatible with the DEC Ada 83 attribute of the same name.

UET_Address

The `UET_Address` attribute can only be used for a prefix which denotes a library package. It yields the address of the unit exception table when zero cost exception handling is used. This attribute is intended only for use within the GNAT implementation. See the unit `Ada.Exceptions` in files `'a-except.ads'` and `'a-except.adb'` for details on how this attribute is used in the implementation.

Unconstrained_Array

The `Unconstrained_Array` attribute can be used with a prefix that denotes any type or subtype. It is a static attribute that yields `True` if the prefix designates an unconstrained array, and `False` otherwise. In a generic instance, the result is still static, and yields the result of applying this test to the generic actual.

Universal_Literal_String

The prefix of `Universal_Literal_String` must be a named number. The static result is the string consisting of the characters of the number as defined in the original source. This allows the user program to access the actual text of named numbers without intermediate conversions and without the need to enclose the strings in quotes (which would preclude

their use as numbers). This is used internally for the construction of values of the floating-point attributes from the file ‘`ttypef.ads`’, but may also be used by user programs.

For example, the following program prints the first 50 digits of pi:

```
with Text_IO; use Text_IO;
with Ada.Numerics;
procedure Pi is
begin
  Put (Ada.Numerics.Pi'Universal_Literal_String);
end;
```

Unrestricted_Access

The `Unrestricted_Access` attribute is similar to `Access` except that all accessibility and aliased view checks are omitted. This is a user-beware attribute. It is similar to `Address`, for which it is a desirable replacement where the value desired is an access type. In other words, its effect is identical to first applying the `Address` attribute and then doing an unchecked conversion to a desired access type. In GNAT, but not necessarily in other implementations, the use of static chains for inner level subprograms means that `Unrestricted_Access` applied to a subprogram yields a value that can be called as long as the subprogram is in scope (normal Ada accessibility rules restrict this usage).

It is possible to use `Unrestricted_Access` for any type, but care must be exercised if it is used to create pointers to unconstrained objects. In this case, the resulting pointer has the same scope as the context of the attribute, and may not be returned to some enclosing scope. For instance, a function cannot use `Unrestricted_Access` to create a unconstrained pointer and then return that value to the caller.

VADS_Size

The `'VADS_Size` attribute is intended to make it easier to port legacy code which relies on the semantics of `'Size` as implemented by the VADS Ada 83 compiler. GNAT makes a best effort at duplicating the same semantic interpretation. In particular, `'VADS_Size` applied to a predefined or other primitive type with no `Size` clause yields the `Object_Size` (for example, `Natural'Size` is 32 rather than 31 on typical machines). In addition `'VADS_Size` applied to an object gives the result that would be obtained by applying the attribute to the corresponding type.

Value_Size

`type'Value_Size` is the number of bits required to represent a value of the given subtype. It is the same as `type'Size`, but, unlike `Size`, may be set for non-first subtypes.

Wchar_T_Size

`Standard'Wchar_T_Size` (`Standard` is the only permissible prefix) provides the size in bits of the C `wchar_t` type primarily for constructing the definition of this type in package `Interfaces.C`.

Word_Size

`Standard'Word_Size` (`Standard` is the only permissible prefix) provides the value `System.Word_Size`.

3 Implementation Advice

The main text of the Ada Reference Manual describes the required behavior of all Ada compilers, and the GNAT compiler conforms to these requirements.

In addition, there are sections throughout the Ada Reference Manual headed by the phrase “Implementation advice”. These sections are not normative, i.e., they do not specify requirements that all compilers must follow. Rather they provide advice on generally desirable behavior. You may wonder why they are not requirements. The most typical answer is that they describe behavior that seems generally desirable, but cannot be provided on all systems, or which may be undesirable on some systems.

As far as practical, GNAT follows the implementation advice sections in the Ada Reference Manual. This chapter contains a table giving the reference manual section number, paragraph number and several keywords for each advice. Each entry consists of the text of the advice followed by the GNAT interpretation of this advice. Most often, this simply says “followed”, which means that GNAT follows the advice. However, in a number of cases, GNAT deliberately deviates from this advice, in which case the text describes what GNAT does and why.

1.1.3(20): Error Detection

If an implementation detects the use of an unsupported Specialized Needs Annex feature at run time, it should raise `Program_Error` if feasible.

Not relevant. All specialized needs annex features are either supported, or diagnosed at compile time.

1.1.3(31): Child Units

If an implementation wishes to provide implementation-defined extensions to the functionality of a language-defined library unit, it should normally do so by adding children to the library unit.

Followed.

1.1.5(12): Bounded Errors

If an implementation detects a bounded error or erroneous execution, it should raise `Program_Error`.

Followed in all cases in which the implementation detects a bounded error or erroneous execution. Not all such situations are detected at runtime.

2.8(16): Pragmas

Normally, implementation-defined pragmas should have no semantic effect for error-free programs; that is, if the implementation-defined pragmas are removed from a working program, the program should still be legal, and should still have the same semantics.

The following implementation defined pragmas are exceptions to this rule:

<code>Abort_Defer</code>	Affects semantics
<code>Ada_83</code>	Affects legality
<code>Assert</code>	Affects semantics
<code>CPP_Class</code>	Affects semantics
<code>CPP_Constructor</code>	Affects semantics
<code>Debug</code>	Affects semantics
<code>Interface_Name</code>	Affects semantics
<code>Machine_Attribute</code>	Affects semantics
<code>Unimplemented_Unit</code>	Affects legality
<code>Unchecked_Union</code>	Affects semantics

In each of the above cases, it is essential to the purpose of the pragma that this advice not be followed. For details see the separate section on implementation defined pragmas.

2.8(17-19): Pragmas

Normally, an implementation should not define pragmas that can make an illegal program legal, except as follows:

A pragma used to complete a declaration, such as a pragma `Import`;

A pragma used to configure the environment by adding, removing, or replacing `library_items`.

See response to paragraph 16 of this same section.

3.5.2(5): Alternative Character Sets

If an implementation supports a mode with alternative interpretations for `Character` and `Wide_Character`, the set of graphic characters of `Character` should nevertheless remain a proper subset of the set of graphic characters of `Wide_Character`. Any character set “localizations” should be reflected in the results of the subprograms defined in the language-defined package `Characters.Handling` (see A.3) available in such a mode. In a mode with an alternative interpretation of `Character`, the implementation should also support a corresponding change in what is a legal `identifier_letter`.

Not all wide character modes follow this advice, in particular the JIS and IEC modes reflect standard usage in Japan, and in these encoding, the upper half of the Latin-1 set is not part of the wide-character subset, since the most significant bit is used for wide character encoding. However, this only applies to the external forms. Internally there is no such restriction.

3.5.4(28): Integer Types

An implementation should support `Long_Integer` in addition to `Integer` if the target machine supports 32-bit (or longer) arithmetic. No other named integer subtypes are recommended for package `Standard`. Instead, appropriate named integer subtypes should be provided in the library package `Interfaces` (see B.2).

`Long_Integer` is supported. Other standard integer types are supported so this advice is not fully followed. These types are supported for convenient interface to C, and so that all hardware types of the machine are easily available.

3.5.4(29): Integer Types

An implementation for a two’s complement machine should support modular types with a binary modulus up to `System.Max_Int*2+2`. An implementation should support a non-binary modules up to `Integer’Last`.

Followed.

3.5.5(8): Enumeration Values

For the evaluation of a call on *S'Pos* for an enumeration subtype, if the value of the operand does not correspond to the internal code for any enumeration literal of its type (perhaps due to an un-initialized variable), then the implementation should raise **Program_Error**. This is particularly important for enumeration types with noncontiguous internal codes specified by an `enumeration_representation_clause`.

Followed.

3.5.7(17): Float Types

An implementation should support `Long_Float` in addition to `Float` if the target machine supports 11 or more digits of precision. No other named floating point subtypes are recommended for package **Standard**. Instead, appropriate named floating point subtypes should be provided in the library package **Interfaces** (see B.2).

`Short_Float` and `Long_Long_Float` are also provided. The former provides improved compatibility with other implementations supporting this type. The latter corresponds to the highest precision floating-point type supported by the hardware. On most machines, this will be the same as `Long_Float`, but on some machines, it will correspond to the IEEE extended form. The notable case is all ia32 (x86) implementations, where `Long_Long_Float` corresponds to the 80-bit extended precision format supported in hardware on this processor. Note that the 128-bit format on SPARC is not supported, since this is a software rather than a hardware format.

3.6.2(11): Multidimensional Arrays

An implementation should normally represent multidimensional arrays in row-major order, consistent with the notation used for multidimensional array aggregates (see 4.3.3). However, if a pragma **Convention (Fortran, ...)** applies to a multidimensional array type, then column-major order should be used instead (see B.5, “Interfacing with Fortran”).

Followed.

9.6(30-31): Duration'Small

Whenever possible in an implementation, the value of `Duration'Small` should be no greater than 100 microseconds.

Followed. (`Duration'Small = 10**(-9)`).

The time base for `delay_relative_statements` should be monotonic; it need not be the same time base as used for `Calendar.Clock`.

Followed.

10.2.1(12): Consistent Representation

In an implementation, a type declared in a pre-elaborated package should have the same representation in every elaboration of a given version of the package, whether the elaborations occur in distinct executions of the same program, or in executions of distinct programs or partitions that include the given version.

Followed, except in the case of tagged types. Tagged types involve implicit pointers to a local copy of a dispatch table, and these pointers have representations which thus depend on a particular elaboration of the package. It is not easy to see how it would be possible to follow this advice without severely impacting efficiency of execution.

11.4.1(19): Exception Information

`Exception_Message` by default and `Exception_Information` should produce information useful for debugging. `Exception_Message` should be short, about one line. `Exception_Information` can be long. `Exception_Message` should not include the `Exception_Name`. `Exception_Information` should include both the `Exception_Name` and the `Exception_Message`.

Followed. For each exception that doesn't have a specified `Exception_Message`, the compiler generates one containing the location of the raise statement. This location has the form "file:line", where file is the short file name (without path information) and line is the line number in the file. Note that in the case of the Zero Cost Exception mechanism, these messages become redundant with the `Exception_Information` that contains a full backtrace of the calling sequence, so they are disabled. To disable explicitly the generation of the source location message, use the `Pragma Discard_Names`.

11.5(28): Suppression of Checks

The implementation should minimize the code executed for checks that have been suppressed.

Followed.

13.1 (21-24): Representation Clauses

The recommended level of support for all representation items is qualified as follows:

An implementation need not support representation items containing non-static expressions, except that an implementation should support a representation item for a given entity if each non-static expression in the representation item is a name that statically denotes a constant declared before the entity.

Followed. In fact, GNAT goes beyond the recommended level of support by allowing non-static expressions in some representation clauses even without the need to declare constants initialized with the values of such expressions. For example:

```
X : Integer;  
Y : Float;  
for Y'Address use X'Address;>>
```

An implementation need not support a specification for the **Size** for a given composite subtype, nor the size or storage place for an object (including a component) of a given composite subtype, unless the constraints on the subtype and its composite subcomponents (if any) are all static constraints.

Followed. Size Clauses are not permitted on non-static components, as described above.

An aliased component, or a component whose type is by-reference, should always be allocated at an addressable location.

Followed.

13.2(6-8): Packed Types

If a type is packed, then the implementation should try to minimize storage allocated to objects of the type, possibly at the expense of speed of accessing components, subject to reasonable complexity in addressing calculations.

The recommended level of support pragma **Pack** is:

For a packed record type, the components should be packed as tightly as possible subject to the Sizes of the component subtypes, and subject to any **record_representation_clause** that applies to the type; the implementation may, but need not, reorder components or cross aligned word boundaries to improve the packing. A component whose **Size** is greater than the word size may be allocated an integral number of words.

Followed. Tight packing of arrays is supported for all component sizes up to 64-bits. If the array component size is 1 (that is to say, if the component is a boolean type or an enumeration type with two values) then values of the type are implicitly initialized to zero. This happens both for objects of the packed type, and for objects that have a subcomponent of the packed type.

An implementation should support **Address** clauses for imported subprograms.

Followed.

13.3(14-19): Address Clauses

For an array X , X' **Address** should point at the first component of the array, and not at the array bounds.

Followed.

The recommended level of support for the **Address** attribute is:

X' **Address** should produce a useful result if X is an object that is aliased or of a by-reference type, or is an entity whose **Address** has been specified.

Followed. A valid address will be produced even if none of those conditions have been met. If necessary, the object is forced into memory to ensure the address is valid.

An implementation should support **Address** clauses for imported subprograms.

Followed.

Objects (including subcomponents) that are aliased or of a by-reference type should be allocated on storage element boundaries.

Followed.

If the **Address** of an object is specified, or it is imported or exported, then the implementation should not perform optimizations based on assumptions of no aliases.

Followed.

13.3(29-35): Alignment Clauses

The recommended level of support for the **Alignment** attribute for subtypes is:

An implementation should support specified **Alignments** that are factors and multiples of the number of storage elements per word, subject to the following:

Followed.

An implementation need not support specified **Alignments** for combinations of **Sizes** and **Alignments** that cannot be easily loaded and stored by available machine instructions.

Followed.

An implementation need not support specified **Alignments** that are greater than the maximum **Alignment** the implementation ever returns by default.

Followed.

The recommended level of support for the **Alignment** attribute for objects is:
Same as above, for subtypes, but in addition:

Followed.

For stand-alone library-level objects of statically constrained subtypes, the implementation should support all **Alignments** supported by the target linker. For example, page alignment is likely to be supported for such objects, but not for subtypes.

Followed.

13.3(42-43): Size Clauses

The recommended level of support for the **Size** attribute of objects is:

A **Size** clause should be supported for an object if the specified **Size** is at least as large as its subtype's **Size**, and corresponds to a size in storage elements that is a multiple of the object's **Alignment** (if the **Alignment** is nonzero).

Followed.

13.3(50-56): Size Clauses

If the **Size** of a subtype is specified, and allows for efficient independent addressability (see 9.10) on the target architecture, then the **Size** of the following objects of the subtype should equal the **Size** of the subtype:

Aliased objects (including components).

Followed.

Size clause on a composite subtype should not affect the internal layout of components.

Followed. But note that this can be overridden by use of the implementation pragma `Implicit_Packing` in the case of packed arrays.

The recommended level of support for the **Size** attribute of subtypes is:

The **Size** (if not specified) of a static discrete or fixed point subtype should be the number of bits needed to represent each value belonging to the subtype using an unbiased representation, leaving space for a sign bit only if the subtype contains negative values. If such a subtype is a first subtype, then an implementation should support a specified **Size** for it that reflects this representation.

Followed.

For a subtype implemented with levels of indirection, the **Size** should include the size of the pointers, but not the size of what they point at.

Followed.

13.3(71-73): Component Size Clauses

The recommended level of support for the `Component_Size` attribute is:

An implementation need not support specified `Component_Sizes` that are less than the `Size` of the component subtype.

Followed.

An implementation should support specified `Component_Sizes` that are factors and multiples of the word size. For such `Component_Sizes`, the array should contain no gaps between components. For other `Component_Sizes` (if supported), the array should contain no gaps between components when packing is also specified; the implementation should forbid this combination in cases where it cannot support a no-gaps representation.

Followed.

13.4(9-10): Enumeration Representation Clauses

The recommended level of support for enumeration representation clauses is:

An implementation need not support enumeration representation clauses for boolean types, but should at minimum support the internal codes in the range `System.Min_Int..System.Max_Int`.

Followed.

13.5.1(17-22): Record Representation Clauses

The recommended level of support for `record_representation_clauses` is:

An implementation should support storage places that can be extracted with a load, mask, shift sequence of machine code, and set with a load, shift, mask, store sequence, given the available machine instructions and run-time model.

Followed.

A storage place should be supported if its size is equal to the **Size** of the component subtype, and it starts and ends on a boundary that obeys the **Alignment** of the component subtype.

Followed.

If the default bit ordering applies to the declaration of a given type, then for a component whose subtype's **Size** is less than the word size, any storage place that does not cross an aligned word boundary should be supported.

Followed.

An implementation may reserve a storage place for the tag field of a tagged type, and disallow other components from overlapping that place.

Followed. The storage place for the tag field is the beginning of the tagged record, and its size is `Address'Size`. GNAT will reject an explicit component clause for the tag field.

An implementation need not support a `component_clause` for a component of an extension part if the storage place is not after the storage places of all components of the parent type, whether or not those storage places had been specified.

Followed. The above advice on record representation clauses is followed, and all mentioned features are implemented.

13.5.2(5): Storage Place Attributes

If a component is represented using some form of pointer (such as an offset) to the actual data of the component, and this data is contiguous with the rest of the object, then the storage place attributes should reflect the place of the actual data, not the pointer. If a component is allocated discontinuously from the rest of the object, then a warning should be generated upon reference to one of its storage place attributes.

Followed. There are no such components in GNAT.

13.5.3(7-8): Bit Ordering

The recommended level of support for the non-default bit ordering is:

If `Word_Size = Storage_Unit`, then the implementation should support the non-default bit ordering in addition to the default bit ordering.

Followed. Word size does not equal storage size in this implementation. Thus non-default bit ordering is not supported.

13.7(37): Address as Private

`Address` should be of a private type.

Followed.

13.7.1(16): Address Operations

Operations in `System` and its children should reflect the target environment semantics as closely as is reasonable. For example, on most machines, it makes sense for address arithmetic to “wrap around”. Operations that do not make sense should raise `Program_Error`.

Followed. Address arithmetic is modular arithmetic that wraps around. No operation raises `Program_Error`, since all operations make sense.

13.9(14-17): Unchecked Conversion

The `Size` of an array object should not include its bounds; hence, the bounds should not be part of the converted data.

Followed.

The implementation should not generate unnecessary run-time checks to ensure that the representation of `S` is a representation of the target type. It should take advantage of the permission to return by reference when possible. Restrictions on unchecked conversions should be avoided unless required by the target environment.

Followed. There are no restrictions on unchecked conversion. A warning is generated if the source and target types do not have the same size since the semantics in this case may be target dependent.

The recommended level of support for unchecked conversions is:

Unchecked conversions should be supported and should be reversible in the cases where this clause defines the result. To enable meaningful use of unchecked conversion, a contiguous representation should be used for elementary subtypes, for statically constrained array subtypes whose component subtype is one of the subtypes described in this paragraph, and for record subtypes without discriminants whose component subtypes are described in this paragraph.

Followed.

13.11(23-25): Implicit Heap Usage

An implementation should document any cases in which it dynamically allocates heap storage for a purpose other than the evaluation of an allocator.

Followed, the only other points at which heap storage is dynamically allocated are as follows:

- At initial elaboration time, to allocate dynamically sized global objects.
- To allocate space for a task when a task is created.
- To extend the secondary stack dynamically when needed. The secondary stack is used for returning variable length results.

A default (implementation-provided) storage pool for an access-to-constant type should not have overhead to support deallocation of individual objects.

Followed.

A storage pool for an anonymous access type should be created at the point of an allocator for the type, and be reclaimed when the designated object becomes inaccessible.

Followed.

13.11.2(17): Unchecked De-allocation

For a standard storage pool, **Free** should actually reclaim the storage.

Followed.

13.13.2(17): Stream Oriented Attributes

If a stream element is the same size as a storage element, then the normal in-memory representation should be used by `Read` and `Write` for scalar objects. Otherwise, `Read` and `Write` should use the smallest number of stream elements needed to represent all values in the base range of the scalar type.

Followed. By default, GNAT uses the interpretation suggested by AI-195, which specifies using the size of the first subtype. However, such an implementation is based on direct binary representations and is therefore target- and endianness-dependent. To address this issue, GNAT also supplies an alternate implementation of the stream attributes `Read` and `Write`, which uses the target-independent XDR standard representation for scalar types. The XDR implementation is provided as an alternative body of the `System.Stream_Attributes` package, in the file ‘`s-strxdr.adb`’ in the GNAT library. There is no ‘`s-strxdr.ads`’ file. In order to install the XDR implementation, do the following:

1. Replace the default implementation of the `System.Stream_Attributes` package with the XDR implementation. For example on a Unix platform issue the commands:

```
$ mv s-stratt.adb s-strold.adb
$ mv s-strxdr.adb s-stratt.adb
```

2. Rebuild the GNAT run-time library as documented in [Section “GNAT and Libraries” in *GNAT User’s Guide*](#).

A.1(52): Names of Predefined Numeric Types

If an implementation provides additional named predefined integer types, then the names should end with ‘`Integer`’ as in ‘`Long_Integer`’. If an implementation provides additional named predefined floating point types, then the names should end with ‘`Float`’ as in ‘`Long_Float`’.

Followed.

A.3.2(49): `Ada.Characters.Handling`

If an implementation provides a localized definition of `Character` or `Wide_Character`, then the effects of the subprograms in `Characters.Handling` should reflect the localizations. See also 3.5.2.

Followed. GNAT provides no such localized definitions.

A.4.4(106): Bounded-Length String Handling

Bounded string objects should not be implemented by implicit pointers and dynamic allocation.

Followed. No implicit pointers or dynamic allocation are used.

A.5.2(46-47): Random Number Generation

Any storage associated with an object of type **Generator** should be reclaimed on exit from the scope of the object.

Followed.

If the generator period is sufficiently long in relation to the number of distinct initiator values, then each possible value of **Initiator** passed to **Reset** should initiate a sequence of random numbers that does not, in a practical sense, overlap the sequence initiated by any other value. If this is not possible, then the mapping between initiator values and generator states should be a rapidly varying function of the initiator value.

Followed. The generator period is sufficiently long for the first condition here to hold true.

A.10.7(23): Get_Immediate

The **Get_Immediate** procedures should be implemented with unbuffered input. For a device such as a keyboard, input should be *available* if a key has already been typed, whereas for a disk file, input should always be available except at end of file. For a file associated with a keyboard-like device, any line-editing features of the underlying operating system should be disabled during the execution of **Get_Immediate**.

Followed on all targets except VxWorks. For VxWorks, there is no way to provide this functionality that does not result in the input buffer being flushed before the **Get_Immediate** call. A special unit **Interfaces.Vxworks.IO** is provided that contains routines to enable this functionality.

B.1(39-41): Pragma Export

If an implementation supports pragma **Export** to a given language, then it should also allow the main subprogram to be written in that language. It should support some mechanism for invoking the elaboration of the Ada library units included in the system, and for invoking the finalization of the environment task. On typical systems, the recommended mechanism is to provide two subprograms whose link names are **adainit** and **adafinal**. **adainit** should contain the elaboration code for library units. **adafinal** should contain the finalization code. These subprograms should have no effect the second and subsequent time they are called.

Followed.

Automatic elaboration of pre-elaborated packages should be provided when pragma **Export** is supported.

Followed when the main program is in Ada. If the main program is in a foreign language, then **adainit** must be called to elaborate pre-elaborated packages.

For each supported convention *L* other than **Intrinsic**, an implementation should support **Import** and **Export** pragmas for objects of *L*-compatible types and for subprograms, and pragma **Convention** for *L*-eligible types and for subprograms, presuming the other language has corresponding features. Pragma **Convention** need not be supported for scalar types.

Followed.

B.2(12-13): Package Interfaces

For each implementation-defined convention identifier, there should be a child package of package **Interfaces** with the corresponding name. This package should contain any declarations that would be useful for interfacing to the language (implementation) represented by the convention. Any declarations useful for interfacing to any language on the given hardware architecture should be provided directly in **Interfaces**.

Followed. An additional package not defined in the Ada Reference Manual is **Interfaces.CPP**, used for interfacing to C++.

An implementation supporting an interface to C, COBOL, or Fortran should provide the corresponding package or packages described in the following clauses.

Followed. GNAT provides all the packages described in this section.

B.3(63-71): Interfacing with C

An implementation should support the following interface correspondences between Ada and C.

Followed.

An Ada procedure corresponds to a void-returning C function.

Followed.

An Ada function corresponds to a non-void C function.

Followed.

An Ada `in` scalar parameter is passed as a scalar argument to a C function.

Followed.

An Ada `in` parameter of an access-to-object type with designated type T is passed as a $\mathbf{t*}$ argument to a C function, where t is the C type corresponding to the Ada type T .

Followed.

An Ada access T parameter, or an Ada `out` or `in out` parameter of an elementary type T , is passed as a $\mathbf{t*}$ argument to a C function, where t is the C type corresponding to the Ada type T . In the case of an elementary `out` or `in out` parameter, a pointer to a temporary copy is used to preserve by-copy semantics.

Followed.

An Ada parameter of a record type T , of any mode, is passed as a $\mathbf{t*}$ argument to a C function, where t is the C structure corresponding to the Ada type T .

Followed. This convention may be overridden by the use of the `C.Pass.By.Copy` pragma, or `Convention`, or by explicitly specifying the mechanism for a given call using an extended `import` or `export` pragma.

An Ada parameter of an array type with component type T , of any mode, is passed as a t^* argument to a C function, where t is the C type corresponding to the Ada type T .

Followed.

An Ada parameter of an access-to-subprogram type is passed as a pointer to a C function whose prototype corresponds to the designated subprogram's specification.

Followed.

B.4(95-98): Interfacing with COBOL

An Ada implementation should support the following interface correspondences between Ada and COBOL.

Followed.

An Ada access T parameter is passed as a 'BY REFERENCE' data item of the COBOL type corresponding to T .

Followed.

An Ada in scalar parameter is passed as a 'BY CONTENT' data item of the corresponding COBOL type.

Followed.

Any other Ada parameter is passed as a 'BY REFERENCE' data item of the COBOL type corresponding to the Ada parameter type; for scalars, a local copy is used if necessary to ensure by-copy semantics.

Followed.

B.5(22-26): Interfacing with Fortran

An Ada implementation should support the following interface correspondences between Ada and Fortran:

Followed.

An Ada procedure corresponds to a Fortran subroutine.

Followed.

An Ada function corresponds to a Fortran function.

Followed.

An Ada parameter of an elementary, array, or record type T is passed as a T argument to a Fortran procedure, where T is the Fortran type corresponding to the Ada type T , and where the `INTENT` attribute of the corresponding dummy argument matches the Ada formal parameter mode; the Fortran implementation's parameter passing conventions are used. For elementary types, a local copy is used if necessary to ensure by-copy semantics.

Followed.

An Ada parameter of an access-to-subprogram type is passed as a reference to a Fortran procedure whose interface corresponds to the designated subprogram's specification.

Followed.

C.1(3-5): Access to Machine Operations

The machine code or intrinsic support should allow access to all operations normally available to assembly language programmers for the target environment, including privileged instructions, if any.

Followed.

The interfacing pragmas (see Annex B) should support interface to assembler; the default assembler should be associated with the convention identifier **Assembler**.

Followed.

If an entity is exported to assembly language, then the implementation should allocate it at an addressable location, and should ensure that it is retained by the linking process, even if not otherwise referenced from the Ada code. The implementation should assume that any call to a machine code or assembler subprogram is allowed to read or update every object that is specified as exported.

Followed.

C.1(10-16): Access to Machine Operations

The implementation should ensure that little or no overhead is associated with calling intrinsic and machine-code subprograms.

Followed for both intrinsics and machine-code subprograms.

It is recommended that intrinsic subprograms be provided for convenient access to any machine operations that provide special capabilities or efficiency and that are not otherwise available through the language constructs.

Followed. A full set of machine operation intrinsic subprograms is provided.

Atomic read-modify-write operations—e.g., test and set, compare and swap, decrement and test, enqueue/dequeue.

Followed on any target supporting such operations.

Standard numeric functions—e.g., sin, log.

Followed on any target supporting such operations.

String manipulation operations—e.g., translate and test.

Followed on any target supporting such operations.

Vector operations—e.g., compare vector against thresholds.

Followed on any target supporting such operations.

Direct operations on I/O ports.

Followed on any target supporting such operations.

C.3(28): Interrupt Support

If the `Ceiling_Locking` policy is not in effect, the implementation should provide means for the application to specify which interrupts are to be blocked during protected actions, if the underlying system allows for a finer-grain control of interrupt blocking.

Followed. The underlying system does not allow for finer-grain control of interrupt blocking.

C.3.1(20-21): Protected Procedure Handlers

Whenever possible, the implementation should allow interrupt handlers to be called directly by the hardware.

Followed on any target where the underlying operating system permits such direct calls.

Whenever practical, violations of any implementation-defined restrictions should be detected before run time.

Followed. Compile time warnings are given when possible.

C.3.2(25): Package Interrupts

If implementation-defined forms of interrupt handler procedures are supported, such as protected procedures with parameters, then for each such form of a handler, a type analogous to `Parameterless_Handler` should be specified in a child package of `Interrupts`, with the same operations as in the predefined package `Interrupts`.

Followed.

C.4(14): Pre-elaboration Requirements

It is recommended that pre-elaborated packages be implemented in such a way that there should be little or no code executed at run time for the elaboration of entities not already covered by the Implementation Requirements.

Followed. Executable code is generated in some cases, e.g. loops to initialize large arrays.

C.5(8): Pragma Discard_Names

If the pragma applies to an entity, then the implementation should reduce the amount of storage used for storing names associated with that entity.

Followed.

C.7.2(30): The Package Task_Attributes

Some implementations are targeted to domains in which memory use at run time must be completely deterministic. For such implementations, it is recommended that the storage for task attributes will be pre-allocated statically and not from the heap. This can be accomplished by either placing restrictions on the number and the size of the task's attributes, or by using the pre-allocated storage for the first N attribute objects, and the heap for the others. In the latter case, N should be documented.

Not followed. This implementation is not targeted to such a domain.

D.3(17): Locking Policies

The implementation should use names that end with ‘_Locking’ for locking policies defined by the implementation.

Followed. A single implementation-defined locking policy is defined, whose name (`Inheritance_Locking`) follows this suggestion.

D.4(16): Entry Queuing Policies

Names that end with ‘_Queuing’ should be used for all implementation-defined queuing policies.

Followed. No such implementation-defined queuing policies exist.

D.6(9-10): Preemptive Abort

Even though the `abort_statement` is included in the list of potentially blocking operations (see 9.5.1), it is recommended that this statement be implemented in a way that never requires the task executing the `abort_statement` to block.

Followed.

On a multi-processor, the delay associated with aborting a task on another processor should be bounded; the implementation should use periodic polling, if necessary, to achieve this.

Followed.

D.7(21): Tasking Restrictions

When feasible, the implementation should take advantage of the specified restrictions to produce a more efficient implementation.

GNAT currently takes advantage of these restrictions by providing an optimized run time when the Ravenscar profile and the GNAT restricted run time set of restrictions are specified. See pragma `Profile (Ravenscar)` and pragma `Profile (Restricted)` for more details.

D.8(47-49): Monotonic Time

When appropriate, implementations should provide configuration mechanisms to change the value of `Tick`.

Such configuration mechanisms are not appropriate to this implementation and are thus not supported.

It is recommended that `Calendar.Clock` and `Real_Time.Clock` be implemented as transformations of the same time base.

Followed.

It is recommended that the *best* time base which exists in the underlying system be available to the application through `Clock`. *Best* may mean highest accuracy or largest range.

Followed.

E.5(28-29): Partition Communication Subsystem

Whenever possible, the PCS on the called partition should allow for multiple tasks to call the RPC-receiver with different messages and should allow them to block until the corresponding subprogram body returns.

Followed by GLADE, a separately supplied PCS that can be used with GNAT.

The `Write` operation on a stream of type `Params_Stream_Type` should raise `Storage_Error` if it runs out of space trying to write the `Item` into the stream.

Followed by GLADE, a separately supplied PCS that can be used with GNAT.

F(7): COBOL Support

If COBOL (respectively, C) is widely supported in the target environment, implementations supporting the Information Systems Annex should provide the child package `Interfaces.COBOL` (respectively, `Interfaces.C`) specified in Annex B and should support a `convention_identifier` of COBOL (respectively, C) in the interfacing pragmas (see Annex B), thus allowing Ada programs to interface with programs written in that language.

Followed.

F.1(2): Decimal Radix Support

Packed decimal should be used as the internal representation for objects of subtype *S* when `S'Machine_Radix = 10`.

Not followed. GNAT ignores `S'Machine_Radix` and always uses binary representations.

G: Numerics

If Fortran (respectively, C) is widely supported in the target environment, implementations supporting the Numerics Annex should provide the child package `Interfaces.Fortran` (respectively, `Interfaces.C`) specified in Annex B and should support a `convention_identifier` of Fortran (respectively, C) in the interfacing pragmas (see Annex B), thus allowing Ada programs to interface with programs written in that language.

Followed.

G.1.1(56-58): Complex Types

Because the usual mathematical meaning of multiplication of a complex operand and a real operand is that of the scaling of both components of the former by the latter, an implementation should not perform this operation by first promoting the real operand to complex type and then performing a full complex multiplication. In systems that, in the future, support an Ada binding to IEC 559:1989, the latter technique will not generate the required result when one of the components of the complex operand is infinite. (Explicit multiplication of the infinite component by the zero component obtained during promotion yields a NaN that propagates into the final result.) Analogous advice applies in the case of multiplication of a complex operand and a pure-imaginary operand, and in the case of division of a complex operand by a real or pure-imaginary operand.

Not followed.

Similarly, because the usual mathematical meaning of addition of a complex operand and a real operand is that the imaginary operand remains unchanged, an implementation should not perform this operation by first promoting the real operand to complex type and then performing a full complex addition. In implementations in which the **Signed_Zeros** attribute of the component type is **True** (and which therefore conform to IEC 559:1989 in regard to the handling of the sign of zero in predefined arithmetic operations), the latter technique will not generate the required result when the imaginary component of the complex operand is a negatively signed zero. (Explicit addition of the negative zero to the zero obtained during promotion yields a positive zero.) Analogous advice applies in the case of addition of a complex operand and a pure-imaginary operand, and in the case of subtraction of a complex operand and a real or pure-imaginary operand.

Not followed.

Implementations in which **Real'Signed_Zeros** is **True** should attempt to provide a rational treatment of the signs of zero results and result components. As one example, the result of the **Argument** function should have the sign of the imaginary component of the parameter **X** when the point represented by that parameter lies on the positive real axis; as another, the sign of the imaginary component of the **Compose_From_Polar** function should be the same as (respectively, the opposite of) that of the **Argument** parameter when that parameter has a value of zero and the **Modulus** parameter has a nonnegative (respectively, negative) value.

Followed.

G.1.2(49): Complex Elementary Functions

Implementations in which `Complex_Types.Real'Signed_Zeros` is `True` should attempt to provide a rational treatment of the signs of zero results and result components. For example, many of the complex elementary functions have components that are odd functions of one of the parameter components; in these cases, the result component should have the sign of the parameter component at the origin. Other complex elementary functions have zero components whose sign is opposite that of a parameter component at the origin, or is always positive or always negative.

Followed.

G.2.4(19): Accuracy Requirements

The versions of the forward trigonometric functions without a `Cycle` parameter should not be implemented by calling the corresponding version with a `Cycle` parameter of `2.0*Numerics.Pi`, since this will not provide the required accuracy in some portions of the domain. For the same reason, the version of `Log` without a `Base` parameter should not be implemented by calling the corresponding version with a `Base` parameter of `Numerics.e`.

Followed.

G.2.6(15): Complex Arithmetic Accuracy

The version of the `Compose_From_Polar` function without a `Cycle` parameter should not be implemented by calling the corresponding version with a `Cycle` parameter of `2.0*Numerics.Pi`, since this will not provide the required accuracy in some portions of the domain.

Followed.

4 Implementation Defined Characteristics

In addition to the implementation dependent pragmas and attributes, and the implementation advice, there are a number of other Ada features that are potentially implementation dependent. These are mentioned throughout the Ada Reference Manual, and are summarized in Annex M.

A requirement for conforming Ada compilers is that they provide documentation describing how the implementation deals with each of these issues. In this chapter, you will find each point in Annex M listed followed by a description in *italic font* of how GNAT handles the implementation dependence.

You can use this chapter as a guide to minimizing implementation dependent features in your programs if portability to other compilers and other operating systems is an important consideration. The numbers in each section below correspond to the paragraph number in the Ada Reference Manual.

2. Whether or not each recommendation given in Implementation Advice is followed. See 1.1.2(37).

See [Chapter 3 \[Implementation Advice\]](#), page 75.

3. Capacity limitations of the implementation. See 1.1.3(3).

The complexity of programs that can be processed is limited only by the total amount of available virtual memory, and disk space for the generated object files.

4. Variations from the standard that are impractical to avoid given the implementation's execution environment. See 1.1.3(6).

There are no variations from the standard.

5. Which `code_statements` cause external interactions. See 1.1.3(10).

Any `code_statement` can potentially cause external interactions.

6. The coded representation for the text of an Ada program. See 2.1(4).

See separate section on source representation.

7. The control functions allowed in comments. See 2.1(14).

See separate section on source representation.

8. The representation for an end of line. See 2.2(2).

See separate section on source representation.

9. Maximum supported line length and lexical element length. See 2.2(15).

The maximum line length is 255 characters and the maximum length of a lexical element is also 255 characters.

10. Implementation defined pragmas. See 2.8(14).

See [Chapter 1 \[Implementation Defined Pragmas\]](#), page 5.

11. Effect of pragma `Optimize`. See 2.8(27).

Pragma `Optimize`, if given with a `Time` or `Space` parameter, checks that the optimization flag is set, and aborts if it is not.

12. The sequence of characters of the value returned by `S'Image` when some of the graphic characters of `S'Wide_Image` are not defined in `Character`. See 3.5(37).

The sequence of characters is as defined by the wide character encoding method used for the source. See section on source representation for further details.

13. The predefined integer types declared in `Standard`. See 3.5.4(25).

`Short_Short_Integer`
8 bit signed

`Short_Integer`
(Short) 16 bit signed

`Integer` 32 bit signed

`Long_Integer`
64 bit signed (Alpha OpenVMS only) 32 bit signed (all other targets)

`Long_Long_Integer`
64 bit signed

14. Any nonstandard integer types and the operators defined for them. See 3.5.4(26).

There are no nonstandard integer types.

15. Any nonstandard real types and the operators defined for them. See 3.5.6(8).

There are no nonstandard real types.

16. What combinations of requested decimal precision and range are supported for floating point types. See 3.5.7(7).

The precision and range is as defined by the IEEE standard.

17. The predefined floating point types declared in **Standard**. See 3.5.7(16).

Short_Float

32 bit IEEE short

Float (Short) 32 bit IEEE short

Long_Float

64 bit IEEE long

Long_Long_Float

64 bit IEEE long (80 bit IEEE long on x86 processors)

18. The small of an ordinary fixed point type. See 3.5.9(8).

Fine_Delta is $2^{*(-63)}$

19. What combinations of small, range, and digits are supported for fixed point types. See 3.5.9(10).

Any combinations are permitted that do not result in a small less than **Fine_Delta** and do not result in a mantissa larger than 63 bits. If the mantissa is larger than 53 bits on machines where **Long_Long_Float** is 64 bits (true of all architectures except ia32), then the output from **Text_IO** is accurate to only 53 bits, rather than the full mantissa. This is because floating-point conversions are used to convert fixed point.

20. The result of **Tags.Expanded_Name** for types declared within an unnamed **block_statement**. See 3.9(10).

Block numbers of the form **Bnnn**, where *nnn* is a decimal integer are allocated.

21. Implementation-defined attributes. See 4.1.4(12).

See [Chapter 2 \[Implementation Defined Attributes\]](#), page 63.

22. Any implementation-defined time types. See 9.6(6).

There are no implementation-defined time types.

23. The time base associated with relative delays.

See 9.6(20). The time base used is that provided by the C library function `gettimeofday`.

24. The time base of the type `Calendar.Time`. See 9.6(23).

The time base used is that provided by the C library function `gettimeofday`.

25. The time zone used for package `Calendar` operations. See 9.6(24).

The time zone used by package `Calendar` is the current system time zone setting for local time, as accessed by the C library function `localtime`.

26. Any limit on `delay_until_statements` of `select_statements`. See 9.6(29).

There are no such limits.

27. Whether or not two non-overlapping parts of a composite object are independently addressable, in the case where packing, record layout, or `Component_Size` is specified for the object. See 9.10(1).

Separate components are independently addressable if they do not share overlapping storage units.

28. The representation for a compilation. See 10.1(2).

A compilation is represented by a sequence of files presented to the compiler in a single invocation of the `gcc` command.

29. Any restrictions on compilations that contain multiple compilation_units. See 10.1(4).

No single file can contain more than one compilation unit, but any sequence of files can be presented to the compiler as a single compilation.

30. The mechanisms for creating an environment and for adding and replacing compilation units. See 10.1.4(3).

See separate section on compilation model.

31. The manner of explicitly assigning library units to a partition. See 10.2(2).

If a unit contains an Ada main program, then the Ada units for the partition are determined by recursive application of the rules in the Ada Reference Manual section 10.2(2-6). In other words, the Ada units will be those that are needed by the main program, and then this definition of need is applied recursively to those units, and the partition contains the transitive closure determined by this relationship. In short, all the necessary units are included, with no need to explicitly specify the list. If additional units are required, e.g. by foreign language units, then all units must be mentioned in the context clause of one of the needed Ada units.

If the partition contains no main program, or if the main program is in a language other than Ada, then GNAT provides the binder options ‘`-z`’ and ‘`-n`’ respectively, and in this case a list of units can be explicitly supplied to the binder for inclusion in the partition (all units needed by these units will also be included automatically). For full details on the use of these options, refer to [Section “The GNAT Make Program gnatmake” in *GNAT User’s Guide*](#).

32. The implementation-defined means, if any, of specifying which compilation units are needed by a given compilation unit. See 10.2(2).

The units needed by a given compilation unit are as defined in the Ada Reference Manual section 10.2(2-6). There are no implementation-defined pragmas or other implementation-defined means for specifying needed units.

33. The manner of designating the main subprogram of a partition. See 10.2(7).

The main program is designated by providing the name of the corresponding ‘`ALI`’ file as the input parameter to the binder.

34. The order of elaboration of `library_items`. See 10.2(18).

The first constraint on ordering is that it meets the requirements of Chapter 10 of the Ada Reference Manual. This still leaves some implementation dependent choices, which are resolved by first elaborating bodies as early as possible (i.e., in preference to specs where there is a choice), and second by evaluating the immediate with clauses of a unit to determine the probably best choice, and third by elaborating in alphabetical order of unit names where a choice still remains.

35. Parameter passing and function return for the main subprogram. See 10.2(21).

The main program has no parameters. It may be a procedure, or a function returning an integer type. In the latter case, the returned integer value is the return code of the program (overriding any value that may have been set by a call to `Ada.Command_Line.Set_Exit_Status`).

36. The mechanisms for building and running partitions. See 10.2(24).

GNAT itself supports programs with only a single partition. The GNATDIST tool provided with the GLADE package (which also includes an implementation of the PCS) provides a completely flexible method for building and running programs consisting of multiple partitions. See the separate GLADE manual for details.

37. The details of program execution, including program termination. See 10.2(25).

See separate section on compilation model.

38. The semantics of any non-active partitions supported by the implementation. See 10.2(28).

Passive partitions are supported on targets where shared memory is provided by the operating system. See the GLADE reference manual for further details.

39. The information returned by `Exception_Message`. See 11.4.1(10).

Exception message returns the null string unless a specific message has been passed by the program.

40. The result of `Exceptions.Exception_Name` for types declared within an unnamed `block_statement`. See 11.4.1(12).

Blocks have implementation defined names of the form **Bnnn** where *nnn* is an integer.

41. The information returned by **Exception_Information**. See 11.4.1(13).

Exception_Information returns a string in the following format:

```
Exception_Name: nnnnn
Message: mmmmm
PID: ppp
Call stack traceback locations:
0xhhhh 0xhhhh 0xhhhh ... 0xhhh
```

where

- **nnnn** is the fully qualified name of the exception in all upper case letters. This line is always present.
- **mmmm** is the message (this line present only if message is non-null)
- **ppp** is the Process Id value as a decimal integer (this line is present only if the Process Id is nonzero). Currently we are not making use of this field.
- The Call stack traceback locations line and the following values are present only if at least one traceback location was recorded. The values are given in C style format, with lower case letters for a-f, and only as many digits present as are necessary.

The line terminator sequence at the end of each line, including the last line is a single LF character (16#0A#).

42. Implementation-defined check names. See 11.5(27).

The implementation defined check name **Alignment_Check** controls checking of address clause values for proper alignment (that is, the address supplied must be consistent with the alignment of the type).

In addition, a user program can add implementation-defined check names by means of the pragma **Check_Name**.

43. The interpretation of each aspect of representation. See 13.1(20).

See separate section on data representations.

44. Any restrictions placed upon representation items. See 13.1(20).

See separate section on data representations.

45. The meaning of **Size** for indefinite subtypes. See 13.3(48).

Size for an indefinite subtype is the maximum possible size, except that for the case of a subprogram parameter, the size of the parameter object is the actual size.

46. The default external representation for a type tag. See 13.3(75).

The default external representation for a type tag is the fully expanded name of the type in upper case letters.

47. What determines whether a compilation unit is the same in two different partitions. See 13.3(76).

A compilation unit is the same in two different partitions if and only if it derives from the same source file.

48. Implementation-defined components. See 13.5.1(15).

The only implementation defined component is the tag for a tagged type, which contains a pointer to the dispatching table.

49. If `Word_Size = Storage_Unit`, the default bit ordering. See 13.5.3(5).

`Word_Size` (32) is not the same as `Storage_Unit` (8) for this implementation, so no non-default bit ordering is supported. The default bit ordering corresponds to the natural endianness of the target architecture.

50. The contents of the visible part of package `System` and its language-defined children. See 13.7(2).

See the definition of these packages in files `'system.ads'` and `'s-stoele.ads'`.

51. The contents of the visible part of package `System.Machine_Code`, and the meaning of `code_statements`. See 13.8(7).

See the definition and documentation in file `'s-maccod.ads'`.

52. The effect of unchecked conversion. See 13.9(11).

Unchecked conversion between types of the same size results in an uninterpreted transmission of the bits from one type to the other. If the types are of unequal sizes, then in the

case of discrete types, a shorter source is first zero or sign extended as necessary, and a shorter target is simply truncated on the left. For all non-discrete types, the source is first copied if necessary to ensure that the alignment requirements of the target are met, then a pointer is constructed to the source value, and the result is obtained by dereferencing this pointer after converting it to be a pointer to the target type. Unchecked conversions where the target subtype is an unconstrained array are not permitted. If the target alignment is greater than the source alignment, then a copy of the result is made with appropriate alignment

53. The manner of choosing a storage pool for an access type when `Storage_Pool` is not specified for the type. See 13.11(17).

There are 3 different standard pools used by the compiler when `Storage_Pool` is not specified depending whether the type is local to a subprogram or defined at the library level and whether `Storage_Size` is specified or not. See documentation in the runtime library units `System.Pool_Global`, `System.Pool_Size` and `System.Pool_Local` in files ‘s-poosiz.ads’, ‘s-pooglo.ads’ and ‘s-pooloc.ads’ for full details on the default pools used.

54. Whether or not the implementation provides user-accessible names for the standard pool type(s). See 13.11(17).

See documentation in the sources of the run time mentioned in paragraph **53** . All these pools are accessible by means of `with`ing these units.

55. The meaning of `Storage_Size`. See 13.11(18).

`Storage_Size` is measured in storage units, and refers to the total space available for an access type collection, or to the primary stack space for a task.

56. Implementation-defined aspects of storage pools. See 13.11(22).

See documentation in the sources of the run time mentioned in paragraph **53** for details on GNAT-defined aspects of storage pools.

57. The set of restrictions allowed in a pragma `Restrictions`. See 13.12(7).

All RM defined Restriction identifiers are implemented. The following additional restriction identifiers are provided. There are two separate lists of implementation dependent restriction identifiers. The first set requires consistency throughout a partition (in other

words, if the restriction identifier is used for any compilation unit in the partition, then all compilation units in the partition must obey the restriction.

Simple_Barriers

This restriction ensures at compile time that barriers in entry declarations for protected types are restricted to either static boolean expressions or references to simple boolean variables defined in the private part of the protected type. No other form of entry barriers is permitted. This is one of the restrictions of the Ravenscar profile for limited tasking (see also pragma **Profile (Ravenscar)**).

Max_Entry_Queue_Length => Expr

This restriction is a declaration that any protected entry compiled in the scope of the restriction has at most the specified number of tasks waiting on the entry at any one time, and so no queue is required. This restriction is not checked at compile time. A program execution is erroneous if an attempt is made to queue more than the specified number of tasks on such an entry.

No_Calendar

This restriction ensures at compile time that there is no implicit or explicit dependence on the package **Ada.Calendar**.

No_Default_Initialization

This restriction prohibits any instance of default initialization of variables. The binder implements a consistency rule which prevents any unit compiled without the restriction from with'ing a unit with the restriction (this allows the generation of initialization procedures to be skipped, since you can be sure that no call is ever generated to an initialization procedure in a unit with the restriction active). If used in conjunction with **Initialize_Scalars** or **Normalize_Scalars**, the effect is to prohibit all cases of variables declared without a specific initializer (including the case of OUT scalar parameters).

No_Direct_Boolean_Operators

This restriction ensures that no logical (and/or/xor) are used on operands of type **Boolean** (or any type derived from **Boolean**). This is intended for use in safety critical programs where the certification protocol requires the use of short-circuit (and then, or else) forms for all composite boolean operations.

No_Dispatching_Calls

This restriction ensures at compile time that the code generated by the compiler involves no dispatching calls. The use of this restriction allows the safe use of record extensions, classwide membership tests and other classwide features not involving implicit dispatching. This restriction ensures that the code contains no indirect calls through a dispatching mechanism. Note that this includes internally-generated calls created by the compiler, for example in the implementation of class-wide objects assignments. The membership test is allowed in the presence of this restriction, because its implementation requires no dispatching. This restriction is comparable to the official Ada restriction **No_Dispatch** except that it is a bit less restrictive in that it allows all class-wide constructs that do not imply dispatching. The following example indicates constructs that violate this restriction.


```

package Pkg is
  type T is tagged record
    Data : Natural;
  end record;
  procedure P (X : T);

  type DT is new T with record
    More_Data : Natural;
  end record;
  procedure Q (X : DT);
end Pkg;

with Pkg; use Pkg;
procedure Example is
  procedure Test (O : T'Class) is
    N : Natural := O'Size;-- Error: Dispatching call
    C : T'Class := O;      -- Error: implicit Dispatching Call
  begin
    if O in DT'Class then -- OK : Membership test
      Q (DT (O));         -- OK : Type conversion plus direct call
    else
      P (O);              -- Error: Dispatching call
    end if;
  end Test;

  Obj : DT;
begin
  P (Obj);                -- OK : Direct call
  P (T (Obj));            -- OK : Type conversion plus direct call
  P (T'Class (Obj));      -- Error: Dispatching call

  Test (Obj);             -- OK : Type conversion

  if Obj in T'Class then -- OK : Membership test
    null;
  end if;
end Example;

```

No_Dynamic_Attachment

This restriction ensures that there is no call to any of the operations defined in package Ada.Interrupts.

No_Enumeration_Maps

This restriction ensures at compile time that no operations requiring enumeration maps are used (that is Image and Value attributes applied to enumeration types).

No_Entry_Calls_In_Elaboration_Code

This restriction ensures at compile time that no task or protected entry calls are made during elaboration code. As a result of the use of this restriction, the compiler can assume that no code past an accept statement in a task can be executed at elaboration time.

No_Exception_Handlers

This restriction ensures at compile time that there are no explicit exception handlers. It also indicates that no exception propagation will be provided. In this mode, exceptions may be raised but will result in an immediate call to

the last chance handler, a routine that the user must define with the following profile:

```
procedure Last_Chance_Handler
  (Source_Location : System.Address; Line : Integer);
pragma Export (C, Last_Chance_Handler,
  "__gnat_last_chance_handler");
```

The parameter is a C null-terminated string representing a message to be associated with the exception (typically the source location of the raise statement generated by the compiler). The Line parameter when nonzero represents the line number in the source program where the raise occurs.

No_Exception_Propagation

This restriction guarantees that exceptions are never propagated to an outer subprogram scope). The only case in which an exception may be raised is when the handler is statically in the same subprogram, so that the effect of a raise is essentially like a goto statement. Any other raise statement (implicit or explicit) will be considered unhandled. Exception handlers are allowed, but may not contain an exception occurrence identifier (exception choice). In addition use of the package GNAT.Current_Exception is not permitted, and reraise statements (raise with no operand) are not permitted.

No_Exception_Registration

This restriction ensures at compile time that no stream operations for types Exception_Id or Exception_Occurrence are used. This also makes it impossible to pass exceptions to or from a partition with this restriction in a distributed environment. If this exception is active, then the generated code is simplified by omitting the otherwise-required global registration of exceptions when they are declared.

No_Implicit_Conditionals

This restriction ensures that the generated code does not contain any implicit conditionals, either by modifying the generated code where possible, or by rejecting any construct that would otherwise generate an implicit conditional. Note that this check does not include run time constraint checks, which on some targets may generate implicit conditionals as well. To control the latter, constraint checks can be suppressed in the normal manner. Constructs generating implicit conditionals include comparisons of composite objects and the Max/Min attributes.

No_Implicit_Dynamic_Code

This restriction prevents the compiler from building “trampolines”. This is a structure that is built on the stack and contains dynamic code to be executed at run time. On some targets, a trampoline is built for the following features: **Access**, **Unrestricted_Access**, or **Address** of a nested subprogram; nested task bodies; primitive operations of nested tagged types. Trampolines do not work on machines that prevent execution of stack data. For example, on windows systems, enabling DEP (data execution protection) will cause trampolines to raise an exception. Trampolines are also quite slow at run time.

On many targets, trampolines have been largely eliminated. Look at the version of system.ads for your target — if it has Always-Compatible_Rep equal

to `False`, then trampolines are largely eliminated. In particular, a trampoline is built for the following features: `Address` of a nested subprogram; `Access` or `Unrestricted_Access` of a nested subprogram, but only if pragma `Favor_Top_Level` applies, or the access type has a foreign-language convention; primitive operations of nested tagged types.

`No_Implicit_Loops`

This restriction ensures that the generated code does not contain any implicit `for` loops, either by modifying the generated code where possible, or by rejecting any construct that would otherwise generate an implicit `for` loop. If this restriction is active, it is possible to build large array aggregates with all static components without generating an intermediate temporary, and without generating a loop to initialize individual components. Otherwise, a loop is created for arrays larger than about 5000 scalar components.

`No_InitializeScalars`

This restriction ensures that no unit in the partition is compiled with pragma `InitializeScalars`. This allows the generation of more efficient code, and in particular eliminates dummy null initialization routines that are otherwise generated for some record and array types.

`No_Local_Protected_Objects`

This restriction ensures at compile time that protected objects are only declared at the library level.

`No_Protected_Type_Allocators`

This restriction ensures at compile time that there are no allocator expressions that attempt to allocate protected objects.

`No_Secondary_Stack`

This restriction ensures at compile time that the generated code does not contain any reference to the secondary stack. The secondary stack is used to implement functions returning unconstrained objects (arrays or records) on some targets.

`No_Select_Statements`

This restriction ensures at compile time no select statements of any kind are permitted, that is the keyword `select` may not appear. This is one of the restrictions of the Ravenscar profile for limited tasking (see also pragma `Profile (Ravenscar)`).

`No_Standard_Storage_Pools`

This restriction ensures at compile time that no access types use the standard default storage pool. Any access type declared must have an explicit `Storage_Pool` attribute defined specifying a user-defined storage pool.

`No_Streams`

This restriction ensures at compile/bind time that there are no stream objects created and no use of stream attributes. This restriction does not forbid dependences on the package `Ada.Streams`. So it is permissible to with `Ada.Streams` (or another package that does so itself) as long as no actual stream objects are created and no stream attributes are used.

Note that the use of restriction allows optimization of tagged types, since they do not need to worry about dispatching stream operations. To take maximum advantage of this space-saving optimization, any unit declaring a tagged type should be compiled with the restriction, though this is not required.

No_Task_Attributes_Package

This restriction ensures at compile time that there are no implicit or explicit dependencies on the package `Ada.Task_Attributes`.

No_Task_Termination

This restriction ensures at compile time that no terminate alternatives appear in any task body.

No_Tasking

This restriction prevents the declaration of tasks or task types throughout the partition. It is similar in effect to the use of `Max_Tasks => 0` except that violations are caught at compile time and cause an error message to be output either by the compiler or binder.

Static_Priorities

This restriction ensures at compile time that all priority expressions are static, and that there are no dependencies on the package `Ada.Dynamic_Priorities`.

Static_Storage_Size

This restriction ensures at compile time that any expression appearing in a `Storage_Size` pragma or attribute definition clause is static.

The second set of implementation dependent restriction identifiers does not require partition-wide consistency. The restriction may be enforced for a single compilation unit without any effect on any of the other compilation units in the partition.

No_Elaboration_Code

This restriction ensures at compile time that no elaboration code is generated. Note that this is not the same condition as is enforced by pragma `Preelaborate`. There are cases in which pragma `Preelaborate` still permits code to be generated (e.g. code to initialize a large array to all zeroes), and there are cases of units which do not meet the requirements for pragma `Preelaborate`, but for which no elaboration code is generated. Generally, it is the case that preelaborable units will meet the restrictions, with the exception of large aggregates initialized with an `others`-clause, and exception declarations (which generate calls to a run-time registry procedure). This restriction is enforced on a unit by unit basis, it need not be obeyed consistently throughout a partition.

In the case of aggregates with `others`, if the aggregate has a dynamic size, there is no way to eliminate the elaboration code (such dynamic bounds would be incompatible with `Preelaborate` in any case). If the bounds are static, then use of this restriction actually modifies the code choice of the compiler to avoid generating a loop, and instead generate the aggregate statically if possible, no matter how many times the data for the `others` clause must be repeatedly generated.

It is not possible to precisely document the constructs which are compatible with this restriction, since, unlike most other restrictions, this is not a restriction on

the source code, but a restriction on the generated object code. For example, if the source contains a declaration:

```
Val : constant Integer := X;
```

where X is not a static constant, it may be possible, depending on complex optimization circuitry, for the compiler to figure out the value of X at compile time, in which case this initialization can be done by the loader, and requires no initialization code. It is not possible to document the precise conditions under which the optimizer can figure this out.

Note that this the implementation of this restriction requires full code generation. If it is used in conjunction with "semantics only" checking, then some cases of violations may be missed.

No_Entry_Queue

This restriction is a declaration that any protected entry compiled in the scope of the restriction has at most one task waiting on the entry at any one time, and so no queue is required. This restriction is not checked at compile time. A program execution is erroneous if an attempt is made to queue a second task on such an entry.

No_Implementation_Attributes

This restriction checks at compile time that no GNAT-defined attributes are present. With this restriction, the only attributes that can be used are those defined in the Ada Reference Manual.

No_Implementation_Pragmas

This restriction checks at compile time that no GNAT-defined pragmas are present. With this restriction, the only pragmas that can be used are those defined in the Ada Reference Manual.

No_Implementation_Restrictions

This restriction checks at compile time that no GNAT-defined restriction identifiers (other than `No_Implementation_Restrictions` itself) are present. With this restriction, the only other restriction identifiers that can be used are those defined in the Ada Reference Manual.

No_Wide_Characters

This restriction ensures at compile time that no uses of the types `Wide_Character` or `Wide_String` or corresponding wide wide types appear, and that no wide or wide wide string or character literals appear in the program (that is literals representing characters not in type `Character`).

58. The consequences of violating limitations on `Restrictions` pragmas. See 13.12(9).

Restrictions that can be checked at compile time result in illegalities if violated. Currently there are no other consequences of violating restrictions.

59. The representation used by the `Read` and `Write` attributes of elementary types in terms of stream elements. See 13.13.2(9).

The representation is the in-memory representation of the base type of the type, using the number of bits corresponding to the `type'Size` value, and the natural ordering of the machine.

60. The names and characteristics of the numeric subtypes declared in the visible part of package `Standard`. See A.1(3).

See items describing the integer and floating-point types supported.

61. The accuracy actually achieved by the elementary functions. See A.5.1(1).

The elementary functions correspond to the functions available in the C library. Only fast math mode is implemented.

62. The sign of a zero result from some of the operators or functions in `Numerics.Generic_Elementary_Functions`, when `Float_Type'Signed_Zeros` is `True`. See A.5.1(46).

The sign of zeroes follows the requirements of the IEEE 754 standard on floating-point.

63. The value of `Numerics.Float_Random.Max_Image_Width`. See A.5.2(27).

Maximum image width is 649, see library file '`a-numran.ads`'.

64. The value of `Numerics.Discrete_Random.Max_Image_Width`. See A.5.2(27).

Maximum image width is 80, see library file '`a-nudira.ads`'.

65. The algorithms for random number generation. See A.5.2(32).

The algorithm is documented in the source files '`a-numran.ads`' and '`a-numran.adb`'.

66. The string representation of a random number generator's state. See A.5.2(38).

See the documentation contained in the file '`a-numran.adb`'.

67. The minimum time interval between calls to the time-dependent Reset procedure that are guaranteed to initiate different random number sequences. See A.5.2(45).

The minimum period between reset calls to guarantee distinct series of random numbers is one microsecond.

68. The values of the `Model_Mantissa`, `Model_Emin`, `Model_Epsilon`, `Model`, `Safe_First`, and `Safe_Last` attributes, if the Numerics Annex is not supported. See A.5.3(72).

See the source file `'ttypef.ads'` for the values of all numeric attributes.

69. Any implementation-defined characteristics of the input-output packages. See A.7(14).

There are no special implementation defined characteristics for these packages.

70. The value of `Buffer_Size` in `Storage_IO`. See A.9(10).

All type representations are contiguous, and the `Buffer_Size` is the value of `type'Size` rounded up to the next storage unit boundary.

71. External files for standard input, standard output, and standard error See A.10(5).

These files are mapped onto the files provided by the C streams libraries. See source file `'i-cstrea.ads'` for further details.

72. The accuracy of the value produced by `Put`. See A.10.9(36).

If more digits are requested in the output than are represented by the precision of the value, zeroes are output in the corresponding least significant digit positions.

73. The meaning of `Argument_Count`, `Argument`, and `Command_Name`. See A.15(1).

These are mapped onto the `argv` and `argc` parameters of the main program in the natural manner.

74. Implementation-defined convention names. See B.1(11).

The following convention names are supported

Ada Ada

Assembler	Assembly language
Asm	Synonym for Assembler
Assembly	Synonym for Assembler
C	C
C_Pass_By_Copy	Allowed only for record types, like C, but also notes that record is to be passed by copy rather than reference.
COBOL	COBOL
C_Plus_Plus (or CPP)	C++
Default	Treated the same as C
External	Treated the same as C
Fortran	Fortran
Intrinsic	For support of pragma Import with convention Intrinsic , see separate section on Intrinsic Subprograms .
Stdcall	Stdcall (used for Windows implementations only). This convention correspond to the WINAPI (previously called Pascal convention) C/C++ convention under Windows. A function with this convention cleans the stack before exit.
DLL	Synonym for Stdcall
Win32	Synonym for Stdcall
Stubbed	Stubbed is a special convention used to indicate that the body of the subprogram will be entirely ignored. Any call to the subprogram is converted into a raise of the Program_Error exception. If a pragma Import specifies convention stubbed then no body need be present at all. This convention is useful during development for the inclusion of subprograms whose body has not yet been written.

In addition, all otherwise unrecognized convention names are also treated as being synonymous with convention C. In all implementations except for VMS, use of such other names results in a warning. In VMS implementations, these names are accepted silently.

75. The meaning of link names. See B.1(36).

Link names are the actual names used by the linker.

76. The manner of choosing link names when neither the link name nor the address of an imported or exported entity is specified. See B.1(36).

The default linker name is that which would be assigned by the relevant external language, interpreting the Ada name as being in all lower case letters.

77. The effect of pragma `Linker_Options`. See B.1(37).

The string passed to `Linker_Options` is presented uninterpreted as an argument to the link command, unless it contains ASCII.NUL characters. NUL characters if they appear act as argument separators, so for example

```
pragma Linker_Options ("-labc" & ASCII.NUL & "-ldef");
```

causes two separate arguments `-labc` and `-ldef` to be passed to the linker. The order of linker options is preserved for a given unit. The final list of options passed to the linker is in reverse order of the elaboration order. For example, linker options for a body always appear before the options from the corresponding package spec.

78. The contents of the visible part of package `Interfaces` and its language-defined descendants. See B.2(1).

See files with prefix ‘i-’ in the distributed library.

79. Implementation-defined children of package `Interfaces`. The contents of the visible part of package `Interfaces`. See B.2(11).

See files with prefix ‘i-’ in the distributed library.

80. The types `Floating`, `Long_Floating`, `Binary`, `Long_Binary`, `Decimal_Element`, and `COBOL_Character`; and the initialization of the variables `Ada_To_COBOL` and `COBOL_To_Ada`, in `Interfaces.COBOLE`. See B.4(50).

`Floating` `Float`

`Long_Floating`
 (`Floating`) `Long_Float`

`Binary` `Integer`

`Long_Binary`
 `Long_Long_Integer`

`Decimal_Element`
 `Character`

`COBOL_Character`
 `Character`

For initialization, see the file ‘i-cobol.ads’ in the distributed library.

81. Support for access to machine instructions. See C.1(1).

See documentation in file ‘`s-maccod.ads`’ in the distributed library.

82. Implementation-defined aspects of access to machine operations. See C.1(9).

See documentation in file ‘`s-maccod.ads`’ in the distributed library.

83. Implementation-defined aspects of interrupts. See C.3(2).

Interrupts are mapped to signals or conditions as appropriate. See definition of unit `Ada.Interrupt_Names` in source file ‘`a-intnam.ads`’ for details on the interrupts supported on a particular target.

84. Implementation-defined aspects of pre-elaboration. See C.4(13).

GNAT does not permit a partition to be restarted without reloading, except under control of the debugger.

85. The semantics of pragma `Discard_Names`. See C.5(7).

Pragma `Discard_Names` causes names of enumeration literals to be suppressed. In the presence of this pragma, the `Image` attribute provides the image of the `Pos` of the literal, and `Value` accepts `Pos` values.

86. The result of the `Task_Identification.Image` attribute. See C.7.1(7).

The result of this attribute is a string that identifies the object or component that denotes a given task. If a variable `Var` has a task type, the image for this task will have the form `Var_XXXXXXX`, where the suffix is the hexadecimal representation of the virtual address of the corresponding task control block. If the variable is an array of tasks, the image of each task will have the form of an indexed component indicating the position of a given task in the array, e.g. `Group(5)_XXXXXXX`. If the task is a component of a record, the image of the task will have the form of a selected component. These rules are fully recursive, so that the image of a task that is a subcomponent of a composite object corresponds to the expression that designates this task. If a task is created by an allocator, its image depends on the context. If the allocator is part of an object declaration, the rules described above are used to construct its image, and this image is not affected by subsequent assignments. If the allocator appears within an expression, the image includes only the name of the task type. If the configuration pragma `Discard_Names` is present, or if the restriction `No_Implicit_Heap_Allocation` is in

effect, the image reduces to the numeric suffix, that is to say the hexadecimal representation of the virtual address of the control block of the task.

87. The value of `Current_Task` when in a protected entry or interrupt handler. See C.7.1(17).

Protected entries or interrupt handlers can be executed by any convenient thread, so the value of `Current_Task` is undefined.

88. The effect of calling `Current_Task` from an entry body or interrupt handler. See C.7.1(19).

The effect of calling `Current_Task` from an entry body or interrupt handler is to return the identification of the task currently executing the code.

89. Implementation-defined aspects of `Task_Attributes`. See C.7.2(19).

There are no implementation-defined aspects of `Task_Attributes`.

90. Values of all `Metrics`. See D(2).

The metrics information for GNAT depends on the performance of the underlying operating system. The sources of the run-time for tasking implementation, together with the output from ‘-gnatG’ can be used to determine the exact sequence of operating systems calls made to implement various tasking constructs. Together with appropriate information on the performance of the underlying operating system, on the exact target in use, this information can be used to determine the required metrics.

91. The declarations of `Any_Priority` and `Priority`. See D.1(11).

See declarations in file ‘`system.ads`’.

92. Implementation-defined execution resources. See D.1(15).

There are no implementation-defined execution resources.

93. Whether, on a multiprocessor, a task that is waiting for access to a protected object keeps its processor busy. See D.2.1(3).

On a multi-processor, a task that is waiting for access to a protected object does not keep its processor busy.

94. The affect of implementation defined execution resources on task dispatching. See D.2.1(9).

Tasks map to threads in the threads package used by GNAT. Where possible and appropriate, these threads correspond to native threads of the underlying operating system.

95. Implementation-defined `policy_identifiers` allowed in a pragma `Task_Dispatching_Policy`. See D.2.2(3).

There are no implementation-defined policy-identifiers allowed in this pragma.

96. Implementation-defined aspects of priority inversion. See D.2.2(16).

Execution of a task cannot be preempted by the implementation processing of delay expirations for lower priority tasks.

97. Implementation defined task dispatching. See D.2.2(18).

The policy is the same as that of the underlying threads implementation.

98. Implementation-defined `policy_identifiers` allowed in a pragma `Locking_Policy`. See D.3(4).

The only implementation defined policy permitted in GNAT is `Inheritance_Locking`. On targets that support this policy, locking is implemented by inheritance, i.e. the task owning the lock operates at a priority equal to the highest priority of any task currently requesting the lock.

99. Default ceiling priorities. See D.3(10).

The ceiling priority of protected objects of the type `System.Interrupt_Priority'Last` as described in the Ada Reference Manual D.3(10),

100. The ceiling of any protected object used internally by the implementation. See D.3(16).

The ceiling priority of internal protected objects is `System.Priority'Last`.

101. Implementation-defined queuing policies. See D.4(1).

There are no implementation-defined queuing policies.

102. On a multiprocessor, any conditions that cause the completion of an aborted construct to be delayed later than what is specified for a single processor. See D.6(3).

The semantics for abort on a multi-processor is the same as on a single processor, there are no further delays.

103. Any operations that implicitly require heap storage allocation. See D.7(8).

The only operation that implicitly requires heap storage allocation is task creation.

104. Implementation-defined aspects of pragma **Restrictions**. See D.7(20).

There are no such implementation-defined aspects.

105. Implementation-defined aspects of package **Real_Time**. See D.8(17).

There are no implementation defined aspects of package **Real_Time**.

106. Implementation-defined aspects of **delay_statements**. See D.9(8).

Any difference greater than one microsecond will cause the task to be delayed (see D.9(7)).

107. The upper bound on the duration of interrupt blocking caused by the implementation. See D.12(5).

The upper bound is determined by the underlying operating system. In no cases is it more than 10 milliseconds.

108. The means for creating and executing distributed programs. See E(5).

The GLADE package provides a utility GNATDIST for creating and executing distributed programs. See the GLADE reference manual for further details.

109. Any events that can result in a partition becoming inaccessible. See E.1(7).

See the GLADE reference manual for full details on such events.

110. The scheduling policies, treatment of priorities, and management of shared resources between partitions in certain cases. See E.1(11).

See the GLADE reference manual for full details on these aspects of multi-partition execution.

111. Events that cause the version of a compilation unit to change. See E.3(5).

Editing the source file of a compilation unit, or the source files of any units on which it is dependent in a significant way cause the version to change. No other actions cause the version number to change. All changes are significant except those which affect only layout, capitalization or comments.

112. Whether the execution of the remote subprogram is immediately aborted as a result of cancellation. See E.4(13).

See the GLADE reference manual for details on the effect of abort in a distributed application.

113. Implementation-defined aspects of the PCS. See E.5(25).

See the GLADE reference manual for a full description of all implementation defined aspects of the PCS.

114. Implementation-defined interfaces in the PCS. See E.5(26).

See the GLADE reference manual for a full description of all implementation defined interfaces.

115. The values of named numbers in the package `Decimal`. See F.2(7).

```
Max_Scale
    +18
Min_Scale
    -18
Min_Delta
    1.0E-18
```

```

Max_Delta
    1.0E+18

Max_Decimal_Digits
    18

```

116. The value of `Max_Picture_Length` in the package `Text_IO Editing`. See F.3.3(16).

64

117. The value of `Max_Picture_Length` in the package `Wide_Text_IO Editing`. See F.3.4(5).

64

118. The accuracy actually achieved by the complex elementary functions and by other complex arithmetic operations. See G.1(1).

Standard library functions are used for the complex arithmetic operations. Only fast math mode is currently supported.

119. The sign of a zero result (or a component thereof) from any operator or function in `Numerics.Generic_Complex_Types`, when `Real'Signed_Zeros` is `True`. See G.1.1(53).

The signs of zero values are as recommended by the relevant implementation advice.

120. The sign of a zero result (or a component thereof) from any operator or function in `Numerics.Generic_Complex_Elementary_Functions`, when `Real'Signed_Zeros` is `True`. See G.1.2(45).

The signs of zero values are as recommended by the relevant implementation advice.

121. Whether the strict mode or the relaxed mode is the default. See G.2(2).

The strict mode is the default. There is no separate relaxed mode. GNAT provides a highly efficient implementation of strict mode.

122. The result interval in certain cases of fixed-to-float conversion. See G.2.1(10).

For cases where the result interval is implementation dependent, the accuracy is that provided by performing all operations in 64-bit IEEE floating-point format.

123. The result of a floating point arithmetic operation in overflow situations, when the `Machine_Overflows` attribute of the result type is `False`. See G.2.1(13).

Infinite and NaN values are produced as dictated by the IEEE floating-point standard.

Note that on machines that are not fully compliant with the IEEE floating-point standard, such as Alpha, the `-mieee` compiler flag must be used for achieving IEEE confirming behavior (although at the cost of a significant performance penalty), so infinite and NaN values are properly generated.

124. The result interval for division (or exponentiation by a negative exponent), when the floating point hardware implements division as multiplication by a reciprocal. See G.2.1(16).

Not relevant, division is IEEE exact.

125. The definition of close result set, which determines the accuracy of certain fixed point multiplications and divisions. See G.2.3(5).

Operations in the close result set are performed using IEEE long format floating-point arithmetic. The input operands are converted to floating-point, the operation is done in floating-point, and the result is converted to the target type.

126. Conditions on a `universal_real` operand of a fixed point multiplication or division for which the result shall be in the perfect result set. See G.2.3(22).

The result is only defined to be in the perfect result set if the result can be computed by a single scaling operation involving a scale factor representable in 64-bits.

127. The result of a fixed point arithmetic operation in overflow situations, when the `Machine_Overflows` attribute of the result type is `False`. See G.2.3(27).

Not relevant, `Machine_Overflows` is `True` for fixed-point types.

128. The result of an elementary function reference in overflow situations, when the `Machine_Overflows` attribute of the result type is `False`. See G.2.4(4).

IEEE infinite and Nan values are produced as appropriate.

129. The value of the angle threshold, within which certain elementary functions, complex arithmetic operations, and complex elementary functions yield results conforming to a maximum relative error bound. See G.2.4(10).

Information on this subject is not yet available.

130. The accuracy of certain elementary functions for parameters beyond the angle threshold. See G.2.4(10).

Information on this subject is not yet available.

131. The result of a complex arithmetic operation or complex elementary function reference in overflow situations, when the `Machine_Overflows` attribute of the corresponding real type is `False`. See G.2.6(5).

IEEE infinite and Nan values are produced as appropriate.

132. The accuracy of certain complex arithmetic operations and certain complex elementary functions for parameters (or components thereof) beyond the angle threshold. See G.2.6(8).

Information on those subjects is not yet available.

133. Information regarding bounded errors and erroneous execution. See H.2(1).

Information on this subject is not yet available.

134. Implementation-defined aspects of pragma `Inspection_Point`. See H.3.2(8).

Pragma `Inspection_Point` ensures that the variable is live and can be examined by the debugger at the inspection point.

135. Implementation-defined aspects of pragma `Restrictions`. See H.4(25).

There are no implementation-defined aspects of pragma `Restrictions`. The use of pragma `Restrictions` [`No_Exceptions`] has no effect on the generated code. Checks must be suppressed by use of pragma `Suppress`.

136. Any restrictions on pragma `Restrictions`. See H.4(27).

There are no restrictions on pragma `Restrictions`.

5 Intrinsic Subprograms

GNAT allows a user application program to write the declaration:

```
pragma Import (Intrinsic, name);
```

providing that the name corresponds to one of the implemented intrinsic subprograms in GNAT, and that the parameter profile of the referenced subprogram meets the requirements. This chapter describes the set of implemented intrinsic subprograms, and the requirements on parameter profiles. Note that no body is supplied; as with other uses of `pragma Import`, the body is supplied elsewhere (in this case by the compiler itself). Note that any use of this feature is potentially non-portable, since the Ada standard does not require Ada compilers to implement this feature.

5.1 Intrinsic Operators

All the predefined numeric operators in package `Standard` in `pragma Import (Intrinsic,...)` declarations. In the binary operator case, the operands must have the same size. The operand or operands must also be appropriate for the operator. For example, for addition, the operands must both be floating-point or both be fixed-point, and the right operand for `"**"` must have a root type of `Standard.Integer'Base`. You can use an intrinsic operator declaration as in the following example:

```
type Int1 is new Integer;
type Int2 is new Integer;

function "+" (X1 : Int1; X2 : Int2) return Int1;
function "+" (X1 : Int1; X2 : Int2) return Int2;
pragma Import (Intrinsic, "+");
```

This declaration would permit “mixed mode” arithmetic on items of the differing types `Int1` and `Int2`. It is also possible to specify such operators for private types, if the full views are appropriate arithmetic types.

5.2 Enclosing_Entity

This intrinsic subprogram is used in the implementation of the library routine `GNAT.Source_Info`. The only useful use of the intrinsic import in this case is the one in this unit, so an application program should simply call the function `GNAT.Source_Info.Enclosing_Entity` to obtain the name of the current subprogram, package, task, entry, or protected subprogram.

5.3 Exception_Information

This intrinsic subprogram is used in the implementation of the library routine `GNAT.Current_Exception`. The only useful use of the intrinsic import in this case is the one in this unit, so an application program should simply call the function `GNAT.Current_Exception.Exception_Information` to obtain the exception information associated with the current exception.

5.4 Exception_Message

This intrinsic subprogram is used in the implementation of the library routine `GNAT.Current_Exception`. The only useful use of the intrinsic import in this case is the one in this unit, so an application program should simply call the function `GNAT.Current_Exception.Exception_Message` to obtain the message associated with the current exception.

5.5 Exception_Name

This intrinsic subprogram is used in the implementation of the library routine `GNAT.Current_Exception`. The only useful use of the intrinsic import in this case is the one in this unit, so an application program should simply call the function `GNAT.Current_Exception.Exception_Name` to obtain the name of the current exception.

5.6 File

This intrinsic subprogram is used in the implementation of the library routine `GNAT.Source_Info`. The only useful use of the intrinsic import in this case is the one in this unit, so an application program should simply call the function `GNAT.Source_Info.File` to obtain the name of the current file.

5.7 Line

This intrinsic subprogram is used in the implementation of the library routine `GNAT.Source_Info`. The only useful use of the intrinsic import in this case is the one in this unit, so an application program should simply call the function `GNAT.Source_Info.Line` to obtain the number of the current source line.

5.8 Rotate_Left

In standard Ada, the `Rotate_Left` function is available only for the predefined modular types in package `Interfaces`. However, in GNAT it is possible to define a `Rotate_Left` function for a user defined modular type or any signed integer type as in this example:

```
function Shift_Left
  (Value  : My_Modular_Type;
   Amount : Natural)
  return  My_Modular_Type;
```

The requirements are that the profile be exactly as in the example above. The only modifications allowed are in the formal parameter names, and in the type of `Value` and the return type, which must be the same, and must be either a signed integer type, or a modular integer type with a binary modulus, and the size must be 8, 16, 32 or 64 bits.

5.9 Rotate_Right

A `Rotate_Right` function can be defined for any user defined binary modular integer type, or signed integer type, as described above for `Rotate_Left`.

5.10 Shift_Left

A `Shift_Left` function can be defined for any user defined binary modular integer type, or signed integer type, as described above for `Rotate_Left`.

5.11 Shift_Right

A `Shift_Right` function can be defined for any user defined binary modular integer type, or signed integer type, as described above for `Rotate_Left`.

5.12 Shift_Right_Arithmetic

A `Shift_Right_Arithmetic` function can be defined for any user defined binary modular integer type, or signed integer type, as described above for `Rotate_Left`.

5.13 Source_Location

This intrinsic subprogram is used in the implementation of the library routine `GNAT.Source_Info`. The only useful use of the intrinsic import in this case is the one in this unit, so an application program should simply call the function `GNAT.Source_Info.Source_Location` to obtain the current source file location.

6 Representation Clauses and Pragmas

This section describes the representation clauses accepted by GNAT, and their effect on the representation of corresponding data objects.

GNAT fully implements Annex C (Systems Programming). This means that all the implementation advice sections in chapter 13 are fully implemented. However, these sections only require a minimal level of support for representation clauses. GNAT provides much more extensive capabilities, and this section describes the additional capabilities provided.

6.1 Alignment Clauses

GNAT requires that all alignment clauses specify a power of 2, and all default alignments are always a power of 2. The default alignment values are as follows:

- *Primitive Types.* For primitive types, the alignment is the minimum of the actual size of objects of the type divided by `Storage_Unit`, and the maximum alignment supported by the target. (This maximum alignment is given by the GNAT-specific attribute `Standard'Maximum_Alignment`; see [\[Maximum_Alignment\]](#), page 68.) For example, for type `Long_Float`, the object size is 8 bytes, and the default alignment will be 8 on any target that supports alignments this large, but on some targets, the maximum alignment may be smaller than 8, in which case objects of type `Long_Float` will be maximally aligned.
- *Arrays.* For arrays, the alignment is equal to the alignment of the component type for the normal case where no packing or component size is given. If the array is packed, and the packing is effective (see separate section on packed arrays), then the alignment will be one for long packed arrays, or arrays whose length is not known at compile time. For short packed arrays, which are handled internally as modular types, the alignment will be as described for primitive types, e.g. a packed array of length 31 bits will have an object size of four bytes, and an alignment of 4.
- *Records.* For the normal non-packed case, the alignment of a record is equal to the maximum alignment of any of its components. For tagged records, this includes the implicit access type used for the tag. If a pragma `Pack` is used and all components are packable (see separate section on pragma `Pack`), then the resulting alignment is 1, unless the layout of the record makes it profitable to increase it.

A special case is when:

- the size of the record is given explicitly, or a full record representation clause is given, and
- the size of the record is 2, 4, or 8 bytes.

In this case, an alignment is chosen to match the size of the record. For example, if we have:

```
type Small is record
  A, B : Character;
end record;
for Small'Size use 16;
```

then the default alignment of the record type `Small` is 2, not 1. This leads to more efficient code when the record is treated as a unit, and also allows the type to be specified as `Atomic` on architectures requiring strict alignment.

An alignment clause may specify a larger alignment than the default value up to some maximum value dependent on the target (obtainable by using the attribute reference `Standard'Maximum_Alignment`). It may also specify a smaller alignment than the default value for enumeration, integer and fixed point types, as well as for record types, for example

```
type V is record
  A : Integer;
end record;

for V'alignment use 1;
```

The default alignment for the type `V` is 4, as a result of the `Integer` field in the record, but it is permissible, as shown, to override the default alignment of the record with a smaller value.

6.2 Size Clauses

The default size for a type `T` is obtainable through the language-defined attribute `T'Size` and also through the equivalent GNAT-defined attribute `T'Value_Size`. For objects of type `T`, GNAT will generally increase the type size so that the object size (obtainable through the GNAT-defined attribute `T'Object_Size`) is a multiple of `T'Alignment * Storage_Unit`. For example

```
type Smallint is range 1 .. 6;

type Rec is record
  Y1 : integer;
  Y2 : boolean;
end record;
```

In this example, `Smallint'Size = Smallint'Value_Size = 3`, as specified by the RM rules, but objects of this type will have a size of 8 (`Smallint'Object_Size = 8`), since objects by default occupy an integral number of storage units. On some targets, notably older versions of the Digital Alpha, the size of stand alone objects of this type may be 32, reflecting the inability of the hardware to do byte load/stores.

Similarly, the size of type `Rec` is 40 bits (`Rec'Size = Rec'Value_Size = 40`), but the alignment is 4, so objects of this type will have their size increased to 64 bits so that it is a multiple of the alignment (in bits). This decision is in accordance with the specific Implementation Advice in RM 13.3(43):

A **Size** clause should be supported for an object if the specified **Size** is at least as large as its subtype's **Size**, and corresponds to a size in storage elements that is a multiple of the object's **Alignment** (if the **Alignment** is nonzero).

An explicit size clause may be used to override the default size by increasing it. For example, if we have:

```
type My_Boolean is new Boolean;
for My_Boolean'Size use 32;
```

then values of this type will always be 32 bits long. In the case of discrete types, the size can be increased up to 64 bits, with the effect that the entire specified field is used to hold the value, sign- or zero-extended as appropriate. If more than 64 bits is specified, then padding space is allocated after the value, and a warning is issued that there are unused bits.

Similarly the size of records and arrays may be increased, and the effect is to add padding bits after the value. This also causes a warning message to be generated.

The largest Size value permitted in GNAT is $2^{31}-1$. Since this is a Size in bits, this corresponds to an object of size 256 megabytes (minus one). This limitation is true on all targets. The reason for this limitation is that it improves the quality of the code in many cases if it is known that a Size value can be accommodated in an object of type Integer.

6.3 Storage_Size Clauses

For tasks, the **Storage_Size** clause specifies the amount of space to be allocated for the task stack. This cannot be extended, and if the stack is exhausted, then **Storage_Error** will be raised (if stack checking is enabled). Use a **Storage_Size** attribute definition clause, or a **Storage_Size** pragma in the task definition to set the appropriate required size. A useful technique is to include in every task definition a pragma of the form:

```
pragma Storage_Size (Default_Stack_Size);
```

Then **Default_Stack_Size** can be defined in a global package, and modified as required. Any tasks requiring stack sizes different from the default can have an appropriate alternative reference in the pragma.

You can also use the ‘-d’ binder switch to modify the default stack size.

For access types, the **Storage_Size** clause specifies the maximum space available for allocation of objects of the type. If this space is exceeded then **Storage_Error** will be raised by an allocation attempt. In the case where the access type is declared local to a subprogram, the use of a **Storage_Size** clause triggers automatic use of a special predefined storage pool (**System.Pool_Size**) that ensures that all space for the pool is automatically reclaimed on exit from the scope in which the type is declared.

A special case recognized by the compiler is the specification of a **Storage_Size** of zero for an access type. This means that no items can be allocated from the pool, and this is recognized at compile time, and all the overhead normally associated with maintaining a fixed size storage pool is eliminated. Consider the following example:

```
procedure p is
  type R is array (Natural) of Character;
  type P is access all R;
  for P'Storage_Size use 0;
  -- Above access type intended only for interfacing purposes

  y : P;

  procedure g (m : P);
  pragma Import (C, g);

  -- ...

begin
  -- ...
  y := new R;
end;
```

As indicated in this example, these dummy storage pools are often useful in connection with interfacing where no object will ever be allocated. If you compile the above example, you get the warning:

```
p.adb:16:09: warning: allocation from empty storage pool
p.adb:16:09: warning: Storage_Error will be raised at run time
```

Of course in practice, there will not be any explicit allocators in the case of such an access declaration.

6.4 Size of Variant Record Objects

In the case of variant record objects, there is a question whether `Size` gives information about a particular variant, or the maximum size required for any variant. Consider the following program

```
with Text_IO; use Text_IO;
procedure q is
  type R1 (A : Boolean := False) is record
    case A is
      when True  => X : Character;
      when False => null;
    end case;
  end record;

  V1 : R1 (False);
  V2 : R1;

begin
  Put_Line (Integer'Image (V1'Size));
  Put_Line (Integer'Image (V2'Size));
end q;
```

Here we are dealing with a variant record, where the `True` variant requires 16 bits, and the `False` variant requires 8 bits. In the above example, both `V1` and `V2` contain the `False` variant, which is only 8 bits long. However, the result of running the program is:

```
8
16
```

The reason for the difference here is that the discriminant value of `V1` is fixed, and will always be `False`. It is not possible to assign a `True` variant value to `V1`, therefore 8 bits is sufficient. On the other hand, in the case of `V2`, the initial discriminant value is `False` (from the default), but it is possible to assign a `True` variant value to `V2`, therefore 16 bits must be allocated for `V2` in the general case, even fewer bits may be needed at any particular point during the program execution.

As can be seen from the output of this program, the `'Size` attribute applied to such an object in GNAT gives the actual allocated size of the variable, which is the largest size of any of the variants. The Ada Reference Manual is not completely clear on what choice should be made here, but the GNAT behavior seems most consistent with the language in the RM.

In some cases, it may be desirable to obtain the size of the current variant, rather than the size of the largest variant. This can be achieved in GNAT by making use of the fact that in the case of a subprogram parameter, GNAT does indeed return the size of the current variant (because a subprogram has no way of knowing how much space is actually allocated for the actual).

Consider the following modified version of the above program:

```
with Text_IO; use Text_IO;
```

```

procedure q is
  type R1 (A : Boolean := False) is record
    case A is
      when True  => X : Character;
      when False => null;
    end case;
  end record;

  V2 : R1;

  function Size (V : R1) return Integer is
  begin
    return V'Size;
  end Size;

begin
  Put_Line (Integer'Image (V2'Size));
  Put_Line (Integer'Image (Size (V2)));
  V2 := (True, 'x');
  Put_Line (Integer'Image (V2'Size));
  Put_Line (Integer'Image (Size (V2)));
end q;

```

The output from this program is

```

16
8
16
16

```

Here we see that while the `'Size` attribute always returns the maximum size, regardless of the current variant value, the `Size` function does indeed return the size of the current variant value.

6.5 Biased Representation

In the case of scalars with a range starting at other than zero, it is possible in some cases to specify a size smaller than the default minimum value, and in such cases, GNAT uses an unsigned biased representation, in which zero is used to represent the lower bound, and successive values represent successive values of the type.

For example, suppose we have the declaration:

```

type Small is range -7 .. -4;
for Small'Size use 2;

```

Although the default size of type `Small` is 4, the `Size` clause is accepted by GNAT and results in the following representation scheme:

```

-7 is represented as 2#00#
-6 is represented as 2#01#
-5 is represented as 2#10#
-4 is represented as 2#11#

```

Biased representation is only used if the specified `Size` clause cannot be accepted in any other manner. These reduced sizes that force biased representation can be used for all discrete types except for enumeration types for which a representation clause is given.

6.6 Value_Size and Object_Size Clauses

In Ada 95 and Ada 2005, `T'Size` for a type `T` is the minimum number of bits required to hold values of type `T`. Although this interpretation was allowed in Ada 83, it was not required, and this requirement in practice can cause some significant difficulties. For example, in most Ada 83 compilers, `Natural'Size` was 32. However, in Ada 95 and Ada 2005, `Natural'Size` is typically 31. This means that code may change in behavior when moving from Ada 83 to Ada 95 or Ada 2005. For example, consider:

```
type Rec is record;
  A : Natural;
  B : Natural;
end record;

for Rec use record
  at 0 range 0 .. Natural'Size - 1;
  at 0 range Natural'Size .. 2 * Natural'Size - 1;
end record;
```

In the above code, since the typical size of `Natural` objects is 32 bits and `Natural'Size` is 31, the above code can cause unexpected inefficient packing in Ada 95 and Ada 2005, and in general there are cases where the fact that the object size can exceed the size of the type causes surprises.

To help get around this problem GNAT provides two implementation defined attributes, `Value_Size` and `Object_Size`. When applied to a type, these attributes yield the size of the type (corresponding to the RM defined size attribute), and the size of objects of the type respectively.

The `Object_Size` is used for determining the default size of objects and components. This size value can be referred to using the `Object_Size` attribute. The phrase “is used” here means that it is the basis of the determination of the size. The backend is free to pad this up if necessary for efficiency, e.g. an 8-bit stand-alone character might be stored in 32 bits on a machine with no efficient byte access instructions such as the Alpha.

The default rules for the value of `Object_Size` for discrete types are as follows:

- The `Object_Size` for base subtypes reflect the natural hardware size in bits (run the compiler with ‘-gnatS’ to find those values for numeric types). Enumeration types and fixed-point base subtypes have 8, 16, 32 or 64 bits for this size, depending on the range of values to be stored.
- The `Object_Size` of a subtype is the same as the `Object_Size` of the type from which it is obtained.
- The `Object_Size` of a derived base type is copied from the parent base type, and the `Object_Size` of a derived first subtype is copied from the parent first subtype.

The `Value_Size` attribute is the (minimum) number of bits required to store a value of the type. This value is used to determine how tightly to pack records or arrays with components of this type, and also affects the semantics of unchecked conversion (unchecked conversions where the `Value_Size` values differ generate a warning, and are potentially target dependent).

The default rules for the value of `Value_Size` are as follows:

- The `Value_Size` for a base subtype is the minimum number of bits required to store all values of the type (including the sign bit only if negative values are possible).

- If a subtype statically matches the first subtype of a given type, then it has by default the same **Value_Size** as the first subtype. This is a consequence of RM 13.1(14) (“if two subtypes statically match, then their subtype-specific aspects are the same”).
- All other subtypes have a **Value_Size** corresponding to the minimum number of bits required to store all values of the subtype. For dynamic bounds, it is assumed that the value can range down or up to the corresponding bound of the ancestor

The RM defined attribute **Size** corresponds to the **Value_Size** attribute.

The **Size** attribute may be defined for a first-named subtype. This sets the **Value_Size** of the first-named subtype to the given value, and the **Object_Size** of this first-named subtype to the given value padded up to an appropriate boundary. It is a consequence of the default rules above that this **Object_Size** will apply to all further subtypes. On the other hand, **Value_Size** is affected only for the first subtype, any dynamic subtypes obtained from it directly, and any statically matching subtypes. The **Value_Size** of any other static subtypes is not affected.

Value_Size and **Object_Size** may be explicitly set for any subtype using an attribute definition clause. Note that the use of these attributes can cause the RM 13.1(14) rule to be violated. If two access types reference aliased objects whose subtypes have differing **Object_Size** values as a result of explicit attribute definition clauses, then it is erroneous to convert from one access subtype to the other.

At the implementation level, **Esize** stores the **Object_Size** and the **RM_Size** field stores the **Value_Size** (and hence the value of the **Size** attribute, which, as noted above, is equivalent to **Value_Size**).

To get a feel for the difference, consider the following examples (note that in each case the base is **Short_Short_Integer** with a size of 8):

	Object_Size	Value_Size
type x1 is range 0 .. 5;	8	3
type x2 is range 0 .. 5; for x2'size use 12;	16	12
subtype x3 is x2 range 0 .. 3;	16	2
subtype x4 is x2'base range 0 .. 10;	8	4
subtype x5 is x2 range 0 .. dynamic;	16	3*
subtype x6 is x2'base range 0 .. dynamic;	8	3*

Note: the entries marked “3*” are not actually specified by the Ada Reference Manual, but it seems in the spirit of the RM rules to allocate the minimum number of bits (here 3, given the range for x2) known to be large enough to hold the given range of values.

So far, so good, but GNAT has to obey the RM rules, so the question is under what conditions must the RM **Size** be used. The following is a list of the occasions on which the RM **Size** must be used:

- Component size for packed arrays or records
- Value of the attribute **Size** for a type

- Warning about sizes not matching for unchecked conversion

For record types, the `Object_Size` is always a multiple of the alignment of the type (this is true for all types). In some cases the `Value_Size` can be smaller. Consider:

```
type R is record
  X : Integer;
  Y : Character;
end record;
```

On a typical 32-bit architecture, the X component will be four bytes, and require four-byte alignment, and the Y component will be one byte. In this case `R'Value_Size` will be 40 (bits) since this is the minimum size required to store a value of this type, and for example, it is permissible to have a component of type R in an outer array whose component size is specified to be 48 bits. However, `R'Object_Size` will be 64 (bits), since it must be rounded up so that this value is a multiple of the alignment (4 bytes = 32 bits).

For all other types, the `Object_Size` and `Value_Size` are the same (and equivalent to the RM attribute `Size`). Only `Size` may be specified for such types.

6.7 Component_Size Clauses

Normally, the value specified in a component size clause must be consistent with the subtype of the array component with regard to size and alignment. In other words, the value specified must be at least equal to the size of this subtype, and must be a multiple of the alignment value.

In addition, component size clauses are allowed which cause the array to be packed, by specifying a smaller value. A first case is for component size values in the range 1 through 63. The value specified must not be smaller than the Size of the subtype. GNAT will accurately honor all packing requests in this range. For example, if we have:

```
type r is array (1 .. 8) of Natural;
for r'Component_Size use 31;
```

then the resulting array has a length of 31 bytes (248 bits = 8 * 31). Of course access to the components of such an array is considerably less efficient than if the natural component size of 32 is used. A second case is when the subtype of the component is a record type padded because of its default alignment. For example, if we have:

```
type r is record
  i : Integer;
  j : Integer;
  b : Boolean;
end record;

type a is array (1 .. 8) of r;
for a'Component_Size use 72;
```

then the resulting array has a length of 72 bytes, instead of 96 bytes if the alignment of the record (4) was obeyed.

Note that there is no point in giving both a component size clause and a pragma `Pack` for the same array type. If such duplicate clauses are given, the pragma `Pack` will be ignored.

6.8 Bit_Order Clauses

For record subtypes, GNAT permits the specification of the `Bit_Order` attribute. The specification may either correspond to the default bit order for the target, in which case the specification has no effect and places no additional restrictions, or it may be for the non-standard setting (that is the opposite of the default).

In the case where the non-standard value is specified, the effect is to renumber bits within each byte, but the ordering of bytes is not affected. There are certain restrictions placed on component clauses as follows:

- Components fitting within a single storage unit. These are unrestricted, and the effect is merely to renumber bits. For example if we are on a little-endian machine with `Low_Order_First` being the default, then the following two declarations have exactly the same effect:

```

type R1 is record
  A : Boolean;
  B : Integer range 1 .. 120;
end record;

for R1 use record
  A at 0 range 0 .. 0;
  B at 0 range 1 .. 7;
end record;

type R2 is record
  A : Boolean;
  B : Integer range 1 .. 120;
end record;

for R2'Bit_Order use High_Order_First;

for R2 use record
  A at 0 range 7 .. 7;
  B at 0 range 0 .. 6;
end record;
```

The useful application here is to write the second declaration with the `Bit_Order` attribute definition clause, and know that it will be treated the same, regardless of whether the target is little-endian or big-endian.

- Components occupying an integral number of bytes. These are components that exactly fit in two or more bytes. Such component declarations are allowed, but have no effect, since it is important to realize that the `Bit_Order` specification does not affect the ordering of bytes. In particular, the following attempt at getting an endian-independent integer does not work:

```

type R2 is record
  A : Integer;
end record;

for R2'Bit_Order use High_Order_First;

for R2 use record
  A at 0 range 0 .. 31;
end record;
```

This declaration will result in a little-endian integer on a little-endian machine, and a big-endian integer on a big-endian machine. If byte flipping is required for interoperability between big- and little-endian machines, this must be explicitly programmed. This capability is not provided by `Bit_Order`.

- Components that are positioned across byte boundaries but do not occupy an integral number of bytes. Given that bytes are not reordered, such fields would occupy a non-contiguous sequence of bits in memory, requiring non-trivial code to reassemble. They are for this reason not permitted, and any component clause specifying such a layout will be flagged as illegal by GNAT.

Since the misconception that `Bit_Order` automatically deals with all endian-related incompatibilities is a common one, the specification of a component field that is an integral number of bytes will always generate a warning. This warning may be suppressed using `pragma Warnings (Off)` if desired. The following section contains additional details regarding the issue of byte ordering.

6.9 Effect of `Bit_Order` on Byte Ordering

In this section we will review the effect of the `Bit_Order` attribute definition clause on byte ordering. Briefly, it has no effect at all, but a detailed example will be helpful. Before giving this example, let us review the precise definition of the effect of defining `Bit_Order`. The effect of a non-standard bit order is described in section 15.5.3 of the Ada Reference Manual:

2 A bit ordering is a method of interpreting the meaning of the storage place attributes.

To understand the precise definition of storage place attributes in this context, we visit section 13.5.1 of the manual:

13 A `record_representation_clause` (without the `mod_clause`) specifies the layout. The storage place attributes (see 13.5.2) are taken from the values of the `position`, `first_bit`, and `last_bit` expressions after normalizing those values so that `first_bit` is less than `Storage_Unit`.

The critical point here is that storage places are taken from the values after normalization, not before. So the `Bit_Order` interpretation applies to normalized values. The interpretation is described in the later part of the 15.5.3 paragraph:

2 A bit ordering is a method of interpreting the meaning of the storage place attributes. `High_Order_First` (known in the vernacular as “big endian”) means that the first bit of a storage element (bit 0) is the most significant bit (interpreting the sequence of bits that represent a component as an unsigned integer value). `Low_Order_First` (known in the vernacular as “little endian”) means the opposite: the first bit is the least significant.

Note that the numbering is with respect to the bits of a storage unit. In other words, the specification affects only the numbering of bits within a single storage unit.

We can make the effect clearer by giving an example.

Suppose that we have an external device which presents two bytes, the first byte presented, which is the first (low addressed byte) of the two byte record is called Master, and the second byte is called Slave.

The left most (most significant bit is called Control for each byte, and the remaining 7 bits are called V1, V2, . . . V7, where V7 is the rightmost (least significant) bit.

On a big-endian machine, we can write the following representation clause

```

type Data is record
  Master_Control : Bit;
  Master_V1      : Bit;
  Master_V2      : Bit;
  Master_V3      : Bit;
  Master_V4      : Bit;
  Master_V5      : Bit;
  Master_V6      : Bit;
  Master_V7      : Bit;
  Slave_Control  : Bit;
  Slave_V1       : Bit;
  Slave_V2       : Bit;
  Slave_V3       : Bit;
  Slave_V4       : Bit;
  Slave_V5       : Bit;
  Slave_V6       : Bit;
  Slave_V7       : Bit;
end record;

for Data use record
  Master_Control at 0 range 0 .. 0;
  Master_V1      at 0 range 1 .. 1;
  Master_V2      at 0 range 2 .. 2;
  Master_V3      at 0 range 3 .. 3;
  Master_V4      at 0 range 4 .. 4;
  Master_V5      at 0 range 5 .. 5;
  Master_V6      at 0 range 6 .. 6;
  Master_V7      at 0 range 7 .. 7;
  Slave_Control  at 1 range 0 .. 0;
  Slave_V1       at 1 range 1 .. 1;
  Slave_V2       at 1 range 2 .. 2;
  Slave_V3       at 1 range 3 .. 3;
  Slave_V4       at 1 range 4 .. 4;
  Slave_V5       at 1 range 5 .. 5;
  Slave_V6       at 1 range 6 .. 6;
  Slave_V7       at 1 range 7 .. 7;
end record;

```

Now if we move this to a little endian machine, then the bit ordering within the byte is backwards, so we have to rewrite the record rep clause as:

```

for Data use record
  Master_Control at 0 range 7 .. 7;
  Master_V1      at 0 range 6 .. 6;
  Master_V2      at 0 range 5 .. 5;
  Master_V3      at 0 range 4 .. 4;
  Master_V4      at 0 range 3 .. 3;
  Master_V5      at 0 range 2 .. 2;
  Master_V6      at 0 range 1 .. 1;
  Master_V7      at 0 range 0 .. 0;
  Slave_Control  at 1 range 7 .. 7;
  Slave_V1       at 1 range 6 .. 6;
  Slave_V2       at 1 range 5 .. 5;
  Slave_V3       at 1 range 4 .. 4;
  Slave_V4       at 1 range 3 .. 3;

```

```

Slave_V5      at 1 range 2 .. 2;
Slave_V6      at 1 range 1 .. 1;
Slave_V7      at 1 range 0 .. 0;
end record;

```

It is a nuisance to have to rewrite the clause, especially if the code has to be maintained on both machines. However, this is a case that we can handle with the `Bit_Order` attribute if it is implemented. Note that the implementation is not required on byte addressed machines, but it is indeed implemented in GNAT. This means that we can simply use the first record clause, together with the declaration

```
for Data'Bit_Order use High_Order_First;
```

and the effect is what is desired, namely the layout is exactly the same, independent of whether the code is compiled on a big-endian or little-endian machine.

The important point to understand is that byte ordering is not affected. A `Bit_Order` attribute definition never affects which byte a field ends up in, only where it ends up in that byte. To make this clear, let us rewrite the record rep clause of the previous example as:

```

for Data'Bit_Order use High_Order_First;
for Data use record
  Master_Control at 0 range 0 .. 0;
  Master_V1      at 0 range 1 .. 1;
  Master_V2      at 0 range 2 .. 2;
  Master_V3      at 0 range 3 .. 3;
  Master_V4      at 0 range 4 .. 4;
  Master_V5      at 0 range 5 .. 5;
  Master_V6      at 0 range 6 .. 6;
  Master_V7      at 0 range 7 .. 7;
  Slave_Control  at 0 range 8 .. 8;
  Slave_V1       at 0 range 9 .. 9;
  Slave_V2       at 0 range 10 .. 10;
  Slave_V3       at 0 range 11 .. 11;
  Slave_V4       at 0 range 12 .. 12;
  Slave_V5       at 0 range 13 .. 13;
  Slave_V6       at 0 range 14 .. 14;
  Slave_V7       at 0 range 15 .. 15;
end record;

```

This is exactly equivalent to saying (a repeat of the first example):

```

for Data'Bit_Order use High_Order_First;
for Data use record
  Master_Control at 0 range 0 .. 0;
  Master_V1      at 0 range 1 .. 1;
  Master_V2      at 0 range 2 .. 2;
  Master_V3      at 0 range 3 .. 3;
  Master_V4      at 0 range 4 .. 4;
  Master_V5      at 0 range 5 .. 5;
  Master_V6      at 0 range 6 .. 6;
  Master_V7      at 0 range 7 .. 7;
  Slave_Control  at 1 range 0 .. 0;
  Slave_V1       at 1 range 1 .. 1;
  Slave_V2       at 1 range 2 .. 2;
  Slave_V3       at 1 range 3 .. 3;
  Slave_V4       at 1 range 4 .. 4;
  Slave_V5       at 1 range 5 .. 5;
  Slave_V6       at 1 range 6 .. 6;
  Slave_V7       at 1 range 7 .. 7;
end record;

```

Why are they equivalent? Well take a specific field, the `Slave_V2` field. The storage place attributes are obtained by normalizing the values given so that the `First_Bit` value is less than 8. After normalizing the values (0,10,10) we get (1,2,2) which is exactly what we specified in the other case.

Now one might expect that the `Bit_Order` attribute might affect bit numbering within the entire record component (two bytes in this case, thus affecting which byte fields end up in), but that is not the way this feature is defined, it only affects numbering of bits, not which byte they end up in.

Consequently it never makes sense to specify a starting bit number greater than 7 (for a byte addressable field) if an attribute definition for `Bit_Order` has been given, and indeed it may be actively confusing to specify such a value, so the compiler generates a warning for such usage.

If you do need to control byte ordering then appropriate conditional values must be used. If in our example, the slave byte came first on some machines we might write:

```
Master_Byte_First constant Boolean := ...;

Master_Byte : constant Natural :=
    1 - Boolean'Pos (Master_Byte_First);
Slave_Byte  : constant Natural :=
    Boolean'Pos (Master_Byte_First);

for Data'Bit_Order use High_Order_First;
for Data use record
    Master_Control at Master_Byte range 0 .. 0;
    Master_V1      at Master_Byte range 1 .. 1;
    Master_V2      at Master_Byte range 2 .. 2;
    Master_V3      at Master_Byte range 3 .. 3;
    Master_V4      at Master_Byte range 4 .. 4;
    Master_V5      at Master_Byte range 5 .. 5;
    Master_V6      at Master_Byte range 6 .. 6;
    Master_V7      at Master_Byte range 7 .. 7;
    Slave_Control  at Slave_Byte  range 0 .. 0;
    Slave_V1       at Slave_Byte  range 1 .. 1;
    Slave_V2       at Slave_Byte  range 2 .. 2;
    Slave_V3       at Slave_Byte  range 3 .. 3;
    Slave_V4       at Slave_Byte  range 4 .. 4;
    Slave_V5       at Slave_Byte  range 5 .. 5;
    Slave_V6       at Slave_Byte  range 6 .. 6;
    Slave_V7       at Slave_Byte  range 7 .. 7;
end record;
```

Now to switch between machines, all that is necessary is to set the boolean constant `Master_Byte_First` in an appropriate manner.

6.10 Pragma Pack for Arrays

Pragma `Pack` applied to an array has no effect unless the component type is packable. For a component type to be packable, it must be one of the following cases:

- Any scalar type
- Any type whose size is specified with a size clause
- Any packed array type with a static size
- Any record type padded because of its default alignment

For all these cases, if the component subtype size is in the range 1 through 63, then the effect of the pragma **Pack** is exactly as though a component size were specified giving the component subtype size. For example if we have:

```
type r is range 0 .. 17;

type ar is array (1 .. 8) of r;
pragma Pack (ar);
```

Then the component size of **ar** will be set to 5 (i.e. to **r'size**, and the size of the array **ar** will be exactly 40 bits.

Note that in some cases this rather fierce approach to packing can produce unexpected effects. For example, in Ada 95 and Ada 2005, subtype **Natural** typically has a size of 31, meaning that if you pack an array of **Natural**, you get 31-bit close packing, which saves a few bits, but results in far less efficient access. Since many other Ada compilers will ignore such a packing request, GNAT will generate a warning on some uses of pragma **Pack** that it guesses might not be what is intended. You can easily remove this warning by using an explicit **Component_Size** setting instead, which never generates a warning, since the intention of the programmer is clear in this case.

GNAT treats packed arrays in one of two ways. If the size of the array is known at compile time and is less than 64 bits, then internally the array is represented as a single modular type, of exactly the appropriate number of bits. If the length is greater than 63 bits, or is not known at compile time, then the packed array is represented as an array of bytes, and the length is always a multiple of 8 bits.

Note that to represent a packed array as a modular type, the alignment must be suitable for the modular type involved. For example, on typical machines a 32-bit packed array will be represented by a 32-bit modular integer with an alignment of four bytes. If you explicitly override the default alignment with an alignment clause that is too small, the modular representation cannot be used. For example, consider the following set of declarations:

```
type R is range 1 .. 3;
type S is array (1 .. 31) of R;
for S'Component_Size use 2;
for S'Size use 62;
for S'Alignment use 1;
```

If the alignment clause were not present, then a 62-bit modular representation would be chosen (typically with an alignment of 4 or 8 bytes depending on the target). But the default alignment is overridden with the explicit alignment clause. This means that the modular representation cannot be used, and instead the array of bytes representation must be used, meaning that the length must be a multiple of 8. Thus the above set of declarations will result in a diagnostic rejecting the size clause and noting that the minimum size allowed is 64.

One special case that is worth noting occurs when the base type of the component size is 8/16/32 and the subtype is one bit less. Notably this occurs with subtype **Natural**. Consider:

```
type Arr is array (1 .. 32) of Natural;
pragma Pack (Arr);
```

In all commonly used Ada 83 compilers, this pragma **Pack** would be ignored, since typically **Natural'Size** is 32 in Ada 83, and in any case most Ada 83 compilers did not attempt 31 bit packing.

In Ada 95 and Ada 2005, `Natural'Size` is required to be 31. Furthermore, GNAT really does pack 31-bit subtype to 31 bits. This may result in a substantial unintended performance penalty when porting legacy Ada 83 code. To help prevent this, GNAT generates a warning in such cases. If you really want 31 bit packing in a case like this, you can set the component size explicitly:

```
type Arr is array (1 .. 32) of Natural;
for Arr'Component_Size use 31;
```

Here 31-bit packing is achieved as required, and no warning is generated, since in this case the programmer intention is clear.

6.11 Pragma Pack for Records

`Pragma Pack` applied to a record will pack the components to reduce wasted space from alignment gaps and by reducing the amount of space taken by components. We distinguish between *packable* components and *non-packable* components. Components of the following types are considered packable:

- All primitive types are packable.
- Small packed arrays, whose size does not exceed 64 bits, and where the size is statically known at compile time, are represented internally as modular integers, and so they are also packable.

All packable components occupy the exact number of bits corresponding to their `Size` value, and are packed with no padding bits, i.e. they can start on an arbitrary bit boundary.

All other types are non-packable, they occupy an integral number of storage units, and are placed at a boundary corresponding to their alignment requirements.

For example, consider the record

```
type Rb1 is array (1 .. 13) of Boolean;
pragma Pack (rb1);

type Rb2 is array (1 .. 65) of Boolean;
pragma Pack (rb2);

type x2 is record
  l1 : Boolean;
  l2 : Duration;
  l3 : Float;
  l4 : Boolean;
  l5 : Rb1;
  l6 : Rb2;
end record;
pragma Pack (x2);
```

The representation for the record `x2` is as follows:

```
for x2'Size use 224;
for x2 use record
  l1 at 0 range 0 .. 0;
  l2 at 0 range 1 .. 64;
  l3 at 12 range 0 .. 31;
  l4 at 16 range 0 .. 0;
  l5 at 16 range 1 .. 13;
  l6 at 18 range 0 .. 71;
end record;
```

Studying this example, we see that the packable fields 11 and 12 are of length equal to their sizes, and placed at specific bit boundaries (and not byte boundaries) to eliminate padding. But 13 is of a non-packable float type, so it is on the next appropriate alignment boundary.

The next two fields are fully packable, so 14 and 15 are minimally packed with no gaps. However, type `Rb2` is a packed array that is longer than 64 bits, so it is itself non-packable. Thus the 16 field is aligned to the next byte boundary, and takes an integral number of bytes, i.e. 72 bits.

6.12 Record Representation Clauses

Record representation clauses may be given for all record types, including types obtained by record extension. Component clauses are allowed for any static component. The restrictions on component clauses depend on the type of the component.

For all components of an elementary type, the only restriction on component clauses is that the size must be at least the `'Size` value of the type (actually the `Value.Size`). There are no restrictions due to alignment, and such components may freely cross storage boundaries.

Packed arrays with a size up to and including 64 bits are represented internally using a modular type with the appropriate number of bits, and thus the same lack of restriction applies. For example, if you declare:

```
type R is array (1 .. 49) of Boolean;
pragma Pack (R);
for R'Size use 49;
```

then a component clause for a component of type `R` may start on any specified bit boundary, and may specify a value of 49 bits or greater.

For packed bit arrays that are longer than 64 bits, there are two cases. If the component size is a power of 2 (1,2,4,8,16,32 bits), including the important case of single bits or boolean values, then there are no limitations on placement of such components, and they may start and end at arbitrary bit boundaries.

If the component size is not a power of 2 (e.g. 3 or 5), then an array of this type longer than 64 bits must always be placed on a storage unit (byte) boundary and occupy an integral number of storage units (bytes). Any component clause that does not meet this requirement will be rejected.

Any aliased component, or component of an aliased type, must have its normal alignment and size. A component clause that does not meet this requirement will be rejected.

The tag field of a tagged type always occupies an address sized field at the start of the record. No component clause may attempt to overlay this tag. When a tagged type appears as a component, the tag field must have proper alignment

In the case of a record extension `T1`, of a type `T`, no component clause applied to the type `T1` can specify a storage location that would overlap the first `T'Size` bytes of the record.

For all other component types, including non-bit-packed arrays, the component can be placed at an arbitrary bit boundary, so for example, the following is permitted:

```
type R is array (1 .. 10) of Boolean;
for R'Size use 80;

type Q is record
  G, H : Boolean;
```

```

    L, M : R;
end record;

for Q use record
  G at 0 range 0 .. 0;
  H at 0 range 1 .. 1;
  L at 0 range 2 .. 81;
  R at 0 range 82 .. 161;
end record;

```

Note: the above rules apply to recent releases of GNAT 5. In GNAT 3, there are more severe restrictions on larger components. For non-primitive types, including packed arrays with a size greater than 64 bits, component clauses must respect the alignment requirement of the type, in particular, always starting on a byte boundary, and the length must be a multiple of the storage unit.

6.13 Enumeration Clauses

The only restriction on enumeration clauses is that the range of values must be representable. For the signed case, if one or more of the representation values are negative, all values must be in the range:

```
System.Min_Int .. System.Max_Int
```

For the unsigned case, where all values are nonnegative, the values must be in the range:

```
0 .. System.Max_Binary_Modulus;
```

A *confirming* representation clause is one in which the values range from 0 in sequence, i.e. a clause that confirms the default representation for an enumeration type. Such a confirming representation is permitted by these rules, and is specially recognized by the compiler so that no extra overhead results from the use of such a clause.

If an array has an index type which is an enumeration type to which an enumeration clause has been applied, then the array is stored in a compact manner. Consider the declarations:

```

type r is (A, B, C);
for r use (A => 1, B => 5, C => 10);
type t is array (r) of Character;

```

The array type `t` corresponds to a vector with exactly three elements and has a default size equal to `3*Character'Size`. This ensures efficient use of space, but means that accesses to elements of the array will incur the overhead of converting representation values to the corresponding positional values, (i.e. the value delivered by the `Pos` attribute).

6.14 Address Clauses

The reference manual allows a general restriction on representation clauses, as found in RM 13.1(22):

An implementation need not support representation items containing nonstatic expressions, except that an implementation should support a representation item for a given entity if each nonstatic expression in the representation item is a name that statically denotes a constant declared before the entity.

In practice this is applicable only to address clauses, since this is the only case in which a non-static expression is permitted by the syntax. As the AARM notes in sections 13.1 (22.a-22.h):

22.a Reason: This is to avoid the following sort of thing:

```
22.b      X : Integer := F(...);
          Y : Address := G(...);
          for X'Address use Y;
```

22.c In the above, we have to evaluate the initialization expression for X before we know where to put the result. This seems like an unreasonable implementation burden.

22.d The above code should instead be written like this:

```
22.e      Y : constant Address := G(...);
          X : Integer := F(...);
          for X'Address use Y;
```

22.f This allows the expression “Y” to be safely evaluated before X is created.

22.g The constant could be a formal parameter of mode in.

22.h An implementation can support other nonstatic expressions if it wants to. Expressions of type Address are hardly ever static, but their value might be known at compile time anyway in many cases.

GNAT does indeed permit many additional cases of non-static expressions. In particular, if the type involved is elementary there are no restrictions (since in this case, holding a temporary copy of the initialization value, if one is present, is inexpensive). In addition, if there is no implicit or explicit initialization, then there are no restrictions. GNAT will reject only the case where all three of these conditions hold:

- The type of the item is non-elementary (e.g. a record or array).
- There is explicit or implicit initialization required for the object. Note that access values are always implicitly initialized, and also in GNAT, certain bit-packed arrays (those having a dynamic length or a length greater than 64) will also be implicitly initialized to zero.
- The address value is non-static. Here GNAT is more permissive than the RM, and allows the address value to be the address of a previously declared stand-alone variable, as long as it does not itself have an address clause.

```
Anchor   : Some_Initialized_Type;
Overlay  : Some_Initialized_Type;
for Overlay'Address use Anchor'Address;
```

However, the prefix of the address clause cannot be an array component, or a component of a discriminated record.

As noted above in section 22.h, address values are typically non-static. In particular the `To_Address` function, even if applied to a literal value, is a non-static function call. To avoid this minor annoyance, GNAT provides the implementation defined attribute `'To_Address`. The following two expressions have identical values:

```
To_Address (16#1234_0000#)
System'To_Address (16#1234_0000#);
```

except that the second form is considered to be a static expression, and thus when used as an address clause value is always permitted.

Additionally, GNAT treats as static an address clause that is an `unchecked_conversion` of a static integer value. This simplifies the porting of legacy code, and provides a portable equivalent to the GNAT attribute `To_Address`.

Another issue with address clauses is the interaction with alignment requirements. When an address clause is given for an object, the address value must be consistent with the alignment of the object (which is usually the same as the alignment of the type of the object). If an address clause is given that specifies an inappropriately aligned address value, then the program execution is erroneous.

Since this source of erroneous behavior can have unfortunate effects, GNAT checks (at compile time if possible, generating a warning, or at execution time with a run-time check) that the alignment is appropriate. If the run-time check fails, then `Program_Error` is raised. This run-time check is suppressed if range checks are suppressed, or if the special GNAT check `Alignment_Check` is suppressed, or if `pragma Restrictions (No_Elaboration_Code)` is in effect.

Finally, GNAT does not permit overlaying of objects of controlled types or composite types containing a controlled component. In most cases, the compiler can detect an attempt at such overlays and will generate a warning at compile time and a `Program_Error` exception at run time.

An address clause cannot be given for an exported object. More understandably the real restriction is that objects with an address clause cannot be exported. This is because such variables are not defined by the Ada program, so there is no external object to export.

It is permissible to give an address clause and a `pragma Import` for the same object. In this case, the variable is not really defined by the Ada program, so there is no external symbol to be linked. The link name and the external name are ignored in this case. The reason that we allow this combination is that it provides a useful idiom to avoid unwanted initializations on objects with address clauses.

When an address clause is given for an object that has implicit or explicit initialization, then by default initialization takes place. This means that the effect of the object declaration is to overwrite the memory at the specified address. This is almost always not what the programmer wants, so GNAT will output a warning:

```
with System;
package G is
  type R is record
    M : Integer := 0;
  end record;

  Ext : R;
  for Ext'Address use System'To_Address (16#1234_1234#);
  |
```

```
>>> warning: implicit initialization of "Ext" may
      modify overlaid storage
>>> warning: use pragma Import for "Ext" to suppress
      initialization (RM B(24))
```

```
end G;
```

As indicated by the warning message, the solution is to use a (dummy) pragma Import to suppress this initialization. The pragma tell the compiler that the object is declared and initialized elsewhere. The following package compiles without warnings (and the initialization is suppressed):

```
with System;
package G is
  type R is record
    M : Integer := 0;
  end record;

  Ext : R;
  for Ext'Address use System'To_Address (16#1234_1234#);
  pragma Import (Ada, Ext);
end G;
```

A final issue with address clauses involves their use for overlaying variables, as in the following example:

```
A : Integer;
B : Integer;
for B'Address use A'Address;
```

or alternatively, using the form recommended by the RM:

```
A      : Integer;
Addr : constant Address := A'Address;
B      : Integer;
for B'Address use Addr;
```

In both of these cases, A and B become aliased to one another via the address clause. This use of address clauses to overlay variables, achieving an effect similar to unchecked conversion was erroneous in Ada 83, but in Ada 95 and Ada 2005 the effect is implementation defined. Furthermore, the Ada RM specifically recommends that in a situation like this, B should be subject to the following implementation advice (RM 13.3(19)):

19 If the Address of an object is specified, or it is imported or exported, then the implementation should not perform optimizations based on assumptions of no aliases.

GNAT follows this recommendation, and goes further by also applying this recommendation to the overlaid variable (A in the above example) in this case. This means that the overlay works "as expected", in that a modification to one of the variables will affect the value of the other.

6.15 Effect of Convention on Representation

Normally the specification of a foreign language convention for a type or an object has no effect on the chosen representation. In particular, the representation chosen for data in GNAT generally meets the standard system conventions, and for example records are laid out in a manner that is consistent with C. This means that specifying convention C (for example) has no effect.

There are four exceptions to this general rule:

- **Convention Fortran and array subtypes** If pragma `Convention Fortran` is specified for an array subtype, then in accordance with the implementation advice in section 3.6.2(11) of the Ada Reference Manual, the array will be stored in a Fortran-compatible column-major manner, instead of the normal default row-major order.
- **Convention C and enumeration types** GNAT normally stores enumeration types in 8, 16, or 32 bits as required to accommodate all values of the type. For example, for the enumeration type declared by:

```
type Color is (Red, Green, Blue);
```

8 bits is sufficient to store all values of the type, so by default, objects of type `Color` will be represented using 8 bits. However, normal C convention is to use 32 bits for all enum values in C, since enum values are essentially of type `int`. If pragma `Convention C` is specified for an Ada enumeration type, then the size is modified as necessary (usually to 32 bits) to be consistent with the C convention for enum values.

Note that this treatment applies only to types. If `Convention C` is given for an enumeration object, where the enumeration type is not `Convention C`, then `Object_Size` bits are allocated. For example, for a normal enumeration type, with less than 256 elements, only 8 bits will be allocated for the object. Since this may be a surprise in terms of what C expects, GNAT will issue a warning in this situation. The warning can be suppressed by giving an explicit size clause specifying the desired size.

- **Convention C/Fortran and Boolean types** In C, the usual convention for boolean values, that is values used for conditions, is that zero represents false, and nonzero values represent true. In Ada, the normal convention is that two specific values, typically 0/1, are used to represent false/true respectively.

Fortran has a similar convention for `LOGICAL` values (any nonzero value represents true).

To accommodate the Fortran and C conventions, if a pragma `Convention` specifies C or Fortran convention for a derived Boolean, as in the following example:

```
type C_Switch is new Boolean;
pragma Convention (C, C_Switch);
```

then the GNAT generated code will treat any nonzero value as true. For truth values generated by GNAT, the conventional value 1 will be used for `True`, but when one of these values is read, any nonzero value is treated as `True`.

- **Access types on OpenVMS** For 64-bit OpenVMS systems, access types (other than those for unconstrained arrays) are 64-bits long. An exception to this rule is for the case of C-convention access types where there is no explicit size clause present (or inherited for derived types). In this case, GNAT chooses to make these pointers 32-bits, which provides an easier path for migration of 32-bit legacy code. size clause specifying 64-bits must be used to obtain a 64-bit pointer.

6.16 Determining the Representations chosen by GNAT

Although the descriptions in this section are intended to be complete, it is often easier to simply experiment to see what GNAT accepts and what the effect is on the layout of types and objects.

As required by the Ada RM, if a representation clause is not accepted, then it must be rejected as illegal by the compiler. However, when a representation clause or pragma is accepted, there can still be questions of what the compiler actually does. For example, if a partial record representation clause specifies the location of some components and not others, then where are the non-specified components placed? Or if pragma `Pack` is used on a record, then exactly where are the resulting fields placed? The section on pragma `Pack` in this chapter can be used to answer the second question, but it is often easier to just see what the compiler does.

For this purpose, GNAT provides the option ‘`-gnatR`’. If you compile with this option, then the compiler will output information on the actual representations chosen, in a format similar to source representation clauses. For example, if we compile the package:

```
package q is
  type r (x : boolean) is tagged record
    case x is
      when True => S : String (1 .. 100);
      when False => null;
    end case;
  end record;

  type r2 is new r (false) with record
    y2 : integer;
  end record;

  for r2 use record
    y2 at 16 range 0 .. 31;
  end record;

  type x is record
    y : character;
  end record;

  type x1 is array (1 .. 10) of x;
  for x1'component_size use 11;

  type ia is access integer;

  type Rb1 is array (1 .. 13) of Boolean;
  pragma Pack (rb1);

  type Rb2 is array (1 .. 65) of Boolean;
  pragma Pack (rb2);

  type x2 is record
    l1 : Boolean;
    l2 : Duration;
    l3 : Float;
    l4 : Boolean;
    l5 : Rb1;
    l6 : Rb2;
  end record;
  pragma Pack (x2);
end q;
```

using the switch ‘`-gnatR`’ we obtain the following output:

```
Representation information for unit q
-----
```

```

for r'Size use ??;
for r'Alignment use 4;
for r use record
  x    at 4 range 0 .. 7;
  _tag at 0 range 0 .. 31;
  s    at 5 range 0 .. 799;
end record;

for r2'Size use 160;
for r2'Alignment use 4;
for r2 use record
  x      at 4 range 0 .. 7;
  _tag   at 0 range 0 .. 31;
  _parent at 0 range 0 .. 63;
  y2     at 16 range 0 .. 31;
end record;

for x'Size use 8;
for x'Alignment use 1;
for x use record
  y at 0 range 0 .. 7;
end record;

for x1'Size use 112;
for x1'Alignment use 1;
for x1'Component_Size use 11;

for rb1'Size use 13;
for rb1'Alignment use 2;
for rb1'Component_Size use 1;

for rb2'Size use 72;
for rb2'Alignment use 1;
for rb2'Component_Size use 1;

for x2'Size use 224;
for x2'Alignment use 4;
for x2 use record
  l1 at 0 range 0 .. 0;
  l2 at 0 range 1 .. 64;
  l3 at 12 range 0 .. 31;
  l4 at 16 range 0 .. 0;
  l5 at 16 range 1 .. 13;
  l6 at 18 range 0 .. 71;
end record;

```

The Size values are actually the `Object_Size`, i.e. the default size that will be allocated for objects of the type. The `??` size for type `r` indicates that we have a variant record, and the actual size of objects will depend on the discriminant value.

The Alignment values show the actual alignment chosen by the compiler for each record or array type.

The record representation clause for type `r` shows where all fields are placed, including the compiler generated tag field (whose location cannot be controlled by the programmer).

The record representation clause for the type extension `r2` shows all the fields present, including the parent field, which is a copy of the fields of the parent type of `r2`, i.e. `r1`.

The component size and size clauses for types `rb1` and `rb2` show the exact effect of pragma `Pack` on these arrays, and the record representation clause for type `x2` shows how pragma `Pack` affects this record type.

In some cases, it may be useful to cut and paste the representation clauses generated by the compiler into the original source to fix and guarantee the actual representation to be used.

7 Standard Library Routines

The Ada Reference Manual contains in Annex A a full description of an extensive set of standard library routines that can be used in any Ada program, and which must be provided by all Ada compilers. They are analogous to the standard C library used by C programs.

GNAT implements all of the facilities described in annex A, and for most purposes the description in the Ada Reference Manual, or appropriate Ada text book, will be sufficient for making use of these facilities.

In the case of the input-output facilities, See [Chapter 8 \[The Implementation of Standard I/O\]](#), [page 167](#), gives details on exactly how GNAT interfaces to the file system. For the remaining packages, the Ada Reference Manual should be sufficient. The following is a list of the packages included, together with a brief description of the functionality that is provided.

For completeness, references are included to other predefined library routines defined in other sections of the Ada Reference Manual (these are cross-indexed from Annex A).

Ada (A.2) This is a parent package for all the standard library packages. It is usually included implicitly in your program, and itself contains no useful data or routines.

Ada.Calendar (9.6)

Calendar provides time of day access, and routines for manipulating times and durations.

Ada.Characters (A.3.1)

This is a dummy parent package that contains no useful entities

Ada.Characters.Handling (A.3.2)

This package provides some basic character handling capabilities, including classification functions for classes of characters (e.g. test for letters, or digits).

Ada.Characters.Latin_1 (A.3.3)

This package includes a complete set of definitions of the characters that appear in type `CHARACTER`. It is useful for writing programs that will run in international environments. For example, if you want an upper case E with an acute accent in a string, it is often better to use the definition of `UC_E_Acute` in this package. Then your program will print in an understandable manner even if your environment does not support these extended characters.

Ada.Command_Line (A.15)

This package provides access to the command line parameters and the name of the current program (analogous to the use of `argc` and `argv` in C), and also allows the exit status for the program to be set in a system-independent manner.

Ada.Decimal (F.2)

This package provides constants describing the range of decimal numbers implemented, and also a decimal divide routine (analogous to the COBOL verb `DIVIDE ... GIVING ... REMAINDER ...`)

Ada.Direct_IO (A.8.4)

This package provides input-output using a model of a set of records of fixed-length, containing an arbitrary definite Ada type, indexed by an integer record number.

Ada.Dynamic_Priorities (D.5)

This package allows the priorities of a task to be adjusted dynamically as the task is running.

Ada.Exceptions (11.4.1)

This package provides additional information on exceptions, and also contains facilities for treating exceptions as data objects, and raising exceptions with associated messages.

Ada.Finalization (7.6)

This package contains the declarations and subprograms to support the use of controlled types, providing for automatic initialization and finalization (analogous to the constructors and destructors of C++)

Ada.Interrupts (C.3.2)

This package provides facilities for interfacing to interrupts, which includes the set of signals or conditions that can be raised and recognized as interrupts.

Ada.Interrupts.Names (C.3.2)

This package provides the set of interrupt names (actually signal or condition names) that can be handled by GNAT.

Ada.IO_Exceptions (A.13)

This package defines the set of exceptions that can be raised by use of the standard IO packages.

Ada.Numerics

This package contains some standard constants and exceptions used throughout the numerics packages. Note that the constants pi and e are defined here, and it is better to use these definitions than rolling your own.

Ada.Numerics.Complex_Elementary_Functions

Provides the implementation of standard elementary functions (such as log and trigonometric functions) operating on complex numbers using the standard `Float` and the `Complex` and `Imaginary` types created by the package `Numerics.Complex_Types`.

Ada.Numerics.Complex_Types

This is a predefined instantiation of `Numerics.Generic_Complex_Types` using `Standard.Float` to build the type `Complex` and `Imaginary`.

Ada.Numerics.Discrete_Random

This package provides a random number generator suitable for generating random integer values from a specified range.

Ada.Numerics.Float_Random

This package provides a random number generator suitable for generating uniformly distributed floating point values.

Ada.Numerics.Generic_Complex_Elementary_Functions

This is a generic version of the package that provides the implementation of standard elementary functions (such as log and trigonometric functions) for an arbitrary complex type.

The following predefined instantiations of this package are provided:

```
Short_Float      Ada.Numerics.Short_Complex_Elementary_Functions
Float           Ada.Numerics.Complex_Elementary_Functions
Long_Float      Ada.Numerics.Long_Complex_Elementary_Functions
```

Ada.Numerics.Generic_Complex_Types

This is a generic package that allows the creation of complex types, with associated complex arithmetic operations.

The following predefined instantiations of this package exist

```
Short_Float      Ada.Numerics.Short_Complex_Complex_Types
Float           Ada.Numerics.Complex_Complex_Types
Long_Float      Ada.Numerics.Long_Complex_Complex_Types
```

Ada.Numerics.Generic_Elementary_Functions

This is a generic package that provides the implementation of standard elementary functions (such as log and trigonometric functions) for an arbitrary float type.

The following predefined instantiations of this package exist

```
Short_Float      Ada.Numerics.Short_Elementary_Functions
Float           Ada.Numerics.Elementary_Functions
Long_Float      Ada.Numerics.Long_Elementary_Functions
```

Ada.Real_Time (D.8)

This package provides facilities similar to those of `Calendar`, but operating with a finer clock suitable for real time control. Note that annex D requires that there be no backward clock jumps, and GNAT generally guarantees this behavior, but of course if the external clock on which the GNAT runtime depends is deliberately reset by some external event, then such a backward jump may occur.

Ada.Sequential_IO (A.8.1)

This package provides input-output facilities for sequential files, which can contain a sequence of values of a single type, which can be any Ada type, including indefinite (unconstrained) types.

Ada.Storage_IO (A.9)

This package provides a facility for mapping arbitrary Ada types to and from a storage buffer. It is primarily intended for the creation of new IO packages.

Ada.Streams (13.13.1)

This is a generic package that provides the basic support for the concept of streams as used by the stream attributes (**Input**, **Output**, **Read** and **Write**).

Ada.Streams.Stream_IO (A.12.1)

This package is a specialization of the type **Streams** defined in package **Streams** together with a set of operations providing **Stream_IO** capability. The **Stream_IO** model permits both random and sequential access to a file which can contain an arbitrary set of values of one or more Ada types.

Ada.Strings (A.4.1)

This package provides some basic constants used by the string handling packages.

Ada.Strings.Bounded (A.4.4)

This package provides facilities for handling variable length strings. The bounded model requires a maximum length. It is thus somewhat more limited than the unbounded model, but avoids the use of dynamic allocation or finalization.

Ada.Strings.Fixed (A.4.3)

This package provides facilities for handling fixed length strings.

Ada.Strings.Maps (A.4.2)

This package provides facilities for handling character mappings and arbitrarily defined subsets of characters. For instance it is useful in defining specialized translation tables.

Ada.Strings.Maps.Constants (A.4.6)

This package provides a standard set of predefined mappings and predefined character sets. For example, the standard upper to lower case conversion table is found in this package. Note that upper to lower case conversion is non-trivial if you want to take the entire set of characters, including extended characters like E with an acute accent, into account. You should use the mappings in this package (rather than adding 32 yourself) to do case mappings.

Ada.Strings.Unbounded (A.4.5)

This package provides facilities for handling variable length strings. The unbounded model allows arbitrary length strings, but requires the use of dynamic allocation and finalization.

Ada.Strings.Wide_Bounded (A.4.7)**Ada.Strings.Wide_Fixed (A.4.7)****Ada.Strings.Wide_Maps (A.4.7)****Ada.Strings.Wide_Maps.Constants (A.4.7)****Ada.Strings.Wide_Unbounded (A.4.7)**

These packages provide analogous capabilities to the corresponding packages without ‘**Wide_**’ in the name, but operate with the types **Wide_String** and **Wide_Character** instead of **String** and **Character**.

`Ada.Strings.Wide_Wide_Bounded` (A.4.7)

`Ada.Strings.Wide_Wide_Fixed` (A.4.7)

`Ada.Strings.Wide_Wide_Maps` (A.4.7)

`Ada.Strings.Wide_Wide_Maps.Constants` (A.4.7)

`Ada.Strings.Wide_Wide_Unbounded` (A.4.7)

These packages provide analogous capabilities to the corresponding packages without ‘Wide_’ in the name, but operate with the types `Wide_Wide_String` and `Wide_Wide_Character` instead of `String` and `Character`.

`Ada.Synchronous_Task_Control` (D.10)

This package provides some standard facilities for controlling task communication in a synchronous manner.

`Ada.Tags` This package contains definitions for manipulation of the tags of tagged values.

`Ada.Task_Attributes`

This package provides the capability of associating arbitrary task-specific data with separate tasks.

`Ada.Text_IO`

This package provides basic text input-output capabilities for character, string and numeric data. The subpackages of this package are listed next.

`Ada.Text_IO.Decimal_IO`

Provides input-output facilities for decimal fixed-point types

`Ada.Text_IO.Enumeration_IO`

Provides input-output facilities for enumeration types.

`Ada.Text_IO.Fixed_IO`

Provides input-output facilities for ordinary fixed-point types.

`Ada.Text_IO.Float_IO`

Provides input-output facilities for float types. The following predefined instantiations of this generic package are available:

`Short_Float`

`Short_Float_Text_IO`

`Float`

`Float_Text_IO`

`Long_Float`

`Long_Float_Text_IO`

`Ada.Text_IO.Integer_IO`

Provides input-output facilities for integer types. The following predefined instantiations of this generic package are available:

`Short_Short_Integer`

`Ada.Short_Short_Integer_Text_IO`

`Short_Integer`

`Ada.Short_Integer_Text_IO`

`Integer`

`Ada.Integer_Text_IO`

`Long_Integer`

`Ada.Long_Integer_Text_IO`

`Long_Long_Integer`

`Ada.Long_Long_Integer_Text_IO`

`Ada.Text_IO.Modular_IO`

Provides input-output facilities for modular (unsigned) types

`Ada.Text_IO.Complex_IO` (G.1.3)

This package provides basic text input-output capabilities for complex data.

`Ada.Text_IO.Editing` (F.3.3)

This package contains routines for edited output, analogous to the use of pictures in COBOL. The picture formats used by this package are a close copy of the facility in COBOL.

`Ada.Text_IO.Text_Streams` (A.12.2)

This package provides a facility that allows `Text_IO` files to be treated as streams, so that the stream attributes can be used for writing arbitrary data, including binary data, to `Text_IO` files.

`Ada.Unchecked_Conversion` (13.9)

This generic package allows arbitrary conversion from one type to another of the same size, providing for breaking the type safety in special circumstances.

If the types have the same `Size` (more accurately the same `Value.Size`), then the effect is simply to transfer the bits from the source to the target type without any modification. This usage is well defined, and for simple types whose representation is typically the same across all implementations, gives a portable method of performing such conversions.

If the types do not have the same size, then the result is implementation defined, and thus may be non-portable. The following describes how GNAT handles such unchecked conversion cases.

If the types are of different sizes, and are both discrete types, then the effect is of a normal type conversion without any constraint checking. In particular if the result type has a larger size, the result will be zero or sign extended. If the result type has a smaller size, the result will be truncated by ignoring high order bits.

If the types are of different sizes, and are not both discrete types, then the conversion works as though pointers were created to the source and target, and the pointer value is converted. The effect is that bits are copied from successive low order storage units and bits of the source up to the length of the target type.

A warning is issued if the lengths differ, since the effect in this case is implementation dependent, and the above behavior may not match that of some other compiler.

A pointer to one type may be converted to a pointer to another type using unchecked conversion. The only case in which the effect is undefined is when one or both pointers are pointers to unconstrained array types. In this case, the

bounds information may get incorrectly transferred, and in particular, GNAT uses double size pointers for such types, and it is meaningless to convert between such pointer types. GNAT will issue a warning if the alignment of the target designated type is more strict than the alignment of the source designated type (since the result may be unaligned in this case).

A pointer other than a pointer to an unconstrained array type may be converted to and from `System.Address`. Such usage is common in Ada 83 programs, but note that `Ada.Address_To_Access_Conversions` is the preferred method of performing such conversions in Ada 95 and Ada 2005. Neither unchecked conversion nor `Ada.Address_To_Access_Conversions` should be used in conjunction with pointers to unconstrained objects, since the bounds information cannot be handled correctly in this case.

`Ada.Unchecked_Deallocation` (13.11.2)

This generic package allows explicit freeing of storage previously allocated by use of an allocator.

`Ada.Wide_Text_IO` (A.11)

This package is similar to `Ada.Text_IO`, except that the external file supports wide character representations, and the internal types are `Wide_Character` and `Wide_String` instead of `Character` and `String`. It contains generic subpackages listed next.

`Ada.Wide_Text_IO.Decimal_IO`

Provides input-output facilities for decimal fixed-point types

`Ada.Wide_Text_IO.Enumeration_IO`

Provides input-output facilities for enumeration types.

`Ada.Wide_Text_IO.Fixed_IO`

Provides input-output facilities for ordinary fixed-point types.

`Ada.Wide_Text_IO.Float_IO`

Provides input-output facilities for float types. The following predefined instantiations of this generic package are available:

```
Short_Float
    Short_Float_Wide_Text_IO

Float      Float_Wide_Text_IO

Long_Float
    Long_Float_Wide_Text_IO
```

`Ada.Wide_Text_IO.Integer_IO`

Provides input-output facilities for integer types. The following predefined instantiations of this generic package are available:

```
Short_Short_Integer
    Ada.Short_Short_Integer_Wide_Text_IO

Short_Integer
    Ada.Short_Integer_Wide_Text_IO
```

```

Integer      Ada.Integer_Wide_Text_IO
Long_Integer
              Ada.Long_Integer_Wide_Text_IO

Long_Long_Integer
              Ada.Long_Long_Integer_Wide_Text_IO

Ada.Wide_Text_IO.Modular_IO
  Provides input-output facilities for modular (unsigned) types

Ada.Wide_Text_IO.Complex_IO (G.1.3)
  This package is similar to Ada.Text_IO.Complex_IO, except that the external
  file supports wide character representations.

Ada.Wide_Text_IO.Editing (F.3.4)
  This package is similar to Ada.Text_IO.Editing, except that the types are
  Wide_Character and Wide_String instead of Character and String.

Ada.Wide_Text_IO.Streams (A.12.3)
  This package is similar to Ada.Text_IO.Streams, except that the types are
  Wide_Character and Wide_String instead of Character and String.

Ada.Wide_Wide_Text_IO (A.11)
  This package is similar to Ada.Text_IO, except that the external file supports
  wide character representations, and the internal types are Wide_Character and
  Wide_String instead of Character and String. It contains generic subpack-
  ages listed next.

Ada.Wide_Wide_Text_IO.Decimal_IO
  Provides input-output facilities for decimal fixed-point types

Ada.Wide_Wide_Text_IO.Enumeration_IO
  Provides input-output facilities for enumeration types.

Ada.Wide_Wide_Text_IO.Fixed_IO
  Provides input-output facilities for ordinary fixed-point types.

Ada.Wide_Wide_Text_IO.Float_IO
  Provides input-output facilities for float types. The following predefined instan-
  tiations of this generic package are available:

    Short_Float
      Short_Float_Wide_Wide_Text_IO

    Float      Float_Wide_Wide_Text_IO

    Long_Float
      Long_Float_Wide_Wide_Text_IO

Ada.Wide_Wide_Text_IO.Integer_IO
  Provides input-output facilities for integer types. The following predefined in-
  stantiations of this generic package are available:

    Short_Short_Integer
      Ada.Short_Short_Integer_Wide_Wide_Text_IO

```

```
Short_Integer
    Ada.Short_Integer_Wide_Wide_Text_IO

Integer    Ada.Integer_Wide_Wide_Text_IO

Long_Integer
    Ada.Long_Integer_Wide_Wide_Text_IO

Long_Long_Integer
    Ada.Long_Long_Integer_Wide_Wide_Text_IO
```

`Ada.Wide_Wide_Text_IO.Modular_IO`

Provides input-output facilities for modular (unsigned) types

`Ada.Wide_Wide_Text_IO.Complex_IO` (G.1.3)

This package is similar to `Ada.Text_IO.Complex_IO`, except that the external file supports wide character representations.

`Ada.Wide_Wide_Text_IO.Editing` (F.3.4)

This package is similar to `Ada.Text_IO.Editing`, except that the types are `Wide_Character` and `Wide_String` instead of `Character` and `String`.

`Ada.Wide_Wide_Text_IO.Streams` (A.12.3)

This package is similar to `Ada.Text_IO.Streams`, except that the types are `Wide_Character` and `Wide_String` instead of `Character` and `String`.

8 The Implementation of Standard I/O

GNAT implements all the required input-output facilities described in A.6 through A.14. These sections of the Ada Reference Manual describe the required behavior of these packages from the Ada point of view, and if you are writing a portable Ada program that does not need to know the exact manner in which Ada maps to the outside world when it comes to reading or writing external files, then you do not need to read this chapter. As long as your files are all regular files (not pipes or devices), and as long as you write and read the files only from Ada, the description in the Ada Reference Manual is sufficient.

However, if you want to do input-output to pipes or other devices, such as the keyboard or screen, or if the files you are dealing with are either generated by some other language, or to be read by some other language, then you need to know more about the details of how the GNAT implementation of these input-output facilities behaves.

In this chapter we give a detailed description of exactly how GNAT interfaces to the file system. As always, the sources of the system are available to you for answering questions at an even more detailed level, but for most purposes the information in this chapter will suffice.

Another reason that you may need to know more about how input-output is implemented arises when you have a program written in mixed languages where, for example, files are shared between the C and Ada sections of the same program. GNAT provides some additional facilities, in the form of additional child library packages, that facilitate this sharing, and these additional facilities are also described in this chapter.

8.1 Standard I/O Packages

The Standard I/O packages described in Annex A for

- `Ada.Text_IO`
- `Ada.Text_IO.Complex_IO`
- `Ada.Text_IO.Text_Streams`
- `Ada.Wide_Text_IO`
- `Ada.Wide_Text_IO.Complex_IO`
- `Ada.Wide_Text_IO.Text_Streams`
- `Ada.Wide_Wide_Text_IO`
- `Ada.Wide_Wide_Text_IO.Complex_IO`
- `Ada.Wide_Wide_Text_IO.Text_Streams`
- `Ada.Stream_IO`
- `Ada.Sequential_IO`
- `Ada.Direct_IO`

are implemented using the C library streams facility; where

- All files are opened using `fopen`.
- All input/output operations use `fread/fwrite`.

There is no internal buffering of any kind at the Ada library level. The only buffering is that provided at the system level in the implementation of the library routines that support streams. This facilitates shared use of these streams by mixed language programs. Note though that system level buffering is explicitly enabled at elaboration of the standard I/O packages and that can have an impact on mixed language programs, in particular those using I/O before calling the Ada elaboration routine (e.g. `adainit`). It is recommended to call the Ada elaboration routine before performing any I/O or when impractical, flush the common I/O streams and in particular `Standard.Output` before elaborating the Ada code.

8.2 FORM Strings

The format of a FORM string in GNAT is:

```
"keyword=value,keyword=value,...,keyword=value"
```

where letters may be in upper or lower case, and there are no spaces between values. The order of the entries is not important. Currently the following keywords defined.

```
TEXT_TRANSLATION=[YES|NO]
SHARED=[YES|NO]
WCEM=[n|h|u|s|e|8|b]
ENCODING=[UTF8|8BITS]
```

The use of these parameters is described later in this section.

8.3 Direct_IO

`Direct_IO` can only be instantiated for definite types. This is a restriction of the Ada language, which means that the records are fixed length (the length being determined by `type'Size`, rounded up to the next storage unit boundary if necessary).

The records of a `Direct_IO` file are simply written to the file in index sequence, with the first record starting at offset zero, and subsequent records following. There is no control information of any kind. For example, if 32-bit integers are being written, each record takes 4-bytes, so the record at index K starts at offset $(K-1)*4$.

There is no limit on the size of `Direct_IO` files, they are expanded as necessary to accommodate whatever records are written to the file.

8.4 Sequential_IO

`Sequential_IO` may be instantiated with either a definite (constrained) or indefinite (unconstrained) type.

For the definite type case, the elements written to the file are simply the memory images of the data values with no control information of any kind. The resulting file should be read using the same type, no validity checking is performed on input.

For the indefinite type case, the elements written consist of two parts. First is the size of the data item, written as the memory image of a `Interfaces.C.size_t` value, followed by the memory image of the data value. The resulting file can only be read using the same (unconstrained) type. Normal assignment checks are performed on these read operations, and if these checks fail, `Data_Error` is raised. In particular, in the array case, the lengths must match, and in the variant record case, if the variable for a particular read operation is constrained, the discriminants must match.

Note that it is not possible to use `Sequential_IO` to write variable length array items, and then read the data back into different length arrays. For example, the following will raise `Data_Error`:

```
package IO is new Sequential_IO (String);
F : IO.File_Type;
S : String (1..4);
...
IO.Create (F)
IO.Write (F, "hello!")
IO.Reset (F, Mode=>In_File);
IO.Read (F, S);
Put_Line (S);
```

On some Ada implementations, this will print `hell`, but the program is clearly incorrect, since there is only one element in the file, and that element is the string `hello!`.

In Ada 95 and Ada 2005, this kind of behavior can be legitimately achieved using `Stream_IO`, and this is the preferred mechanism. In particular, the above program fragment rewritten to use `Stream_IO` will work correctly.

8.5 Text_IO

`Text_IO` files consist of a stream of characters containing the following special control characters:

```
LF (line feed, 16#0A#) Line Mark
FF (form feed, 16#0C#) Page Mark
```

A canonical `Text_IO` file is defined as one in which the following conditions are met:

- The character `LF` is used only as a line mark, i.e. to mark the end of the line.
- The character `FF` is used only as a page mark, i.e. to mark the end of a page and consequently can appear only immediately following a `LF` (line mark) character.
- The file ends with either `LF` (line mark) or `LF-FF` (line mark, page mark). In the former case, the page mark is implicitly assumed to be present.

A file written using `Text_IO` will be in canonical form provided that no explicit `LF` or `FF` characters are written using `Put` or `Put_Line`. There will be no `FF` character at the end of the file unless an explicit `New_Page` operation was performed before closing the file.

A canonical `Text_IO` file that is a regular file (i.e., not a device or a pipe) can be read using any of the routines in `Text_IO`. The semantics in this case will be exactly as defined in the Ada Reference Manual, and all the routines in `Text_IO` are fully implemented.

A text file that does not meet the requirements for a canonical `Text_IO` file has one of the following:

- The file contains `FF` characters not immediately following a `LF` character.
- The file contains `LF` or `FF` characters written by `Put` or `Put_Line`, which are not logically considered to be line marks or page marks.
- The file ends in a character other than `LF` or `FF`, i.e. there is no explicit line mark or page mark at the end of the file.

`Text_IO` can be used to read such non-standard text files but subprograms to do with line or page numbers do not have defined meanings. In particular, a `FF` character that does not

follow a LF character may or may not be treated as a page mark from the point of view of page and line numbering. Every LF character is considered to end a line, and there is an implied LF character at the end of the file.

8.5.1 Stream Pointer Positioning

`Ada.Text_IO` has a definition of current position for a file that is being read. No internal buffering occurs in `Text_IO`, and usually the physical position in the stream used to implement the file corresponds to this logical position defined by `Text_IO`. There are two exceptions:

- After a call to `End_Of_Page` that returns `True`, the stream is positioned past the LF (line mark) that precedes the page mark. `Text_IO` maintains an internal flag so that subsequent read operations properly handle the logical position which is unchanged by the `End_Of_Page` call.
- After a call to `End_Of_File` that returns `True`, if the `Text_IO` file was positioned before the line mark at the end of file before the call, then the logical position is unchanged, but the stream is physically positioned right at the end of file (past the line mark, and past a possible page mark following the line mark. Again `Text_IO` maintains internal flags so that subsequent read operations properly handle the logical position.

These discrepancies have no effect on the observable behavior of `Text_IO`, but if a single Ada stream is shared between a C program and Ada program, or shared (using `'shared=yes'` in the form string) between two Ada files, then the difference may be observable in some situations.

8.5.2 Reading and Writing Non-Regular Files

A non-regular file is a device (such as a keyboard), or a pipe. `Text_IO` can be used for reading and writing. Writing is not affected and the sequence of characters output is identical to the normal file case, but for reading, the behavior of `Text_IO` is modified to avoid undesirable look-ahead as follows:

An input file that is not a regular file is considered to have no page marks. Any `Ascii.FF` characters (the character normally used for a page mark) appearing in the file are considered to be data characters. In particular:

- `Get_Line` and `Skip_Line` do not test for a page mark following a line mark. If a page mark appears, it will be treated as a data character.
- This avoids the need to wait for an extra character to be typed or entered from the pipe to complete one of these operations.
- `End_Of_Page` always returns `False`
- `End_Of_File` will return `False` if there is a page mark at the end of the file.

Output to non-regular files is the same as for regular files. Page marks may be written to non-regular files using `New_Page`, but as noted above they will not be treated as page marks on input if the output is piped to another Ada program.

Another important discrepancy when reading non-regular files is that the end of file indication is not “sticky”. If an end of file is entered, e.g. by pressing the EOT key, then end of file is signaled once (i.e. the test `End_Of_File` will yield `True`, or a read will raise `End_Error`), but then reading can resume to read data past that end of file indication, until another end of file indication is entered.

8.5.3 Get_Immediate

Get_Immediate returns the next character (including control characters) from the input file. In particular, Get_Immediate will return LF or FF characters used as line marks or page marks. Such operations leave the file positioned past the control character, and it is thus not treated as having its normal function. This means that page, line and column counts after this kind of Get_Immediate call are set as though the mark did not occur. In the case where a Get_Immediate leaves the file positioned between the line mark and page mark (which is not normally possible), it is undefined whether the FF character will be treated as a page mark.

8.5.4 Treating Text_IO Files as Streams

The package `Text_IO.Streams` allows a Text_IO file to be treated as a stream. Data written to a Text_IO file in this stream mode is binary data. If this binary data contains bytes `16#0A#` (LF) or `16#0C#` (FF), the resulting file may have non-standard format. Similarly if read operations are used to read from a Text_IO file treated as a stream, then LF and FF characters may be skipped and the effect is similar to that described above for Get_Immediate.

8.5.5 Text_IO Extensions

A package `GNAT.IO_Aux` in the GNAT library provides some useful extensions to the standard `Text_IO` package:

- function `File.Exists (Name : String)` return Boolean; Determines if a file of the given name exists.
- function `Get.Line` return String; Reads a string from the standard input file. The value returned is exactly the length of the line that was read.
- function `Get.Line (File : Ada.Text_IO.File_Type)` return String; Similar, except that the parameter `File` specifies the file from which the string is to be read.

8.5.6 Text_IO Facilities for Unbounded Strings

The package `Ada.Strings.Unbounded.Text_IO` in library files `a-suteio.ads`/`adb` contains some GNAT-specific subprograms useful for Text_IO operations on unbounded strings:

- function `Get.Line (File : File_Type)` return Unbounded_String; Reads a line from the specified file and returns the result as an unbounded string.
- procedure `Put (File : File_Type; U : Unbounded_String)`; Writes the value of the given unbounded string to the specified file Similar to the effect of `Put (To_String (U))` except that an extra copy is avoided.
- procedure `Put_Line (File : File_Type; U : Unbounded_String)`; Writes the value of the given unbounded string to the specified file, followed by a `New_Line`. Similar to the effect of `Put_Line (To_String (U))` except that an extra copy is avoided.

In the above procedures, `File` is of type `Ada.Text_IO.File_Type` and is optional. If the parameter is omitted, then the standard input or output file is referenced as appropriate.

The package `Ada.Strings.Wide_Unbounded.Wide_Text_IO` in library files `'a-swuwti.ads'` and `'a-swuwti.adb'` provides similar extended `Wide_Text_IO` functionality for unbounded wide strings.

The package `Ada.Strings.Wide_Wide_Unbounded.Wide_Wide_Text_IO` in library files `'a-szuzti.ads'` and `'a-szuzti.adb'` provides similar extended `Wide_Wide_Text_IO` functionality for unbounded wide wide strings.

8.6 Wide_Text_IO

`Wide_Text_IO` is similar in most respects to `Text_IO`, except that both input and output files may contain special sequences that represent wide character values. The encoding scheme for a given file may be specified using a FORM parameter:

`WCEM=x`

as part of the FORM string (`WCEM` = wide character encoding method), where `x` is one of the following characters

'h'	Hex ESC encoding
'u'	Upper half encoding
's'	Shift-JIS encoding
'e'	EUC Encoding
'8'	UTF-8 encoding
'b'	Brackets encoding

The encoding methods match those that can be used in a source program, but there is no requirement that the encoding method used for the source program be the same as the encoding method used for files, and different files may use different encoding methods.

The default encoding method for the standard files, and for opened files for which no `WCEM` parameter is given in the FORM string matches the wide character encoding specified for the main program (the default being brackets encoding if no coding method was specified with `-gnatW`).

Hex Coding

In this encoding, a wide character is represented by a five character sequence:

`ESC a b c d`

where `a`, `b`, `c`, `d` are the four hexadecimal characters (using upper case letters) of the wide character code. For example, `ESC A345` is used to represent the wide character with code `16#A345#`. This scheme is compatible with use of the full `Wide_Character` set.

Upper Half Coding

The wide character with encoding `16#abcd#`, where the upper bit is on (i.e. `a` is in the range 8-F) is represented as two bytes `16#ab#` and `16#cd#`. The second byte may never be a format control character, but is not required to be in the upper half. This method can be also used for shift-JIS or EUC where the internal coding matches the external coding.

Shift JIS Coding

A wide character is represented by a two character sequence `16#ab#` and `16#cd#`, with the restrictions described for upper half encoding as described

above. The internal character code is the corresponding JIS character according to the standard algorithm for Shift-JIS conversion. Only characters defined in the JIS code set table can be used with this encoding method.

EUC Coding

A wide character is represented by a two character sequence `16#ab#` and `16#cd#`, with both characters being in the upper half. The internal character code is the corresponding JIS character according to the EUC encoding algorithm. Only characters defined in the JIS code set table can be used with this encoding method.

UTF-8 Coding

A wide character is represented using UCS Transformation Format 8 (UTF-8) as defined in Annex R of ISO 10646-1/Am.2. Depending on the character value, the representation is a one, two, or three byte sequence:

```
16#0000#-16#007f#: 2#0xxxxxx#
16#0080#-16#07ff#: 2#110xxxxx# 2#10xxxxxx#
16#0800#-16#ffff#: 2#1110xxxx# 2#10xxxxxx# 2#10xxxxxx#
```

where the xxx bits correspond to the left-padded bits of the 16-bit character value. Note that all lower half ASCII characters are represented as ASCII bytes and all upper half characters and other wide characters are represented as sequences of upper-half (The full UTF-8 scheme allows for encoding 31-bit characters as 6-byte sequences, but in this implementation, all UTF-8 sequences of four or more bytes length will raise a `Constraint_Error`, as will all invalid UTF-8 sequences.)

Brackets Coding

In this encoding, a wide character is represented by the following eight character sequence:

```
[ " a b c d " ]
```

where `a`, `b`, `c`, `d` are the four hexadecimal characters (using uppercase letters) of the wide character code. For example, `["A345"]` is used to represent the wide character with code `16#A345#`. This scheme is compatible with use of the full `Wide_Character` set. On input, brackets coding can also be used for upper half characters, e.g. `["C1"]` for lower case `a`. However, on output, brackets notation is only used for wide characters with a code greater than `16#FF#`.

Note that brackets coding is not normally used in the context of `Wide_Text_IO` or `Wide_Wide_Text_IO`, since it is really just designed as a portable way of encoding source files. In the context of `Wide_Text_IO` or `Wide_Wide_Text_IO`, it can only be used if the file does not contain any instance of the left bracket character other than to encode wide character values using the brackets encoding method. In practice it is expected that some standard wide character encoding method such as UTF-8 will be used for text input output.

If brackets notation is used, then any occurrence of a left bracket in the input file which is not the start of a valid wide character sequence will cause `Constraint_Error` to be raised. It is possible to encode a left bracket as `["5B"]` and `Wide_Text_IO` and `Wide_Wide_Text_IO` input will interpret this as a left bracket.

However, when a left bracket is output, it will be output as a left bracket and not as ["5B"]. We make this decision because for normal use of `Wide_Text_IO` for outputting messages, it is unpleasant to clobber left brackets. For example, if we write:

```
Put_Line ("Start of output [first run]");
```

we really do not want to have the left bracket in this message clobbered so that the output reads:

```
Start of output ["5B"]first run]
```

In practice brackets encoding is reasonably useful for normal `Put_Line` use since we won't get confused between left brackets and wide character sequences in the output. But for input, or when files are written out and read back in, it really makes better sense to use one of the standard encoding methods such as UTF-8.

For the coding schemes other than UTF-8, Hex, or Brackets encoding, not all wide character values can be represented. An attempt to output a character that cannot be represented using the encoding scheme for the file causes `Constraint_Error` to be raised. An invalid wide character sequence on input also causes `Constraint_Error` to be raised.

8.6.1 Stream Pointer Positioning

`Ada.Wide_Text_IO` is similar to `Ada.Text_IO` in its handling of stream pointer positioning (see [Section 8.5 \[Text_IO\]](#), page 169). There is one additional case:

If `Ada.Wide_Text_IO.Look_Ahead` reads a character outside the normal lower ASCII set (i.e. a character in the range:

```
Wide_Character'Val (16#0080#) .. Wide_Character'Val (16#FFFF#)
```

then although the logical position of the file pointer is unchanged by the `Look_Ahead` call, the stream is physically positioned past the wide character sequence. Again this is to avoid the need for buffering or backup, and all `Wide_Text_IO` routines check the internal indication that this situation has occurred so that this is not visible to a normal program using `Wide_Text_IO`. However, this discrepancy can be observed if the wide text file shares a stream with another file.

8.6.2 Reading and Writing Non-Regular Files

As in the case of `Text_IO`, when a non-regular file is read, it is assumed that the file contains no page marks (any form characters are treated as data characters), and `End_Of_Page` always returns `False`. Similarly, the end of file indication is not sticky, so it is possible to read beyond an end of file.

8.7 Wide_Wide_Text_IO

`Wide_Wide_Text_IO` is similar in most respects to `Text_IO`, except that both input and output files may contain special sequences that represent wide wide character values. The encoding scheme for a given file may be specified using a FORM parameter:

```
WCEM=x
```

as part of the FORM string (`WCEM` = wide character encoding method), where `x` is one of the following characters

'h'	Hex ESC encoding
'u'	Upper half encoding
's'	Shift-JIS encoding
'e'	EUC Encoding
'8'	UTF-8 encoding
'b'	Brackets encoding

The encoding methods match those that can be used in a source program, but there is no requirement that the encoding method used for the source program be the same as the encoding method used for files, and different files may use different encoding methods.

The default encoding method for the standard files, and for opened files for which no WCEM parameter is given in the FORM string matches the wide character encoding specified for the main program (the default being brackets encoding if no coding method was specified with -gnatW).

UTF-8 Coding

A wide character is represented using UCS Transformation Format 8 (UTF-8) as defined in Annex R of ISO 10646-1/Am.2. Depending on the character value, the representation is a one, two, three, or four byte sequence:

```
16#000000#-16#00007f#: 2#0xxxxxxx#
16#000080#-16#0007ff#: 2#110xxxxx# 2#10xxxxxx#
16#000800#-16#00ffff#: 2#1110xxxx# 2#10xxxxxx# 2#10xxxxxx#
16#010000#-16#10ffff#: 2#11110xxx# 2#10xxxxxx# 2#10xxxxxx# 2#10xxxxxx#
```

where the xxx bits correspond to the left-padded bits of the 21-bit character value. Note that all lower half ASCII characters are represented as ASCII bytes and all upper half characters and other wide characters are represented as sequences of upper-half characters.

Brackets Coding

In this encoding, a wide wide character is represented by the following eight character sequence if is in wide character range

```
[ " a b c d " ]
```

and by the following ten character sequence if not

```
[ " a b c d e f " ]
```

where a, b, c, d, e, and f are the four or six hexadecimal characters (using uppercase letters) of the wide wide character code. For example, ["01A345"] is used to represent the wide wide character with code 16#01A345#.

This scheme is compatible with use of the full Wide_Wide_Character set. On input, brackets coding can also be used for upper half characters, e.g. ["C1"] for lower case a. However, on output, brackets notation is only used for wide characters with a code greater than 16#FF#.

If is also possible to use the other Wide_Character encoding methods, such as Shift-JIS, but the other schemes cannot support the full range of wide wide characters. An attempt to output a character that cannot be represented using the encoding scheme for the file causes Constraint_Error to be raised. An invalid wide character sequence on input also causes Constraint_Error to be raised.

8.7.1 Stream Pointer Positioning

`Ada.Wide_Wide_Text_IO` is similar to `Ada.Text_IO` in its handling of stream pointer positioning (see [Section 8.5 \[Text_IO\]](#), page 169). There is one additional case:

If `Ada.Wide_Wide_Text_IO.Look_Ahead` reads a character outside the normal lower ASCII set (i.e. a character in the range:

```
Wide_Wide_Character'Val (16#0080#) .. Wide_Wide_Character'Val (16#10FFFF#)
```

then although the logical position of the file pointer is unchanged by the `Look_Ahead` call, the stream is physically positioned past the wide character sequence. Again this is to avoid the need for buffering or backup, and all `Wide_Wide_Text_IO` routines check the internal indication that this situation has occurred so that this is not visible to a normal program using `Wide_Wide_Text_IO`. However, this discrepancy can be observed if the wide text file shares a stream with another file.

8.7.2 Reading and Writing Non-Regular Files

As in the case of `Text_IO`, when a non-regular file is read, it is assumed that the file contains no page marks (any form characters are treated as data characters), and `End_Of_Page` always returns `False`. Similarly, the end of file indication is not sticky, so it is possible to read beyond an end of file.

8.8 Stream_IO

A stream file is a sequence of bytes, where individual elements are written to the file as described in the Ada Reference Manual. The type `Stream_Element` is simply a byte. There are two ways to read or write a stream file.

- The operations `Read` and `Write` directly read or write a sequence of stream elements with no control information.
- The stream attributes applied to a stream file transfer data in the manner described for stream attributes.

8.9 Text Translation

`'Text_Translation=xxx'` may be used as the `Form` parameter passed to `Text_IO.Create` and `Text_IO.Open`: `'Text_Translation=Yes'` is the default, which means to translate LF to/from CR/LF on Windows systems. `'Text_Translation=No'` disables this translation; i.e. it uses binary mode. For output files, `'Text_Translation=No'` may be used to create Unix-style files on Windows. `'Text_Translation=xxx'` has no effect on Unix systems.

8.10 Shared Files

Section A.14 of the Ada Reference Manual allows implementations to provide a wide variety of behavior if an attempt is made to access the same external file with two or more internal files.

To provide a full range of functionality, while at the same time minimizing the problems of portability caused by this implementation dependence, GNAT handles file sharing as follows:

- In the absence of a `'shared=xxx'` form parameter, an attempt to open two or more files with the same full name is considered an error and is not supported. The exception

`Use_Error` will be raised. Note that a file that is not explicitly closed by the program remains open until the program terminates.

- If the form parameter `'shared=no'` appears in the form string, the file can be opened or created with its own separate stream identifier, regardless of whether other files sharing the same external file are opened. The exact effect depends on how the C stream routines handle multiple accesses to the same external files using separate streams.
- If the form parameter `'shared=yes'` appears in the form string for each of two or more files opened using the same full name, the same stream is shared between these files, and the semantics are as described in Ada Reference Manual, Section A.14.

When a program that opens multiple files with the same name is ported from another Ada compiler to GNAT, the effect will be that `Use_Error` is raised.

The documentation of the original compiler and the documentation of the program should then be examined to determine if file sharing was expected, and `'shared=xxx'` parameters added to `Open` and `Create` calls as required.

When a program is ported from GNAT to some other Ada compiler, no special attention is required unless the `'shared=xxx'` form parameter is used in the program. In this case, you must examine the documentation of the new compiler to see if it supports the required file sharing semantics, and form strings modified appropriately. Of course it may be the case that the program cannot be ported if the target compiler does not support the required functionality. The best approach in writing portable code is to avoid file sharing (and hence the use of the `'shared=xxx'` parameter in the form string) completely.

One common use of file sharing in Ada 83 is the use of instantiations of `Sequential_IO` on the same file with different types, to achieve heterogeneous input-output. Although this approach will work in GNAT if `'shared=yes'` is specified, it is preferable in Ada to use `Stream_IO` for this purpose (using the stream attributes)

8.11 Filenames encoding

An encoding form parameter can be used to specify the filename encoding `'encoding=xxx'`.

- If the form parameter `'encoding=utf8'` appears in the form string, the filename must be encoded in UTF-8.
- If the form parameter `'encoding=8bits'` appears in the form string, the filename must be a standard 8bits string.

In the absence of a `'encoding=xxx'` form parameter, the encoding is controlled by the `'GNAT_CODE_PAGE'` environment variable. And if not set `'utf8'` is assumed.

`'CP_ACP'` The current system Windows ANSI code page.

`'CP_UTF8'` UTF-8 encoding

This encoding form parameter is only supported on the Windows platform. On the other Operating Systems the run-time is supporting UTF-8 natively.

8.12 Open Modes

Open and Create calls result in a call to `fopen` using the mode shown in the following table:

Open and Create Call Modes		
	OPEN	CREATE
Append_File	"r+"	"w+"
In_File	"r"	"w+"
Out_File (Direct_IO)	"r+"	"w"
Out_File (all other cases)	"w"	"w"
Inout_File	"r+"	"w+"

If text file translation is required, then either 'b' or 't' is added to the mode, depending on the setting of Text. Text file translation refers to the mapping of CR/LF sequences in an external file to LF characters internally. This mapping only occurs in DOS and DOS-like systems, and is not relevant to other systems.

A special case occurs with Stream_IO. As shown in the above table, the file is initially opened in 'r' or 'w' mode for the In_File and Out_File cases. If a Set_Mode operation subsequently requires switching from reading to writing or vice-versa, then the file is reopened in 'r+' mode to permit the required operation.

8.13 Operations on C Streams

The package Interfaces.C.Streams provides an Ada program with direct access to the C library functions for operations on C streams:

```
package Interfaces.C.Streams is
  -- Note: the reason we do not use the types that are in
  -- Interfaces.C is that we want to avoid dragging in the
  -- code in this unit if possible.
  subtype chars is System.Address;
  -- Pointer to null-terminated array of characters
  subtype FILEs is System.Address;
  -- Corresponds to the C type FILE*
  subtype voids is System.Address;
  -- Corresponds to the C type void*
  subtype int is Integer;
  subtype long is Long_Integer;
  -- Note: the above types are subtypes deliberately, and it
  -- is part of this spec that the above correspondences are
  -- guaranteed. This means that it is legitimate to, for
  -- example, use Integer instead of int. We provide these
  -- synonyms for clarity, but in some cases it may be
  -- convenient to use the underlying types (for example to
  -- avoid an unnecessary dependency of a spec on the spec
  -- of this unit).
  type size_t is mod 2 ** Standard'Address_Size;
  NULL_Stream : constant FILEs;
  -- Value returned (NULL in C) to indicate an
  -- fdopen/fopen/tmpfile error
  -----
  -- Constants Defined in stdio.h --
  -----
  EOF : constant int;
  -- Used by a number of routines to indicate error or
```

```

-- end of file
IOFBF : constant int;
IOLBF : constant int;
IONBF : constant int;
-- Used to indicate buffering mode for setvbuf call
SEEK_CUR : constant int;
SEEK_END : constant int;
SEEK_SET : constant int;
-- Used to indicate origin for fseek call
function stdin return FILEs;
function stdout return FILEs;
function stderr return FILEs;
-- Streams associated with standard files
-----
-- Standard C functions --
-----
-- The functions selected below are ones that are
-- available in DOS, OS/2, UNIX and Xenix (but not
-- necessarily in ANSI C). These are very thin interfaces
-- which copy exactly the C headers. For more
-- documentation on these functions, see the Microsoft C
-- "Run-Time Library Reference" (Microsoft Press, 1990,
-- ISBN 1-55615-225-6), which includes useful information
-- on system compatibility.
procedure clearerr (stream : FILEs);
function fclose (stream : FILEs) return int;
function fdopen (handle : int; mode : chars) return FILEs;
function feof (stream : FILEs) return int;
function ferror (stream : FILEs) return int;
function fflush (stream : FILEs) return int;
function fgetc (stream : FILEs) return int;
function fgets (strng : chars; n : int; stream : FILEs)
    return chars;
function fileno (stream : FILEs) return int;
function fopen (filename : chars; Mode : chars)
    return FILEs;
-- Note: to maintain target independence, use
-- text_translation_required, a boolean variable defined in
-- a-sysdep.c to deal with the target dependent text
-- translation requirement. If this variable is set,
-- then b/t should be appended to the standard mode
-- argument to set the text translation mode off or on
-- as required.
function fputc (C : int; stream : FILEs) return int;
function fputs (Strng : chars; Stream : FILEs) return int;
function fread
    (buffer : voids;
     size : size_t;
     count : size_t;
     stream : FILEs)
    return size_t;
function freopen
    (filename : chars;
     mode : chars;
     stream : FILEs)
    return FILEs;
function fseek
    (stream : FILEs;

```

```

    offset : long;
    origin : int)
    return int;
function ftell (stream : FILEs) return long;
function fwrite
  (buffer : voids;
   size : size_t;
   count : size_t;
   stream : FILEs)
  return size_t;
function isatty (handle : int) return int;
procedure mktemp (template : chars);
-- The return value (which is just a pointer to template)
-- is discarded
procedure rewind (stream : FILEs);
function rmtmp return int;
function setvbuf
  (stream : FILEs;
   buffer : chars;
   mode : int;
   size : size_t)
  return int;

function tmpfile return FILEs;
function ungetc (c : int; stream : FILEs) return int;
function unlink (filename : chars) return int;
-----
-- Extra functions --
-----
-- These functions supply slightly thicker bindings than
-- those above. They are derived from functions in the
-- C Run-Time Library, but may do a bit more work than
-- just directly calling one of the Library functions.
function is_regular_file (handle : int) return int;
-- Tests if given handle is for a regular file (result 1)
-- or for a non-regular file (pipe or device, result 0).
-----
-- Control of Text/Binary Mode --
-----
-- If text_translation_required is true, then the following
-- functions may be used to dynamically switch a file from
-- binary to text mode or vice versa. These functions have
-- no effect if text_translation_required is false (i.e. in
-- normal UNIX mode). Use fileno to get a stream handle.
procedure set_binary_mode (handle : int);
procedure set_text_mode (handle : int);
-----
-- Full Path Name support --
-----
procedure full_name (nam : chars; buffer : chars);
-- Given a NUL terminated string representing a file
-- name, returns in buffer a NUL terminated string
-- representing the full path name for the file name.
-- On systems where it is relevant the drive is also
-- part of the full path name. It is the responsibility
-- of the caller to pass an actual parameter for buffer
-- that is big enough for any full path name. Use
-- max_path_len given below as the size of buffer.

```

```

max_path_len : integer;
-- Maximum length of an allowable full path name on the
-- system, including a terminating NUL character.
end Interfaces.C_Streams;

```

8.14 Interfacing to C Streams

The packages in this section permit interfacing Ada files to C Stream operations.

```

with Interfaces.C_Streams;
package Ada.Sequential_IO.C_Streams is
  function C_Stream (F : File_Type)
    return Interfaces.C_Streams.FILES;
  procedure Open
    (File : in out File_Type;
     Mode : in File_Mode;
     C_Stream : in Interfaces.C_Streams.FILES;
     Form : in String := "");
end Ada.Sequential_IO.C_Streams;

with Interfaces.C_Streams;
package Ada.Direct_IO.C_Streams is
  function C_Stream (F : File_Type)
    return Interfaces.C_Streams.FILES;
  procedure Open
    (File : in out File_Type;
     Mode : in File_Mode;
     C_Stream : in Interfaces.C_Streams.FILES;
     Form : in String := "");
end Ada.Direct_IO.C_Streams;

with Interfaces.C_Streams;
package Ada.Text_IO.C_Streams is
  function C_Stream (F : File_Type)
    return Interfaces.C_Streams.FILES;
  procedure Open
    (File : in out File_Type;
     Mode : in File_Mode;
     C_Stream : in Interfaces.C_Streams.FILES;
     Form : in String := "");
end Ada.Text_IO.C_Streams;

with Interfaces.C_Streams;
package Ada.Wide_Text_IO.C_Streams is
  function C_Stream (F : File_Type)
    return Interfaces.C_Streams.FILES;
  procedure Open
    (File : in out File_Type;
     Mode : in File_Mode;
     C_Stream : in Interfaces.C_Streams.FILES;
     Form : in String := "");
end Ada.Wide_Text_IO.C_Streams;

with Interfaces.C_Streams;
package Ada.Wide_Wide_Text_IO.C_Streams is
  function C_Stream (F : File_Type)
    return Interfaces.C_Streams.FILES;
  procedure Open
    (File : in out File_Type;

```

```

        Mode : in File_Mode;
        C_Stream : in Interfaces.C_Streams.FILES;
        Form : in String := "");
end Ada.Wide_Wide_Text_IO.C_Streams;

with Interfaces.C_Streams;
package Ada.Stream_IO.C_Streams is
  function C_Stream (F : File_Type)
    return Interfaces.C_Streams.FILES;
  procedure Open
    (File : in out File_Type;
     Mode : in File_Mode;
     C_Stream : in Interfaces.C_Streams.FILES;
     Form : in String := "");
end Ada.Stream_IO.C_Streams;

```

In each of these six packages, the `C_Stream` function obtains the `FILE` pointer from a currently opened Ada file. It is then possible to use the `Interfaces.C_Streams` package to operate on this stream, or the stream can be passed to a C program which can operate on it directly. Of course the program is responsible for ensuring that only appropriate sequences of operations are executed.

One particular use of relevance to an Ada program is that the `setvbuf` function can be used to control the buffering of the stream used by an Ada file. In the absence of such a call the standard default buffering is used.

The `Open` procedures in these packages open a file giving an existing C Stream instead of a file name. Typically this stream is imported from a C program, allowing an Ada file to operate on an existing C file.

9 The GNAT Library

The GNAT library contains a number of general and special purpose packages. It represents functionality that the GNAT developers have found useful, and which is made available to GNAT users. The packages described here are fully supported, and upwards compatibility will be maintained in future releases, so you can use these facilities with the confidence that the same functionality will be available in future releases.

The chapter here simply gives a brief summary of the facilities available. The full documentation is found in the spec file for the package. The full sources of these library packages, including both spec and body, are provided with all GNAT releases. For example, to find out the full specifications of the SPITBOL pattern matching capability, including a full tutorial and extensive examples, look in the ‘`g-spipat.ads`’ file in the library.

For each entry here, the package name (as it would appear in a `with` clause) is given, followed by the name of the corresponding spec file in parentheses. The packages are children in four hierarchies, `Ada`, `Interfaces`, `System`, and `GNAT`, the latter being a GNAT-specific hierarchy.

Note that an application program should only use packages in one of these four hierarchies if the package is defined in the Ada Reference Manual, or is listed in this section of the GNAT Programmers Reference Manual. All other units should be considered internal implementation units and should not be directly `with`ed by application code. The use of a `with` statement that references one of these internal implementation units makes an application potentially dependent on changes in versions of GNAT, and will generate a warning message.

9.1 `Ada.Characters.Latin_9` (‘`a-chlat9.ads`’)

This child of `Ada.Characters` provides a set of definitions corresponding to those in the RM-defined package `Ada.Characters.Latin_1` but with the few modifications required for `Latin_9`. The provision of such a package is specifically authorized by the Ada Reference Manual (RM A.3.3(27)).

9.2 `Ada.Characters.Wide_Latin_1` (‘`a-cwila1.ads`’)

This child of `Ada.Characters` provides a set of definitions corresponding to those in the RM-defined package `Ada.Characters.Latin_1` but with the types of the constants being `Wide_Character` instead of `Character`. The provision of such a package is specifically authorized by the Ada Reference Manual (RM A.3.3(27)).

9.3 `Ada.Characters.Wide_Latin_9` (‘`a-cwila1.ads`’)

This child of `Ada.Characters` provides a set of definitions corresponding to those in the GNAT defined package `Ada.Characters.Latin_9` but with the types of the constants being `Wide_Character` instead of `Character`. The provision of such a package is specifically authorized by the Ada Reference Manual (RM A.3.3(27)).

9.4 `Ada.Characters.Wide_Wide_Latin_1` ('a-chzla1.ads')

This child of `Ada.Characters` provides a set of definitions corresponding to those in the RM-defined package `Ada.Characters.Latin_1` but with the types of the constants being `Wide_Wide_Character` instead of `Character`. The provision of such a package is specifically authorized by the Ada Reference Manual (RM A.3.3(27)).

9.5 `Ada.Characters.Wide_Wide_Latin_9` ('a-chzla9.ads')

This child of `Ada.Characters` provides a set of definitions corresponding to those in the GNAT defined package `Ada.Characters.Latin_9` but with the types of the constants being `Wide_Wide_Character` instead of `Character`. The provision of such a package is specifically authorized by the Ada Reference Manual (RM A.3.3(27)).

9.6 `Ada.Command_Line.Environment` ('a-colien.ads')

This child of `Ada.Command_Line` provides a mechanism for obtaining environment values on systems where this concept makes sense.

9.7 `Ada.Command_Line.Remove` ('a-colire.ads')

This child of `Ada.Command_Line` provides a mechanism for logically removing arguments from the argument list. Once removed, an argument is not visible to further calls on the subprograms in `Ada.Command_Line` will not see the removed argument.

9.8 `Ada.Command_Line.Response_File` ('a-clrefi.ads')

This child of `Ada.Command_Line` provides a mechanism facilities for getting command line arguments from a text file, called a "response file". Using a response file allow passing a set of arguments to an executable longer than the maximum allowed by the system on the command line.

9.9 `Ada.Direct_IO.C_Streams` ('a-diocst.ads')

This package provides subprograms that allow interfacing between C streams and `Direct_IO`. The stream identifier can be extracted from a file opened on the Ada side, and an Ada file can be constructed from a stream opened on the C side.

9.10 `Ada.Exceptions.Is_Null_Occurrence` ('a-einuoc.ads')

This child subprogram provides a way of testing for the null exception occurrence (`Null_Occurrence`) without raising an exception.

9.11 `Ada.Exceptions.Last_Chance_Handler` ('a-elchha.ads')

This child subprogram is used for handling otherwise unhandled exceptions (hence the name last chance), and perform clean ups before terminating the program. Note that this subprogram never returns.

9.12 Ada.Exceptions.Traceback ('a-exctra.ads')

This child package provides the subprogram (Tracebacks) to give a traceback array of addresses based on an exception occurrence.

9.13 Ada.Sequential_IO.C_Streams ('a-siocst.ads')

This package provides subprograms that allow interfacing between C streams and Sequential_IO. The stream identifier can be extracted from a file opened on the Ada side, and an Ada file can be constructed from a stream opened on the C side.

9.14 Ada.Streams.Stream_IO.C_Streams ('a-ssicst.ads')

This package provides subprograms that allow interfacing between C streams and Stream_IO. The stream identifier can be extracted from a file opened on the Ada side, and an Ada file can be constructed from a stream opened on the C side.

9.15 Ada.Strings.Unbounded.Text_IO ('a-suteio.ads')

This package provides subprograms for Text_IO for unbounded strings, avoiding the necessity for an intermediate operation with ordinary strings.

9.16 Ada.Strings.Wide_Unbounded.Wide_Text_IO ('a-swuwti.ads')

This package provides subprograms for Text_IO for unbounded wide strings, avoiding the necessity for an intermediate operation with ordinary wide strings.

9.17 Ada.Strings.Wide_Wide_Unbounded.Wide_Wide_Text_IO ('a-szuzti.ads')

This package provides subprograms for Text_IO for unbounded wide wide strings, avoiding the necessity for an intermediate operation with ordinary wide wide strings.

9.18 Ada.Text_IO.C_Streams ('a-tiocst.ads')

This package provides subprograms that allow interfacing between C streams and Text_IO. The stream identifier can be extracted from a file opened on the Ada side, and an Ada file can be constructed from a stream opened on the C side.

9.19 Ada.Text_IO.Reset_Standard_Files ('a-tirsfi.ads')

This procedure is used to reset the status of the standard files used by Ada.Text_IO. This is useful in a situation (such as a restart in an embedded application) where the status of the files may change during execution (for example a standard input file may be redefined to be interactive).

9.20 Ada.Wide_Characters.Unicode ('a-wichun.ads')

This package provides subprograms that allow categorization of Wide_Character values according to Unicode categories.

9.21 Ada.Wide_Text_IO.C_Streams ('a-wtcstr.ads')

This package provides subprograms that allow interfacing between C streams and `Wide_Text_IO`. The stream identifier can be extracted from a file opened on the Ada side, and an Ada file can be constructed from a stream opened on the C side.

9.22 Ada.Wide_Text_IO.Reset_Standard_Files ('a-wrstfi.ads')

This procedure is used to reset the status of the standard files used by `Ada.Wide_Text_IO`. This is useful in a situation (such as a restart in an embedded application) where the status of the files may change during execution (for example a standard input file may be redefined to be interactive).

9.23 Ada.Wide_Wide_Characters.Unicode ('a-zchuni.ads')

This package provides subprograms that allow categorization of `Wide_Wide_Character` values according to Unicode categories.

9.24 Ada.Wide_Wide_Text_IO.C_Streams ('a-ztcstr.ads')

This package provides subprograms that allow interfacing between C streams and `Wide_Wide_Text_IO`. The stream identifier can be extracted from a file opened on the Ada side, and an Ada file can be constructed from a stream opened on the C side.

9.25 Ada.Wide_Wide_Text_IO.Reset_Standard_Files ('a-zrstfi.ads')

This procedure is used to reset the status of the standard files used by `Ada.Wide_Wide_Text_IO`. This is useful in a situation (such as a restart in an embedded application) where the status of the files may change during execution (for example a standard input file may be redefined to be interactive).

9.26 GNAT.Altivec ('g-altive.ads')

This is the root package of the GNAT Altivec binding. It provides definitions of constants and types common to all the versions of the binding.

9.27 GNAT.Altivec.Conversions ('g-altcon.ads')

This package provides the Vector/View conversion routines.

9.28 GNAT.Altivec.Vector_Operations ('g-alveop.ads')

This package exposes the Ada interface to the Altivec operations on vector objects. A soft emulation is included by default in the GNAT library. The hard binding is provided as a separate package. This unit is common to both bindings.

9.29 GNAT.Altivec.Vector_Types ('g-alvety.ads')

This package exposes the various vector types part of the Ada binding to Altivec facilities.

9.30 GNAT.Altivec.Vector_Views ('g-alvevi.ads')

This package provides public 'View' data types from/to which private vector representations can be converted via GNAT.Altivec.Conversions. This allows convenient access to individual vector elements and provides a simple way to initialize vector objects.

9.31 GNAT.Array_Split ('g-arrspl.ads')

Useful array-manipulation routines: given a set of separators, split an array wherever the separators appear, and provide direct access to the resulting slices.

9.32 GNAT.AWK ('g-awk.ads')

Provides AWK-like parsing functions, with an easy interface for parsing one or more files containing formatted data. The file is viewed as a database where each record is a line and a field is a data element in this line.

9.33 GNAT.Bounded_Buffers ('g-boubuf.ads')

Provides a concurrent generic bounded buffer abstraction. Instances are useful directly or as parts of the implementations of other abstractions, such as mailboxes.

9.34 GNAT.Bounded_Mailboxes ('g-boumai.ads')

Provides a thread-safe asynchronous intertask mailbox communication facility.

9.35 GNAT.Bubble_Sort ('g-bubsor.ads')

Provides a general implementation of bubble sort usable for sorting arbitrary data items. Exchange and comparison procedures are provided by passing access-to-procedure values.

9.36 GNAT.Bubble_Sort_A ('g-busora.ads')

Provides a general implementation of bubble sort usable for sorting arbitrary data items. Move and comparison procedures are provided by passing access-to-procedure values. This is an older version, retained for compatibility. Usually GNAT.Bubble_Sort will be preferable.

9.37 GNAT.Bubble_Sort_G ('g-busorg.ads')

Similar to Bubble_Sort_A except that the move and sorting procedures are provided as generic parameters, this improves efficiency, especially if the procedures can be inlined, at the expense of duplicating code for multiple instantiations.

9.38 GNAT.Byte_Order_Mark ('g-byorma.ads')

Provides a routine which given a string, reads the start of the string to see whether it is one of the standard byte order marks (BOM's) which signal the encoding of the string. The routine includes detection of special XML sequences for various UCS input formats.

9.39 GNAT.Byte_Swapping ('g-bytswa.ads')

General routines for swapping the bytes in 2-, 4-, and 8-byte quantities. Machine-specific implementations are available in some cases.

9.40 GNAT.Calendar ('g-calend.ads')

Extends the facilities provided by `Ada.Calendar` to include handling of days of the week, an extended `Split` and `Time_Of` capability. Also provides conversion of `Ada.Calendar.Time` values to and from the C `timeval` format.

9.41 GNAT.Calendar.Time_IO ('g-catiio.ads')

9.42 GNAT.CRC32 ('g-crc32.ads')

This package implements the CRC-32 algorithm. For a full description of this algorithm see "Computation of Cyclic Redundancy Checks via Table Look-Up", *Communications of the ACM*, Vol. 31 No. 8, pp. 1008-1013, Aug. 1988. Sarwate, D.V.

9.43 GNAT.Case_Util ('g-casuti.ads')

A set of simple routines for handling upper and lower casing of strings without the overhead of the full casing tables in `Ada.Characters.Handling`.

9.44 GNAT.CGI ('g-cgi.ads')

This is a package for interfacing a GNAT program with a Web server via the Common Gateway Interface (CGI). Basically this package parses the CGI parameters, which are a set of key/value pairs sent by the Web server. It builds a table whose index is the key and provides some services to deal with this table.

9.45 GNAT.CGI.Cookie ('g-cgicoo.ads')

This is a package to interface a GNAT program with a Web server via the Common Gateway Interface (CGI). It exports services to deal with Web cookies (piece of information kept in the Web client software).

9.46 GNAT.CGI.Debug ('g-cgideb.ads')

This is a package to help debugging CGI (Common Gateway Interface) programs written in Ada.

9.47 GNAT.Command_Line ('g-comlin.ads')

Provides a high level interface to `Ada.Command_Line` facilities, including the ability to scan for named switches with optional parameters and expand file names using wild card notations.

9.48 GNAT.Compiler_Version ('g-comver.ads')

Provides a routine for obtaining the version of the compiler used to compile the program. More accurately this is the version of the binder used to bind the program (this will normally be the same as the version of the compiler if a consistent tool set is used to compile all units of a partition).

9.49 GNAT.Ctrl_C ('g-ctrl_c.ads')

Provides a simple interface to handle Ctrl-C keyboard events.

9.50 GNAT.Current_Exception ('g-curexc.ads')

Provides access to information on the current exception that has been raised without the need for using the Ada 95 / Ada 2005 exception choice parameter specification syntax. This is particularly useful in simulating typical facilities for obtaining information about exceptions provided by Ada 83 compilers.

9.51 GNAT.Debug_Pools ('g-debpoo.ads')

Provide a debugging storage pools that helps tracking memory corruption problems. See [Section “The GNAT Debug Pool Facility” in *GNAT User’s Guide*](#).

9.52 GNAT.Debug_Uutilities ('g-debuti.ads')

Provides a few useful utilities for debugging purposes, including conversion to and from string images of address values. Supports both C and Ada formats for hexadecimal literals.

9.53 GNAT.Decode_String ('g-decstr.ads')

A generic package providing routines for decoding wide character and wide wide character strings encoded as sequences of 8-bit characters using a specified encoding method. Includes validation routines, and also routines for stepping to next or previous encoded character in an encoded string. Useful in conjunction with Unicode character coding. Note there is a preinstantiation for UTF-8. See next entry.

9.54 GNAT.Decode_UTF8_String ('g-deutst.ads')

A preinstantiation of GNAT.Decode.Strings for UTF-8 encoding.

9.55 GNAT.Directory_Operations ('g-dirope.ads')

Provides a set of routines for manipulating directories, including changing the current directory, making new directories, and scanning the files in a directory.

9.56 GNAT.Directory_Operations.Iteration ('g-diopit.ads')

A child unit of GNAT.Directory_Operations providing additional operations for iterating through directories.

9.57 GNAT.Dynamic_HTables ('g-dynhta.ads')

A generic implementation of hash tables that can be used to hash arbitrary data. Provided in two forms, a simple form with built in hash functions, and a more complex form in which the hash function is supplied.

This package provides a facility similar to that of `GNAT.HTable`, except that this package declares a type that can be used to define dynamic instances of the hash table, while an instantiation of `GNAT.HTable` creates a single instance of the hash table.

9.58 GNAT.Dynamic_Tables ('g-dyntab.ads')

A generic package providing a single dimension array abstraction where the length of the array can be dynamically modified.

This package provides a facility similar to that of `GNAT.Table`, except that this package declares a type that can be used to define dynamic instances of the table, while an instantiation of `GNAT.Table` creates a single instance of the table type.

9.59 GNAT.Encode_String ('g-encstr.ads')

A generic package providing routines for encoding wide character and wide wide character strings as sequences of 8-bit characters using a specified encoding method. Useful in conjunction with Unicode character coding. Note there is a preinstantiation for UTF-8. See next entry.

9.60 GNAT.Encode_UTF8_String ('g-enutst.ads')

A preinstantiation of `GNAT.Encode_Strings` for UTF-8 encoding.

9.61 GNAT.Exception_Actions ('g-excact.ads')

Provides callbacks when an exception is raised. Callbacks can be registered for specific exceptions, or when any exception is raised. This can be used for instance to force a core dump to ease debugging.

9.62 GNAT.Exception_Traces ('g-extra.ads')

Provides an interface allowing to control automatic output upon exception occurrences.

9.63 GNAT.Exceptions ('g-expect.ads')

Normally it is not possible to raise an exception with a message from a subprogram in a pure package, since the necessary types and subprograms are in `Ada.Exceptions` which is not a pure unit. `GNAT.Exceptions` provides a facility for getting around this limitation for a few predefined exceptions, and for example allow raising `Constraint_Error` with a message from a pure subprogram.

9.64 GNAT.Expect ('g-expect.ads')

Provides a set of subprograms similar to what is available with the standard Tcl Expect tool. It allows you to easily spawn and communicate with an external process. You can send

commands or inputs to the process, and compare the output with some expected regular expression. Currently `GNAT.Expect` is implemented on all native GNAT ports except for OpenVMS. It is not implemented for cross ports, and in particular is not implemented for VxWorks or LynxOS.

9.65 GNAT.Float_Control ('g-flocon.ads')

Provides an interface for resetting the floating-point processor into the mode required for correct semantic operation in Ada. Some third party library calls may cause this mode to be modified, and the `Reset` procedure in this package can be used to reestablish the required mode.

9.66 GNAT.Heap_Sort ('g-heasor.ads')

Provides a general implementation of heap sort usable for sorting arbitrary data items. Exchange and comparison procedures are provided by passing access-to-procedure values. The algorithm used is a modified heap sort that performs approximately $N \cdot \log(N)$ comparisons in the worst case.

9.67 GNAT.Heap_Sort_A ('g-hesora.ads')

Provides a general implementation of heap sort usable for sorting arbitrary data items. Move and comparison procedures are provided by passing access-to-procedure values. The algorithm used is a modified heap sort that performs approximately $N \cdot \log(N)$ comparisons in the worst case. This differs from `GNAT.Heap_Sort` in having a less convenient interface, but may be slightly more efficient.

9.68 GNAT.Heap_Sort_G ('g-hesorg.ads')

Similar to `Heap_Sort_A` except that the move and sorting procedures are provided as generic parameters, this improves efficiency, especially if the procedures can be inlined, at the expense of duplicating code for multiple instantiations.

9.69 GNAT.HTable ('g-htable.ads')

A generic implementation of hash tables that can be used to hash arbitrary data. Provides two approaches, one a simple static approach, and the other allowing arbitrary dynamic hash tables.

9.70 GNAT.IO ('g-io.ads')

A simple preelaborable input-output package that provides a subset of simple `Text_IO` functions for reading characters and strings from `Standard_Input`, and writing characters, strings and integers to either `Standard_Output` or `Standard_Error`.

9.71 GNAT.IO_Aux ('g-io_aux.ads')

Provides some auxiliary functions for use with `Text_IO`, including a test for whether a file exists, and functions for reading a line of text.

9.72 GNAT.Lock_Files ('g-locfil.ads')

Provides a general interface for using files as locks. Can be used for providing program level synchronization.

9.73 GNAT.MD5 ('g-md5.ads')

Implements the MD5 Message-Digest Algorithm as described in RFC 1321.

9.74 GNAT.Memory_Dump ('g-memdum.ads')

Provides a convenient routine for dumping raw memory to either the standard output or standard error files. Uses GNAT.IO for actual output.

9.75 GNAT.Most_Recent_Exception ('g-moreex.ads')

Provides access to the most recently raised exception. Can be used for various logging purposes, including duplicating functionality of some Ada 83 implementation dependent extensions.

9.76 GNAT.OS_Lib ('g-os_lib.ads')

Provides a range of target independent operating system interface functions, including time/date management, file operations, subprocess management, including a portable spawn procedure, and access to environment variables and error return codes.

9.77 GNAT.Perfect_Hash_Generators ('g-pehage.ads')

Provides a generator of static minimal perfect hash functions. No collisions occur and each item can be retrieved from the table in one probe (perfect property). The hash table size corresponds to the exact size of the key set and no larger (minimal property). The key set has to be known in advance (static property). The hash functions are also order preserving. If w2 is inserted after w1 in the generator, their hashcode are in the same order. These hashing functions are very convenient for use with realtime applications.

9.78 GNAT.Random_Numbers ('g-rannum.ads')

Provides random number capabilities which extend those available in the standard Ada library and are more convenient to use.

9.79 GNAT.Regexp ('g-regexp.ads')

A simple implementation of regular expressions, using a subset of regular expression syntax copied from familiar Unix style utilities. This is the simplest of the three pattern matching packages provided, and is particularly suitable for "file globbing" applications.

9.80 GNAT.Registry ('g-regist.ads')

This is a high level binding to the Windows registry. It is possible to do simple things like reading a key value, creating a new key. For full registry API, but at a lower level of abstraction, refer to the Win32.Winreg package provided with the Win32Ada binding

9.81 GNAT.Regpat ('g-regpat.ads')

A complete implementation of Unix-style regular expression matching, copied from the original V7 style regular expression library written in C by Henry Spencer (and binary compatible with this C library).

9.82 GNAT.Secondary_Stack_Info ('g-sestin.ads')

Provide the capability to query the high water mark of the current task's secondary stack.

9.83 GNAT.Semaphores ('g-semaph.ads')

Provides classic counting and binary semaphores using protected types.

9.84 GNAT.Serial_Communications ('g-sercom.ads')

Provides a simple interface to send and receive data over a serial port. This is only supported on GNU/Linux and Windows.

9.85 GNAT.SHA1 ('g-sha1.ads')

Implements the SHA-1 Secure Hash Algorithm as described in FIPS PUB 180-3 and RFC 3174.

9.86 GNAT.SHA224 ('g-sha224.ads')

Implements the SHA-224 Secure Hash Algorithm as described in FIPS PUB 180-3.

9.87 GNAT.SHA256 ('g-sha256.ads')

Implements the SHA-256 Secure Hash Algorithm as described in FIPS PUB 180-3.

9.88 GNAT.SHA384 ('g-sha384.ads')

Implements the SHA-384 Secure Hash Algorithm as described in FIPS PUB 180-3.

9.89 GNAT.SHA512 ('g-sha512.ads')

Implements the SHA-512 Secure Hash Algorithm as described in FIPS PUB 180-3.

9.90 GNAT.Signals ('g-signal.ads')

Provides the ability to manipulate the blocked status of signals on supported targets.

9.91 GNAT.Sockets ('g-socket.ads')

A high level and portable interface to develop sockets based applications. This package is based on the sockets thin binding found in `GNAT.Sockets.Thin`. Currently `GNAT.Sockets` is implemented on all native GNAT ports except for OpenVMS. It is not implemented for the LynxOS cross port.

9.92 GNAT.Source_Info ('g-souinf.ads')

Provides subprograms that give access to source code information known at compile time, such as the current file name and line number.

9.93 GNAT.Spelling_Checker ('g-speche.ads')

Provides a function for determining whether one string is a plausible near misspelling of another string.

9.94 GNAT.Spelling_Checker_Generic ('g-spchge.ads')

Provides a generic function that can be instantiated with a string type for determining whether one string is a plausible near misspelling of another string.

9.95 GNAT.Spitbol.Patterns ('g-spipat.ads')

A complete implementation of SNOBOL4 style pattern matching. This is the most elaborate of the pattern matching packages provided. It fully duplicates the SNOBOL4 dynamic pattern construction and matching capabilities, using the efficient algorithm developed by Robert Dewar for the SPITBOL system.

9.96 GNAT.Spitbol ('g-spitbo.ads')

The top level package of the collection of SPITBOL-style functionality, this package provides basic SNOBOL4 string manipulation functions, such as Pad, Reverse, Trim, Substr capability, as well as a generic table function useful for constructing arbitrary mappings from strings in the style of the SNOBOL4 TABLE function.

9.97 GNAT.Spitbol.Table_Boolean ('g-sptabo.ads')

A library level of instantiation of GNAT.Spitbol.Patterns.Table for type `Standard.Boolean`, giving an implementation of sets of string values.

9.98 GNAT.Spitbol.Table_Integer ('g-sptain.ads')

A library level of instantiation of GNAT.Spitbol.Patterns.Table for type `Standard.Integer`, giving an implementation of maps from string to integer values.

9.99 GNAT.Spitbol.Table_VString ('g-sptavs.ads')

A library level of instantiation of GNAT.Spitbol.Patterns.Table for a variable length string type, giving an implementation of general maps from strings to strings.

9.100 GNAT.SSE ('g-sse.ads')

Root of a set of units aimed at offering Ada bindings to a subset of the Intel(r) Streaming SIMD Extensions with GNAT on the x86 family of targets. It exposes vector component types together with a general introduction to the binding contents and use.

9.101 GNAT.SSE.Vector_Types ('g-ssvety.ads')

SSE vector types for use with SSE related intrinsics.

9.102 GNAT.Strings ('g-string.ads')

Common String access types and related subprograms. Basically it defines a string access and an array of string access types.

9.103 GNAT.String_Split ('g-strspl.ads')

Useful string manipulation routines: given a set of separators, split a string wherever the separators appear, and provide direct access to the resulting slices. This package is instantiated from GNAT.Array_Split.

9.104 GNAT.Table ('g-table.ads')

A generic package providing a single dimension array abstraction where the length of the array can be dynamically modified.

This package provides a facility similar to that of GNAT.Dynamic_Tables, except that this package declares a single instance of the table type, while an instantiation of GNAT.Dynamic_Tables creates a type that can be used to define dynamic instances of the table.

9.105 GNAT.Task_Lock ('g-tasloc.ads')

A very simple facility for locking and unlocking sections of code using a single global task lock. Appropriate for use in situations where contention between tasks is very rarely expected.

9.106 GNAT.Time_Stamp ('g-timsta.ads')

Provides a simple function that returns a string YYYY-MM-DD HH:MM:SS.SS that represents the current date and time in ISO 8601 format. This is a very simple routine with minimal code and there are no dependencies on any other unit.

9.107 GNAT.Threads ('g-thread.ads')

Provides facilities for dealing with foreign threads which need to be known by the GNAT run-time system. Consult the documentation of this package for further details if your program has threads that are created by a non-Ada environment which then accesses Ada code.

9.108 GNAT.Traceback ('g-traceb.ads')

Provides a facility for obtaining non-symbolic traceback information, useful in various debugging situations.

9.109 GNAT.Traceback.Symbolic ('g-trasym.ads')

9.110 GNAT.UTF_32 ('g-table.ads')

This is a package intended to be used in conjunction with the `Wide_Character` type in Ada 95 and the `Wide_Wide_Character` type in Ada 2005 (available in GNAT in Ada 2005 mode). This package contains Unicode categorization routines, as well as lexical categorization routines corresponding to the Ada 2005 lexical rules for identifiers and strings, and also a lower case to upper case fold routine corresponding to the Ada 2005 rules for identifier equivalence.

9.111 GNAT.Wide_Spelling_Checker ('g-u3spch.ads')

Provides a function for determining whether one wide wide string is a plausible near misspelling of another wide wide string, where the strings are represented using the `UTF_32_String` type defined in `System.Wch_Cnv`.

9.112 GNAT.Wide_Spelling_Checker ('g-wispch.ads')

Provides a function for determining whether one wide string is a plausible near misspelling of another wide string.

9.113 GNAT.Wide_String_Split ('g-wistsp.ads')

Useful wide string manipulation routines: given a set of separators, split a wide string wherever the separators appear, and provide direct access to the resulting slices. This package is instantiated from `GNAT.Array_Split`.

9.114 GNAT.Wide_Wide_Spelling_Checker ('g-zspche.ads')

Provides a function for determining whether one wide wide string is a plausible near misspelling of another wide wide string.

9.115 GNAT.Wide_Wide_String_Split ('g-zistsp.ads')

Useful wide wide string manipulation routines: given a set of separators, split a wide wide string wherever the separators appear, and provide direct access to the resulting slices. This package is instantiated from `GNAT.Array_Split`.

9.116 Interfaces.C.Extensions ('i-cexten.ads')

This package contains additional C-related definitions, intended for use with either manually or automatically generated bindings to C libraries.

9.117 Interfaces.C.Streams ('i-cstrea.ads')

This package is a binding for the most commonly used operations on C streams.

9.118 Interfaces.CPP ('i-cpp.ads')

This package provides facilities for use in interfacing to C++. It is primarily intended to be used in connection with automated tools for the generation of C++ interfaces.

9.119 Interfaces.Packed_Decimal ('i-pacdec.ads')

This package provides a set of routines for conversions to and from a packed decimal format compatible with that used on IBM mainframes.

9.120 Interfaces.VxWorks ('i-vxwork.ads')

This package provides a limited binding to the VxWorks API. In particular, it interfaces with the VxWorks hardware interrupt facilities.

9.121 Interfaces.VxWorks.IO ('i-vxwoio.ads')

This package provides a binding to the ioctl (IO/Control) function of VxWorks, defining a set of option values and function codes. A particular use of this package is to enable the use of Get_Immediate under VxWorks.

9.122 System.Address_Image ('s-addima.ads')

This function provides a useful debugging function that gives an (implementation dependent) string which identifies an address.

9.123 System.Assertions ('s-assert.ads')

This package provides the declaration of the exception raised by a run-time assertion failure, as well as the routine that is used internally to raise this assertion.

9.124 System.Memory ('s-memory.ads')

This package provides the interface to the low level routines used by the generated code for allocation and freeing storage for the default storage pool (analogous to the C routines malloc and free. It also provides a reallocation interface analogous to the C routine realloc. The body of this unit may be modified to provide alternative allocation mechanisms for the default pool, and in addition, direct calls to this unit may be made for low level allocation uses (for example see the body of GNAT.Tables).

9.125 System.Partition_Interface ('s-parint.ads')

This package provides facilities for partition interfacing. It is used primarily in a distribution context when using Annex E with GLADE.

9.126 System.Pool_Global ('s-pooglo.ads')

This package provides a storage pool that is equivalent to the default storage pool used for access types for which no pool is specifically declared. It uses malloc/free to allocate/free and does not attempt to do any automatic reclamation.

9.127 System.Pool_Local ('s-poolloc.ads')

This package provides a storage pool that is intended for use with locally defined access types. It uses malloc/free for allocate/free, and maintains a list of allocated blocks, so that all storage allocated for the pool can be freed automatically when the pool is finalized.

9.128 `System.Restrictions` ('s-restri.ads')

This package provides facilities for accessing at run time the status of restrictions specified at compile time for the partition. Information is available both with regard to actual restrictions specified, and with regard to compiler determined information on which restrictions are violated by one or more packages in the partition.

9.129 `System.Rident` ('s-rident.ads')

This package provides definitions of the restrictions identifiers supported by GNAT, and also the format of the restrictions provided in package `System.Restrictions`. It is not normally necessary to `with` this generic package since the necessary instantiation is included in package `System.Restrictions`.

9.130 `System.Strings.Stream_Ops` ('s-ststop.ads')

This package provides a set of stream subprograms for standard string types. It is intended primarily to support implicit use of such subprograms when stream attributes are applied to string types, but the subprograms in this package can be used directly by application programs.

9.131 `System.Task_Info` ('s-tasinf.ads')

This package provides target dependent functionality that is used to support the `Task_Info` pragma

9.132 `System.Wch_Cnv` ('s-wchcnv.ads')

This package provides routines for converting between wide and wide wide characters and a representation as a value of type `Standard.String`, using a specified wide character encoding method. It uses definitions in package `System.Wch_Con`.

9.133 `System.Wch_Con` ('s-wchcon.ads')

This package provides definitions and descriptions of the various methods used for encoding wide characters in ordinary strings. These definitions are used by the package `System.Wch_Cnv`.

10 Interfacing to Other Languages

The facilities in annex B of the Ada Reference Manual are fully implemented in GNAT, and in addition, a full interface to C++ is provided.

10.1 Interfacing to C

Interfacing to C with GNAT can use one of two approaches:

- The types in the package `Interfaces.C` may be used.
- Standard Ada types may be used directly. This may be less portable to other compilers, but will work on all GNAT compilers, which guarantee correspondence between the C and Ada types.

Pragma `Convention C` may be applied to Ada types, but mostly has no effect, since this is the default. The following table shows the correspondence between Ada scalar types and the corresponding C types.

<code>Integer</code>	<code>int</code>
<code>Short_Integer</code>	<code>short</code>
<code>Short_Short_Integer</code>	<code>signed char</code>
<code>Long_Integer</code>	<code>long</code>
<code>Long_Long_Integer</code>	<code>long long</code>
<code>Short_Float</code>	<code>float</code>
<code>Float</code>	<code>float</code>
<code>Long_Float</code>	<code>double</code>
<code>Long_Long_Float</code>	

This is the longest floating-point type supported by the hardware.

Additionally, there are the following general correspondences between Ada and C types:

- Ada enumeration types map to C enumeration types directly if pragma `Convention C` is specified, which causes them to have `int` length. Without pragma `Convention C`, Ada enumeration types map to 8, 16, or 32 bits (i.e. C types `signed char`, `short`, `int`, respectively) depending on the number of values passed. This is the only case in which pragma `Convention C` affects the representation of an Ada type.
- Ada access types map to C pointers, except for the case of pointers to unconstrained types in Ada, which have no direct C equivalent.
- Ada arrays map directly to C arrays.
- Ada records map directly to C structures.
- Packed Ada records map to C structures where all members are bit fields of the length corresponding to the `type'Size` value in Ada.

10.2 Interfacing to C++

The interface to C++ makes use of the following pragmas, which are primarily intended to be constructed automatically using a binding generator tool, although it is possible to construct them by hand. No suitable binding generator tool is supplied with GNAT though.

Using these pragmas it is possible to achieve complete inter-operability between Ada tagged types and C++ class definitions. See [Chapter 1 \[Implementation Defined Pragmas\]](#), [page 5](#), for more details.

`pragma CPP_Class ([Entity =>] LOCAL_NAME)`

The argument denotes an entity in the current declarative region that is declared as a tagged or untagged record type. It indicates that the type corresponds to an externally declared C++ class type, and is to be laid out the same way that C++ would lay out the type.

Note: Pragma `CPP_Class` is currently obsolete. It is supported for backward compatibility but its functionality is available using pragma `Import` with `Convention = CPP`.

`pragma CPP_Constructor ([Entity =>] LOCAL_NAME)`

This pragma identifies an imported function (imported in the usual way with pragma `Import`) as corresponding to a C++ constructor.

10.3 Interfacing to COBOL

Interfacing to COBOL is achieved as described in section B.4 of the Ada Reference Manual.

10.4 Interfacing to Fortran

Interfacing to Fortran is achieved as described in section B.5 of the Ada Reference Manual. The pragma `Convention Fortran`, applied to a multi-dimensional array causes the array to be stored in column-major order as required for convenient interface to Fortran.

10.5 Interfacing to non-GNAT Ada code

It is possible to specify the convention `Ada` in a pragma `Import` or pragma `Export`. However this refers to the calling conventions used by GNAT, which may or may not be similar enough to those used by some other Ada 83 / Ada 95 / Ada 2005 compiler to allow interoperation.

If arguments types are kept simple, and if the foreign compiler generally follows system calling conventions, then it may be possible to integrate files compiled by other Ada compilers, provided that the elaboration issues are adequately addressed (for example by eliminating the need for any load time elaboration).

In particular, GNAT running on VMS is designed to be highly compatible with the DEC Ada 83 compiler, so this is one case in which it is possible to import foreign units of this type, provided that the data items passed are restricted to simple scalar values or simple record types without variants, or simple array types with fixed bounds.

11 Specialized Needs Annexes

Ada 95 and Ada 2005 define a number of Specialized Needs Annexes, which are not required in all implementations. However, as described in this chapter, GNAT implements all of these annexes:

Systems Programming (Annex C)

The Systems Programming Annex is fully implemented.

Real-Time Systems (Annex D)

The Real-Time Systems Annex is fully implemented.

Distributed Systems (Annex E)

Stub generation is fully implemented in the GNAT compiler. In addition, a complete compatible PCS is available as part of the GLADE system, a separate product. When the two products are used in conjunction, this annex is fully implemented.

Information Systems (Annex F)

The Information Systems annex is fully implemented.

Numerics (Annex G)

The Numerics Annex is fully implemented.

Safety and Security / High-Integrity Systems (Annex H)

The Safety and Security Annex (termed the High-Integrity Systems Annex in Ada 2005) is fully implemented.

12 Implementation of Specific Ada Features

This chapter describes the GNAT implementation of several Ada language facilities.

12.1 Machine Code Insertions

Package `Machine_Code` provides machine code support as described in the Ada Reference Manual in two separate forms:

- Machine code statements, consisting of qualified expressions that fit the requirements of RM section 13.8.
- An intrinsic callable procedure, providing an alternative mechanism of including machine instructions in a subprogram.

The two features are similar, and both are closely related to the mechanism provided by the `asm` instruction in the GNU C compiler. Full understanding and use of the facilities in this package requires understanding the `asm` instruction, see [Section “Assembler Instructions with C Expression Operands” in *Using the GNU Compiler Collection \(GCC\)*](#).

Calls to the function `Asm` and the procedure `Asm` have identical semantic restrictions and effects as described below. Both are provided so that the procedure call can be used as a statement, and the function call can be used to form a `code_statement`.

The first example given in the GCC documentation is the C `asm` instruction:

```
asm ("fsinx %1 %0" : "=f" (result) : "f" (angle));
```

The equivalent can be written for GNAT as:

```
Asm ("fsinx %1 %0",
    My_Float'Asm_Output ("=f", result),
    My_Float'Asm_Input  ("f",  angle));
```

The first argument to `Asm` is the assembler template, and is identical to what is used in GNU C. This string must be a static expression. The second argument is the output operand list. It is either a single `Asm_Output` attribute reference, or a list of such references enclosed in parentheses (technically an array aggregate of such references).

The `Asm_Output` attribute denotes a function that takes two parameters. The first is a string, the second is the name of a variable of the type designated by the attribute prefix. The first (string) argument is required to be a static expression and designates the constraint for the parameter (e.g. what kind of register is required). The second argument is the variable to be updated with the result. The possible values for constraint are the same as those used in the RTL, and are dependent on the configuration file used to build the GCC back end. If there are no output operands, then this argument may either be omitted, or explicitly given as `No_Output_Operands`.

The second argument of `my_float'Asm_Output` functions as though it were an `out` parameter, which is a little curious, but all names have the form of expressions, so there is no syntactic irregularity, even though normally functions would not be permitted `out` parameters. The third argument is the list of input operands. It is either a single `Asm_Input` attribute reference, or a list of such references enclosed in parentheses (technically an array aggregate of such references).

The `Asm_Input` attribute denotes a function that takes two parameters. The first is a string, the second is an expression of the type designated by the prefix. The first (string)

argument is required to be a static expression, and is the constraint for the parameter, (e.g. what kind of register is required). The second argument is the value to be used as the input argument. The possible values for the constant are the same as those used in the RTL, and are dependent on the configuration file used to built the GCC back end.

If there are no input operands, this argument may either be omitted, or explicitly given as `No_Input_Operands`. The fourth argument, not present in the above example, is a list of register names, called the *clobber* argument. This argument, if given, must be a static string expression, and is a space or comma separated list of names of registers that must be considered destroyed as a result of the `Asm` call. If this argument is the null string (the default value), then the code generator assumes that no additional registers are destroyed.

The fifth argument, not present in the above example, called the *volatile* argument, is by default `False`. It can be set to the literal value `True` to indicate to the code generator that all optimizations with respect to the instruction specified should be suppressed, and that in particular, for an instruction that has outputs, the instruction will still be generated, even if none of the outputs are used. See [Section “Assembler Instructions with C Expression Operands” in *Using the GNU Compiler Collection \(GCC\)*](#), for the full description. Generally it is strongly advisable to use `Volatile` for any ASM statement that is missing either input or output operands, or when two or more ASM statements appear in sequence, to avoid unwanted optimizations. A warning is generated if this advice is not followed.

The `Asm` subprograms may be used in two ways. First the procedure forms can be used anywhere a procedure call would be valid, and correspond to what the RM calls “intrinsic” routines. Such calls can be used to intersperse machine instructions with other Ada statements. Second, the function forms, which return a dummy value of the limited private type `Asm_Insn`, can be used in code statements, and indeed this is the only context where such calls are allowed. Code statements appear as aggregates of the form:

```
Asm_Insn'(Asm (...));
Asm_Insn'(Asm_Volatile (...));
```

In accordance with RM rules, such code statements are allowed only within subprograms whose entire body consists of such statements. It is not permissible to intermix such statements with other Ada statements.

Typically the form using intrinsic procedure calls is more convenient and more flexible. The code statement form is provided to meet the RM suggestion that such a facility should be made available. The following is the exact syntax of the call to `Asm`. As usual, if named notation is used, the arguments may be given in arbitrary order, following the normal rules for use of positional and named arguments)

```
ASM_CALL ::= Asm (
    [Template =>] static_string_EXPRESSION
    [, [Outputs =>] OUTPUT_OPERAND_LIST      ]
    [, [Inputs  =>] INPUT_OPERAND_LIST       ]
    [, [Clobber =>] static_string_EXPRESSION ]
    [, [Volatile =>] static_boolean_EXPRESSION] )

OUTPUT_OPERAND_LIST ::=
    [PREFIX.]No_Output_Operands
  | OUTPUT_OPERAND_ATTRIBUTE
  | (OUTPUT_OPERAND_ATTRIBUTE {, OUTPUT_OPERAND_ATTRIBUTE})

OUTPUT_OPERAND_ATTRIBUTE ::=
```

```

SUBTYPE_MARK'Asm_Output (static_string_EXPRESSION, NAME)

INPUT_OPERAND_LIST ::=
  [PREFIX.]No_Input_Operands
| INPUT_OPERAND_ATTRIBUTE
| (INPUT_OPERAND_ATTRIBUTE {,INPUT_OPERAND_ATTRIBUTE})

INPUT_OPERAND_ATTRIBUTE ::=
  SUBTYPE_MARK'Asm_Input (static_string_EXPRESSION, EXPRESSION)

```

The identifiers `No_Input_Operands` and `No_Output_Operands` are declared in the package `Machine_Code` and must be referenced according to normal visibility rules. In particular if there is no `use` clause for this package, then appropriate package name qualification is required.

12.2 GNAT Implementation of Tasking

This chapter outlines the basic GNAT approach to tasking (in particular, a multi-layered library for portability) and discusses issues related to compliance with the Real-Time Systems Annex.

12.2.1 Mapping Ada Tasks onto the Underlying Kernel Threads

GNAT's run-time support comprises two layers:

- GNARL (GNAT Run-time Layer)
- GNULL (GNAT Low-level Library)

In GNAT, Ada's tasking services rely on a platform and OS independent layer known as GNARL. This code is responsible for implementing the correct semantics of Ada's task creation, rendezvous, protected operations etc.

GNARL decomposes Ada's tasking semantics into simpler lower level operations such as create a thread, set the priority of a thread, yield, create a lock, lock/unlock, etc. The spec for these low-level operations constitutes GNULLI, the GNULL Interface. This interface is directly inspired from the POSIX real-time API.

If the underlying executive or OS implements the POSIX standard faithfully, the GNULL Interface maps as is to the services offered by the underlying kernel. Otherwise, some target dependent glue code maps the services offered by the underlying kernel to the semantics expected by GNARL.

Whatever the underlying OS (VxWorks, UNIX, OS/2, Windows NT, etc.) the key point is that each Ada task is mapped on a thread in the underlying kernel. For example, in the case of VxWorks, one Ada task = one VxWorks task.

In addition Ada task priorities map onto the underlying thread priorities. Mapping Ada tasks onto the underlying kernel threads has several advantages:

- The underlying scheduler is used to schedule the Ada tasks. This makes Ada tasks as efficient as kernel threads from a scheduling standpoint.
- Interaction with code written in C containing threads is eased since at the lowest level Ada tasks and C threads map onto the same underlying kernel concept.
- When an Ada task is blocked during I/O the remaining Ada tasks are able to proceed.
- On multiprocessor systems Ada tasks can execute in parallel.

Some threads libraries offer a mechanism to fork a new process, with the child process duplicating the threads from the parent. GNAT does not support this functionality when the parent contains more than one task.

12.2.2 Ensuring Compliance with the Real-Time Annex

Although mapping Ada tasks onto the underlying threads has significant advantages, it does create some complications when it comes to respecting the scheduling semantics specified in the real-time annex (Annex D).

For instance the Annex D requirement for the `FIFO_Within_Priorities` scheduling policy states:

When the active priority of a ready task that is not running changes, or the setting of its base priority takes effect, the task is removed from the ready queue for its old active priority and is added at the tail of the ready queue for its new active priority, except in the case where the active priority is lowered due to the loss of inherited priority, in which case the task is added at the head of the ready queue for its new active priority.

While most kernels do put tasks at the end of the priority queue when a task changes its priority, (which respects the main `FIFO_Within_Priorities` requirement), almost none keep a thread at the beginning of its priority queue when its priority drops from the loss of inherited priority.

As a result most vendors have provided incomplete Annex D implementations.

The GNAT run-time, has a nice cooperative solution to this problem which ensures that accurate `FIFO_Within_Priorities` semantics are respected.

The principle is as follows. When an Ada task T is about to start running, it checks whether some other Ada task R with the same priority as T has been suspended due to the loss of priority inheritance. If this is the case, T yields and is placed at the end of its priority queue. When R arrives at the front of the queue it executes.

Note that this simple scheme preserves the relative order of the tasks that were ready to execute in the priority queue where R has been placed at the end.

12.3 GNAT Implementation of Shared Passive Packages

GNAT fully implements the pragma `Shared_Passive` for the purpose of designating shared passive packages. This allows the use of passive partitions in the context described in the Ada Reference Manual; i.e., for communication between separate partitions of a distributed application using the features in Annex E.

However, the implementation approach used by GNAT provides for more extensive usage as follows:

Communication between separate programs

This allows separate programs to access the data in passive partitions, using protected objects for synchronization where needed. The only requirement is that the two programs have a common shared file system. It is even possible for programs running on different machines with different architectures (e.g. different endianness) to communicate via the data in a passive partition.

Persistence between program runs

The data in a passive package can persist from one run of a program to another, so that a later program sees the final values stored by a previous run of the same program.

The implementation approach used is to store the data in files. A separate stream file is created for each object in the package, and an access to an object causes the corresponding file to be read or written.

The environment variable `SHARED_MEMORY_DIRECTORY` should be set to the directory to be used for these files. The files in this directory have names that correspond to their fully qualified names. For example, if we have the package

```
package X is
  pragma Shared_Passive (X);
  Y : Integer;
  Z : Float;
end X;
```

and the environment variable is set to `/stemp/`, then the files created will have the names:

```
/stemp/x.y
/stemp/x.z
```

These files are created when a value is initially written to the object, and the files are retained until manually deleted. This provides the persistence semantics. If no file exists, it means that no partition has assigned a value to the variable; in this case the initial value declared in the package will be used. This model ensures that there are no issues in synchronizing the elaboration process, since elaboration of passive packages elaborates the initial values, but does not create the files.

The files are written using normal `Stream_IO` access. If you want to be able to communicate between programs or partitions running on different architectures, then you should use the XDR versions of the stream attribute routines, since these are architecture independent.

If active synchronization is required for access to the variables in the shared passive package, then as described in the Ada Reference Manual, the package may contain protected objects used for this purpose. In this case a lock file (whose name is ‘`___lock`’ (three underscores)) is created in the shared memory directory. This is used to provide the required locking semantics for proper protected object synchronization.

As of January 2003, GNAT supports shared passive packages on all platforms except for OpenVMS.

12.4 Code Generation for Array Aggregates

Aggregates have a rich syntax and allow the user to specify the values of complex data structures by means of a single construct. As a result, the code generated for aggregates can be quite complex and involve loops, case statements and multiple assignments. In the simplest cases, however, the compiler will recognize aggregates whose components and constraints are fully static, and in those cases the compiler will generate little or no executable code. The following is an outline of the code that GNAT generates for various aggregate constructs. For further details, you will find it useful to examine the output produced by the `-gnatG` flag to see the expanded source that is input to the code generator. You may also want to examine the assembly code generated at various levels of optimization.

The code generated for aggregates depends on the context, the component values, and the type. In the context of an object declaration the code generated is generally simpler than in the case of an assignment. As a general rule, static component values and static subtypes also lead to simpler code.

12.4.1 Static constant aggregates with static bounds

For the declarations:

```
type One_Dim is array (1..10) of integer;
ar0 : constant One_Dim := (1, 2, 3, 4, 5, 6, 7, 8, 9, 0);
```

GNAT generates no executable code: the constant ar0 is placed in static memory. The same is true for constant aggregates with named associations:

```
Cr1 : constant One_Dim := (4 => 16, 2 => 4, 3 => 9, 1 => 1, 5 .. 10 => 0);
Cr3 : constant One_Dim := (others => 7777);
```

The same is true for multidimensional constant arrays such as:

```
type two_dim is array (1..3, 1..3) of integer;
Unit : constant two_dim := ( (1,0,0), (0,1,0), (0,0,1));
```

The same is true for arrays of one-dimensional arrays: the following are static:

```
type ar1b is array (1..3) of boolean;
type ar_ar is array (1..3) of ar1b;
None : constant ar1b := (others => false);      -- fully static
None2 : constant ar_ar := (1..3 => None);      -- fully static
```

However, for multidimensional aggregates with named associations, GNAT will generate assignments and loops, even if all associations are static. The following two declarations generate a loop for the first dimension, and individual component assignments for the second dimension:

```
Zero1: constant two_dim := (1..3 => (1..3 => 0));
Zero2: constant two_dim := (others => (others => 0));
```

12.4.2 Constant aggregates with unconstrained nominal types

In such cases the aggregate itself establishes the subtype, so that associations with **others** cannot be used. GNAT determines the bounds for the actual subtype of the aggregate, and allocates the aggregate statically as well. No code is generated for the following:

```
type One_Unc is array (natural range <>) of integer;
Cr_Unc : constant One_Unc := (12,24,36);
```

12.4.3 Aggregates with static bounds

In all previous examples the aggregate was the initial (and immutable) value of a constant. If the aggregate initializes a variable, then code is generated for it as a combination of individual assignments and loops over the target object. The declarations

```
Cr_Var1 : One_Dim := (2, 5, 7, 11, 0, 0, 0, 0, 0, 0);
Cr_Var2 : One_Dim := (others > -1);
```

generate the equivalent of

```
Cr_Var1 (1) := 2;
Cr_Var1 (2) := 5;
Cr_Var1 (3) := 7;
Cr_Var1 (4) := 11;

for I in Cr_Var2'range loop
```

```

        Cr_Var2 (I) := -1;
    end loop;

```

12.4.4 Aggregates with non-static bounds

If the bounds of the aggregate are not statically compatible with the bounds of the nominal subtype of the target, then constraint checks have to be generated on the bounds. For a multidimensional array, constraint checks may have to be applied to sub-arrays individually, if they do not have statically compatible subtypes.

12.4.5 Aggregates in assignment statements

In general, aggregate assignment requires the construction of a temporary, and a copy from the temporary to the target of the assignment. This is because it is not always possible to convert the assignment into a series of individual component assignments. For example, consider the simple case:

```

    A := (A(2), A(1));

```

This cannot be converted into:

```

    A(1) := A(2);
    A(2) := A(1);

```

So the aggregate has to be built first in a separate location, and then copied into the target. GNAT recognizes simple cases where this intermediate step is not required, and the assignments can be performed in place, directly into the target. The following sufficient criteria are applied:

- The bounds of the aggregate are static, and the associations are static.
- The components of the aggregate are static constants, names of simple variables that are not renamings, or expressions not involving indexed components whose operands obey these rules.

If any of these conditions are violated, the aggregate will be built in a temporary (created either by the front-end or the code generator) and then that temporary will be copied onto the target.

12.5 The Size of Discriminated Records with Default Discriminants

If a discriminated type *T* has discriminants with default values, it is possible to declare an object of this type without providing an explicit constraint:

```

type Size is range 1..100;

type Rec (D : Size := 15) is record
    Name : String (1..D);
end T;

Word : Rec;

```

Such an object is said to be *unconstrained*. The discriminant of the object can be modified by a full assignment to the object, as long as it preserves the relation between the value of the discriminant, and the value of the components that depend on it:

```

Word := (3, "yes");

Word := (5, "maybe");

Word := (5, "no"); -- raises Constraint_Error

```

In order to support this behavior efficiently, an unconstrained object is given the maximum size that any value of the type requires. In the case above, `Word` has storage for the discriminant and for a `String` of length 100. It is important to note that unconstrained objects do not require dynamic allocation. It would be an improper implementation to place on the heap those components whose size depends on discriminants. (This improper implementation was used by some Ada83 compilers, where the `Name` component above would have been stored as a pointer to a dynamic string). Following the principle that dynamic storage management should never be introduced implicitly, an Ada compiler should reserve the full size for an unconstrained declared object, and place it on the stack.

This maximum size approach has been a source of surprise to some users, who expect the default values of the discriminants to determine the size reserved for an unconstrained object: “If the default is 15, why should the object occupy a larger size?” The answer, of course, is that the discriminant may be later modified, and its full range of values must be taken into account. This is why the declaration:

```

type Rec (D : Positive := 15) is record
  Name : String (1..D);
end record;

```

```

Too_Large : Rec;

```

is flagged by the compiler with a warning: an attempt to create `Too_Large` will raise `Storage_Error`, because the required size includes `Positive'Last` bytes. As the first example indicates, the proper approach is to declare an index type of “reasonable” range so that unconstrained objects are not too large.

One final wrinkle: if the object is declared to be *aliased*, or if it is created in the heap by means of an allocator, then it is *not* unconstrained: it is constrained by the default values of the discriminants, and those values cannot be modified by full assignment. This is because in the presence of aliasing all views of the object (which may be manipulated by different tasks, say) must be consistent, so it is imperative that the object, once created, remain invariant.

12.6 Strict Conformance to the Ada Reference Manual

The dynamic semantics defined by the Ada Reference Manual impose a set of run-time checks to be generated. By default, the GNAT compiler will insert many run-time checks into the compiled code, including most of those required by the Ada Reference Manual. However, there are three checks that are not enabled in the default mode for efficiency reasons: arithmetic overflow checking for integer operations (including division by zero), checks for access before elaboration on subprogram calls, and stack overflow checking (most operating systems do not perform this check by default).

Strict conformance to the Ada Reference Manual can be achieved by adding three compiler options for overflow checking for integer operations (`-gnato`), dynamic checks for access-before-elaboration on subprogram calls and generic instantiations (`-gnatE`), and stack overflow checking (`-fstack-check`).

Note that the result of a floating point arithmetic operation in overflow and invalid situations, when the `Machine_Overflows` attribute of the result type is `False`, is to generate IEEE NaN and infinite values. This is the case for machines compliant with the IEEE floating-point standard, but on machines that are not fully compliant with this standard, such as Alpha, the `'-mieee'` compiler flag must be used for achieving IEEE confirming behavior (although at the cost of a significant performance penalty), so infinite and NaN values are properly generated.

13 Project File Reference

This chapter describes the syntax and semantics of project files. Project files specify the options to be used when building a system. Project files can specify global settings for all tools, as well as tool-specific settings. See [Section “Examples of Project Files” in *GNAT User’s Guide*](#), for examples of use.

13.1 Reserved Words

All Ada reserved words are reserved in project files, and cannot be used as variable names or project names. In addition, the following are also reserved in project files:

- `extends`
- `external`
- `project`

13.2 Lexical Elements

Rules for identifiers are the same as in Ada. Identifiers are case-insensitive. Strings are case sensitive, except where noted. Comments have the same form as in Ada.

Syntax:

```
simple_name ::=
    identifier

name ::=
    simple_name { . simple_name }
```

13.3 Declarations

Declarations introduce new entities that denote types, variables, attributes, and packages. Some declarations can only appear immediately within a project declaration. Others can appear within a project or within a package.

Syntax:

```
declarative_item ::=
    simple_declarative_item |
    typed_string_declaration |
    package_declaration

simple_declarative_item ::=
    variable_declaration |
    typed_variable_declaration |
    attribute_declaration |
    case_construction |
    empty_declaration
```

13.4 Empty declarations

```
empty_declaration ::=
    null ;
```

An empty declaration is allowed anywhere a declaration is allowed. It has no effect.

13.5 Typed string declarations

Typed strings are sequences of string literals. Typed strings are the only named types in project files. They are used in case constructions, where they provide support for conditional attribute definitions.

Syntax:

```
typed_string_declaration ::=
  type <typed_string>_simple_name is
    ( string_literal {, string_literal} );
```

A typed string declaration can only appear immediately within a project declaration.

All the string literals in a typed string declaration must be distinct.

13.6 Variables

Variables denote values, and appear as constituents of expressions.

```
typed_variable_declaration ::=
  <typed_variable>_simple_name : <typed_string>_name := string_expression ;

variable_declaration ::=
  <variable>_simple_name := expression;
```

The elaboration of a variable declaration introduces the variable and assigns to it the value of the expression. The name of the variable is available after the assignment symbol.

A `typed_variable` can only be declare once.

a non-typed variable can be declared multiple times.

Before the completion of its first declaration, the value of variable is the null string.

13.7 Expressions

An expression is a formula that defines a computation or retrieval of a value. In a project file the value of an expression is either a string or a list of strings. A string value in an expression is either a literal, the current value of a variable, an external value, an attribute reference, or a concatenation operation.

Syntax:

```
expression ::=
  term {& term}

term ::=
  string_literal |
  string_list |
  <variable>_name |
  external_value |
  attribute_reference

string_literal ::=
  (same as Ada)

string_list ::=
  ( <string>_expression { , <string>_expression } )
```


13.7.1 Concatenation

The following concatenation functions are defined:

```
function "&" (X : String;      Y : String)      return String;
function "&" (X : String_List; Y : String)      return String_List;
function "&" (X : String_List; Y : String_List) return String_List;
```

13.8 Attributes

An attribute declaration defines a property of a project or package. This property can later be queried by means of an attribute reference. Attribute values are strings or string lists.

Some attributes are associative arrays. These attributes are mappings whose domain is a set of strings. These attributes are declared one association at a time, by specifying a point in the domain and the corresponding image of the attribute. They may also be declared as a full associative array, getting the same associations as the corresponding attribute in an imported or extended project.

Attributes that are not associative arrays are called simple attributes.

Syntax:

```
attribute_declaration ::=
  full_associative_array_declaration |
  for attribute_designator use expression ;

full_associative_array_declaration ::=
  for <associative_array_attribute>simple_name use
  <project>simple_name [ . <package>simple_Name ] ' <attribute>simple_name ;

attribute_designator ::=
  <simple_attribute>simple_name |
  <associative_array_attribute>simple_name ( string_literal )
```

Some attributes are project-specific, and can only appear immediately within a project declaration. Others are package-specific, and can only appear within the proper package.

The expression in an attribute definition must be a string or a string_list. The string literal appearing in the attribute_designator of an associative array attribute is case-insensitive.

13.9 Project Attributes

The following attributes apply to a project. All of them are simple attributes.

Object_Dir

Expression must be a path name. The attribute defines the directory in which the object files created by the build are to be placed. If not specified, object files are placed in the project directory.

Exec_Dir

Expression must be a path name. The attribute defines the directory in which the executables created by the build are to be placed. If not specified, executables are placed in the object directory.

Source_Dirs

Expression must be a list of path names. The attribute defines the directories in which the source files for the project are to be found. If not specified, source

files are found in the project directory. If a string in the list ends with `"/**"`, then the directory that precedes `"/**"` and all of its subdirectories (recursively) are included in the list of source directories.

Excluded_Source_Dirs

Expression must be a list of strings. Each entry designates a directory that is not to be included in the list of source directories of the project. This is normally used when there are strings ending with `"/**"` in the value of attribute `Source_Dirs`.

Source_Files

Expression must be a list of file names. The attribute defines the individual files, in the project directory, which are to be used as sources for the project. File names are path_names that contain no directory information. If the project has no sources the attribute must be declared explicitly with an empty list.

Excluded_Source_Files (Locally_Removed_Files)

Expression must be a list of strings that are legal file names. Each file name must designate a source that would normally be a source file in the source directories of the project or, if the project file is an extending project file, inherited by the current project file. It cannot designate an immediate source that is not inherited. Each of the source files in the list are not considered to be sources of the project file: they are not inherited. Attribute `Locally_Removed_Files` is obsolescent, attribute `Excluded_Source_Files` is preferred.

Source_List_File

Expression must be a single path name. The attribute defines a text file that contains a list of source file names to be used as sources for the project

Library_Dir

Expression must be a path name. The attribute defines the directory in which a library is to be built. The directory must exist, must be distinct from the project's object directory, and must be writable.

Library_Name

Expression must be a string that is a legal file name, without extension. The attribute defines a string that is used to generate the name of the library to be built by the project.

Library_Kind

Argument must be a string value that must be one of the following `"static"`, `"dynamic"` or `"relocatable"`. This string is case-insensitive. If this attribute is not specified, the library is a static library. Otherwise, the library may be dynamic or relocatable. This distinction is operating-system dependent.

Library_Version

Expression must be a string value whose interpretation is platform dependent. On UNIX, it is used only for dynamic/relocatable libraries as the internal name of the library (the `"soname"`). If the library file name (built from the `Library_Name`) is different from the `Library_Version`, then the library file will be a symbolic link to the actual file whose name will be `Library_Version`.

Library_Interface

Expression must be a string list. Each element of the string list must designate a unit of the project. If this attribute is present in a Library Project File, then the project file is a Stand-alone Library_Project_File.

Library_Auto_Init

Expression must be a single string "true" or "false", case-insensitive. If this attribute is present in a Stand-alone Library Project File, it indicates if initialization is automatic when the dynamic library is loaded.

Library_Options

Expression must be a string list. Indicates additional switches that are to be used when building a shared library.

Library_GCC

Expression must be a single string. Designates an alternative to "gcc" for building shared libraries.

Library_Src_Dir

Expression must be a path name. The attribute defines the directory in which the sources of the interfaces of a Stand-alone Library will be copied. The directory must exist, must be distinct from the project's object directory and source directories of all projects in the project tree, and must be writable.

Library_Src_Dir

Expression must be a path name. The attribute defines the directory in which the ALI files of a Library will be copied. The directory must exist, must be distinct from the project's object directory and source directories of all projects in the project tree, and must be writable.

Library_Symbol_File

Expression must be a single string. Its value is the single file name of a symbol file to be created when building a stand-alone library when the symbol policy is either "compliant", "controlled" or "restricted", on platforms that support symbol control, such as VMS. When symbol policy is "direct", then a file with this name must exist in the object directory.

Library_Reference_Symbol_File

Expression must be a single string. Its value is the path name of a reference symbol file that is read when the symbol policy is either "compliant" or "controlled", on platforms that support symbol control, such as VMS, when building a stand-alone library. The path may be an absolute path or a path relative to the project directory.

Library_Symbol_Policy

Expression must be a single string. Its case-insensitive value can only be "autonomous", "default", "compliant", "controlled", "restricted" or "direct".

This attribute is not taken into account on all platforms. It controls the policy for exported symbols and, on some platforms (like VMS) that have the notions of major and minor IDs built in the library files, it controls the setting of these IDs.

"autonomous" or "default": exported symbols are not controlled.

"compliant": if attribute `Library_Reference_Symbol_File` is not defined, then it is equivalent to policy "autonomous". If there are exported symbols in the reference symbol file that are not in the object files of the interfaces, the major ID of the library is increased. If there are symbols in the object files of the interfaces that are not in the reference symbol file, these symbols are put at the end of the list in the newly created symbol file and the minor ID is increased.

"controlled": the attribute `Library_Reference_Symbol_File` must be defined. The library will fail to build if the exported symbols in the object files of the interfaces do not match exactly the symbol in the symbol file.

"restricted": The attribute `Library_Symbol_File` must be defined. The library will fail to build if there are symbols in the symbol file that are not in the exported symbols of the object files of the interfaces. Additional symbols in the object files are not added to the symbol file.

"direct": The attribute `Library_Symbol_File` must be defined and must designate an existing file in the object directory. This symbol file is passed directly to the underlying linker without any symbol processing.

Main Expression must be a list of strings that are legal file names. These file names designate existing compilation units in the source directory that are legal main subprograms.

When a project file is elaborated, as part of the execution of a `gnatmake` command, one or several executables are built and placed in the `Exec.Dir`. If the `gnatmake` command does not include explicit file names, the executables that are built correspond to the files specified by this attribute.

Externally_Built

Expression must be a single string. Its value must be either "true" or "false", case-insensitive. The default is "false". When the value of this attribute is "true", no attempt is made to compile the sources or to build the library, when the project is a library project.

Main_Language

This is a simple attribute. Its value is a string that specifies the language of the main program.

Languages

Expression must be a string list. Each string designates a programming language that is known to GNAT. The strings are case-insensitive.

13.10 Attribute References

Attribute references are used to retrieve the value of previously defined attribute for a package or project. Syntax:

```
attribute_reference ::=
  attribute_prefix ' <simple_attribute>simple_name [ ( string_literal ) ]

attribute_prefix ::=
  project |
```

```

<project_simple_name | package_identifier |
<project_>simple_name . package_identifier

```

If an attribute has not been specified for a given package or project, its value is the null string or the empty list.

13.11 External Values

An external value is an expression whose value is obtained from the command that invoked the processing of the current project file (typically a gnatmake command).

Syntax:

```

external_value ::=
  external ( string_literal [, string_literal] )

```

The first `string_literal` is the string to be used on the command line or in the environment to specify the external value. The second `string_literal`, if present, is the default to use if there is no specification for this external value either on the command line or in the environment.

13.12 Case Construction

A case construction supports attribute and variable declarations that depend on the value of a previously declared variable.

Syntax:

```

case_construction ::=
  case <typed_variable_>name is
    {case_item}
  end case ;

case_item ::=
  when discrete_choice_list =>
    {case_construction |
     attribute_declaration |
     variable_declaration |
     empty_declaration}

discrete_choice_list ::=
  string_literal { | string_literal } |
  others

```

Inside a case construction, variable declarations must be for variables that have already been declared before the case construction.

All choices in a choice list must be distinct. The choice lists of two distinct alternatives must be disjoint. Unlike Ada, the choice lists of all alternatives do not need to include all values of the type. An `others` choice must appear last in the list of alternatives.

13.13 Packages

A package provides a grouping of variable declarations and attribute declarations to be used when invoking various GNAT tools. The name of the package indicates the tool(s) to which it applies. Syntax:

```

package_declaration ::=
  package_spec | package_renaming

package_spec ::=

```

```

package package_identifier is
  {simple_declarative_item}
end package_identifier ;

package_identifier ::=
  Naming | Builder | Compiler | Binder |
  Linker | Finder  | Cross_Reference |
  gnats | IDE      | Pretty_Printer | Check

```

13.13.1 Package Naming

The attributes of a **Naming** package specifies the naming conventions that apply to the source files in a project. When invoking other GNAT tools, they will use the sources in the source directories that satisfy these naming conventions.

The following attributes apply to a **Naming** package:

Casing This is a simple attribute whose value is a string. Legal values of this string are "lowercase", "uppercase" or "mixedcase". These strings are themselves case insensitive.

If **Casing** is not specified, then the default is "lowercase".

Dot_Replacement

This is a simple attribute whose string value satisfies the following requirements:

- It must not be empty
- It cannot start or end with an alphanumeric character
- It cannot be a single underscore
- It cannot start with an underscore followed by an alphanumeric
- It cannot contain a dot '.' if longer than one character

If **Dot_Replacement** is not specified, then the default is "-".

Spec_Suffix

This is an associative array attribute, defined on language names, whose image is a string that must satisfy the following conditions:

- It must not be empty
- It cannot start with an alphanumeric character
- It cannot start with an underscore followed by an alphanumeric character

For Ada, the attribute denotes the suffix used in file names that contain library unit declarations, that is to say units that are package and subprogram declarations. If **Spec_Suffix** ("Ada") is not specified, then the default is ".ads".

For C and C++, the attribute denotes the suffix used in file names that contain prototypes.

Body_Suffix

This is an associative array attribute defined on language names, whose image is a string that must satisfy the following conditions:

- It must not be empty
- It cannot start with an alphanumeric character

- It cannot start with an underscore followed by an alphanumeric character
- It cannot be a suffix of `Spec_Suffix`

For Ada, the attribute denotes the suffix used in file names that contain library bodies, that is to say units that are package and subprogram bodies. If `Body_Suffix` ("Ada") is not specified, then the default is ".adb".

For C and C++, the attribute denotes the suffix used in file names that contain source code.

`Separate_Suffix`

This is a simple attribute whose value satisfies the same conditions as `Body_Suffix`.

This attribute is specific to Ada. It denotes the suffix used in file names that contain separate bodies. If it is not specified, then it defaults to same value as `Body_Suffix` ("Ada").

`Spec`

This is an associative array attribute, specific to Ada, defined over compilation unit names. The image is a string that is the name of the file that contains that library unit. The file name is case sensitive if the conventions of the host operating system require it.

`Body`

This is an associative array attribute, specific to Ada, defined over compilation unit names. The image is a string that is the name of the file that contains the library unit body for the named unit. The file name is case sensitive if the conventions of the host operating system require it.

`Specification_Exceptions`

This is an associative array attribute defined on language names, whose value is a list of strings.

This attribute is not significant for Ada.

For C and C++, each string in the list denotes the name of a file that contains prototypes, but whose suffix is not necessarily the `Spec_Suffix` for the language.

`Implementation_Exceptions`

This is an associative array attribute defined on language names, whose value is a list of strings.

This attribute is not significant for Ada.

For C and C++, each string in the list denotes the name of a file that contains source code, but whose suffix is not necessarily the `Body_Suffix` for the language.

The following attributes of package `Naming` are obsolescent. They are kept as synonyms of other attributes for compatibility with previous versions of the Project Manager.

`Specification_Suffix`

This is a synonym of `Spec_Suffix`.

`Implementation_Suffix`

This is a synonym of `Body_Suffix`.

Specification

This is a synonym of `Spec`.

Implementation

This is a synonym of `Body`.

13.13.2 package Compiler

The attributes of the `Compiler` package specify the compilation options to be used by the underlying compiler.

Default_Switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies the compilation options to be used when compiling a component written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies the compilation options to be used when compiling the named file. If a file is not specified in the `Switches` attribute, it is compiled with the options specified by `Default_Switches` of its language, if defined.

Local_Configuration_Pragmas.

This is a simple attribute, whose value is a path name that designates a file containing configuration pragmas to be used for all invocations of the compiler for immediate sources of the project.

13.13.3 package Builder

The attributes of package `Builder` specify the compilation, binding, and linking options to be used when building an executable for a project. The following attributes apply to package `Builder`:

Default_Switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when building a main written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when building the named main file. If a main file is not specified in the `Switches` attribute, it is built with the options specified by `Default_Switches` of its language, if defined.

Global_Configuration_Pragmas

This is a simple attribute, whose value is a path name that designates a file that contains configuration pragmas to be used in every build of an executable. If both local and global configuration pragmas are specified, a compilation makes use of both sets.

Executable

This is an associative array attribute. Its domain is a set of main source file names. Its range is a simple string that specifies the executable file name to be used when linking the specified main source. If a main source is not specified in

the Executable attribute, the executable file name is deducted from the main source file name. This attribute has no effect if its value is the empty string.

Executable_Suffix

This is a simple attribute whose value is the suffix to be added to the executables that don't have an attribute Executable specified.

13.13.4 package Gnatls

The attributes of package **Gnatls** specify the tool options to be used when invoking the library browser **gnatls**. The following attributes apply to package **Gnatls**:

Switches This is a single attribute with a string list value. Each nonempty string in the list is an option when invoking **gnatls**.

13.13.5 package Binder

The attributes of package **Binder** specify the options to be used when invoking the binder in the construction of an executable. The following attributes apply to package **Binder**:

Default_Switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when binding a main written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when binding the named main file. If a main file is not specified in the **Switches** attribute, it is bound with the options specified by **Default_Switches** of its language, if defined.

13.13.6 package Linker

The attributes of package **Linker** specify the options to be used when invoking the linker in the construction of an executable. The following attributes apply to package **Linker**:

Default_Switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when linking a main written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when linking the named main file. If a main file is not specified in the **Switches** attribute, it is linked with the options specified by **Default_Switches** of its language, if defined.

Linker_Options

This is a string list attribute. Its value specifies additional options that be given to the linker when linking an executable. This attribute is not used in the main project, only in projects imported directly or indirectly.

13.13.7 package Cross_Reference

The attributes of package **Cross_Reference** specify the tool options to be used when invoking the library tool **gnatxref**. The following attributes apply to package **Cross_Reference**:

Default_Switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when calling **gnatxref** on a source written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when calling **gnatxref** on the named main source. If a source is not specified in the **Switches** attribute, **gnatxref** will be called with the options specified by **Default_Switches** of its language, if defined.

13.13.8 package Finder

The attributes of package **Finder** specify the tool options to be used when invoking the search tool **gnatfind**. The following attributes apply to package **Finder**:

Default_Switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when calling **gnatfind** on a source written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when calling **gnatfind** on the named main source. If a source is not specified in the **Switches** attribute, **gnatfind** will be called with the options specified by **Default_Switches** of its language, if defined.

13.13.9 package Check

The attributes of package **Check** specify the checking rule options to be used when invoking the checking tool **gnatcheck**. The following attributes apply to package **Check**:

Default_switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when calling **gnatcheck** on a source written in that language. The first string in the range should always be **"-rules"** to specify that all the other options belong to the **-rules** section of the parameters of **gnatcheck** call.

13.13.10 package Pretty_Printer

The attributes of package **Pretty_Printer** specify the tool options to be used when invoking the formatting tool **gnatpp**. The following attributes apply to package **Pretty_Printer**:

Default_switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when calling **gnatpp** on a source written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when calling **gnatpp** on the named main source. If a source is not specified in the **Switches** attribute, **gnatpp** will be called with the options specified by **Default_Switches** of its language, if defined.

13.13.11 package **gnatstub**

The attributes of package **gnatstub** specify the tool options to be used when invoking the tool **gnatstub**. The following attributes apply to package **gnatstub**:

Default_switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when calling **gnatstub** on a source written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when calling **gnatstub** on the named main source. If a source is not specified in the **Switches** attribute, **gnatpp** will be called with the options specified by **Default_Switches** of its language, if defined.

13.13.12 package **Eliminate**

The attributes of package **Eliminate** specify the tool options to be used when invoking the tool **gnatelim**. The following attributes apply to package **Eliminate**:

Default_switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when calling **gnatelim** on a source written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when calling **gnatelim** on the named main source. If a source is not specified in the **Switches** attribute, **gnatelim** will be called with the options specified by **Default_Switches** of its language, if defined.

13.13.13 package **Metrics**

The attributes of package **Metrics** specify the tool options to be used when invoking the tool **gnatmetric**. The following attributes apply to package **Metrics**:

Default_switches

This is an associative array attribute. Its domain is a set of language names. Its range is a string list that specifies options to be used when calling **gnatmetric** on a source written in that language, for which no file-specific switches have been specified.

Switches This is an associative array attribute. Its domain is a set of file names. Its range is a string list that specifies options to be used when calling **gnatmetric**

on the named main source. If a source is not specified in the Switches attribute, `gnatmetric` will be called with the options specified by `Default_Switches` of its language, if defined.

13.13.14 package IDE

The attributes of package IDE specify the options to be used when using an Integrated Development Environment such as GPS.

Remote_Host

This is a simple attribute. Its value is a string that designates the remote host in a cross-compilation environment, to be used for remote compilation and debugging. This field should not be specified when running on the local machine.

Program_Host

This is a simple attribute. Its value is a string that specifies the name of IP address of the embedded target in a cross-compilation environment, on which the program should execute.

Communication_Protocol

This is a simple string attribute. Its value is the name of the protocol to use to communicate with the target in a cross-compilation environment, e.g. `"wtx"` or `"vxworks"`.

Compiler_Command

This is an associative array attribute, whose domain is a language name. Its value is string that denotes the command to be used to invoke the compiler. The value of `Compiler_Command ("Ada")` is expected to be compatible with `gnatmake`, in particular in the handling of switches.

Debugger_Command

This is simple attribute, Its value is a string that specifies the name of the debugger to be used, such as `gdb`, `powerpc-wrs-vxworks-gdb` or `gdb-4`.

Default_Switches

This is an associative array attribute. Its indexes are the name of the external tools that the GNAT Programming System (GPS) is supporting. Its value is a list of switches to use when invoking that tool.

Gnatlist This is a simple attribute. Its value is a string that specifies the name of the `gnatls` utility to be used to retrieve information about the predefined path; e.g., `"gnatls"`, `"powerpc-wrs-vxworks-gnatls"`.

VCS_Kind This is a simple attribute. Its value is a string used to specify the Version Control System (VCS) to be used for this project, e.g. `CVS`, `RCS` `ClearCase` or `Perforce`.

VCS_File_Check

This is a simple attribute. Its value is a string that specifies the command used by the VCS to check the validity of a file, either when the user explicitly asks for a check, or as a sanity check before doing the check-in.

VCS_Log_Check

This is a simple attribute. Its value is a string that specifies the command used by the VCS to check the validity of a log file.

VCS_Repository_Root

The VCS repository root path. This is used to create tags or branches of the repository. For subversion the value should be the URL as specified to check-out the working copy of the repository.

VCS_Patch_Root

The local root directory to use for building patch file. All patch chunks will be relative to this path. The root project directory is used if this value is not defined.

13.14 Package Renamings

A package can be defined by a renaming declaration. The new package renames a package declared in a different project file, and has the same attributes as the package it renames. Syntax:

```
package_renaming ::=
  package package_identifier renames
    <project_>simple_name.package_identifier ;
```

The package_identifier of the renamed package must be the same as the package_identifier. The project whose name is the prefix of the renamed package must contain a package declaration with this name. This project must appear in the context_clause of the enclosing project declaration, or be the parent project of the enclosing child project.

13.15 Projects

A project file specifies a set of rules for constructing a software system. A project file can be self-contained, or depend on other project files. Dependencies are expressed through a context clause that names other projects.

Syntax:

```
project ::=
  context_clause project_declaration

project_declaration ::=
  simple_project_declaration | project_extension

simple_project_declaration ::=
  project <project_>simple_name is
    {declarative_item}
  end <project_>simple_name;

context_clause ::=
  {with_clause}

with_clause ::=
  [limited] with path_name { , path_name } ;

path_name ::=
  string_literal
```

A path name denotes a project file. A path name can be absolute or relative. An absolute path name includes a sequence of directories, in the syntax of the host operating system, that identifies uniquely the project file in the file system. A relative path name identifies the project file, relative to the directory that contains the current project, or relative to a directory listed in the environment variable `ADA_PROJECT_PATH`. Path names are case sensitive if file names in the host operating system are case sensitive.

The syntax of the environment variable `ADA_PROJECT_PATH` is a list of directory names separated by colons (semicolons on Windows).

A given project name can appear only once in a `context_clause`.

It is illegal for a project imported by a context clause to refer, directly or indirectly, to the project in which this context clause appears (the dependency graph cannot contain cycles), except when one of the `with_clause` in the cycle is a **limited with**.

13.16 Project Extensions

A project extension introduces a new project, which inherits the declarations of another project. Syntax:

```
project_extension ::=
  project <project_>simple_name extends path_name is
    {declarative_item}
  end <project_>simple_name;
```

The project extension declares a child project. The child project inherits all the declarations and all the files of the parent project. These inherited declaration can be overridden in the child project, by means of suitable declarations.

13.17 Project File Elaboration

A project file is processed as part of the invocation of a gnat tool that uses the project option. Elaboration of the process file consists in the sequential elaboration of all its declarations. The computed values of attributes and variables in the project are then used to establish the environment in which the gnat tool will execute.

14 Obsolescent Features

This chapter describes features that are provided by GNAT, but are considered obsolescent since there are preferred ways of achieving the same effect. These features are provided solely for historical compatibility purposes.

14.1 `pragma No_Run_Time`

The `pragma No_Run_Time` is used to achieve an affect similar to the use of the "Zero Foot Print" configurable run time, but without requiring a specially configured run time. The result of using this pragma, which must be used for all units in a partition, is to restrict the use of any language features requiring run-time support code. The preferred usage is to use an appropriately configured run-time that includes just those features that are to be made accessible.

14.2 `pragma Ravenscar`

The `pragma Ravenscar` has exactly the same effect as `pragma Profile (Ravenscar)`. The latter usage is preferred since it is part of the new Ada 2005 standard.

14.3 `pragma Restricted_Run_Time`

The `pragma Restricted_Run_Time` has exactly the same effect as `pragma Profile (Restricted)`. The latter usage is preferred since the Ada 2005 `pragma Profile` is intended for this kind of implementation dependent addition.

GNU Free Documentation License

Version 1.2, November 2002

Copyright © 2000,2001,2002 Free Software Foundation, Inc.
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document *free* in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or non-commercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft”, which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The “Document”, below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “you”. You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A “Modified Version” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “Secondary Section” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The “Invariant Sections” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released

under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none. The “Cover Texts” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A “Transparent” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not “Transparent” is called “Opaque”.

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The “Title Page” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

A section “Entitled XYZ” means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as “Acknowledgements”, “Dedications”, “Endorsements”, or “History”.) To “Preserve the Title” of such a section when you modify the Document means that it remains a section “Entitled XYZ” according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and

that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.

- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
- C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
- D. Preserve all the copyright notices of the Document.
- E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- H. Include an unaltered copy of this License.
- I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
- K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
- L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
- N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
- O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled “Endorsements”, provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled “History” in the various original documents, forming one section Entitled “History”; likewise combine any sections Entitled “Acknowledgements”, and any sections Entitled “Dedications”. You must delete all sections Entitled “Endorsements.”

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called

an “aggregate” if the copyright resulting from the compilation is not used to limit the legal rights of the compilation’s users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document’s Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled “Acknowledgements”, “Dedications”, or “History”, the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided for under this License. Any other attempt to copy, modify, sublicense or distribute the Document is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation.

ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

```
Copyright (C)  year  your name.
Permission is granted to copy, distribute and/or modify this document
under the terms of the GNU Free Documentation License, Version 1.2
or any later version published by the Free Software Foundation;
with no Invariant Sections, no Front-Cover Texts, and no Back-Cover
Texts. A copy of the license is included in the section entitled ‘‘GNU
Free Documentation License’’.
```

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the “with...Texts.” line with this:

```
with the Invariant Sections being list their titles, with
the Front-Cover Texts being list, and with the Back-Cover Texts
being list.
```

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

Index

-
- '-gnatR' switch 153
-
- '__lock' file (for shared passive packages).... 207
- A**
- Abort_Defer 5
- Abort_Signal 63
- Access values, testing for 67
- Access, unrestricted 73
- Accuracy requirements 100
- Accuracy, complex arithmetic 100
- Ada 2005 Language Reference Manual 2
- Ada 83 attributes 65, 66, 68, 71
- Ada 95 Language Reference Manual 2
- Ada.Characters.Handling 88
- Ada.Characters.Latin_9 ('a-chlat9.ads')... 183
- Ada.Characters.Wide_Latin_1 ('a-cwila1.ads')
..... 183
- Ada.Characters.Wide_Latin_9 ('a-cwila1.ads')
..... 183
- Ada.Characters.Wide_Wide_Latin_1
('a-chzla1.ads') 184
- Ada.Characters.Wide_Wide_Latin_9
('a-chzla9.ads') 184
- Ada.Command_Line.Environment ('a-colien.ads')
..... 184
- Ada.Command_Line.Remove ('a-colire.ads').. 184
- Ada.Command_Line.Response_File
('a-clrefi.ads') 184
- Ada.Direct_IO.C_Streams ('a-diocst.ads').. 184
- Ada.Exceptions.Is_Null_Occurrence
('a-einuoc.ads') 184
- Ada.Exceptions.Last_Chance_Handler
('a-elchha.ads') 184
- Ada.Exceptions.Traceback ('a-extra.ads')
..... 185
- Ada.Sequential_IO.C_Streams ('a-siocst.ads')
..... 185
- Ada.Streams.Stream_IO.C_Streams
('a-ssicst.ads') 185
- Ada.Strings.Unbounded.Text_IO
('a-suteio.ads') 185
- Ada.Strings.Wide_Unbounded.Wide_Text_IO
('a-swuwti.ads') 185
- Ada.Strings.Wide_Wide_Unbounded.Wide_Wide_
Text_IO ('a-szuzti.ads') 185
- Ada.Text_IO.C_Streams ('a-tiocst.ads').... 185
- Ada.Text_IO.Reset_Standard_Files
('a-tirsfi.ads') 185
- Ada.Wide_Characters.Unicode ('a-wichun.ads')
..... 185
- Ada.Wide_Text_IO.C_Streams ('a-wtcstr.ads')
..... 186
- Ada.Wide_Text_IO.Reset_Standard_Files
('a-wrstfi.ads') 186
- Ada.Wide_Wide_Characters.Unicode
('a-zchuni.ads') 186
- Ada.Wide_Wide_Text_IO.C_Streams
('a-ztcstr.ads') 186
- Ada.Wide_Wide_Text_IO.Reset_Standard_Files
('a-zrstfi.ads') 186
- Ada_05 6
- Ada_2005 6
- Ada_83 5
- Ada_95 5
- Address Clause 149
- Address clauses 81
- Address image 197
- Address of subprogram code 64
- Address, as private type 86
- Address, operations of 86
- Address_Size 63
- Alignment Clause 133
- Alignment clauses 82
- Alignment, default 134
- Alignment, default settings 40
- Alignment, maximum 68
- Alignments of components 12
- Alternative Character Sets 77
- Altivec 186, 187
- Annex E 206
- Annotate 6
- Argument passing mechanisms 18
- Array packing 26
- Array splitter 187
- Arrays, extendable 190, 195
- Arrays, multidimensional 78
- Asm_Input 63
- Asm_Output 63
- Assert 6
- Assert.Failure, exception 197
- Assertions 8, 197
- Assertions, control 9
- Assume_No_Invalid_Values 7
- Ast_Entry 8
- AST_Entry 64
- Attribute 151
- AWK 187
- B**
- Biased representation 137
- Big endian 65
- Bit 64

bit ordering	141
Bit ordering	85
Bit_Order Clause	141
Bit_Position	64
Bounded Buffers	187
Bounded errors	75
Bounded-length strings	88
Bubble sort	187
byte ordering	142
Byte swapping	188

C

C streams, interfacing	196
C Streams, Interfacing with Direct_IO	184
C Streams, Interfacing with Sequential_IO	185
C Streams, Interfacing with Stream_IO	185
C Streams, Interfacing with Text_IO	185
C Streams, Interfacing with Wide_Text_IO	186
C Streams, Interfacing with Wide_Wide_Text_IO	186
C++ interfacing	196
C, interfacing with	91
C_Pass_By_Copy	8
Calendar	188
Capturing Old values	69
Casing of External names	22
Casing utilities	188
CGI (Common Gateway Interface)	188
CGI (Common Gateway Interface) cookie support	188
CGI (Common Gateway Interface) debugging	188
Character handling (GNAT.Case_Util)	188
Character Sets	77
Check	8, 9
Check names, defining	9
Check pragma control	9
Check_Name	9
Checks, postconditions	42
Checks, preconditions	44
Checks, suppression of	79
Child Units	75
COBOL support	98
COBOL, interfacing with	92
Code_Address	64
Command line	188
Command line, argument removal	184
Command line, handling long command lines ..	184
Command line, response file	184
Comment	10
Common_Object	10
Compile_Time_Error	10
Compile_Time_Warning	11
Compiler Version	189
Compiler_Unit	11
Compiler_Version	64
Complete_Representation	11

Complex arithmetic accuracy	100
Complex elementary functions	99
Complex types	99
Complex_Representation	12
Component Clause	148
Component_Alignment	12
Component_Size	12
Component_Size Clause	140
Component_Size clauses	83
Component_Size_4	12
Controlling assertions	9
Convention, effect on representation	152
Convention_Identifier	13
Conventions, synonyms	13
Conventions, typographical	2
Cookie support in CGI	188
CPP_Class	13
CPP_Constructor	14
CPP_Virtual	14
CPP_Vtable	14
CRC32	188
Current exception	189
Current time	195
Cyclic Redundancy Check	188

D

Debug	15
Debug pools	189
Debug_Policy	15
Debugging	189, 190
debugging with Initialize Scalars	29
Dec Ada 83	21
Dec Ada 83 casing compatibility	22
Decimal radix support	98
Decoding strings	189
Decoding UTF-8 strings	189
Default_Bit_Order	65
Deferring aborts	5
Defining check names	9
Detect_Blocking	15
Directory operations	189
Directory operations iteration	189
Discriminants, testing for	67
Distribution Systems Annex	206
Dump Memory	192
Duration'Small	78

E

Elab_Body	65
Elab_Spec	65
Elaborated	65
Elaboration control	15
Elaboration_Checks	15
Eliminate	15
Elimination of unused subprograms	15
Emax	65

- Enabled 66
 - Enclosing_Entity 129
 - Encoding strings 190
 - Encoding UTF-8 strings 190
 - Endian 188
 - Entry queuing policies 96
 - Enum_Rep 66
 - Enum_Val 66
 - Enumeration representation clauses 84
 - Enumeration values 77
 - Environment entries 184
 - Epsilon 66
 - Error detection 75
 - Exception actions 190
 - Exception information 79
 - Exception retrieval 189
 - Exception traces 190
 - Exception, obtaining most recent 192
 - Exception_Information' 129
 - Exception_Message 130
 - Exception_Name 130
 - Exceptions, Pure 190
 - Export 89, 151
 - Export_Exception 17
 - Export_Function 18
 - Export_Object 19
 - Export_Procedure 19
 - Export_Value 20
 - Export_Valued_Procedure 20
 - Extend_System 21
 - External 22
 - External Names, casing 22
 - External_Name_Casing 22
- F**
- Fast_Math 23
 - Favor_Top_Level 24
 - FDL, GNU Free Documentation License 231
 - File 130
 - File locking 192
 - Finalize_Storage_Only 24
 - Fixed_Value 67
 - Float types 78
 - Float_Representation 24
 - Floating-Point Processor 191
 - Foreign threads 195
 - Forking a new process 206
 - Fortran, interfacing with 92
- G**
- Get_Immediate 89
 - Get_Immediate 171
 - Get_Immediate, VxWorks 197
 - Global storage pool 197
 - GNAT.Altivec ('g-altivec.ads') 186
 - GNAT.Altivec.Conversions ('g-altcon.ads') 186
 - GNAT.Altivec.Vector_Operations ('g-alveop.ads') 186
 - GNAT.Altivec.Vector_Types ('g-alvety.ads') 186
 - GNAT.Altivec.Vector_Views ('g-alvevi.ads') 187
 - GNAT.Array_Split ('g-arrspl.ads') 187
 - GNAT.AWK ('g-awk.ads') 187
 - GNAT.Bounded_Buffers ('g-boubuf.ads') 187
 - GNAT.Bounded-Mailboxes ('g-boumai.ads') 187
 - GNAT.Bubble_Sort ('g-bubsort.ads') 187
 - GNAT.Bubble_Sort_A ('g-busora.ads') 187
 - GNAT.Bubble_Sort_G ('g-busorg.ads') 187
 - GNAT.Byte_Order_Mark ('g-byorma.ads') 187
 - GNAT.Byte_Swapping ('g-bytswa.ads') 188
 - GNAT.Calendar ('g-calend.ads') 188
 - GNAT.Calendar.Time_IO ('g-catiio.ads') 188
 - GNAT.Case_Util ('g-casuti.ads') 188
 - GNAT.CGI ('g-cgi.ads') 188
 - GNAT.CGI.Cookie ('g-cgicoo.ads') 188
 - GNAT.CGI.Debug ('g-cgideb.ads') 188
 - GNAT.Command_Line ('g-comlin.ads') 188
 - GNAT.Compiler_Version ('g-comver.ads') 189
 - GNAT.CRC32 ('g-crc32.ads') 188
 - GNAT.Ctrl_C ('g-ctrl.c.ads') 189
 - GNAT.Current_Exception ('g-curexc.ads') 189
 - GNAT.Debug_Pools ('g-debpoo.ads') 189
 - GNAT.Debug_Uutilities ('g-debuti.ads') 189
 - GNAT.Decode_String ('g-decstr.ads') 189
 - GNAT.Decode_UTF8_String ('g-deutst.ads') 189
 - GNAT.Directory_Operations ('g-dirope.ads') 189
 - GNAT.Directory_Operations.Iteration ('g-diopit.ads') 189
 - GNAT.Dynamic_HTables ('g-dynhta.ads') 190
 - GNAT.Dynamic_Tables ('g-dyntab.ads') 190
 - GNAT.Encode_String ('g-encstr.ads') 190
 - GNAT.Encode_UTF8_String ('g-enutst.ads') 190
 - GNAT.Exception_Actions ('g-exact.ads') 190
 - GNAT.Exception_Traces ('g-extra.ads') 190
 - GNAT.Exceptions ('g-expect.ads') 190
 - GNAT.Expect ('g-expect.ads') 190
 - GNAT.Float_Control ('g-flocon.ads') 191
 - GNAT.Heap_Sort ('g-heasor.ads') 191
 - GNAT.Heap_Sort_A ('g-hesora.ads') 191
 - GNAT.Heap_Sort_G ('g-hesorg.ads') 191
 - GNAT.HTable ('g-htable.ads') 191
 - GNAT.IO ('g-io.ads') 191
 - GNAT.IO_Aux ('g-io_aux.ads') 191
 - GNAT.Lock_Files ('g-locfil.ads') 192
 - GNAT.MD5 ('g-md5.ads') 192
 - GNAT.Memory_Dump ('g-memdum.ads') 192
 - GNAT.Most_Recent_Exception ('g-moreex.ads') 192
 - GNAT.OS_Lib ('g-os_lib.ads') 192

GNAT.Perfect_Hash_Generators ('g-pehage.ads')	192
GNAT.Random_Numbers ('g-rannum.ads')	192
GNAT.Regexp ('g-regexp.ads')	192
GNAT.Registry ('g-regist.ads')	192
GNAT.Regpat ('g-regpat.ads')	193
GNAT.Secondary_Stack_Info ('g-sestin.ads')	193
GNAT.Semaphores ('g-semaph.ads')	193
GNAT.Serial_Communications ('g-sercom.ads')	193
GNAT.SHA1 ('g-sha1.ads')	193
GNAT.SHA224 ('g-sha224.ads')	193
GNAT.SHA256 ('g-sha256.ads')	193
GNAT.SHA384 ('g-sha384.ads')	193
GNAT.SHA512 ('g-sha512.ads')	193
GNAT.Signals ('g-signal.ads')	193
GNAT.Sockets ('g-socket.ads')	193
GNAT.Source_Info ('g-souinf.ads')	194
GNAT.Spelling_Checker ('g-speche.ads')	194
GNAT.Spelling_Checker_Generic ('g-spchge.ads')	194
GNAT.Spitbol ('g-spitbo.ads')	194
GNAT.Spitbol.Patterns ('g-spiPAT.ads')	194
GNAT.Spitbol.Table_Boolean ('g-sptabo.ads')	194
GNAT.Spitbol.Table_Integer ('g-sptain.ads')	194
GNAT.Spitbol.Table_VString ('g-sptavs.ads')	194
GNAT.SSE ('g-sse.ads')	194
GNAT.SSE.Vector_Types ('g-ssvety.ads')	195
GNAT.String_Split ('g-strspl.ads')	195
GNAT.Strings ('g-string.ads')	195
GNAT.Table ('g-table.ads')	195
GNAT.Task_Lock ('g-tasloc.ads')	195
GNAT.Threads ('g-thread.ads')	195
GNAT.Time_Stamp ('g-timsta.ads')	195
GNAT.Traceback ('g-traceb.ads')	195
GNAT.Traceback.Symbolic ('g-trasym.ads')	195
GNAT.UTF_32 ('g-table.ads')	196
GNAT.Wide_Spelling_Checker ('g-u3spch.ads')	196
GNAT.Wide_Spelling_Checker ('g-wispch.ads')	196
GNAT.Wide_String_Split ('g-wistsp.ads')	196
GNAT.Wide_Wide_Spelling_Checker ('g-zspche.ads')	196
GNAT.Wide_Wide_String_Split ('g-zistsp.ads')	196

H

Has_Access_Values	67
Has_Discriminants	67
Hash functions	192
Hash tables	190, 191
Heap usage, implicit	87

I

IBM Packed Format	197
Ident	25
Image, of an address	197
Img	67
Implementation-dependent features	1
Implemented_By_Entry	25
Implicit_Packing	25
Import	151
Import_Exception	26
Import_Function	26
Import_Object	27
Import_Procedure	28
Import_Valued_Procedure	29
Initialization, suppression of	54
Initialize_Scalars	29
Inline_Always	30
Inline_Generic	30
Input/Output facilities	191
Integer maps	194
Integer types	77
Integer_Value	67
Interface	31
Interface_Name	31
Interfaces	90
Interfaces.C.Extensions ('i-cexten.ads')	196
Interfaces.C.Streams ('i-cstrea.ads')	196
Interfaces.CPP ('i-cpp.ads')	196
Interfaces.Packed_Decimal ('i-pacdec.ads')	197
Interfaces.VxWorks ('i-vxwork.ads')	197
Interfaces.VxWorks.IO ('i-vxwoio.ads')	197
Interfacing to C++	14
Interfacing to VxWorks	197
Interfacing to VxWorks' I/O	197
Interfacing with C++	13, 14
Interfacing, to C++	196
Interrupt	189
Interrupt priority, maximum	68
Interrupt support	94
Interrupt_Handler	31
Interrupt_State	31
Interrupts	95
Intrinsic operator	129
Intrinsic Subprograms	129
Invalid representations	7
Invalid values	7
Invalid_Value	68

K

Keep_Names	32
------------	----

L

Large	68
Latin_1 constants for Wide_Character	183
Latin_1 constants for Wide_Wide_Character	184

Latin_9 constants for Character..... 183
 Latin_9 constants for Wide_Character..... 183
 Latin_9 constants for Wide_Wide_Character.. 184
 License..... 33
 License checking..... 33
 Line..... 130
 Link_With..... 34
 Linker_Alias..... 34
 Linker_Constructor..... 34
 Linker_Destructor..... 35
 Linker_Section..... 35
 Little endian..... 65
 Local storage pool..... 197
 Locking..... 195
 Locking Policies..... 96
 Locking using files..... 192
 Long_Float..... 36

M

Machine Code insertions..... 203
 Machine operations..... 93
 Machine_Attribute..... 36
 Machine_Size..... 68
 Mailboxes..... 187
 Main..... 36
 Main_Storage..... 37
 Mantissa..... 68
 Maps..... 194
 Max_Entry_Queue_Length..... 110
 Max_Interrupt_Priority..... 68
 Max_Priority..... 68
 Maximum_Alignment..... 68
 Maximum_Alignment attribute..... 133
 Mechanism_Code..... 68
 Memory allocation..... 197
 Memory corruption debugging..... 189
 Message Digest MD5..... 192
 Multidimensional arrays..... 78

N

Named assertions..... 8, 9
 Named numbers, representation of..... 72
 No_Body..... 37
 No_Calendar..... 110
 No_Default_Initialization..... 110
 No_Direct_Boolean_Operators..... 110
 No_Dispatching_Calls..... 110
 No_Dynamic_Attachment..... 111
 No_Elaboration_Code..... 114
 No_Entry_Calls_In_Elaboration_Code..... 111
 No_Entry_Queue..... 115
 No_Enumeration_Maps..... 111
 No_Exception_Handlers..... 111
 No_Exception_Propagation..... 112
 No_Exception_Registration..... 112
 No_Implementation_Attributes..... 115

No_Implementation_Pragmas..... 115
 No_Implementation_Restrictions..... 115
 No_Implicit_Conditionals..... 112
 No_Implicit_Dynamic_Code..... 112
 No_Implicit_Loops..... 113
 No_Initialize_Scalars..... 113
 No_Local_Protected_Objects..... 113
 No_Protected_Type_Allocators..... 113
 No_Return..... 37
 No_Secondary_Stack..... 113
 No_Select_Statements..... 113
 No_Standard_Storage_Pools..... 113
 No_Streams..... 113
 No_Strict_Aliasing..... 38
 No_Task_Attributes_Package..... 114
 No_Task_Termination..... 114
 No_Tasking..... 114
 No_Wide_Characters..... 115
 Normalize_Scalars..... 38
 Null_Occurrence, testing for..... 184
 Null_Parameter..... 69
 Numerics..... 98

O

Object_Size..... 69, 138
 Obsolescent..... 39
 OpenVMS..... 8, 12, 17, 19, 20, 21, 24, 25, 26, 27,
 36, 37, 56, 64, 69
 Operating System interface..... 192
 Operations, on Address..... 86
 Optimize_Alignment..... 40
 ordering, of bits..... 141
 ordering, of bytes..... 142
 Overlaying of objects..... 152

P

Package Interfaces..... 90
 Package Interrupts..... 95
 Package_Task_Attributes..... 96
 Packed Decimal..... 197
 Packed types..... 80
 Parameters, passing mechanism..... 68
 Parameters, when passed by reference..... 70
 Parsing..... 187
 Partition communication subsystem..... 97
 Partition interfacing functions..... 197
 Passed_By_Reference..... 70
 Passing by copy..... 8
 Passing by descriptor..... 19, 20, 21, 27
 Passive..... 41
 Pattern matching..... 192, 193, 194
 PCS..... 97
 Persistent_BSS..... 41
 Polling..... 42
 Pool_Address..... 70
 Portability..... 1

Postconditions	42
Postconditions.....	69
Pragma Pack (for arrays).....	145
Pragma Pack (for records).....	147
Pragma Pack (for type Natural).....	146
Pragma Pack warning.....	146
pragma Shared_Passive	206
Pragma, representation.....	133
Pragmas.....	75
Pre-elaboration requirements.....	95
Preconditions	44
Preemptive abort.....	96
Priority, maximum.....	68
Protected procedure handlers.....	95
Psect_Object	48
Pure	49
Pure packages, exceptions.....	190
Pure_Function	48

R

Random number generation.....	89, 192
Range_Length	70
Ravenscar	45
Read attribute.....	88
Real-Time Systems Annex compliance.....	206
Record Representation Clause.....	148
Record representation clauses.....	84
Regular expressions.....	192, 193
Removing command line arguments.....	184
Representation Clause.....	133
Representation clauses.....	79
Representation Clauses.....	133
Representation clauses, enumeration.....	84
Representation clauses, records.....	84
Representation of enums.....	66
Representation of wide characters.....	198
Representation Pragma.....	133
Representation, determination of.....	153
Response file for command line.....	184
Restricted Run Time	47
Restriction_Warnings	49
Restrictions definitions.....	198
Result	70
Return values, passing mechanism.....	68
Rotate_Left	130
Rotate_Right	130
Run-time restrictions access.....	198

S

Safe_Emax	71
Safe_Large	71
Secondary Stack Info.....	193
Secure Hash Algorithm SHA-1.....	193
Secure Hash Algorithm SHA-224.....	193
Secure Hash Algorithm SHA-256.....	193
Secure Hash Algorithm SHA-384.....	193

Secure Hash Algorithm SHA-512.....	193
Semaphores.....	193
Serial_Communications.....	193
Sets of strings.....	194
Shared	49
Shared passive packages.....	206
SHARED_MEMORY_DIRECTORY environment variable.....	207
Shift_Left	131
Shift_Right	131
Shift_Right_Arithmetic	131
Short_Circuit_And_Or	49
Signals.....	193
Simple I/O.....	191
Simple_Barriers	110
Size Clause.....	134
Size clauses	82
Size for biased representation.....	137
Size of Address	63
Size, of objects.....	138
Size , setting for not-first subtype.....	73
Size, used for objects.....	69
Size , VADS compatibility.....	59, 73
Size, variant record objects.....	136
Small	71
Sockets.....	193
Sorting.....	187, 191
Source Information.....	194
Source_File_Name	49
Source_File_Name_Project	51
Source_Location	131
Source_Reference	51
Spawn capability.....	192
Spell checking.....	194, 196
SPITBOL interface.....	194
SPITBOL pattern matching.....	194
SPITBOL Tables.....	194
Static_Priorities	114
Static_Storage_Size	114
Storage place attributes.....	85
Storage pool, global.....	197
Storage pool, local.....	197
Storage_Size Clause	135
Storage_Unit	12, 71
Stream files.....	171
Stream operations.....	198
Stream oriented attributes.....	87, 88
Stream_Convert	51
String decoding.....	189
String encoding.....	190
String maps.....	194
String splitter.....	195
String stream operations.....	198
Stub_Type	71
Style_Checks	52
Subprogram address.....	64
Subtitle	53
Suppress	53

Suppress_All 54
 Suppress_Exception_Locations 54
 Suppress_Initialization 54
 Suppressing external name 19, 20, 21
 Suppressing initialization 54
 Suppression of checks 79
 system, extending 21
 System.Address_Image ('s-addima.ads') 197
 System.Assertions ('s-assert.ads') 197
 System.Memory ('s-memory.ads') 197
 System.Partition_Interface ('s-parint.ads') 197
 System.Pool_Global ('s-pooglo.ads') 197
 System.Pool_Local ('s-pooloc.ads') 197
 System.Restrictions ('s-restri.ads') 198
 System.Rident ('s-rident.ads') 198
 System.Strings.Stream_Ops ('s-ststop.ads') 198
 System.Task_Info ('s-tasinf.ads') 198
 System.Wch_Cnv ('s-wchcnv.ads') 198
 System.Wch_Con ('s-wchcon.ads') 198

T

Table implementation 190, 195
 Target_Name 71
 Task locking 195
 Task specific storage 56
 Task synchronization 195
 Task_Attributes 96
 Task_Info 54
 Task_Info pragma 198
 Task_Name 55
 Task_Storage 55
 Tasking restrictions 97
 Text_IO 191
 Text_IO extensions 171
 Text_IO for unbounded strings 171
 Text_IO resetting standard files 185
 Text_IO, extensions for unbounded strings 185
 Text_IO, extensions for unbounded wide strings 185
 Text_IO, extensions for unbounded wide wide strings 185
 Thread_Local_Storage 56
 Threads, foreign 195
 Tick 71
 Time 188
 Time stamp 195
 Time, monotonic 97
 Time_Slice 56
 Title 56
 TLS (Thread Local Storage) 56
 To_Address 72, 151
 Trace back facilities 195
 Traceback for Exception Occurrence 185
 trampoline 112
 Type_Class 72

Typographical conventions 2

U

UET_Address 72
 Unbounded_String, IO support 185
 Unbounded_String, Text_IO operations 171
 Unbounded_Wide_String, IO support 185
 Unbounded_Wide_Wide_String, IO support 185
 Unchecked conversion 86
 Unchecked deallocation 87
 Unchecked_Union 56
 Unconstrained_Array 72
 Unicode 189, 190
 Unicode categorization, Wide_Character 185
 Unicode categorization, Wide_Wide_Character 186
 Unimplemented_Unit 57
 Unions in C 56
 Universal_Aliasing 57
 Universal_Data 57
 Universal_Literal_String 72
 Unmodified 57
 Unreferenced 58
 Unreferenced_Objects 58
 Unreserve_All_Interrupts 59
 Unrestricted_Access 73
 Unsuppress 59
 Use_VADS_Size 59
 UTF-8 189, 190
 UTF-8 representation 187
 UTF-8 string decoding 189
 UTF-8 string encoding 190

V

VADS_Size 73
 Validity_Checks 60
 Value_Size 73, 138
 Variant record objects, size 136
 Version, of compiler 189
 Volatile 60
 VxWorks, Get_Immediate 197
 VxWorks, I/O interfacing 197
 VxWorks, interfacing 197

W

Warnings 60
 Warnings, unmodified 57
 Warnings, unreferenced 58
 Wchar_T_Size 73
 Weak_External 62
 Wide character representations 187
 Wide character codes 196
 Wide character decoding 189
 Wide character encoding 189, 190
 Wide Character, Representation 198

Wide String, Conversion..... 198
Wide_Character-Encoding..... 62
Wide_String splitter..... 196
Wide_Text_IO resetting standard files..... 186
Wide_Wide_String splitter..... 196
Wide_Wide_Text_IO resetting standard files... 186
Windows Registry..... 192
Word_Size..... 74
Write attribute..... 88

X

XDR representation..... 88

Z

Zero address, passing..... 69

Table of Contents

About This Guide	1
What This Reference Manual Contains	1
Conventions	2
Related Information	2
1 Implementation Defined Pragmas	5
Pragma Abort_Defer	5
Pragma Ada.83	5
Pragma Ada.95	5
Pragma Ada.05	6
Pragma Ada.2005	6
Pragma Annotate	6
Pragma Assert	6
Pragma Assume_No_Invalid_Values	7
Pragma Ast_Entry	8
Pragma C_Pass_By_Copy	8
Pragma Check	8
Pragma Check_Name	9
Pragma Check_Policy	9
Pragma Comment	10
Pragma Common_Object	10
Pragma Compile_Time_Error	10
Pragma Compile_Time_Warning	11
Pragma Compiler_Unit	11
Pragma Complete_Representation	11
Pragma Complex_Representation	12
Pragma Component_Alignment	12
Pragma Convention_Identifier	13
Pragma CPP_Class	13
Pragma CPP_Constructor	14
Pragma CPP_Virtual	14
Pragma CPP_Vtable	14
Pragma Debug	15
Pragma Debug_Policy	15
Pragma Detect_Blocking	15
Pragma Elaboration_Checks	15
Pragma Eliminate	15
Pragma Export_Exception	17
Pragma Export_Function	18
Pragma Export_Object	19
Pragma Export_Procedure	19
Pragma Export_Value	20
Pragma Export_Valued_Procedure	20

Pragma Extend_System	21
Pragma External	22
Pragma External_Name_Casing	22
Pragma Fast_Math	23
Pragma Favor_Top_Level	24
Pragma Finalize_Storage_Only	24
Pragma Float_Representation	24
Pragma Ident	25
Pragma Implemented_By_Entry	25
Pragma Implicit_Packing	25
Pragma Import_Exception	26
Pragma Import_Function	26
Pragma Import_Object	27
Pragma Import_Procedure	28
Pragma Import_Valued_Procedure	29
Pragma Initialize_Scalars	29
Pragma Inline_Always	30
Pragma Inline_Generic	30
Pragma Interface	31
Pragma Interface_Name	31
Pragma Interrupt_Handler	31
Pragma Interrupt_State	31
Pragma Keep_Names	32
Pragma License	33
Pragma Link_With	34
Pragma Linker_Alias	34
Pragma Linker_Constructor	34
Pragma Linker_Destructor	35
Pragma Linker_Section	35
Pragma Long_Float	36
Pragma Machine_Attribute	36
Pragma Main	36
Pragma Main_Storage	37
Pragma No_Body	37
Pragma No_Return	37
Pragma No_Strict_Aliasing	38
Pragma Normalize_Scalars	38
Pragma Obsolescent	39
Pragma Optimize_Alignment	40
Pragma Passive	41
Pragma Persistent_BSS	41
Pragma Polling	42
Pragma Postcondition	42
Pragma Precondition	44
Pragma Profile (Ravenscar)	45
Pragma Profile (Restricted)	47
Pragma Psect_Object	48
Pragma Pure_Function	48

Pragma Restriction_Warnings	49
Pragma Shared	49
Pragma Short_Circuit_And_Or	49
Pragma Source_File_Name	49
Pragma Source_File_Name_Project	51
Pragma Source_Reference	51
Pragma Stream_Convert	51
Pragma Style_Checks	52
Pragma Subtitle	53
Pragma Suppress	53
Pragma Suppress_All	54
Pragma Suppress_Exception_Locations	54
Pragma Suppress_Initialization	54
Pragma Task_Info	54
Pragma Task_Name	55
Pragma Task_Storage	55
Pragma Thread_Local_Storage	56
Pragma Time_Slice	56
Pragma Title	56
Pragma Unchecked_Union	56
Pragma Unimplemented_Unit	57
Pragma Universal_Aliasing	57
Pragma Universal_Data	57
Pragma Unmodified	57
Pragma Unreferenced	58
Pragma Unreferenced_Objects	58
Pragma Unreserve_All_Interrupts	59
Pragma Unsuppress	59
Pragma Use_VADS_Size	59
Pragma Validity_Checks	60
Pragma Volatile	60
Pragma Warnings	60
Pragma Weak_External	62
Pragma Wide_Character-Encoding	62

2 Implementation Defined Attributes..... 63

Abort_Signal	63
Address_Size	63
Asm_Input	63
Asm_Output	63
AST_Entry	64
Bit	64
Bit_Position	64
Compiler_Version	64
Code_Address	64
Default_Bit_Order	65
Elaborated	65
Elab_Body	65

Elab_Spec	65
Emax	65
Enabled	66
Enum_Rep	66
Enum_Val	66
Epsilon	66
Fixed_Value	67
Has_Access_Values	67
Has_Discriminants	67
Img	67
Integer_Value	67
Invalid_Value	68
Large	68
Machine_Size	68
Mantissa	68
Max_Interrupt_Priority	68
Max_Priority	68
Maximum_Alignment	68
Mechanism_Code	68
Null_Parameter	69
Object_Size	69
Old	69
Passed_By_Reference	70
Pool_Address	70
Range_Length	70
Result	70
Safe_Emax	71
Safe_Large	71
Small	71
Storage_Unit	71
Stub_Type	71
Target_Name	71
Tick	71
To_Address	72
Type_Class	72
UET_Address	72
Unconstrained_Array	72
Universal_Literal_String	72
Unrestricted_Access	73
VADS_Size	73
Value_Size	73
Wchar_T_Size	73
Word_Size	74

3	Implementation Advice	75
1.1.3(20):	Error Detection	75
1.1.3(31):	Child Units	75
1.1.5(12):	Bounded Errors	75
2.8(16):	Pragmas	76
2.8(17-19):	Pragmas	76
3.5.2(5):	Alternative Character Sets	77
3.5.4(28):	Integer Types	77
3.5.4(29):	Integer Types	77
3.5.5(8):	Enumeration Values	78
3.5.7(17):	Float Types	78
3.6.2(11):	Multidimensional Arrays	78
9.6(30-31):	Duration'Small	78
10.2.1(12):	Consistent Representation	79
11.4.1(19):	Exception Information	79
11.5(28):	Suppression of Checks	79
13.1 (21-24):	Representation Clauses	80
13.2(6-8):	Packed Types	80
13.3(14-19):	Address Clauses	81
13.3(29-35):	Alignment Clauses	82
13.3(42-43):	Size Clauses	83
13.3(50-56):	Size Clauses	83
13.3(71-73):	Component Size Clauses	84
13.4(9-10):	Enumeration Representation Clauses	84
13.5.1(17-22):	Record Representation Clauses	84
13.5.2(5):	Storage Place Attributes	85
13.5.3(7-8):	Bit Ordering	85
13.7(37):	Address as Private	86
13.7.1(16):	Address Operations	86
13.9(14-17):	Unchecked Conversion	86
13.11(23-25):	Implicit Heap Usage	87
13.11.2(17):	Unchecked De-allocation	87
13.13.2(17):	Stream Oriented Attributes	88
A.1(52):	Names of Predefined Numeric Types	88
A.3.2(49):	Ada.Characters.Handling	88
A.4.4(106):	Bounded-Length String Handling	89
A.5.2(46-47):	Random Number Generation	89
A.10.7(23):	Get_Immediate	89
B.1(39-41):	Pragma Export	90
B.2(12-13):	Package Interfaces	90
B.3(63-71):	Interfacing with C	91
B.4(95-98):	Interfacing with COBOL	92
B.5(22-26):	Interfacing with Fortran	92
C.1(3-5):	Access to Machine Operations	93
C.1(10-16):	Access to Machine Operations	94
C.3(28):	Interrupt Support	95
C.3.1(20-21):	Protected Procedure Handlers	95
C.3.2(25):	Package Interrupts	95

C.4(14): Pre-elaboration Requirements	95
C.5(8): Pragma <code>Discard_Names</code>	96
C.7.2(30): The Package <code>Task_Attributes</code>	96
D.3(17): Locking Policies	96
D.4(16): Entry Queuing Policies	96
D.6(9-10): Preemptive Abort	96
D.7(21): Tasking Restrictions	97
D.8(47-49): Monotonic Time	97
E.5(28-29): Partition Communication Subsystem	98
F(7): COBOL Support	98
F.1(2): Decimal Radix Support	98
G: Numerics	98
G.1.1(56-58): Complex Types	99
G.1.2(49): Complex Elementary Functions	100
G.2.4(19): Accuracy Requirements	100
G.2.6(15): Complex Arithmetic Accuracy	100
4 Implementation Defined Characteristics ...	101
5 Intrinsic Subprograms	129
5.1 Intrinsic Operators	129
5.2 Enclosing_Entity	129
5.3 Exception_Information	129
5.4 Exception_Message	130
5.5 Exception_Name	130
5.6 File	130
5.7 Line	130
5.8 Rotate_Left	130
5.9 Rotate_Right	130
5.10 Shift_Left	131
5.11 Shift_Right	131
5.12 Shift_Right_Arithmetic	131
5.13 Source_Location	131
6 Representation Clauses and Pragmas	133
6.1 Alignment Clauses	133
6.2 Size Clauses	134
6.3 Storage_Size Clauses	135
6.4 Size of Variant Record Objects	136
6.5 Biased Representation	137
6.6 Value_Size and Object_Size Clauses	138
6.7 Component_Size Clauses	140
6.8 Bit_Order Clauses	141
6.9 Effect of Bit_Order on Byte Ordering	142
6.10 Pragma Pack for Arrays	145
6.11 Pragma Pack for Records	147
6.12 Record Representation Clauses	148

6.13	Enumeration Clauses	149
6.14	Address Clauses	149
6.15	Effect of Convention on Representation.....	152
6.16	Determining the Representations chosen by GNAT	153
7	Standard Library Routines	157
8	The Implementation of Standard I/O	167
8.1	Standard I/O Packages	167
8.2	FORM Strings	168
8.3	Direct_IO	168
8.4	Sequential_IO	168
8.5	Text_IO	169
8.5.1	Stream Pointer Positioning	170
8.5.2	Reading and Writing Non-Regular Files	170
8.5.3	Get_Immediate	171
8.5.4	Treating Text_IO Files as Streams	171
8.5.5	Text_IO Extensions	171
8.5.6	Text_IO Facilities for Unbounded Strings	171
8.6	Wide_Text_IO	172
8.6.1	Stream Pointer Positioning	174
8.6.2	Reading and Writing Non-Regular Files	174
8.7	Wide_Wide_Text_IO	174
8.7.1	Stream Pointer Positioning	176
8.7.2	Reading and Writing Non-Regular Files	176
8.8	Stream_IO	176
8.9	Text Translation	176
8.10	Shared Files	176
8.11	Filenames encoding	177
8.12	Open Modes	178
8.13	Operations on C Streams	178
8.14	Interfacing to C Streams	181
9	The GNAT Library	183
9.1	Ada.Characters.Latin_9 ('a-chlat9.ads')	183
9.2	Ada.Characters.Wide_Latin_1 ('a-cwila1.ads')	183
9.3	Ada.Characters.Wide_Latin_9 ('a-cwila1.ads')	183
9.4	Ada.Characters.Wide_Wide_Latin_1 ('a-chzla1.ads')	184
9.5	Ada.Characters.Wide_Wide_Latin_9 ('a-chzla9.ads')	184
9.6	Ada.Command_Line.Environment ('a-colien.ads')	184
9.7	Ada.Command_Line.Remove ('a-colire.ads')	184
9.8	Ada.Command_Line.Response_File ('a-clrefi.ads')	184
9.9	Ada.Direct_IO.C_Streams ('a-diocst.ads')	184
9.10	Ada.Exceptions.Is_Null_Occurrence ('a-einuoc.ads')	184
9.11	Ada.Exceptions.Last_Chance_Handler ('a-elchha.ads')	184
9.12	Ada.Exceptions.Traceback ('a-exctra.ads')	185
9.13	Ada.Sequential_IO.C_Streams ('a-siocst.ads')	185

9.14	Ada.Streams.Stream_IO.C_Streams ('a-ssicst.ads')	185
9.15	Ada.Strings.Unbounded.Text_IO ('a-suteio.ads')	185
9.16	Ada.Strings.Wide_Unbounded.Wide_Text_IO ('a-swuwti.ads')	
		185
9.17	Ada.Strings.Wide_Wide_Unbounded.Wide_Wide_Text_IO	
	('a-szuzti.ads')	185
9.18	Ada.Text_IO.C_Streams ('a-tiocst.ads')	185
9.19	Ada.Text_IO.Reset_Standard_Files ('a-tirsfi.ads')	185
9.20	Ada.Wide_Characters.Unicode ('a-wichun.ads')	185
9.21	Ada.Wide_Text_IO.C_Streams ('a-wtcstr.ads')	186
9.22	Ada.Wide_Text_IO.Reset_Standard_Files ('a-wrstfi.ads')	
		186
9.23	Ada.Wide_Wide_Characters.Unicode ('a-zchuni.ads')	186
9.24	Ada.Wide_Wide_Text_IO.C_Streams ('a-ztcstr.ads')	186
9.25	Ada.Wide_Wide_Text_IO.Reset_Standard_Files	
	('a-zrstfi.ads')	186
9.26	GNAT.Alivec ('g-alive.ads')	186
9.27	GNAT.Alivec.Conversions ('g-altcon.ads')	186
9.28	GNAT.Alivec.Vector_Operations ('g-alveop.ads')	186
9.29	GNAT.Alivec.Vector_Types ('g-alvety.ads')	186
9.30	GNAT.Alivec.Vector_Views ('g-alvevi.ads')	187
9.31	GNAT.Array_Split ('g-arrspl.ads')	187
9.32	GNAT.AWK ('g-awk.ads')	187
9.33	GNAT.Bounded_Buffers ('g-boubuf.ads')	187
9.34	GNAT.Bounded_Mailboxes ('g-boumai.ads')	187
9.35	GNAT.Bubble_Sort ('g-bubsor.ads')	187
9.36	GNAT.Bubble_Sort_A ('g-busora.ads')	187
9.37	GNAT.Bubble_Sort_G ('g-busorg.ads')	187
9.38	GNAT.Byte_Order_Mark ('g-byorma.ads')	187
9.39	GNAT.Byte_Swapping ('g-bytswa.ads')	188
9.40	GNAT.Calendar ('g-calend.ads')	188
9.41	GNAT.Calendar.Time_IO ('g-catiio.ads')	188
9.42	GNAT.CRC32 ('g-crc32.ads')	188
9.43	GNAT.Case_Util ('g-casuti.ads')	188
9.44	GNAT.CGI ('g-cgi.ads')	188
9.45	GNAT.CGI.Cookie ('g-cgicoo.ads')	188
9.46	GNAT.CGI.Debug ('g-cgideb.ads')	188
9.47	GNAT.Command_Line ('g-comlin.ads')	188
9.48	GNAT.Compiler_Version ('g-comver.ads')	189
9.49	GNAT.Ctrl_C ('g-ctrl_c.ads')	189
9.50	GNAT.Current_Exception ('g-curexc.ads')	189
9.51	GNAT.Debug_Pools ('g-debpoo.ads')	189
9.52	GNAT.Debug_Uutilities ('g-debuti.ads')	189
9.53	GNAT.Decode_String ('g-decstr.ads')	189
9.54	GNAT.Decode_UTF8_String ('g-deutst.ads')	189
9.55	GNAT.Directory_Operations ('g-dirope.ads')	189
9.56	GNAT.Directory_Operations.Iteration ('g-diopit.ads')	
		189

9.57	GNAT.Dynamic_HTables ('g-dynhta.ads')	190
9.58	GNAT.Dynamic_Tables ('g-dyntab.ads')	190
9.59	GNAT.Encode_String ('g-encstr.ads')	190
9.60	GNAT.Encode_UTF8_String ('g-enutst.ads')	190
9.61	GNAT.Exception_Actions ('g-excact.ads')	190
9.62	GNAT.Exception_Traces ('g-extra.ads')	190
9.63	GNAT.Exceptions ('g-expect.ads')	190
9.64	GNAT.Expect ('g-expect.ads')	190
9.65	GNAT.Float_Control ('g-flocon.ads')	191
9.66	GNAT.Heap_Sort ('g-heasor.ads')	191
9.67	GNAT.Heap_Sort_A ('g-hesora.ads')	191
9.68	GNAT.Heap_Sort_G ('g-hesorg.ads')	191
9.69	GNAT.HTable ('g-htable.ads')	191
9.70	GNAT.IO ('g-io.ads')	191
9.71	GNAT.IO_Aux ('g-io_aux.ads')	191
9.72	GNAT.Lock_Files ('g-locfil.ads')	192
9.73	GNAT.MD5 ('g-md5.ads')	192
9.74	GNAT.Memory_Dump ('g-memdum.ads')	192
9.75	GNAT.Most_Recent_Exception ('g-moreex.ads')	192
9.76	GNAT.OS_Lib ('g-os_lib.ads')	192
9.77	GNAT.Perfect_Hash_Generators ('g-pehage.ads')	192
9.78	GNAT.Random_Numbers ('g-rannum.ads')	192
9.79	GNAT.Regexp ('g-regexp.ads')	192
9.80	GNAT.Registry ('g-regist.ads')	192
9.81	GNAT.Regpat ('g-regpat.ads')	193
9.82	GNAT.Secondary_Stack_Info ('g-sestin.ads')	193
9.83	GNAT.Semaphores ('g-semaph.ads')	193
9.84	GNAT.Serial_Communications ('g-sercom.ads')	193
9.85	GNAT.SHA1 ('g-sha1.ads')	193
9.86	GNAT.SHA224 ('g-sha224.ads')	193
9.87	GNAT.SHA256 ('g-sha256.ads')	193
9.88	GNAT.SHA384 ('g-sha384.ads')	193
9.89	GNAT.SHA512 ('g-sha512.ads')	193
9.90	GNAT.Signals ('g-signal.ads')	193
9.91	GNAT.Sockets ('g-socket.ads')	193
9.92	GNAT.Source_Info ('g-souinf.ads')	194
9.93	GNAT.Spelling_Checker ('g-speche.ads')	194
9.94	GNAT.Spelling_Checker_Generic ('g-spchge.ads')	194
9.95	GNAT.Spitbol.Patterns ('g-spipat.ads')	194
9.96	GNAT.Spitbol ('g-spitbo.ads')	194
9.97	GNAT.Spitbol.Table_Boolean ('g-sptabo.ads')	194
9.98	GNAT.Spitbol.Table_Integer ('g-sptain.ads')	194
9.99	GNAT.Spitbol.Table_VString ('g-sptavs.ads')	194
9.100	GNAT.SSE ('g-sse.ads')	194
9.101	GNAT.SSE.Vector_Types ('g-ssvety.ads')	195
9.102	GNAT.Strings ('g-string.ads')	195
9.103	GNAT.String_Split ('g-strspl.ads')	195
9.104	GNAT.Table ('g-table.ads')	195

9.105	GNAT.Task_Lock ('g-tasloc.ads').....	195
9.106	GNAT.Time_Stamp ('g-timsta.ads').....	195
9.107	GNAT.Threads ('g-thread.ads').....	195
9.108	GNAT.Traceback ('g-traceb.ads').....	195
9.109	GNAT.Traceback.Symbolic ('g-trasym.ads').....	195
9.110	GNAT.UTF_32 ('g-table.ads').....	196
9.111	GNAT.Wide_Spelling_Checker ('g-u3spch.ads').....	196
9.112	GNAT.Wide_Spelling_Checker ('g-wispch.ads').....	196
9.113	GNAT.Wide_String_Split ('g-wistsp.ads').....	196
9.114	GNAT.Wide_Wide_Spelling_Checker ('g-zspche.ads').....	196
9.115	GNAT.Wide_Wide_String_Split ('g-zistsp.ads').....	196
9.116	Interfaces.C.Extensions ('i-cexten.ads').....	196
9.117	Interfaces.C.Streams ('i-cstrea.ads').....	196
9.118	Interfaces.CPP ('i-cpp.ads').....	196
9.119	Interfaces.Packed_Decimal ('i-pacdec.ads').....	197
9.120	Interfaces.VxWorks ('i-vxwork.ads').....	197
9.121	Interfaces.VxWorks.IO ('i-vxwoio.ads').....	197
9.122	System.Address_Image ('s-addima.ads').....	197
9.123	System.Assertions ('s-assert.ads').....	197
9.124	System.Memory ('s-memory.ads').....	197
9.125	System.Partition_Interface ('s-parint.ads').....	197
9.126	System.Pool_Global ('s-pooglo.ads').....	197
9.127	System.Pool_Local ('s-pooloc.ads').....	197
9.128	System.Restrictions ('s-restri.ads').....	198
9.129	System.Rident ('s-rident.ads').....	198
9.130	System.Strings.Stream_Ops ('s-ststop.ads').....	198
9.131	System.Task_Info ('s-tasinf.ads').....	198
9.132	System.Wch_Cnv ('s-wchcnv.ads').....	198
9.133	System.Wch_Con ('s-wchcon.ads').....	198
10	Interfacing to Other Languages.....	199
10.1	Interfacing to C.....	199
10.2	Interfacing to C++.....	200
10.3	Interfacing to COBOL.....	200
10.4	Interfacing to Fortran.....	200
10.5	Interfacing to non-GNAT Ada code.....	200
11	Specialized Needs Annexes.....	201

12	Implementation of Specific Ada Features	203
12.1	Machine Code Insertions	203
12.2	GNAT Implementation of Tasking	205
12.2.1	Mapping Ada Tasks onto the Underlying Kernel Threads	205
12.2.2	Ensuring Compliance with the Real-Time Annex	206
12.3	GNAT Implementation of Shared Passive Packages	206
12.4	Code Generation for Array Aggregates	207
12.4.1	Static constant aggregates with static bounds	208
12.4.2	Constant aggregates with unconstrained nominal types	208
12.4.3	Aggregates with static bounds	208
12.4.4	Aggregates with non-static bounds	209
12.4.5	Aggregates in assignment statements	209
12.5	The Size of Discriminated Records with Default Discriminants	209
12.6	Strict Conformance to the Ada Reference Manual	210
13	Project File Reference	213
13.1	Reserved Words	213
13.2	Lexical Elements	213
13.3	Declarations	213
13.4	Empty declarations	213
13.5	Typed string declarations	214
13.6	Variables	214
13.7	Expressions	214
13.7.1	Concatenation	215
13.8	Attributes	215
13.9	Project Attributes	215
13.10	Attribute References	218
13.11	External Values	219
13.12	Case Construction	219
13.13	Packages	219
13.13.1	Package Naming	220
13.13.2	package Compiler	222
13.13.3	package Builder	222
13.13.4	package Gnatls	223
13.13.5	package Binder	223
13.13.6	package Linker	223
13.13.7	package Cross_Reference	223
13.13.8	package Finder	224
13.13.9	package Check	224
13.13.10	package Pretty_Printer	224
13.13.11	package gnatstub	225
13.13.12	package Eliminate	225
13.13.13	package Metrics	225
13.13.14	package IDE	226
13.14	Package Renamings	227

13.15	Projects	227
13.16	Project Extensions	228
13.17	Project File Elaboration	228
14	Obsolescent Features	229
14.1	pragma No_Run_Time	229
14.2	pragma Ravenscar	229
14.3	pragma Restricted_Run_Time	229
	GNU Free Documentation License	231
	ADDENDUM: How to use this License for your documents	237
	Index	239